



掌握FreeRTOS™实时内核

实践教程指南

理查德·巴里-和FreeRTOS团队

版本- 1.0

FreeRTOS™、FreeRTOS.org™和FreeRTOS徽标是亚马逊网络服务的商标。奥本托斯®和萨弗托斯®是威滕斯坦高完整性系统有限公司的商标。所有其他品牌或产品名称均为其各自持有人的财产。



除非另有说明，所有文本、源代码和图表都是AmazonWeb服务的专有财产。

©2023Amazon. com, Inc. 或其附属公司。保留所有权利。

<https://www.FreeRTOS.org>

<https://forums.FreeRTOS.org>

<https://github.com/FreeRTOS>

本文是免费提供的。作为回报，我们要求您创建一个PR来提供更正。
在<https://forums.FreeRTOS.org> 发布反馈和评论。

给*Caroline, India*和*Max*。 - R. B.

献给使用*FreeRTOS*构建伟大系统的下一代工程师。 - J. J.

缩略语列表

缩写	意义
ADC	模数转换器
API	应用程序编程接口
DMA	直接存储器访问
FAQ	常见问题
FIFO	先入先出
HMI	人机界面
IDE	综合开发环境
IRQ	中断请求
ISR	中断服务例程
LCD	液晶显示器
MCU	微控制器
RMS	速率单调调度
RTOS	实时操作系统
SIL	安全完整性等级
SPI	串行外设接口
TCB	任务控制块
UART	通用异步收发机

1 前言

1.1 小型嵌入式系统中的多任务处理

1.1.1 关于FreeRTOS内核

FreeRTOS是一个C库的集合，由一个实时内核和一组实现互补功能的模块化库组成。

理查德·巴里最初在2003年左右开发了FreeRTOS。实时工程师有限公司，理查德的公司，继续与世界领先的芯片公司密切合作开发FreeRTOS，直到亚马逊网络服务（AWS）接管管理FreeRTOS 2016。理查德现在继续他作为AWS物联网团队的高级首席工程师在FreeRTOS上的工作。FreeRTOS是MIT授权的开放源代码，可用于任何目的。你不必成为AWS的客户才能从AWS的管理中获益！

FreeRTOS内核非常适合于运行在微控制器或小型微处理器上的深度嵌入式实时应用程序。这种类型的应用程序通常包括硬实时需求和软实时需求的混合体。

软实时需求声明了一个时间截止期限——但是违反截止期限不会使系统无用。例如，对击键的响应太慢可能会让系统看起来失去响应，而实际上不会让它无法使用。

硬实时要求规定了一个时间期限——违反这个时间期限将导致系统的绝对失败。例如，如果驾驶员安全气囊对碰撞传感器输入反应太慢，可能弊大于利。

FreeRTOS内核是一个实时内核（或实时调度器），它使构建在FreeRTOS上的应用程序能够满足它们的硬实时需求。它允许应用程序被组织为一个独立的执行线程的集合。例如，在一个只有一个核心的处理器上，任何时候都只能执行一个执行线程。内核通过检查应用程序设计器分配给每个线程的优先级来决定要执行哪个线程。在最简单的情况下，应用程序设计器可以为实现硬实时需求的线程分配更高的优先级，而为实现软实时需求的线程分配更低的优先级。以这种方式分配优先级将确保硬实时线程总是在软实时线程之前执行，但优先级分配决策并不总是那么简单。

如果你还没有完全理解前一段中的概念，请不要担心。下面的章节提供了详细的解释和许多示例，以帮助您理解如何使用实时内核，特别是FreeRTOS。

1.1.2 价值主张

FreeRTOS内核在全球前所未有的成功来自于其引人注目的价值主张：FreeRTOS是专业开发的，严格的质量控制，健壮，支持，不包含任何知识产权所有权歧义，并且在商业应用中真正自由使用，不需要任何要求

公开您的专有源代码。此外，AWS的管理提供了全球存在、专家安全事件响应程序、大型和多样化的开发团队、正式验证、笔测试、内存安全证明和长期支持方面的专业知识——所有这些都将FreeRTOS作为硬件、开发工具和云服务中立的开源项目。FreeRTOS的开发在GitHub中是透明的和社区驱动的，不需要任何特殊的工具或开发实践。

你可以使用FreeRTOS将产品推向市场，更不用说支付任何费用了，成千上万的公司就这样做了。如果您在任何时候想要获得额外的备份，或者如果您的法律团队需要额外的书面担保或赔偿，那么我们的战略合作伙伴将提供简单的低成本商业许可选项。内心的平静伴随着知识，你可以选择商业路线。

1.1.3 一个关于术语的说明

在FreeRTOS中，每个执行的线程都被称为“任务”。在嵌入式社区中对术语没有共识，但我更喜欢“task”而不是“线程”，因为线程在某些应用领域中可以有更具体的含义。

1.1.4 为什么要使用RTOS？

有许多成熟的技术可以编写不使用多线程内核的良好的嵌入式软件。如果正在开发的系统很简单，那么这些技术可能会提供最合适的解决方案。在更复杂的情况下，使用内核可能更可取，但在交叉点发生的地方，它总是主观的。

如前所述，任务优先级可以帮助确保应用程序满足其处理截止日期，但内核可以带来其他不那么明显的好处。下面非常简要地列出了其中的一些内容。

- 抽象化后的时间信息

RTOS负责执行时间，并为应用程序提供与时间相关的API。这使得应用程序代码的结构更加简单，整体代码的大小也更小。

- 可维护性/可扩展性

抽象化时间细节会导致模块之间的相互依赖关系减少，并允许软件以一种受控和可预测的方式发展。此外，内核还负责时间安排，因此应用程序的性能不太容易受到底层硬件中的变化的影响。

- 模块化

任务是独立的模块，每个模块都应该有一个明确定义的目的。

- 团队发展

任务还应该定义良好的接口，允许更容易的团队开发。

- 更容易的测试

具有定义良好的独立模块和干净接口的任务更容易单独测试。代码重用

- 具有更大的模块化和更少的相互依赖性设计的代码更容易重用。

- 提高效率

使用RTOS的应用程序代码可以完全是由事件驱动的。通过轮询未发生的事件，不需要浪费任何处理时间。

抵消从事件驱动中获得的效率是需要处理RTOS滴答中断并将执行从一个任务切换到另一个任务。然而，不使用RTOS的应用程序通常会包含某种形式的滴答中断。

- 空闲时间

当不存在需要处理的应用程序任务时，自动创建的空闲任务将会执行。空闲任务可以测量备用处理容量，执行后台检查，或将处理器置于低功耗模式。

- 电力管理

使用RTOS所提高效率允许处理器在低功耗模式下花更多的时间。

每次空闲任务运行时，将处理器置于低功耗状态，可以显著降低功耗。FreeRTOS也有一个特殊的减少滴答模式。使用Tick-less模式允许处理器进入比其他方式更低功耗的模式，并在低功率模式下保持更长的时间。

- 灵活的中断处理

中断处理程序可以通过延迟处理到由应用程序编写器创建的任务或自动创建的RTOS守护进程任务（也称为计时器任务），从而保持非常短。

- 混合加工要求

简单的设计模式可以在应用程序中实现周期性、连续和事件驱动的混合处理。此外，通过选择适当的任务和中断优先级，可以满足硬实时和软实时需求。

1.1.5 FreeRTOS内核功能

FreeRTOS内核具有以下标准特性：

- 预防性或合作性操作
- 可选的时间切片

- 非常灵活的任务优先级分配
- 灵活、快速和轻便的任务通知机制
- 队列
- 二进制信号器
- 计数信号器
- 互斥
- 递归性突变
- 软件定时器
- 事件组
- 流缓冲区
- 消息缓冲区
- 协例程（不建议使用）
- 钩子功能
- 怠速挂钩功能
- 堆栈溢出检查
- 跟踪宏
- 任务运行时统计信息收集
- 可选的商业许可和支持
- 全中断嵌套模型（对于某些体系结构）
- 适用于极低功耗的应用程序（对于某些架构）
- 内存保护单元支持隔离任务和提高应用程序安全性（对于某些体系结构）
- 软件管理的中断堆栈在适当时（这可以帮助保存RAM）使用静态或动态分配的内存创建RTOS对象的能力

1.1.6 授权、OpenRTOS和SafeRTOS系列

FreeRTOS MIT开放源码许可旨在确保：

- FreeRTOS可以用于商业应用程序。
- FreeRTOS本身仍然免费提供。
- FreeRTOS用户保留了对其知识产权的所有权。

访问<https://www.FreeRTOS.org/license>。获取最新的开源许可证信息的组织/许可证。

OpenRTOS是由第三方通过亚马逊Web服务许可提供的FreeRTOS的商业授权版本。

SafeRTOS与FreeRTOS具有相同的使用模式，但它是根据要求符合各种国际公认的安全相关标准所需的实践、程序和流程开发的。

1.2 包括源文件和项目

1.2.1 获取了这本书附带的例子

该zip文件可从 <https://www.FreeRTOS.org/Documentation/code>下载。该zip包含了所有的源代码、预配置的项目文件和构建和执行本书中提供的示例所需的说明。注意，zip文件不一定包含FreeRTOS的最新版本。

本书中包含的屏幕截图显示了在微软Windows环境中执行的示例，使用FreeRTOS Windows端口。使用FreeRTOS Windows端口的项目被预先配置为使用免费的Visual Studio社区版进行构建，可从<https://www.visualstudio.com/>获得。请注意，虽然FreeRTOS Windows端口提供了一个方便的评估、测试和开发平台，但它并不能提供真正的实时行为。

2. FreeRTOS内核结构

2.1 介绍

为了帮助用户使用FreeRTOS内核文件和目录来定位自己，本章：

- 提供了FreeRTOS目录结构的顶级视图。
- 描述任何特定的FreeRTOS项目所需的源文件。
- 介绍了演示过程中的应用程序。
- 提供有关如何创建新的FreeRTOS项目的信息。

这里的描述仅涉及到官方的FreeRTOS发行版。这本书附带的例子使用了一个稍微不同的组织。

2.2 了解FreeRTOS的分布

2.2.1 定义： FreeRTOS端口

FreeRTOS可以用大约20种不同的编译器来构建，并且可以在40多种不同的处理器架构上运行。每个受支持的编译器和处理器的组合都被称为FreeRTOS端口。

2.2.2 构建(编译)FreeRTOS

FreeRTOS是一个库，它为原本将是单线程、裸金属的应用程序提供了多任务处理功能。

FreeRTOS是作为一组C源文件提供的。有些源文件是所有端口通用的，而其他文件是特定于端口的。将源文件作为项目的一部分构建，使应用程序可以使用FreeRTOS API。为每个官方的FreeRTOS端口提供了一个可以作为参考的演示应用程序。演示应用程序被预配置为构建正确的源文件，并包含正确的头文件。

在创建时，每个演示应用程序都是“开箱即用”构建的，没有编译器错误或警告。请使用FreeRTOS支持论坛 (<https://forums.FreeRTOS.org>)。让我们知道，如果对构建工具的后续更改是否意味着情况不再如此。第2.3节描述了演示应用程序。

2.2.3 FreeRTOSConfig.h

常量在名为FreeRTOSConfig.h的头文件中定义，用来配置内核。不要直接包括FreeRTOSConfig.h直接在你的源文件中！相反，在你的源文件中包括FreeRTOS.h，这将包括FreeRTOSConfig.h。

FreeRTOSConfig.h用于定制特定应用程序的FreeRTOS内核。例如，FreeRTOSConfig.h包含像configUSE_PREEMPTION这样的常量，它定义了FreeRTOS是使用协作调度还是抢占式调度^[1]。

^[1]：第4.13节描述了调度算法。

FreeRTOSConfig.h为特定的应用程序定制FreeRTOS，因此它应该位于作为应用程序一部分的目录中，而不是在包含FreeRTOS源代码的目录中。

主要的FreeRTOS发行版包含一个用于每个FreeRTOS端口的演示应用程序，并且每个演示应用程序都有自己的FreeRTOSConfig.h文件。建议从开始，然后调整，到配置通过使用的FreeRTOS端口提供的演示应用程序，而不是从头创建文件。

FreeRTOS参考手册和 <https://www.freertos.org/a00110.html>描述在FreeRTOSConfig.h中出现的常量。不必要在FreeRTOSConfig.h中包含所有常量——如果省略，许多会得到一个默认值。

2.2.4 官方发行

包括内核在内的个人FreeRTOS库，可以从它们自己的Github存储库和作为zip文件存档获得。当在生产代码中使用FreeRTOS时，获得单个库的能力是很方便的。但是，最好下载主要的FreeRTOS发行版来开始，因为它同时包含库和示例项目。

主发行版包含所有FreeRTOS库的源代码、所有FreeRTOS内核端口以及所有FreeRTOS演示应用程序的项目文件。不要被文件的数量所推迟！应用程序只需要一个小子集。

使用 <https://github.com/FreeRTOS/FreeRTOS/releases/latest>下载包含最新发行版的zip文件。或者，使用以下Git命令之一从GitHub克隆主发行版，包括从各自的Git存储库中子调制的单个库：

```
git clone https://github.com/FreeRTOS/FreeRTOS.git --recurse-submodules

git clone git@github.com:FreeRTOS/FreeRTOS.git --recurse-submodules
```

图2.1显示了FreeRTOS发行版的第一级和第二级目录：

```
FreeRTOS
|  |
|  |——Source包含FreeRTOS内核源文件
|  |
|  |——Demo  包含预配置的和端口特定的FreeRTOS内核演示
|
FreeRTOS-Plus
|
|——源代码包含其他FreeRTOS和生态系统库的源代码
```



图2. 1FreeRTOS发行版中的顶级目录

该发行版只包含FreeRTOS内核源文件的一个副本；所有演示项目都希望在FreeRTOS/源目录中找到内核源文件，如果目录结构被更改，可能不会构建。

2. 2. 6 Source目录中所有端口共有的FreeRTOS源文件

tasks.c和list.c实现了核心的FreeRTOS内核功能，并且总是必需的。它们直接位于FreeRTOS/Source目录中，如图2. 2所示。同一目录中还包含以下可选的源文件：

•queue.c

同时提供队列服务和信号量服务，如本书后面所述。queue.c几乎总是必需的。

•times.c

提供了软件定时器功能，如本书后面所述。只有在应用程序使用软件定时器时，才需要构建它。

•event_groups.c

提供了事件组功能，如本书后面所述。只有在应用程序使用事件组时，才需要构建它。

•stream_buffer.c

提供了流缓冲区和消息缓冲区功能，如本书后面所述。只有在应用程序使用流或消息缓冲区时才需要构建它。

•croutine.c

croutine.c实现了FreeRTOS的协同例程功能。只有在应用程序使用协同例程时才需要构建它。协同例程是打算用于非常小的微控制器，现在很少使用。因此，它们不再被维护，也不建议在新的设计中使用它们。在这本书中没有描述共同例程。



└─list.c	FreeRTOS源文件-总是必需的
└─queue.c	FreeRTOS源文件-几乎总是必需的
└─timers.c	FreeRTOS源文件-可选的
└─event_groups.c	FreeRTOS源文件-可选的
└─stream_buffer.c	FreeRTOS源文件-可选的
└─croutine.c	FreeRTOS源文件-可选的，并且不再维护

图2.2 FreeRTOS目录树中的核心FreeRTOS源文件

可以认识到zip文件分发中使用的文件名可能导致名称空间冲突，因为许多项目已经使用具有相同名称的文件。如果需要，用户可以更改文件名，但名称不能在发行版中更改，因为这样做将破坏与现有用户的项目以及支持FreeRTOS的开发工具的兼容性。

2.2.7 特定于一个端口的FreeRTOS源文件

FreeRTOS/Source/portable包含特定于FreeRTOS端口的源文件。可移植目录被安排为一个层次结构，首先是编译器，然后是处理器架构。图2.3显示了该层次结构。

要在使用编译器“compiler”的具有架构“arch”的处理器上运行FreeRTOS，除了核心的FreeRTOS源文件外，您还必须构建位于FreeRTOS/Source/portable/[compiler]/[arch]目录的文件。

如第3章“堆内存管理”中所述，FreeRTOS还将堆内存分配视为可移植层的一部分。如果`configSUPPORT_DYNAMIC_ALLOCATION`设置为0，则不会在项目中包含堆内存分配方案。

FreeRTOS在FreeRTOS/Source/portable/MemMang目录中提供了堆分配方案的示例。如果将“FreeRTOS”配置为使用动态内存分配，则需要在项目中包含来自该目录中的一个堆实现源文件，或者提供您自己的实现。

!不要在项目中包含多个示例堆分配实现。



```

|   └─[architecture 3] Contains files for the compiler 1 architecture 3 port
|
└─[compiler 2] Directory containing port files specific to compiler 2
    |
    └─[architecture 1] Contains files for the compiler 2 architecture 1 port
        └─[architecture 2] Contains files for the compiler 2 architecture 2 port
            └─[etc.]

```

图2.3 FreeRTOS目录树中的端口特定的源文件

2.2.8 include路径

FreeRTOS要求在编译器的包含路径中包含三个目录。这些是：

1. 核心FreeRTOS内核头文件的路径，FreeRTOS/Source/include。
2. 特定于正在使用的FreeRTOS端口的源文件的路径，
FreeRTOS/Source/portable/[compiler]/[architecture]。
3. 正确的FreeRTOSConfig.h头文件路径。

2.2.9 头文件

使用FreeRTOS API的源文件必须包含FreeRTOS.h，后面是包含API函数原型的头文件——task.h，queue.h，semphr.h，timers.h、event_groups.h、stream_buffer.h或croutine.h。不要显式地包括任何其他FreeRTOS头文件——FreeRTOS.h自动包括FreeRTOSConfig.h。

2.3 演示程序

每个FreeRTOS端口都带有至少一个演示应用程序，在其创建时，它是“开箱即用”构建的，没有编译器错误或警告。请使用FreeRTOS支持论坛 (<https://forums.FreeRTOS.org>)。让我们知道，如果对构建工具的后续更改是否意味着情况不再如此。

跨平台支持： FreeRTOS是在Windows、Linux和MacOS系统上开发和测试的，并使用了嵌入式和传统的各种工具链。但是，由于版本的差异或错过的测试，偶尔会出现构建错误。请使用FreeRTOS支持论坛 (<https://forums.FreeRTOS.org>)，以提醒我们有任何此类错误。

演示应用程序有以下几个目的：

- 提供一个工作的和预配置的项目示例，包含正确的文件和设置正确的编译器选项。
- 允许用最少的设置或先验知识进行“开箱即用”的实验。
- 演示如何使用FreeRTOSapi。
- 作为可以创建真实应用程序的基础。

- 来强调测试内核的实现。

每个演示项目都位于FreeRTOS/演示目录下的一个唯一的子目录中。子目录的名称表示演示项目所关联的端口。

FreeRTOS.org网站包含每个演示应用程序包含一个页面。本网页包括以下资料：

- 如何在FreeRTOS目录结构中找到演示版本的项目文件。
- 项目被配置为要使用的硬件或模拟器。
- 如何设置硬件来运行演示程序。
- 如何构建演示版本。
- 演示的预期行为。

所有演示项目都创建一个“公共演示任务”的子集，其现在在FreeRTOS/Demo/Common/Minimal目录中。常见的演示任务是为了演示如何使用FreeRTOS API和测试FreeRTOS内核端口——它们没有实现任何特定的有用的

功能。许多演示项目也可以配置为创建一个简单的“Blinky”风格的启动项目(小灯闪烁程序)，该项目通常会创建两个RTOS任务和一个队列。

每个演示项目都包含一个名为main.c的文件，它包含main()函数，它会在启动FreeRTOS内核之前创建演示应用程序任务。有关特定项目演示的信息，请查看main.c中的注释。

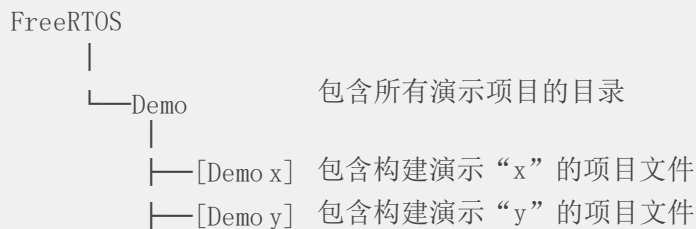


图2.4 演示版目录层次结构

2.4 创建一个FreeRTOS项目

2.4.1 正在调整所提供的演示项目之一

每个FreeRTOS端口都至少带有一个预配置的演示应用程序。建议通过调整这些现有项目中的一个来创建新项目，以确保新项目包含了正确的文件，安装了正确的中断处理程序，以及设置了正确的编译器选项。

要从现有的演示项目中创建一个新的应用程序：

1. 打开所提供的演示项目，并确保它按照预期进行构建和执行。
2. 删除实现演示任务的源文件，这些文件是位于Demo/Common中的文件(应该指演示任务的源文件文件名是Demo/Common中的那些文件，鉴于演示项目都创建一个“公共演示任务”的子集)。
3. 删除main（）中的所有函数调用，除了prvSetupHardware（） and vTaskStartScheduler（），如清单2.1所示。
4. 验证该项目是否仍可以构建中。

当您遵循这些步骤时，您将创建一个包含正确的FreeRTOS源文件的项目，但没有定义任何功能。

```
int main (void) {

    /*可执行任何必要的硬件设置。*/
    prvSetupHardware();

    /* ---应用程序任务可以在这里创建，--- */

    /*启动已创建的任务的运行。*/
    vTaskStartScheduler();

    只有当没有足够的堆来启动调度程序时，/*执行才会到达这里。*/
    for(;;);
    return 0;

}
```

清单2.1 一个新的main（）函数的模板

2.4.2 从头开始创建一个新的项目

如前所述，建议从现有的演示项目中创建新的项目。如果不需要这样做，则请使用以下步骤创建新项目：

1. 使用您所选择的工具链，创建一个还不包含任何FreeRTOS源文件的新项目。
2. 确保新项目构建、下载到目标硬件并执行。
3. 只有当您确定您已经有了一个正在工作的项目时，才能添加表中详细说明了FreeRTOS源文件到项目。
4. 将演示项目使用的FreeRTOSConfig.h头文件复制到您的新项目目录中。
5. 将以下目录添加到项目将搜索的路径中，以查找头文件：

- FreeRTOS/Source/include
- FreeRTOS/Source/portable/[compiler]/[architecture] (其中[编译器]和[体系结构]是为您选择的端口
- 包含FreeRTOSConfig.h头文件的目录

6. 从相关的演示项目中复制编译器设置。
7. 安装可能需要的任何FreeRTOS中断处理程序。使用描述正在使用的端口的网页，以及为正在使用的端口提供的演示项目作为参考。

文件	位置
task.c	FreeRTOS/Source
queue.c	FreeRTOS/Source
list.c	FreeRTOS/Source
timers.c	FreeRTOS/Source
event_groups.c	FreeRTOS/Source
stream_buffer.c	FreeRTOS/Source
所有的C和汇编程序文件	FreeRTOS/Source/portable/[compiler]/[architecture]
heap_n .c	FreeRTOS/Source/portable/MemMang，其中n为1、2、3、4或5

表1要包含在项目中的FreeRTOS源文件

关于堆内存的注意事项：如果configSUPPORT_DYNAMIC_ALLOCATION为0，那么在项目中不包含堆内存分配方案。另外，在项目中包含堆内存分配方案，即heap_n.c文件之一，或一个由你自己提供的。有关更多信息，请参阅第3章，堆的内存管理。

2.5 数据类型和编码样式指南

2.5.1数据类型

FreeRTOS的每个端口都有一个唯一的端口宏。包含（以及其他内容）两种特定于端口的数据类型的定义：TickType_t和BaseType_t。以下列表描述了所使用的宏或类型定义以及实际类型：

- TickType_t

FreeRTOS配置了一个被称为滴答中断的周期性中断。

自FreeRTOS应用程序启动以来发生的滴答中断数称为滴答计数。滴答计数是用来作为时间的衡量标准。

两次滴答中断之间的时间称为滴答周期。时间被指定为滴答周期的倍数。

TickType_t是用于保存计数值和指定时间的数据类型。

TickType_t可以是无符号的16位类型，无符号的32位类型，或无符号的64位类型，根据FreeRTOSConfig.h中configTICK_TYPE_WIDTH_IN_BITS的设置。configTICK_TYPE_WIDTH_IN_BITS的设置依赖于体系结构。FreeRTOS端口还将检查此设置是否有效。

使用16位类型可以大大提高8位和16位体系结构的效率，但严重限制了在FreeRTOS API调用中可以指定的最大阻塞时间。没有理由在32位或64位体系结构上使用16位TickType_t类型。

以前使用的configUSE_16_BIT_TICKS已经被configTICK_TYPE_WIDTH_IN_BITS所取代，以支持大于32位的滴答计数。新的设计应该使用configTICK_TYPE_WIDTH_IN_BITS来代替configUSE_16_BIT_TICKS。

configTICK_TYPE_WIDTH_IN_BITS	8位架构	16位架构	32位架构	64位架构
TICK_TYPE_WIDTH_16_BITS	uint16_t	uint16_t	uint16_t	N/A
TICK_TYPE_WIDTH_32_BITS	uint32_t	uint32_t	uint32_t	N/A
TICK_TYPE_WIDTH_64_BITS	N/A	N/A	uint64_t	uint64_t

表2 TickType_t数据类型和configTICK_TYPE_WIDTH_IN_BITS配置

• BaseType_t

这总是被定义为体系结构中最有效的数据类型。通常，这是64位体系结构上的64位类型，32位体系结构上的32位类型，16位体系结构上的16位类型，以及8位体系结构上的8位类型。

BaseType_t通常用于只取非常有限的值范围的返回类型，以及pdTRUE/pdFALSE类型的布尔类值。

FreeRTOS使用的端口特定数据类型的列表

2.5.2变量名称

变量以其类型作为前缀：“c”表示char，“s”表示int16_t（短），“l”表示int32_t（长），“x”表示BaseType_t和任何其他非标准类型（结构、任务句柄、队列句柄等）。

如果变量为无符号，则它也以“u”作为前缀。如果变量是指针，它也以“p”作为前缀。例如，一个类型为uint8_t的变量将以“uc”为前缀，一个指向char（char*）的指针变量将以“pc”为前缀。

2.5.3 函数名称

函数是返回的类型和前缀在其中定义的文件。例如：

- vTaskPrioritySet() 返回一个空，并在task.c中定义。
- xQueueReceive() 返回一个类型的变量，并在queue.c中定义。
- pvTimerGetTimerID() 返回一个指向void的指针，并在timers.c中定义。

文件范围（私有）函数以“prv”为前缀。

2.5.4 格式化

Tab在一些演示应用程序中使用，其中一个Tab总是设置为等于四个空格。内核不再使用Tab。

2.5.5宏名称

大多数宏都是用大写写的，并用小写字母作为前缀，表示宏的位置。表3提供了一个前缀的列表。

前缀	宏定义的位置
port（例如，portMAX_DELAY）	portable.h或portmacro.h
task（例如，taskENTER_CRITICAL（））	task.h
pd（例如，pdTRUE）	projdefs.h //project define
config（例如，configUSE_PREEMPTION）	FreeRTOSConfig.h
err（例如，errQUEUE_FULL）	projdefs.h

表3 宏前缀

注意，信号量API几乎完全是一组宏，但遵循函数命名约定，而不是宏命名约定。

表4中定义的宏在整个FreeRTOS源代码中都在使用。

宏	值
pdTRUE	1
pdFALSE	0
pdPASS	1

宏 值

pdFAIL 0

表4 常用的宏定义

2.5.6 更多类型转换的基本原理

FreeRTOS源代码使用许多不同的编译器进行编译，其中许多编译器在如何生成警告以及何时生成警告方面有所不同。特别是，不同的编译器希望以不同的方式使用转换。因此，FreeRTOS源代码包含比通常保证的更多的类型转换。

3 堆内存管理

3.1 介绍

3.1.1 先决条件

成为一名称职的C程序员是使用FreeRTOS的先决条件，所以本章假设读者熟悉以下概念：

- 构建一个C项目的不同的编译和链接阶段。
- 堆和栈是什么。
- 标准的C库`malloc()`和`free()`函数。

3.1.2 范围

本章包括：

- 什么时候FreeRTOS分配RAM。
- FreeRTOS提供的五个内存分配方案。
- 要选择的内存分配方案。

3.1.3 在静态和动态内存分配之间的切换

下面的章节将介绍内核对象，如任务、队列、信号量和事件组。保存这些对象所需的RAM可以在编译时静态分配，或在运行时动态分配。动态分配减少了设计和规划工作，简化了API，并最小化了RAM占用。静态分配更具确定性，消除了处理内存分配失败的需要，并消除了堆碎片化的风险（其中堆有足够的可用内存，但不在一个可用的连续块中）。

只有在FreeRTOSConfig.h中，当`configSUPPORT_STATIC_ALLOCATION`被设置为1时，可以使用静态分配的内存创建内核对象的FreeRTOS API函数才可用。FreeRTOS API函数只有在`configSUPPORT_DYNAMIC_ALLOCATION`中设置为1或未定义时，使用动态分配内存创建内核对象的FreeRTOS API函数才可用。将两个常数同时设置为1是有效的。关于

`configSUPPORT_STATIC_ALLOCATION`的更多信息见第3.4节 使用静态内存分配。

3.1.4 使用动态内存分配

动态内存分配是一个C语言编程的概念，而不是一个特定于FreeRTOS或多任务处理的概念。它与FreeRTOS相关，因为内核对象可以选择使用动态分配的内存来创建，而通用的C库`malloc()`和`free()`函数可能不适合。

由于以下一个或多个原因：

- 它们并不总是在小型嵌入式系统上可用。
- 它们的实现可能相对较大，占用了有价值的代码空间。
- 它们很少是线程安全的。
- 它们不是确定性的；执行函数所花费的时间与调用不同。它们可能会出现碎片化（堆有足够的可用内存，但在一个可用的连续块中没有）。
- 它们可能会使连接器的配置复杂化。
- 如果允许堆空间增长到其他变量使用的内存中，它们可能成为难以调试错误的来源。

3.1.5 用于动态内存分配的选项

FreeRTOS的早期版本使用了一种内存池分配方案，其中在编译时预先分配不同大小的内存块的池，然后由内存分配函数返回。尽管块分配在实时系统中很常见，但它被从FreeRTOS中删除了，因为它在非常小的嵌入式系统对RAM的低效使用导致了許多支持请求。

FreeRTOS现在将内存分配视为portable层的一部分（而不是核心代码库的一部分）。这是因为不同的嵌入式系统有不同的动态内存分配和定时要求，因此一个单一的动态内存分配算法将永远只适用于应用程序的一个子集。此外，从核心代码库中删除动态内存分配可以使应用程序编写者能够在适当的时候提供他们自己的特定实现。

当FreeRTOS需要RAM时，它调用`pvPortMalloc()`，而不是`malloc()`。同样地，当FreeRTOS释放先前分配的RAM时，它会调用`vPortFree()`，而不是`free()`。`pvPortMalloc()`与标准C库`malloc()`函数具有相同的原型，而`vPortFree()`与标准C库`free()`函数具有相同的原型。

`pvPortMalloc()`和`vPortFree()`是公共函数，所以它们也可以从应用程序代码中调用。

FreeRTOS提供了五个`pvPortMalloc()`和`vPortFree()`的实现示例，它们都在本章中说明。FreeRTOS应用程序可以使用其中一个示例实现或提供它们自己的实现

这五个例子在`heap_1.c`，`heap_2.c`，`heap_3.c`，`heap_4.c`和`heap_5.c`源文件中定义，所有这些都位于FreeRTOS/Source/portable/MemMang目录中。

3.2 内存分配方案的示例

3.2.1 heap_1.c

对于小型的专用嵌入式系统，在启动FreeRTOS调度程序之前，通常只创建任务和其他内核对象。在这种情况下，只有在应用程序开始执行任何实时功能之前，才会由内核（动态地）分配内存，并且在应用程序的生命周期中仍然分配内存。这意味着所选择的分配方案不必考虑更复杂的内存分配问题，如确定论和碎片化，而可以优先考虑代码大小等属性和简单性。

.c实现了一个非常基本的pvPortMalloc（）版本，而没有实现vPortfree（）。从未删除任务或其他内核对象的应用程序有可能使用heap_1。一些商业关键和安全关键系统将禁止使用动态内存分配，否则也有可能使用heap_1。关键系统通常禁止动态内存分配，因为与不确定性、内存碎片和失败分配相关的不确定性。Heap_1总是确定性的，不能分割内存碎片。

Heap_1的pvPortMalloc（）实现简单地将一个名为FreeRTOS堆的简单uint8_t数组细分为更小的块。FreeRTOSConfig.h的常量configTOTAL_HEAP_SIZE以字节为单位设置数组的大小。将堆作为静态分配的数组实现会使FreeRTOS看起来消耗大量RAM，因为堆会成为FreeRTOS数据的一部分。

每个动态分配的任务都会导致对pvPortMalloc（）的两次调用。第一个分配一个任务控制块（TCB），第二个是分配任务的堆栈。图3.1演示了heap_1如何在创建任务时细分简单数组。

参见图3.1：

- A在创建任何任务之前显示数组一整个数组是空闲的。
- B显示了创建一个任务后的数组。
- C显示了创建三个任务后的数组。

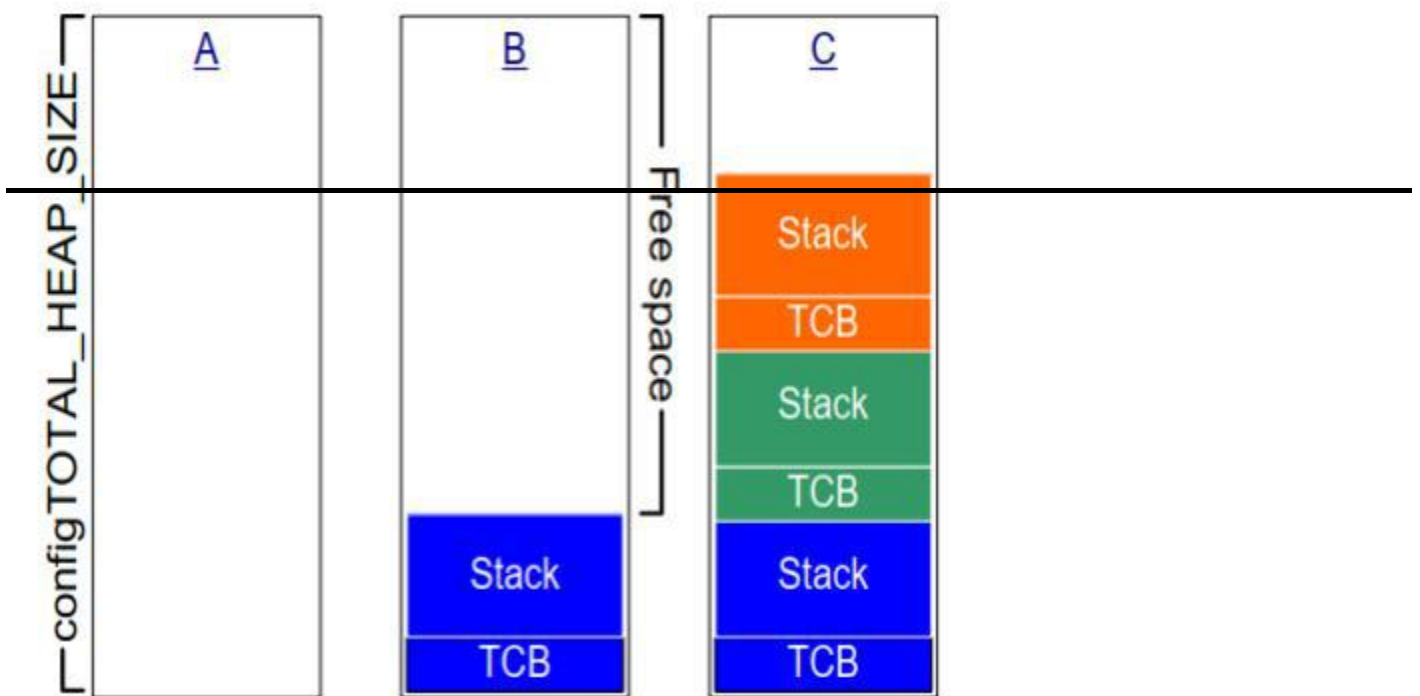


图3.1 每次创建任务时从heap_1数组分配RAM

3.2.3 Heap_2

Heap_2被heap_4所取代，其中包括增强的功能。Heap_2保留在FreeRTOS发行版中，以便向后兼容，不建议用于新的设计。

Heap_2.c也通过用configTOTAL_HEAP_SIZE常数来细分一个数组来工作。它使用了一个最佳匹配(best-fit)的算法来分配内存，并且，与heap_1不同，它确实实现了vPortFree()。同样，将堆实现为静态分配的数组会使FreeRTOS似乎消耗大量RAM，因为堆会成为FreeRTOS数据的一部分。

最佳匹配算法确保pvPortMalloc()使用大小最接近请求字节数的空闲内存块。例如，考虑以下场景：

- 堆包含三个可用内存块，分别为5字节、25字节和100字节。
- ()请求20个字节的内存。

请求的字节数适合的最小的自由内存块是25字节块，因此pv() malloc()将25字节块分成一个20字节块和一个5字节块，然后返回一个指向20字节块[^2]的指针。新的5字节块仍然可用于未来调用pvPortMalloc()。

[^2]：这过于简单化了，因为heap_2在堆区域内存储了关于块大小的信息，所以两个分割块的总和实际上将小于25。

与heap_4不同，heap_2不将相邻的自由块组合成一个更大的块，所以它比heap_4更容易发生碎片化。但是，如果分配的和随后释放的块总是相同的大小，碎片并不是问题。

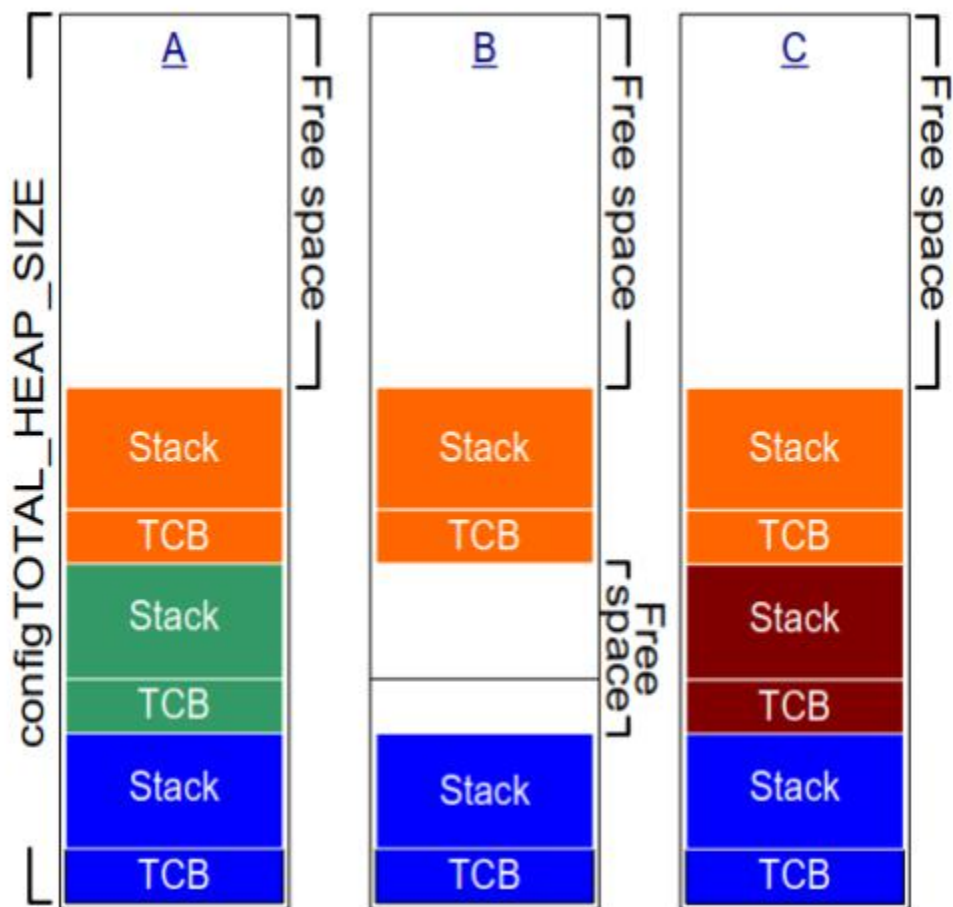


图3.2 在创建和删除任务时，正在从heap_2数组中分配和释放的RAM

图3.2演示了在创建、删除和再次创建任务时，最佳匹配算法如何工作。参见图3.2:

- A显示了在分配了三个任务后的数组。一个很大的自由块仍然保留在数组的顶部。
- B显示了删除其中一个任务后的数组。位于数组顶部的较大的自由块保留了下来。现在还有两个更小的空闲块，它们以前保存着TCB和已删除任务的堆栈。
- C显示了创建另一个任务后的情况。创建该任务会导致从xTaskCreate() API函数中两次调用pvPortMalloc()，一个来分配一个新的TCB，另一个来分配任务堆栈。本书的3.4节描述了xTaskCreate()。

每个TCB的大小都是相同的，因此最佳拟合算法会重用保存已删除任务的TCB的RAM块来保存已创建任务的TCB。

如果分配给新创建任务的堆栈的大小与分配给之前删除任务的堆栈的大小相同，那么最佳匹配算法将重用保存已删除任务堆栈的RAM块来保存已创建任务的堆栈。

位于数组顶部的较大的未分配块保持不变。

Heap_2不是确定性的，但比malloc（）和free（）的大多数标准库实现都要快。

3.2.3 Heap_3

Heap_3.c使用了标准的库malloc（）和free（）函数，因此链接器配置定义了堆的大小，并且不使用configTOTAL_HEAP_SIZE常量。

Heap_3通过在执行期间暂时暂停FreeRTOS调度程序，使malloc（）和free（）线程安全。第8章，资源管理，涵盖了线程安全和调度程序暂停。

3.2.4 Heap_4

与heap_1和heap_2一样，heap_4的工作原理是将一个数组细分为更小的块。与前面一样，数组是由configTOTAL_HEAP_SIZE静态分配和标注的，这使得FreeRTOS在使用FreeRTOS数据时使用了大量RAM。

Heap_4使用了一种first-fit算法来分配内存。与heap_2不同，heap_4将（合并）相邻的空闲内存块合并成一个更大的内存块，从而将内存碎片化的风险降到最低。

first-fit确保pvPortMalloc（）使用第一个空闲内存块，可以保存请求的字节数。例如，考虑以下场景：

- 堆包含三个可用内存块，按照它们在数组中出现的顺序，分别为5字节、200字节和100字节
 - 。pvPortMalloc() 请求20个字节的内存。
- 请求的字节数适合的第一个自由RAM块是200字节块，因此pvPortMalloc()将200字节块分成一个20字节和一个180字节的块^[^3]，然后返回一个指向20字节块的指针。新的180字节块仍然可用于未来调用pvPortMalloc（）。

[^3]：这过于简单化了，因为heap_4在堆区域内存储了关于块大小的信息，所以两个分割块的总和实际上将小于200字节。

Heap_4将（合并）相邻的自由块组合成一个更大的块，最大限度地减少了碎片化的风险，并使其适合于重复分配和释放不同大小的RAM块的应用程序。

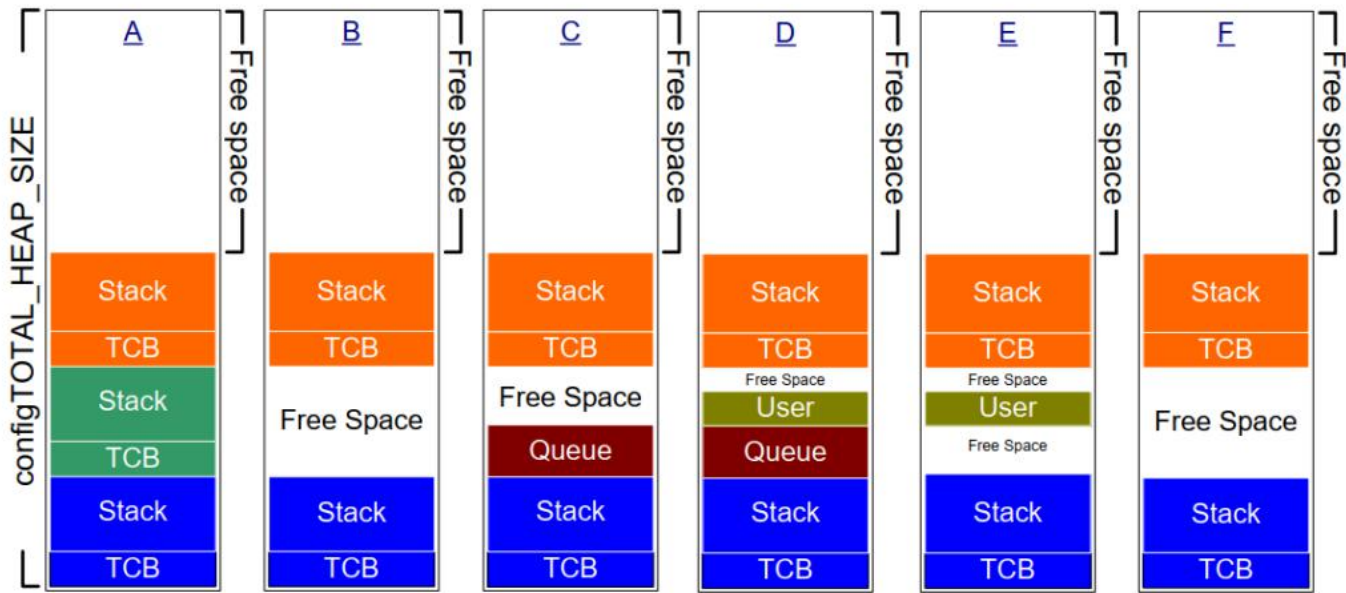


图3. 3正在从heap_4数组中分配和释放的RAM

图3. 3展示了heap_4的first-fit算法与内存合并的工作原理。参见图3. 3:

- A显示了创建三个任务后的数组。一个很大的自由块仍然保留在数组的顶部。
- B显示了删除其中一个任务后的数组。位于数组顶部的较大的自由块保留了下来。现在有另一个空闲块，其中是已删除任务的TCB和堆栈。与heap_2不同，例如，heap_4将先前分别持有TCB和已删除任务堆栈的两个内存块合并为一个更大的单个自由块。
- C显示了创建一个FreeRTOS队列后的情况。这本书的第5. 3节描述了创建用于动态分配队列的xQueueCreate() API函数。xQueueCreate()调用pvPortMalloc()来分配队列使用的RAM。由于heap_4使用了first-fit算法，pvPortMalloc()从第一个大到足以保存队列的空闲RAM块分配RAM，在图3. 3中，是通过删除任务释放的RAM。队列不占用空闲块中的所有RAM，因此块被分成两个，未使用的部分仍然可用于将来呼叫pvPortMalloc()。
- D显示了直接从应用程序代码调用pvPortMalloc()而不是间接调用pvPortMalloc()后的情况

用户分配的块足够小，可以容纳入第一个空闲块，这是分配给队列的内存和后面分配给TCB的内存之间的块。

通过删除任务而释放的内存现在已经分成三个独立的块：第一个块保存队列，第二个块保存用户分配的内存，第三个块保持空闲。

- E显示了删除队列后的情况，它会自动释放分配给已删除队列的内存。现在在用户分配的块的两边都有空闲内存。

- 了释放用户分配的内存后的情况。用户分配的块以前使用的内存与两边的空闲内存相结合，以创建一个更大的单个空闲块。

Heap_4不是确定性的，但比malloc（）和免费（）的大多数标准库实现都要快。

3.2.5 Heap_5

Heap_5使用与heap_4相同的分配算法。heap_4仅限于从单个数组分配内存，而heap_5可以将来自多个分离内存空间的内存合并到单个堆中。当运行FreeRTOS的系统提供的RAM在系统的内存映射中不作为单个连续（无空格）块出现时，Heap_5非常有用。

3.2.6 初始化heap_5: vPortDefineHeapRegions() API函数

vPortDefineHeapRegions() 初始化heap_5通过指定每个单独内存区域的起始地址和大小, 通过它们组成heap_5管理的堆。Heap_5是唯一提供的需要显式初始化的堆分配方案，并且要在调用vPortDefineHeapRegions()之后才能使用。这意味着内核对象，如任务、队列和信号量，不能被动态创建，直到调用vPortDefineHeapRegions()之后。

```
void vPortDefineHeapRegions( const HeapRegion_t * const pxHeapRegions );
```

vPortDefineHeapRegions() API函数原型

vPortDefineHeapRegions() 将一个HeapRegion_t结构数组作为其唯一的参数。每个结构都定义了将成为堆的一部分的内存块的起始地址和大小——整个结构数组定义了整个堆空间。

```
typedef struct HeapRegion
{
    /* The start address of a block of memory that will be part of the heap.*/
    uint8_t *pucStartAddress;

    /* The size of the block of memory in bytes. */
    size_t xSizeInBytes;
} HeapRegion_t;
```

List3.2 HeapRegion_t结构体

参数:

- pxHeapRegions

指向HeapRegion_t结构数组开头的指针。每个结构定义将成为堆一部分的内存块的起始地址和大小。

数组中的HeapRegion_t结构必须按起始地址排序；描述具有最低起始地址的内存区域的HeapRegion_t结构必须是数组中的第一个结构，而描述具有最高起始地址的内存区域的HeapRegion_t结构必须是数组中的最后一个结构。

用HeapRegion_t结构标记数组的末端，该结构的起始地址成员设置为NULL。

举例来说，考虑一下图3. 4中所示的假设的内存映射。A包含三个独立的RAM块： RAM1、RAM2和RAM3。假设可执行代码放置在只读内存中，但没有显示。

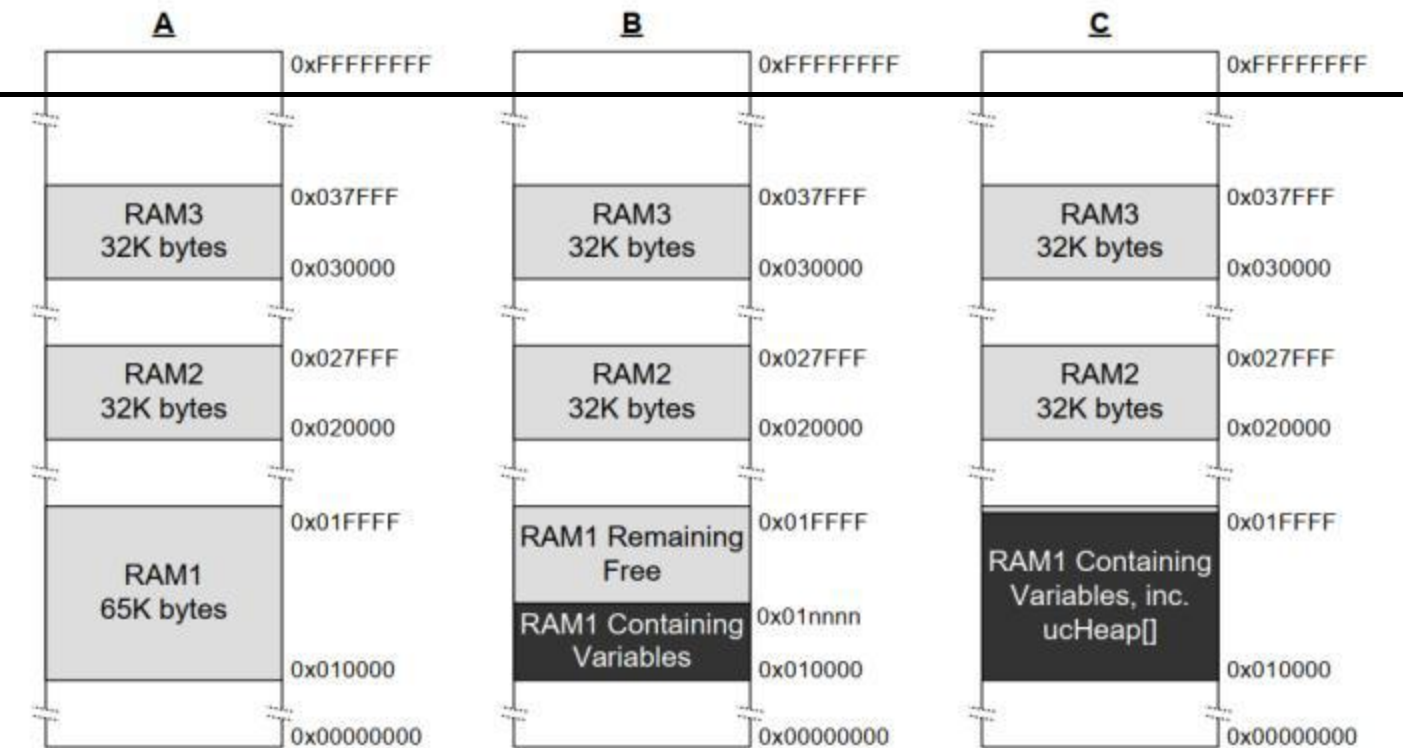


图3. 4 内存图

清单3. 3显示了一个HeapRegion_t结构阵列，它们一起描述了三个RAM块。

```
/*定义三个RAM区域的起始地址和大小。*/
#define RAM1_START_ADDRESS ( (uint_t *) 0x00010000)
#define RAM1_SIZE (64 * 1024)

#define RAM_STAR_ADDRESS ( (uint_t *) 0x00020000)
```

```

#define RAM_SIZE (32 * 1024)

#define RAM_START_ADDRESS ( (uint_t *) 0x00030000)
#define RAM_SIZE (32 * 1024)

/*创建一个HeapRegion_t定义数组，其中包含三个RAM区域中的每一个索引，并使用包含
   NULL地址的HeapRegion_t结构终止该数组。HeapRegion_t结构必须以起始地址的顺序
   出现，其中包含最低起始地址的结构将首先出现。*/
const HeapRegion_t xHeapRegions[] =
{
    { ram1_start_address , ram1_size },
    { ram2_start_address , ram2_size },
    { ram3_start_address , ram3_size },
    { NULL, 0 }
    /* 标记标记数组的末尾。*/
};

int main (void)
{
    /*初始化heap_5。*/
    vPortDefineHeapRegions ( xHeapRegions );

    /*可在这里添加应用程序代码。*/
}

```

清单3.3 一个HeapRegion_t结构数组，它们一起完整地描述了RAM的3个区域

尽管清单3.3正确地描述了RAM，但它没有演示一个可用的示例，因为它将所有RAM分配给堆，没有留下任何RAM供其他变量使用。

构建过程的链接阶段为每个变量分配一个RAM地址。链接器可供使用的RAM通常由链接器配置文件来描述，例如链接器脚本。在图3.4 B中，假设链接器脚本包含了关于RAM1的信息，但不包括关于RAM2或RAM3的信息。因此，链接器在RAM1中放置了变量，只留下RAM1中地址0x0001 nnnn以上的部分可供heap_5使用。0x0001 nnnn的实际值取决于应用程序中包含的所有变量的组合大小。链接器使所有RAM2和所有RAM3都未使用，使得整个RAM2和整个RAM3可供heap_5使用。

清单3.3中显示的代码将导致在地址0x0001 nnnn下面分配给heap_5的RAM与用于保存变量的RAM重叠。如果将xHeapRegions[]数组中第一个HeapRegion_t结构的起始地址设置为0x0001 nnnn，而不是0x00010000，则堆将不会与链接器使用的RAM重叠。但是，这并不是一个推荐的解决方案，因为：

- 起始地址可能不容易确定。
- 链接器使用的RAM数量可能会在未来的构建中发生变化，这将使需要更新到HeapRegion_t结构中使用的起始地址。

- 如果链接器使用的RAM和heap_5使用的RAM重叠，构建工具将不知道，因此不能警告应用程序编写器。

清单3.4展示了一个更方便和可维护的示例。它声明了一个名为ucHeap的数组。ucHeap是一个普通变量，因此它成为了链接器分配给RAM1的数据的一部分。xHeapRegions数组中的第一个HeapRegion_t结构描述了ucHeap的起始地址和大小，因此ucHeap成为由heap_5管理的内存的一部分。ucHeap的大小可以增加，直到连接器使用的RAM消耗掉了所有的RAM1，如图3.4 C所示。

```
/*定义未使用的两个RAM区域的起始地址和大小
   链接器。*/
#define RAM_STAR_ADDRESS ( (uint_t *) 0x00020000)
#define RAM_SIZE (32 * 1024)

#define RAM_STAR_ADDRESS ( (uint_t *) 0x00030000)
#define RAM_SIZE (32 * 1024)

/*声明一个将是heap_5使用的堆的一部分的数组。这个
   阵列将由链接器放置在RAM1中。*/
#define RAM_HEA_SIZE (30 * 1024)
static uint_t ucHeap[ RAM1_HEAP_SIZE ];

/*创建一个HeapRegion_t定义数组。而在清单3.5中
   第一个条目描述了所有的RAM1，所以heap_5已经使用了所有的RAM1，这次第一个条目
   只描述了ucHeap数组，所以heap_5将只使用包含ucHeap数组的部分。HeapRegion_t结
   构必须仍然以起始地址的顺序显示，
   首先出现包含最低起始地址的结构。*/

const HeapRegion_t xheap[]=
{
    { ucHeap, RAM1_HEAP_SIZE },
    { ram2_start_address , ram2_size },
    { ram3_start_address , ram3_size },
    标记标记数组的末尾。*/
};
```

清单3.4一个HeapRegion_t结构的数组，它描述了所有的RAM2，所有的RAM3，但只有RAM1的一部分

清单3.4中展示的技术的优点包括：

- 不需要使用硬编码的起始地址。
- 在HeapRegion_t结构中使用的地址将由链接器自动设置，因此它将始终是正确的，即使链接器使用的RAM数量在未来的构建中发生变化。
- 分配给heap_5的RAM不可能将由链接器放置到RAM1中的数据重叠。
- 如果ucHeap的大小太大，则该应用程序将不会进行链接。

3.3 与堆相关的实用程序函数和宏

3.3.1 定义堆的开始地址

Heap_1、heap_2和heap_4从一个由configTOTAL_HEAP_SIZE标注的静态分配的数组中分配内存。

本节将这些分配方案统称为heap_n。

有时，需要将堆放在一个特定的内存地址上。例如，分配给动态创建的任务的堆栈来自堆，因此可能需要在快速内部内存中定位堆，而不是在慢速外部内存中定位堆。（关于在快速内存中分配任务堆栈的方法，请参阅下面放置任务堆栈的另一种方法）。configAPPLICATION_ALLOCATED_HEAP编译时配置常量使应用程序能够声明该数组，而不是原本将在heap_n.c源文件中的声明。在应用程序代码中声明该数组，使应用程序写入器能够指定其起始地址。

如果configAPPLICATION_ALLOCATED_HEAP在FreeRTOSConfig.h中设置为1，或未定义，使用FreeRTOS的应用程序必须分配一个名为ucHeap的uint8_t数组，并由configTOTAL_HEAP_SIZE常量标注

将变量放置在特定内存地址所需的语法取决于正在使用的编译器，因此请参阅编译器的文档。下面是两个编译器的示例：

- 清单3.5显示了GCC编译器声明该数组并将该数组放在名为.my_heap的内存部分中所需的语法。
- 清单3.6显示了IAR编译器声明该数组并将该数组放在绝对内存地址0x20000000处所需的语法。

```
uint8_t ucHeap[ configTOTAL_HEAP_SIZE ] __attribute__ ( ( section( ".my_heap" ) ) );
```

清单3.5 使用GCC语法声明将被heap_4使用的数组，并将该数组放在一个名为.my_heap的内存部分中

```
uint8_t ucHeap[ configTOTAL_HEAP_SIZE ] @ 0x20000000;
```

清单3.6 使用IAR语法声明heap_4将使用的数组，并将该数组放置在绝对地址0x20000000处

3.3.2 The xPortGetFreeHeapSize() API函数

xPortGetFreeHeapSize() 返回在调用该函数时堆中的空闲字节数。它不提供有关堆碎片化的信息。

xPortGetFreeHeapSize() 没有为heap_3实现。

```
size_t xPortGetFreeHeapSize( void );
```

xPortGetFreeHeapSize() API函数原型

返回值:

- xPortGetFreeHeapSize() 返回调用函数时在堆中未分配的字节数

3.3.3 The xPortGetMinimumEverFreeHeapSize() API函数

函数返回自FreeRTOS应用程序开始执行以来堆中存在的最小未分配的最小字节数。

返回的值表示应用程序曾经接近于堆空间耗尽的距离。例如，如果函数返回200，那么，在应用程序开始执行后的某个时候，它距离堆空间耗尽不到200字节。

函数返回值也可以用来优化堆的大小。例如，如果函数在执行您知道的堆使用率最高的代码后返回2000，那么configTOTAL_HEAP_SIZE最多可以减少2000字节。

该函数只在heap_4和heap_5中实现。

```
size_t xPortGetMinimumEverFreeHeapSize (void);
```

清单3.8 函数原型

返回值:

- 返回自FreeRTOS应用程序开始执行以来堆中存在的最小未分配的最小字节数

3.3.4 The vPortGetHeapStats() API函数

Heap_4和heap_5实现了vPortGetHeapStats_t()，HeapStats_t结构引用作为函数的唯一参数。

清单3.9显示了一个完整的vPortGetHeapStats_t()函数原型。清单3.10显示了HeapStats_t的结构成员

```
void vPortGetHeapStats( HeapStats_t *xHeapStats );
```

清单3.9 The vPortGetHeapStatus() API function prototype

```

/* The vPortGetHeapStatus() 函数的原型。*/
void vPortGetHeapStats( HeapStats_t *xHeapStats );

/*对HeapStats_t结构的定义。以字节数为单位指定的所有大小。*/
typedef struct xHeapStats
{
    /*当前可用的总堆大小 --这是所有堆空闲块大小的总和
    ，而不是最大的可用的块。*/
    size_t xAvailableHeapSpaceInBytes;

    /*在调用vPortGetHeapStats（）时，堆中最大的空闲块的大小
    */
    size_t xSizeOfLargestFreeBlockInBytes;

    /*当时调用vPortGetHeapStats（），堆中最小的自由块的大小
    。*/
    size_t xSizeOfSmallestFreeBlockInBytes;

    /*当时堆中的可用内存块的数量
    vPortGetHeapStats（）调用。*/
    size_t xNumberOfFreeBlocks;

    /*自系统启动以来，堆中存在的最小可用内存总量（所有可用块的总和）*/
    size_t xMinimumEverFreeBytesRemaining;

    /*对已返回有效内存块的pvPortMalloc（）的调用次数。*/
    size_t xNumberOfSuccessfulAllocations;

    /*对成功释放内存块的vPortFree（）的调用次数。*/
    size_t xNumberOfSuccessfulFrees;
} HeapStats_t;

```

清单3.10 HeapStatus_t（）结构

3.3.5 正在收集每个任务堆的使用情况统计信息

在本书的TBD-RB节中记录了vTaskGetInfo（）API函数，它用关于任务的信息填充了一个TaskStatus_t结构。如果在FreeRTOSConfig.h中将configTRACK_TASK_MEMORY_ALLOCATIONS编译时常量设置为1，该结构包括以下附加信息

:

- 任务调用pvPortMalloc（）的次数。
- 任务调用vPortFree（）的次数。

- 在调用vTaskGetInfo（）时，尚未被任何任务释放的任务所分配的堆字节数。
- 自任务开始以来的任何给定时间，任务分配的最大堆内存量。

3.3.6 Malloc失败钩函数

与标准库malloc（）函数一样，pvPortMalloc（）如果不能分配请求的RAM量，则返回NULL。malloc失败的钩子（或回调）是一个应用程序提供的函数，如果pvPortMalloc（）返回NULL，则会被调用。为了使回调发生，在FreeRTOSConfig.h中，必须将configUSE_MALLOC_FAILED_HOOK设置为1。如果失败的钩子在FreeRTOS API函数中调用malloc来创建内核对象，则不会创建该对象。

如果在FreeRTOSConfig.h中将configUSE_MALLOC_FAILED_HOOK设置为1，则应用程序必须提供一个malloc失败的钩子函数，其名称和原型是清单3.11中所示的。该应用程序可以以任何适合于该应用程序的方式来实现该功能。许多提供的FreeRTOS演示应用程序将分配失败视为致命错误，但这对于生产系统来说不是最佳实践，它应该可以优雅地从分配失败中恢复。

```
void vApplicationMallocFailedHook( void );
```

清单3.11 malloc失败的钩子函数名称和原型

3.3.7 在快速内存中放置任务堆栈

因为栈以很高的速率写入和读取，所以它们应该放在快速内存中，但这可能不是您希望堆驻留的位置。

FreeRTOS使用pvPortMallocStack（）和 vPortFreeStack（）宏函数来选择性地启用在FreeRTOS API代码中分配的堆栈有自己的内存分配器。如果您希望堆来自从 pvPortMalloc（）管理的堆，那么保留pvPortMallocStack（）和 vPortFreeStack（）未定义，因为它们默认分别调用pvPortMalloc（）和vPortFree（）。否则，请定义宏以调用应用程序所提供的函数，如清单3.12所示。

/*函数由应用程序写入器提供，而不是从一个快速的内存区域分配和释放内存。*/

```
void *pvMallocFastMemory( size_t xWantedSize );
```

```
void vPortFreeFastMemory( void *pvBlockToFree );
```

/*添加以下内容到FreeRTOSConfig.h，以将pvPortMallocStack（）和 vPortFreeStack（）宏函数映射到使用

*快速内存的函数

*/

```
#define pvPortMallocStack(x) pvMallocFastMemory ( x )
```

```
#define vPortFreeStack(x) vPortFreeFastMemory ( x )
```

清单3.12 Mapping the `pvPortMallocStack()` and `vPortFreeStack()` macros to an application defined memory allocator

3.4 使用静态内存分配

第3.1.4节列出了动态内存分配所附带的一些缺点。为了避免这些问题，静态内存分配允许开发人员明确地创建应用程序所需的每个内存块。这将具有以下优点：

所有所需的内存在编译时都是已知的。

所有的记忆都是确定性的。

还有其他优点，但这些优点也带来一些并发症。主要的复杂是添加了一些额外的用户函数来管理一些内核内存，第二个复杂是需要确保所有静态内存在合适的作用域内声明。

3.4.1 已启用静态内存分配

通过在FreeRTOSConfig中将`configSUPPORT_STATIC_ALLOCATION.h`设置为1来启用静态内存分配。当启用此配置时，内核将启用内核函数的所有静态版本。这些是：

```
xTaskCreateStatic
xEventGroupCreateStatic
xEventGroupGetStaticBuffer
■ xQueueGenericCreateStatic
■ xQueueGenericGetStaticBuffers
■ xQueueCreateMutexStatic
■   ◦if configUSE_MUTEXES is 1
■ xQueueCreateCountingSemaphoreStatic
■   ◦if configUSE_COUNTING_SEMAPHORES is 1
xStreamBufferGenericCreateStatic
■ xStreamBufferGetStaticBuffers
xTimerCreateStatic
■   ◦if configUSE_TIMERS is 1
■ xTimerGetStaticBuffer
■   ◦if configUSE_TIMERS is 1
■
```

这些功能将在这本书的适当章节中进行解释。

3.2.4 静态内核内存

当启用静态内存分配器时，空闲任务和定时器任务（如果启用）将使用由用户函数提供的静态内存分配。这些用户功能包括：

- vApplicationGetTimerTaskMemory
 - if configUSE_TIMERS is 1
- vApplicationGetIdleTaskMemory

3.4.2.1 vApplicationGetTimerTaskMemory

如果configSUPPORT_STATIC_ALLOCATION和configUSE_TIMERS都启用，内核将调用vApplicationGetTimerTaskMemory()，允许应用程序为定时器任务TCB和定时器任务堆栈创建并返回一个内存缓冲区。该函数还将返回计时器任务堆栈的大小。清单3.13显示了计时器任务内存分配函数的可能实现。

```
void vApplicationGetTimerTaskMemory( StaticTask_t **ppxTimerTaskTCBBuffer,
                                     StackType_t **ppxTimerTaskStackBuffer,
                                     uint32_t *pulTimerTaskStackSize )
{
    /*如果要提供给空闲任务的缓冲区在此函数内声明，则它们必须声明为静态 - 否则它们将在堆栈上分配，因此在该函数退出后将不存在*/

    static StaticTask_t xTimerTaskTCB;
    static StackType_t uxTimerTaskStack[ configMINIMAL_STACK_SIZE ];

    /*传递一个指向空闲任务的StaticTask_t结构的指针
    状态将被存储。*/
    *ppxTimerTaskTCBBuffer = &xTimerTaskTCB;

    /*传递将用作空闲任务堆栈的数组。*/
    *ppxTimerTaskStackBuffer = uxTimerTaskStack;

    /*传出 *ppxIdleTaskStackBuffer 指向的数组的堆栈大小。请注意，堆栈大小是 StackType_t 的计数*/
    *pulTimerTaskStackSize = sizeof(uxTimerTaskStack) / sizeof(*uxTimerTaskStack); }

```

清单3.13 vApplicationGetTimerTaskMemory的典型实现

由于包括SMP在内的任何系统中都只有一个定时器任务，因此定时器任务内存问题的有效解决方案是在vApplicationGetTimeTaskMemory()函数中分配静态缓冲区并将缓冲区指针返回到内核

3.4.2.2 vApplicationGetIdleTaskMemory

当核心用完计划的工作时，将运行空闲任务。空闲任务执行一些内务处理，如果启用，还可以触发用户的vTaskIdleHook()。在对称多处理系统（SMP）中，其余每个核心还存在非内务管理空闲任务，但这些任务在内部静态分配到 configMINIMUM_STACK_SIZE 字节。

调用 `vApplicationGetIdleTaskMemory` 函数以允许应用程序为“主”空闲任务创建所需的缓冲区。清单 3.14 显示了 `vApplicationIdleTaskMemory()` 函数的典型实现，它使用静态局部变量来创建所需的缓冲区。

```
void vApplicationGetIdleTaskMemory( StaticTask_t **ppxIdleTaskTCBBuffer,
                                     StackType_t **ppxIdleTaskStackBuffer,
                                     uint32_t *pulIdleTaskStackSize )

{
    static StaticTask_t xIdleTaskTCB;
    static StackType_t uxIdleTaskStack[ configMINIMAL_STACK_SIZE ];;

    *ppxIdleTaskTCBBuffer = &xIdleTaskTCB;
    *ppxIdleTaskStackBuffer = uxIdleTaskStack;
    *pulIdleTaskStackSize = configMINIMAL_STACK_SIZE;
}
```

清单3.14 `vApplicationGetIdleTaskMemory`的典型实现

4 任务管理

4.1 简介

4.1.1 范围

本章包括：

- FreeRTOS如何为应用程序中的每个任务分配处理时间。
- FreeRTOS如何选择在任何给定的时间应该执行哪个任务。
- 每个任务的相对优先级如何影响系统行为。
- 任务可以存在的状态。

本章还讨论：

- 如何实现任务。
- 如何创建一个或多个任务的实例。
- 如何使用任务参数。
- 如何更改已经创建的任务的优先级。
- 如何删除一个任务。
- 如何使用一个任务来实现周期性的处理。（后面的章节描述了如何使用软件定时器进行同样的操作。
- ）空闲任务何时执行以及如何使用它。

本章中介绍的概念是理解如何使用FreeRTOS以及FreeRTOS应用程序如何进行行为的基础。因此，这是书中最详细的一章。

4.2 任务函数

任务作为C函数实现。任务必须实现清单4.1中所示的预期函数原型。它接受一个void指针参数并返回void。

```
void vATaskFunction( void * pvParameters );
```

清单4.1 任务函数原型

每个任务本身都是一个小程序。它有一个入口点，通常会在一个无限的循环中永远运行，并且不会退出。清单4.2显示了一个典型任务的结构。

不能允许FreeRTOS任务以任何方式实现它的函数返回。它不能包含“return”语句，也不能允许在其实现功能结束后执行。如果不再需要一个任务，则应该显式地删除它，如清单4.2所示。

单个任务函数定义可以用于创建任意数量的任务，其中每个创建的任务都是一个单独的执行实例。每个实例都有自己的堆栈，因此也有自己的在任务本身中定义的任何自动（堆栈）变量的副本。

```
void vATaskFunction( void * pvParameters )
{
    /*
     *堆栈分配的变量可以在一个函数中正常声明。
     *使用这个示例函数创建的每个任务都将有它自己的实例lStackVariable。（这是在任务的堆栈上分配的）*/
    long lStackVariable = 0;

    /*
     *与堆栈分配的变量相比，使用`静态`声明的变量
     *关键字由链接器分配到内存中的特定位置。
     *这意味着所有调用vATaskFunction的任务都将共享相同的
     *静态变量实例lStaticVariable。*/
    static long lStaticVariable = 0;

    for( ;; )
    {
        /*实现任务功能的代码将转到这里。*/
    }

    /*
     *如果任务实现要退出上述循环，那么任务
     *必须在达到其实现功能的结束之前被删除。
     *将NULL作为参数传递给vTaskDelete（）API函数时，
     *这表示要删除的任务是调用（此）任务。*/
    vTaskDelete( NULL );
}
```

清单4.2 一个典型的任务函数的结构

3.4 顶级任务状态

一个应用程序可能包含许多任务。如果运行应用程序的处理器包含单个核心，那么在任意给定时间可能只能执行一个任务。这意味着一个任务可能处于两种状态之一：运行和不运行。首先考虑这个简单的模型。在本章的后面，我们将描述不运行状态的几个子状态。

当处理器正在执行该任务的代码时，该任务处于“正在运行”状态。当任务处于“不运行”状态时，任务将暂停并保存其状态，以便在下次调度程序时恢复执行

决定它应该进入“正在运行中”的状态。当任务恢复执行时，它将从离开运行状态之前将要执行的指令中执行。

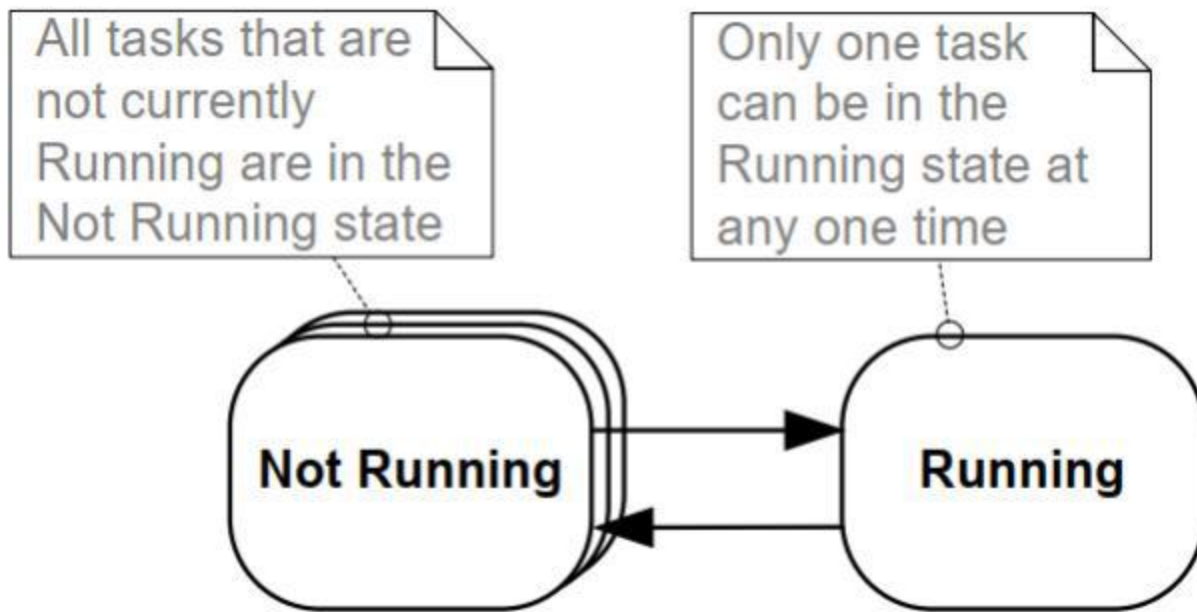


图4.1 顶层任务状态和转变

从“未正在运行”状态转换到“正在运行”状态的任务据说已“切换”或“交换”。相反，从“运行”状态转换到“不运行”状态的任务被称为“关闭”或“交换关闭”。FreeRTOS调度程序是唯一可以将任务切换出“运行”状态的实体。

4.4 任务创建

6个API函数可用于创建任务：`xTaskCreate()`，`xTaskCreateStatic()`，`xTaskCreateRestricted()`，`xTaskCreateRestrictedStatic()`，`xTaskCreateAffinitySet()`，and `xTaskCreateStaticAffinitySet()`

每个任务都需要两个RAM块：一个用于保存其任务控制块（TCB），另一个用于存储其堆栈。名称中有“静态”的FreeRTOS API函数使用传递给函数的预先分配的RAM块作为参数。相反，在名称中没有“静态”的API函数在运行时从系统堆中动态分配所需的RAM。

一些FreeRTOS端口支持以“受限”或“非特权”模式运行的任务。名称中包含“受限”的FreeRTOS API函数创建对系统内存访问有限的任务。在其名称中没有“限制”的API函数可以创建以“特权模式”执行的任务，并可以访问系统的整个内存映射。

支持对称多处理（SMP）的FreeRTOS端口允许在同一CPU的多个核上同时运行不同的任务。对于这些端口，您可以通过使用名称中的“亲和性”函数来指定任务将在哪个核心上运行。

FreeRTOS任务创建API函数相当复杂。本文档中的大多数示例都使用xTaskCreate()，因为它是这些函数中最简单的一个。

4.4.1 xTaskCreate() API函数

清单4.3显示了xTaskCreate() API函数原型。xTaskCreateStatic()有两个额外的参数，它们分别指向预先分配给保存任务的数据结构和堆栈的内存。(第2.5节：数据类型和编码样式指南)描述了所使用的数据类型和命名约定。

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,
                        const char * const pcName,
                        configSTACK_DEPTH_TYPE usStackDepth,
                        void * pvParameters,
                        UBaseType_t uxPriority,
                        TaskHandle_t * pxCreatedTask );
```

清单4.3 The xTaskCreate() API函数原型

参数和返回值：

•pvTaskCode

任务只是从不退出的C函数，因此，通常被实现为一个无限循环。pvTaskCode参数只是一个指向实现该任务的函数的指针（实际上，它只是函数的名称）。

•pcName

该任务的一个描述性名称。FreeRTOS不以任何方式使用它，它纯粹作为调试辅助工具。用人类可读的名称来识别任务比用其句柄来识别任务要简单得多。

应用程序定义的常量configMAX_TASK_NAME_LEN定义了任务名称的最大长度，包括空终止符。提供一个较长的字符串会导致该字符串被截断。

•usStackDepth

指定要分配给任务使用的堆栈的大小。使用xTaskCreateStatic() i，而不是xTaskCreate()来使用预先分配的内存，而不是动态分配的内存。

注意，该值指定了堆栈可以保存的字数，而不是字节数。例如，如果堆栈是32位宽，而usStackDepth是128，那么xTaskCreate()分配512字节的堆栈空间（128 * 4字节）。

configSTACK_DEPTH_TYPE是一个宏，它允许应用程序编写器指定用于保存堆栈大小的数据类型。如果未定义，configSTACK_DEPTH_TYPE默认为uint16_t，可以在FreeRTOSConfig.h 中#define

`configSTACK_DEPTH_TYPE`为`unsigned long`或`size_t`，如果堆栈深度乘以堆栈宽度大于65535（可能的最大的16位数）。（第13.3节：堆栈溢出，介绍了一种选择最佳堆栈大小的实际方法。）

• `pvParameters`

实现任务的函数接受单个`void`指针（`void *`）参数。`pvParameters`是作为该参数传递到任务中的值。

• `uxPriority`

定义任务的优先级。0是最低优先级，（`configMAX_PRIORITIES-1`）是最高优先级。第4.5节描述了用户定义的`configMAX_PRIORITIES`常量。

如果定义了一个单位优先级大于（`configMAX_PRIORITIES-1`），则将其限制为（`configMAX_PRIORITIES-1`）。

• `pxCreatedTask`

指向要存储到已创建任务的句柄的位置的指针。此句柄可以在将来的API调用中使用，例如，更改任务的优先级或删除任务。

`pxcreated`任务是一个可选参数，如果不需要任务的句柄，则可以设置为`NULL`。

• 返回值

有两个可能的返回值：

◦ `pdPASS`

这表明该任务已成功创建。

◦ `pdFAIL`

这表明没有足够的堆内存来创建该任务。第3章提供了关于堆内存管理的更多信息。

示例4.1 创建任务

下面的示例演示了创建两个简单的任务，然后启动新创建的任务所需的步骤。这些任务只是通过使用一个粗忙循环定期打印出一个字符串来创建周期延迟。这两个任务都以相同的优先级创建，除了它们打印出的字符串之外其他都是相同的——它们各自的实现请参见清单4.4和清单4.5。有关在使用`printf()`的警告，请参见第8章

```
void vTask1( void * pvParameters )
{
```

```

/* ulCount is declared volatile to ensure it is not optimized out.*/
volatile unsigned long ulCount;

for( ;; )
{
    /*打印出当前任务任务的名称。*/
    vPrintLine( "Task 1 is running" );

    /*延迟了一段时间。*/
    for (ulCount = 0; ulCount < mainDELAY_LOOP_COUNT; ulCount++)
    { /*
        *这个循环只是一个非常粗糙的延迟实现。有
        *在这里没什么可做的。以后具有适当的延迟/睡眠
        *功能的例子将取代这种循环。
        */
    }
}
}

```

清单4.4 实施例4.1中使用的第一个任务的实现

```

void vTask2( void * pvParameters
){
    /* ulCount is declared volatile to ensure it is not optimized out. */
    volatile unsigned long ulCount;

    /*与大多数任务一样，这个任务是在一个无限循环中实现的。
    */for( ;; ){
        /*请打印出此任务的名称。*/
        vPrintLine( "Task 2 is running" );

        /*延迟了一段时间。*/
        for (ulCount = 0; ulCount < mainDELAY_LOOP_COUNT; ulCount++)
        {
            /*
            *这个循环只是一个非常粗糙的延迟实现。有
            *在这里没什么可做的。以后具有适当的延迟/睡眠
            *功能的例子将取代这种循环。
            */
        }
    }
}

```

清单4.5 实施例4.1中使用的第二个任务的实现

主函数在启动调度程序之前创建任务——其实现请参见清单4.6。

```
int main (void)
{
    /*
     *在启动FreeRTOS后，这里声明的变量可能不再存在
     *调度程序。不要尝试访问在所使用的堆栈上声明的变量
     *由主函数从任务中完成。*/

    /*
     *创建这两个任务中的一个。请注意，一个真正的应用程序应该进行检查
     *xTaskCreate（）调用的返回值，以确保该任务为
     *创建成功。*/
    xTaskCreate( vTask1, /* Pointer to the function that implements the task.*/
                "Task 1", /* Text name for the task. */
                1000,      /*堆栈深度。*/
                NULL,      /*此示例不使用任务参数。*/
                1,         /*此任务将在优先级为1上运行。*/
                NULL )     /*此示例不使用任务句柄。*/

    /*以完全相同的方式和相同的优先级创建其他任务。*/
    xTaskCreate( vTask2, "Task2", 1000, NULL, 1, NULL );

    /*启动调度程序，以便任务开始执行。*/
    vTaskStartScheduler();

    /*
     *如果一切都好，主函数将不会到达这里，因为调度程序现在
     *正在运行已创建的任务。如果主函数确实到达了这里，那么就
     *没有足够的堆内存来创建空闲任务或计时器任务。在本书后面描述）
     *第三章提供了更多的信息。
     */
    for(;;);
}
```

清单4.6 启动示例4.1任务

执行该示例将产生如图4.2所示的输出。

```
C:\Temp>rtosdemo
Task 1 is running
Task 2 is running
Task 1 is running
```

```
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
```

图4.2在执行示例4.1时产生的输出。[⁴]

[⁴]: 屏幕截图显示每个任务在下一个任务执行之前只打印一次消息。这是一个通过使用FreeRTOS Windows模拟器而产生的人工场景。Windows模拟器并不是真正实时的。此外，写入Windows控制台需要较长时间，会导致Windows系统调用。使用快速和非阻塞打印函数在真正的嵌入目标上执行相同的代码可能会导致每个任务在切换之前多次打印其字符串。

图4.2显示了似乎同时执行的两个任务；但是，这两个任务都在同一处理器核心上执行，因此情况并非如此。实际上，这两个任务都在快速进入和退出运行状态。这两个任务以相同的优先级运行，因此在同一个处理器核心上共享时间。图4.3显示了它们的实际执行模式。

图4.3底部的箭头显示了从时间t1开始经过的时间。彩色的线表示在每个时间点上正在执行哪个任务——例如，任务1在时间t1和t2之间执行。

任何时候只能有一个任务处于运行状态。因此，当一个任务进入“运行”状态（任务切换）时，另一个任务进入“不运行”状态（任务切换）。

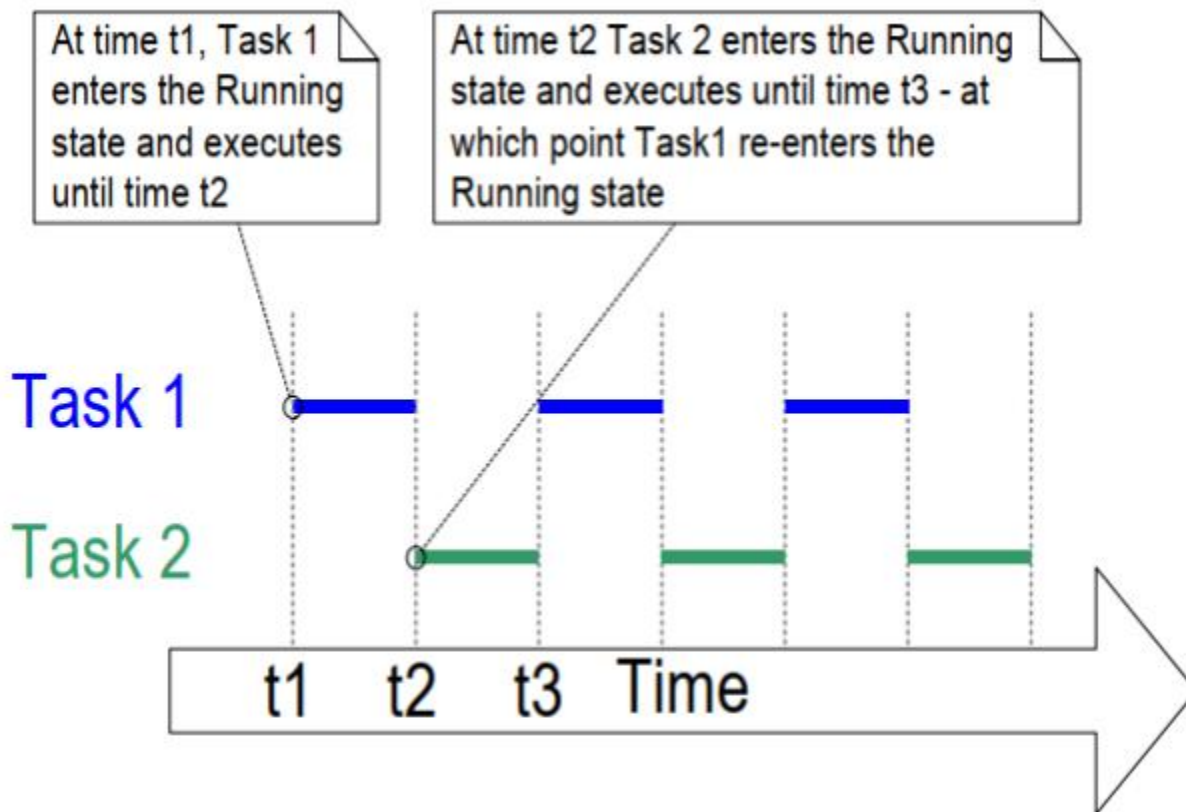


图4.3 这两个示例4.1任务的实际执行模式

示例4.1在启动调度程序之前，从主函数中创建了这两个任务。也可以从另一个任务中创建一个任务。例如，任务2可以从任务1中创建，如清单4.7所示。

```
void vTask1 (void *pvParameters)
{
    const char *pcTaskName = "任务1正在运行\r\n";
    volatile unsigned long ul;

    /*如果这个任务代码正在执行，那么调度程序必须已经
    *启动。在进入无限循环之前，请先创建另一个任务。*/
    xTaskCreate (vTask2, "task2", 1000, NULL, 1, NULL);

    for(;;)
    {
        /*请打印出此任务的名称。*/
        vPrintLine ( pcTaskName );

        /*延迟了一段时间。*/
        for (ul = 0; ul < mainDELA_LOO_COUNT; ul++)
        {
```

```

        /*
        *这个循环只是一个非常粗糙的延迟实现。有
        *在这里没什么可做的。以后的例子将取代这种
        *循环，具有适当的延迟/睡眠功能。
        */
    }
}
}

```

清单4.7 在调度程序启动后从另一个任务中创建一个任务

示例4.2: 使用任务参数

在示例4.1中创建的两个任务几乎是相同的，它们之间唯一的区别是它们打印出的文本字符串。如果您创建了单个任务实现的两个实例，并使用任务参数将该字符串传递到每个实例中，这将移除重复。

示例4.2将名为`vTaskFunction()`的单个任务函数替换为示例4.1中使用的两个任务函数，如清单4.8所示。注意如何将任务参数转换为`char *`以获得任务应该打印出的字符串。

```

void vTaskFunction( void *
pvParameters ) {

    char *pcTaskName;
    volatile unsigned long ul;

    /*
    *要打印出来的字符串将通过该参数传入。将其转换为
    *字符指针。*/
    pcTaskName = ( char * ) pvParameters;

    /*与大多数任务一样，这个任务是在一个无限循环中实现的。*/
    for( ;; ){
        /*打印出此任务的名称。*/
        vPrintLine ( pcTaskName );

        /*延迟了一段时间。*/
        for (ul = 0; ul < mainDELAY_LOOP_COUNT; ul++
        ) {
            /*
            *这个循环只是一个非常粗糙的延迟实现。
            *在这里没什么可做的。以后的实例将取代这种粗糙
            *循环。
            */

```

```

        */
    }
}
}

```

清单4.8 示例4.2中用于创建两个任务的单一任务函数

清单4.9 通过vTaskFunction() 创建了任务的两个实例，它们使用任务的参数向每个字符串中传递不同的字符串。这两个任务在FreeRTOS调度程序的控制下独立执行，并使用自己的堆栈，因此使用自己的pcTaskName和u1变量副本。

```

/*
 *定义将作为任务参数传递进来的字符串
 *为const，而不是在主函数使用的堆栈上，以确保它们仍然
 *在执行任务时有效。*/
static const char * pcTextForTask1 = "Task 1 is running";
static const char * pcTextForTask2 = "Task 2 is running";

int main( void )
{
    /*
     *在启动FreeRTOS后，这里声明的变量可能不再存在
     *调度程序。不要试图从任务中访问主函数使用的堆栈
     *上声明的变量。*/

    /*创建这两个任务之一。*/
    xTaskCreate( vTaskFunction, /* Pointer to the function that implements the task. */

                "Task 1", /*任务的文本名称。这是为了 仅方便调试。*/

                1000, /*堆栈深度-小型微控制器 将使用比这个更少的堆栈。*/

                ( void * ) pcTextForTask1, /*传递要打印到的文本

                1, /* This task will run at priority 1. */

                NULL );

    /*以完全相同的方式创建另一个任务。请注意，这次
     *多个任务正在从相同的任务实现
     *(vTaskFunction)。只有在参数中传递的值才有所不同。
     *正在创建具有相同任务定义的两个实例。*/

    xTaskCreate( vTaskFunction,
                "Task 2",

```

```

        1000,
        (void *) pcTextForTask2,
        1,
        NULL );

/*启动调度程序，以便任务开始执行。*/
vTaskStartScheduler();

/*

*如果一切都好，主函数将不会到达这里，因为调度程序现在
*正在运行已创建的任务。如果主函数确实到达了这里，那么就
*没有足够的堆内存来创建空闲任务或计时器任务。在本书后面描述）
*第三章提供了更多的信息。
*/
for(;;);

}

```

清单4.9 示例2的主函数

示例4.2的输出与图4.2中显示的示例1完全相同。

4.5 任务优先级

FreeRTOS调度程序始终确保可运行的最高优先级任务是选择要进入运行状态的任务。同等优先级的任务依次转换到和退出运行状态。

用于创建任务的API函数的uxPriority参数赋予任务初始优先级。vTaskPrioritySet() API函数在创建任务后更改任务的优先级。

应用程序定义的configMAX_PRIORITIES编译时配置常量可设置可用优先级的数量。低数字优先级值表示低优先级任务，优先级0是可能的最低优先级，因此有效优先级范围从0到（configMAX_PRIORITIES-1）。任意数量的任务都可以共享相同的优先级。

FreeRTOS调度程序有两个用于选择正在运行的状态任务的算法的实现，而configMAX_PRIORITIES的最大允许值取决于所使用的实现：

4.5.1 通用调度程序

通用调度程序是用C编写的，可以与所有FreeRTOS体系结构端口一起使用。它没有对configMAX_PRIORITIES施加一个上限。一般来说，建议最小化configMAX_PRIORITIES，因为更多的值需要更多的RAM，并将导致更长的最坏情况执行时间。

4.5.2 架构-优化的调度程序

体系结构优化的实现是用特定于体系结构的汇编代码编写的，并且比通用的c实现性能更好，并且对于所有的configMAX_PRIORITIES值，最坏情况下的执行时间都是相同的。

架构优化实现使32位架构的configMAX_PRIORITIES最大值为32, 64位架构的configMAX_PRIORITIES最大值为64。与通用方法一样，建议将configMAX_PRIORITIES保持在最小值，因为更高的值需要更多的RAM。

在FreeRTOSConfig.h中设置configUSE_PORT_optimized_TASK_SELECTION为1以使用体系结构优化的实现，或设置为0以使用通用实现。并不是所有的FreeRTOS端口都有一个体系结构优化的实现。如果configUSE_PORT_optimized_TASK_SELECTION未定义为1，则默认为1。没有定义的，如果未定义，默认为0。

4.6 时间测量和计时器中断

Section 4.12, Scheduling Algorithms, 描述了一个被称为“时间切片”的可选特性。到目前为止的例子中使用了时间切片，是在它们产生的输出中观察到的行为。在示例中，两个任务都以相同的优先级创建，并且两个任务总是能够运行。因此，为“时间片”执行的每个任务，在时间片开始时输入“运行”状态，在时间片结束时退出“运行”状态。在图4.3中，t1和t2之间的时间等于单个时间片。

调度程序在每个时间片的末尾执行，以选择下一个要运行的任务^[5]。一个周期性的中断，称为“滴答中断”，被用于这个目的。configTICK_RATE_HZ编译时配置常数设置了滴答中断的频率，因此也设置了每个时间片的长度。例如，将configTICK_RATE_HZ设置为100（Hz）会导致每个时间片持续10毫秒。两次滴答中断之间的时间被称为“滴答周期”——所以一个时间片等于一个滴答周期。

^[5]：需要注意的是，时间片的结束并不是调度程序可以选择要运行的新任务的唯一位置。正如我们将在本书中演示的，调度程序还将选择在当前执行的任务进入阻塞状态之后，或者当中断将更高优先级的任务移动到就绪状态时，立即运行的新任务。

图4.4扩展了图4.3，同时还显示了调度程序的执行情况。在图4.4中，顶部一行显示调度程序的执行时间，细箭头显示从任务到提示中断，然后从提示中断回到其他任务的执行顺序。

configTICK_RATE_HZ的最佳值取决于应用程序，尽管典型的值是100。

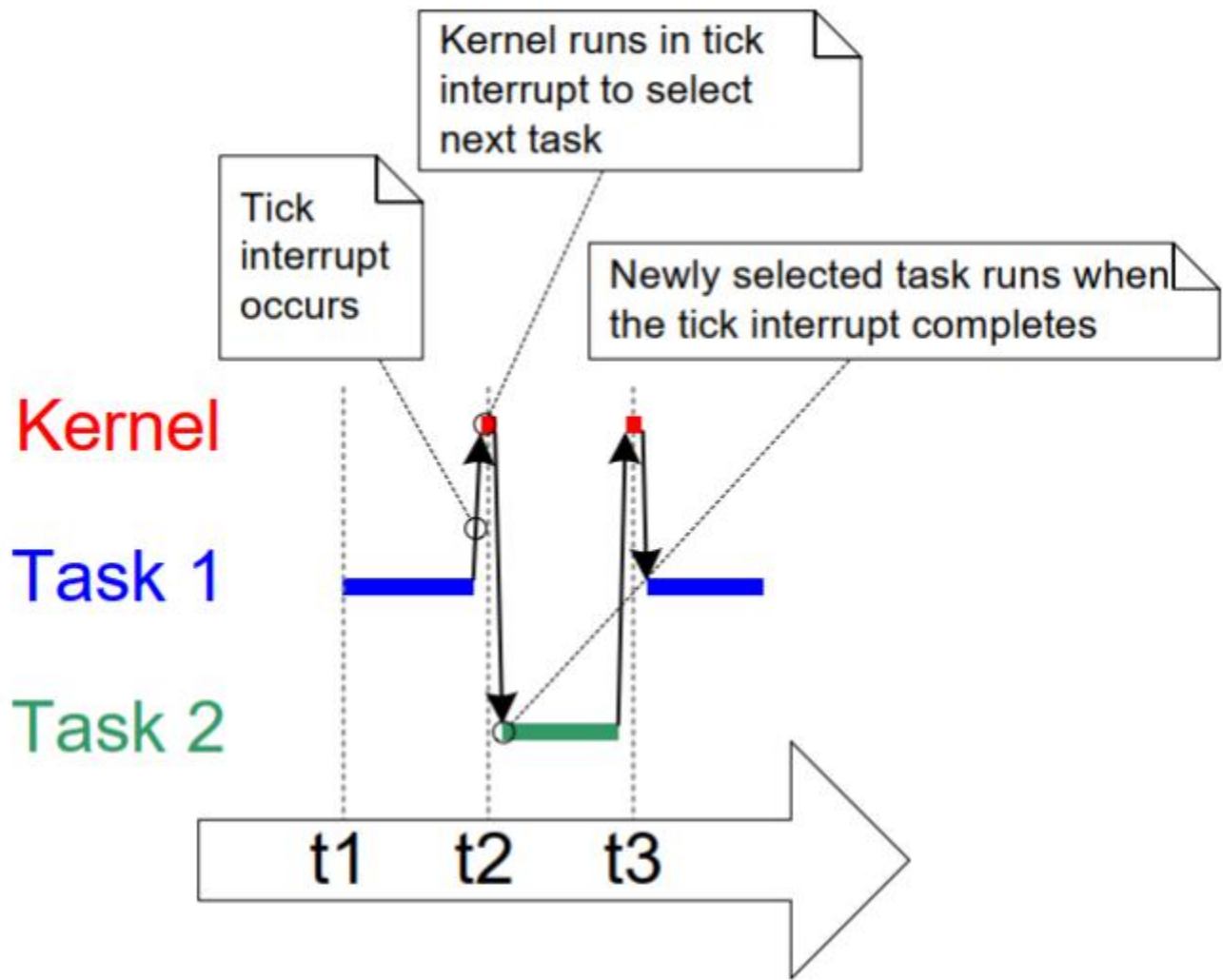


图4.4 执行序列详细地显示了正在执行的滴答中断

FreeRTOS API调用以多个滴答周期指定时间，通常简单地称为“滴答”。`pdMS_TO_TICKS()`宏将以毫秒为单位指定的时间转换为滴答指定的时间。可用的分辨率取决于所定义的滴答频率，如果滴答频率高于1 KHz（如果`configTICK_RATE_HZ`大于1000），则不能使用`pdMS_TO_TICKS()`。清单4.10展示了如何使用`pdMS_TO_TICKS()`将指定为200毫秒的时间转换为在刻度符中指定的等效时间。

```
/*
 * pdMS_TO_TICKS() 以毫秒为单位的时间作为其唯一参数
 * ，并计算为滴答周期的等效时间。此示例显示
 * xTimeInTicks 被设置为等于200毫秒的滴答周期。*/
TickType_t xTimeInTicks = pdMS_TO_TICKS(200);
```

清单4.10 使用`pdMS_TO_TICKS()`宏将200毫秒转换为等效的时间

使用pdMS_TO_TICKS（）以毫秒为单位指定时间，而不是直接作为滴答，确保如果滴答频率发生改变，则在应用程序中指定的时间不会改变

“滴答计数(tick count)”是自调度程序启动以来发生的滴答中断的总数，假设滴答计数没有溢出。用户应用程序在指定延迟期间时不必考虑溢出，因为FreeRTOS在内部管理时间一致性。

第4.12：调度算法描述了影响调度器何时选择要运行的新任务以及何时执行提示中断的配置常量。

示例4.3 试验使用优先级

调度程序将始终确保可以运行的最高优先级任务是选择以进入运行状态的任务。到目前为止，示例创建了两个相同优先级的任务，因此都依次输入和退出运行状态。此示例着眼于当任务具有不同的优先级时会发生什么。清单4.11显示了用于创建任务的代码，第一个代码的优先级为1，第二个代码的优先级为2。实现这两个任务的单个函数没有更改；它仍然会定期打印一个字符串，并使用一个空循环来创建一个延迟。

```
/*
 *定义将作为任务参数传递进来的字符串。
 *这些都是已定义的常量，而不是在堆栈上，以确保它们保持有效
 *当任务正在执行时。*/
static const char * pcTextForTask1 = "Task 1 is running";
static const char * pcTextForTask2 = "Task 2 is running";

int main (void)
{
    /*创建优先级为1的第一个任务。*/
    xTaskCreate( vTaskFunction, /*任务函数*/
                "Task", /*任务名称*/
                1000, /*任务堆栈深度*/
                (void *) pcTextForTask1, /*任务参数*/
                1, /*任务优先级*/
                NULL );

    /*以更高的优先级2创建第二个任务。*/
    xTaskCreate ( vTaskFunction,
                  Task2",
                  1000,
                  (void*) pcTextForTask2,
                  2,
                  NULL );

    /*启动调度程序，以便任务开始执行。*/
    vTaskStartScheduler();
```

```
/*将不会到达这里。*/  
return 0;  
}
```

清单4.11 按不同的优先级创建两个任务

图4.5 显示示例4.3所产生的输出。

调度程序将始终选择可以运行的最高优先级的任务。任务2的优先级高于任务1，并且总是可以运行；因此，调度器总是选择任务2，而任务1从不执行。任务1被任务2称为“缺乏”处理时间——它不能打印其字符串，因为它从未处于运行状态。

```
C:\Temp>rtosdemo  
Task 2 is running  
Task 2 is running  
Task 2 is running  
Task 2 is running  
Task 2 is running  
Task 2 is running  
Task 2 is running  
Task 2 is running  
Task 2 is running  
Task 2 is running  
Task 2 is running  
Task 2 is running  
Task 2 is running  
Task 2 is running  
Task 2 is running  
Task 2 is running
```

图4.5以不同的优先级运行这两个任务

任务2总是可以运行，因为它不需要等待任何事情——它要么绕着空循环循环，要么打印到终端。

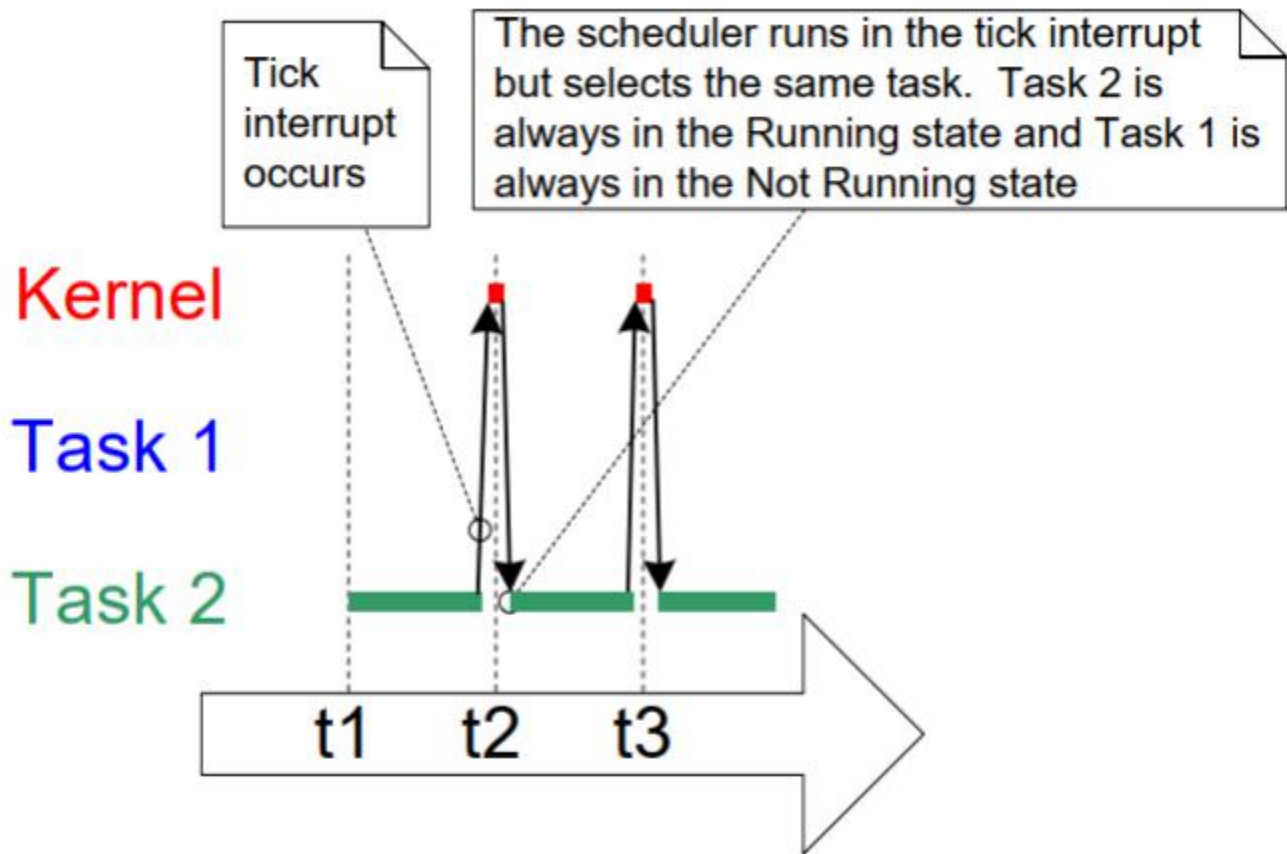


图4.6 示例4.3中一个任务比另一个任务优先级时的执行模式

4.7 扩展未运行的状态

到目前为止，创建的任务总是有处理来执行，从来没有等待任何事情——由于它们从来没有等待任何事情，它们总是能够进入运行状态。这种“连续处理”任务的有用性有限，因为它们只能在非常最低的优先级下创建。如果它们以任何其他优先级运行，它们将完全阻塞较低优先级的任务永远运行。

为了使这些任务有用，必须将它们重写为事件驱动的。事件驱动的任务只有在事件触发后才需要执行的工作（处理），并且在此之前无法进入正在运行中的状态。调度程序总是选择可以运行的最高优先级的任务。如果一个高优先级的任务因为它正在等待一个事件而无法被选择，则调度程序必须选择一个可以运行的较低优先级的任务。因此，编写事件驱动任务意味着任务可以在不同的优先级上创建，而不是最高优先级的任务满足所有处理时间而饿死低优先级任务。

4.7.1 阻塞(阻塞)状态

等待事件的任务称为处于“已阻塞”状态，即“未运行”状态的子状态。任务可以进入阻塞状态，

以等待两种不同类型的事件：

1. 时间（与时间相关）事件—这些事件在延迟期到期或达到绝对时间时发生。例如，一个任务可能会进入阻塞状态，等待10毫秒通过。

2. 同步事件——这些事件源自另一个任务或中断。例如，任务可能进入阻塞状态，等待数据到达队列。同步事件涵盖了广泛的事件类型。

FreeRTOS队列、二进制信号量、计数信号量、**互斥锁**、递归互斥、事件组、流缓冲区、消息缓冲区和直接到任务通知都可以创建同步事件。后面的章节涵盖了大部分的这些特性。

任务可以用超时来阻塞同步事件，有效地同时阻塞两种类型的事件。例如，一个任务可以选择等待最多10毫秒的数据到达队列。如果数据在10毫秒内到达，或者10毫秒没有到达，任务将保持阻塞状态。

4.7.2 暂停(挂起)状态

*暂停(挂起)*也是不运行的一个子状态。处于“已挂起”状态下的任务对调度程序不可用。进入挂起状态的唯一方法是通过调用`vTaskSuspend()` API函数，而唯一的解除方法是通过调用 `vTaskResume()` 或 `xTaskResumeFromISR()` API函数。大多数应用程序都不使用“已挂起”状态。

4.7.3 就绪状态

处于“不运行”状态且未被阻塞或挂起的任务称为处于“就绪”状态。它们可以运行，因此已“准备”运行，但当前未处于运行状态。

4.7.4 正在完成状态转换图

图4.7扩展了简化的状态图，以包括本节中描述的所有“未运行”子状态。在示例中创建的任务迄今尚未使用阻塞或挂起状态。它们只在就绪状态和运行状态之间转换，如图4.7中的粗体行所示。

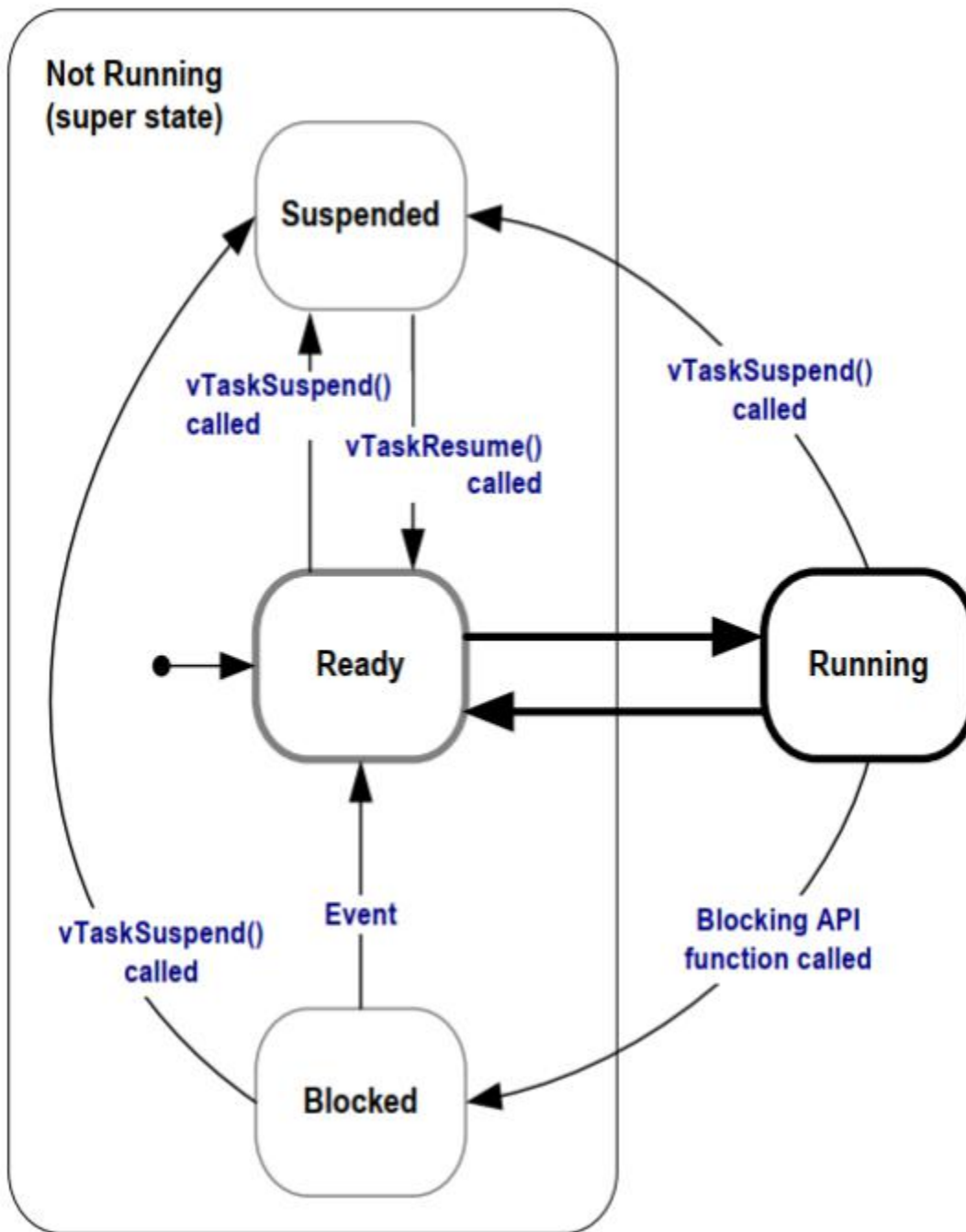


图4. 7完整的任务状态机

示例4. 4 *使用一个被阻塞的状态来创建一个延迟

到目前为止，在这些例子中创建的所有任务都是“周期性的”——它们延迟了一段时间，然后打印出字符串，然后再次延迟，以此类推。延迟已经使用一个零循环非常粗略地生成了——任务轮询一个递增的循环计数器，直到它达到一个固定的值。示例4. 3清楚地说明了该方法的缺点。较高优先级的任务在执行空循环时仍处于运行状态，“饥饿”任何处理时间的较低优先级任务。

任何形式的轮询都有其他几个缺点，尤其是它的效率低下。在轮询期间，该任务实际上没有任何工作要做，但它仍然使用最大的处理时间，因此浪费了处理器周期。示例4.4通过用对vTaskDelay（）API函数的调用替换轮询空循环来纠正这个行为，它的原型如清单4.12所示。新的任务定义如清单4.13所示。请注意，只有在FreeRTOSConfig.h中，当INCLUDE_vTaskDelay被设置为1时，vTaskDelay（）API函数才可用。

vTaskDelay()将调用任务置于阻塞状态，以获得固定数量的滴答中断。该任务在处于“阻塞”状态时不使用任何处理时间，因此该任务只在实际有工作要完成时使用处理时间。

```
void vTaskDelay( TickType_t xTicksToDelay );
```

清单4.12 vTaskDelay（）API函数原型

vTaskDelay()参数:

- xTicksToDelay

在切换回已就绪状态之前，调用任务将处于“阻塞”状态的中断数。

例如，如果一个名为vTaskDelay（100）的任务，当滴答计数为10,000时，它将立即进入阻塞状态，并保持阻塞状态，直到滴答计数达到10,100。

宏pdMS_TO_TICKS（）可用于将以毫秒为单位指定的时间转换为在刻度中指定的时间。例如，调用vTaskDelay（pdMS_TO_TICKS（100））会导致调用任务保持在阻塞状态下，持续100毫秒。

```
void vTaskFunction( void * pvParameters )
{
    char* pcTaskName;
    const TickType_t xDelay250ms = pdMS_TO_TICKS (250);

    /*
     *要打印出来的字符串将通过该参数传入。将其转换为
     *字符指针。*/
    pcTaskName = ( char * ) pvParameters;

    /*与大多数任务一样，这个任务是在一个无限循环中实现的*/

    for ( ; ; ) {

        /*请打印出此任务的名称。*/
        vPrintLine ( pcTaskName );
```

```

/*
 *延迟一段时间。这次调用了vTaskDelay ( )
 *将任务置于阻塞状态，直到延迟时间结束为止
 *该参数需要在“滴度数”中指定的时间，并且
 *使用pdMS_TO_TICKS ( ) 宏(其中xDelay250ms被
 *声明为常量)，将250毫秒转换为一个等效的时间
 *滴答。
 */
vTaskDelay ( xDelay250ms );
}
}

```

清单4.13 调用vTaskDelay () 替换空循环延迟后的示例任务的源代码

尽管这两个任务仍然在不同的优先级下创建，但它们现在都将运行。如图4.8所示的示例4.4的输出确认了预期的行为。

```

C:\Temp>rtosdemo
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running
Task 2 is running
Task 1 is running

```

图4.8 在执行示例4.4时产生的输出

图4.9中所示的执行序列解释了为什么这两个任务都会运行，即使它们是在不同的优先级下创建的。为了简单起见，我们省略了调度程序本身的执行

空闲任务将在调度程序启动时自动创建，以确保始终至少有一个任务可以运行（至少有一个任务处于就绪状态）。第4.8节：空闲任务和空闲任务钩更详细地描述了空闲任务。

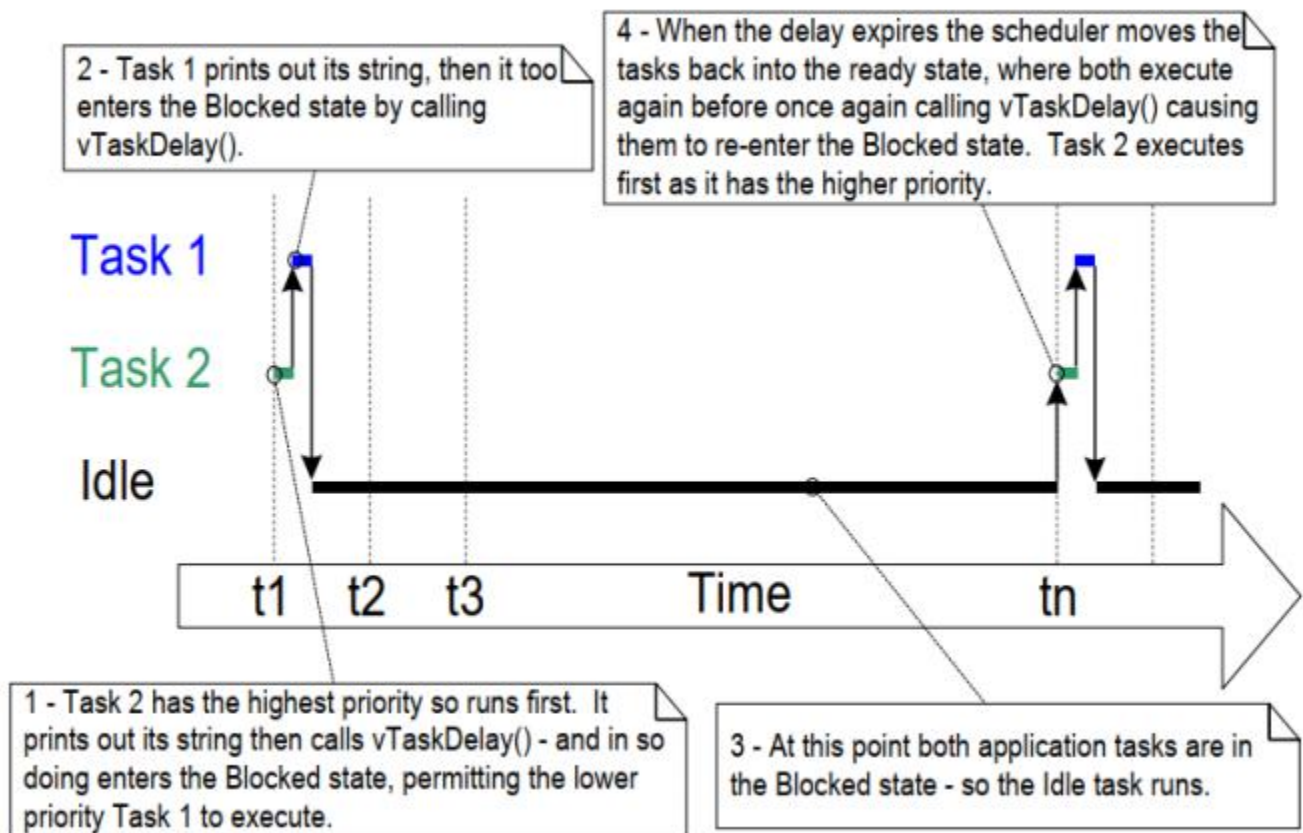


图4.9 当任务使用 `vTaskDelay()` 代替空循环时的执行序列

只有这两个任务的实现发生了变化，而不是它们的功能。比较图4.9和图4.4，清楚地说明了这个功能正在以一种更有效的方式实现。

图4.4显示了当任务使用空循环来创建延迟并始终能够运行时的执行模式。因此，它们之间使用了100%的可用处理器时间。图4.9显示了任务在整个延迟期间内进入阻塞状态时的执行模式。它们只在实际上有需要执行的工作时使用处理器时间（在这种情况下只是要打印出来的消息），因此只使用可用处理时间的一小部分。

在图4.9所示的场景中，每次任务离开阻塞状态时，它们在重新进入阻塞状态之前执行少量的时间。大多数情况下，没有可以运行的应用程序任务（没有处于已就绪状态的应用程序任务），因此，也没有可以选择来进入正在运行状态的应用程序任务。在这种情况下，空闲任务会运行。分配给空闲时间的处理时间是系统中备用处理能力的度量。仅仅通过允许应用程序完全被事件驱动，使用RTOS就可以显著增加备用处理能力。

图4.10中的粗体线显示了示例4.4中任务执行的转换，每个任务在返回到“就绪”状态之前都经过“阻塞”状态。

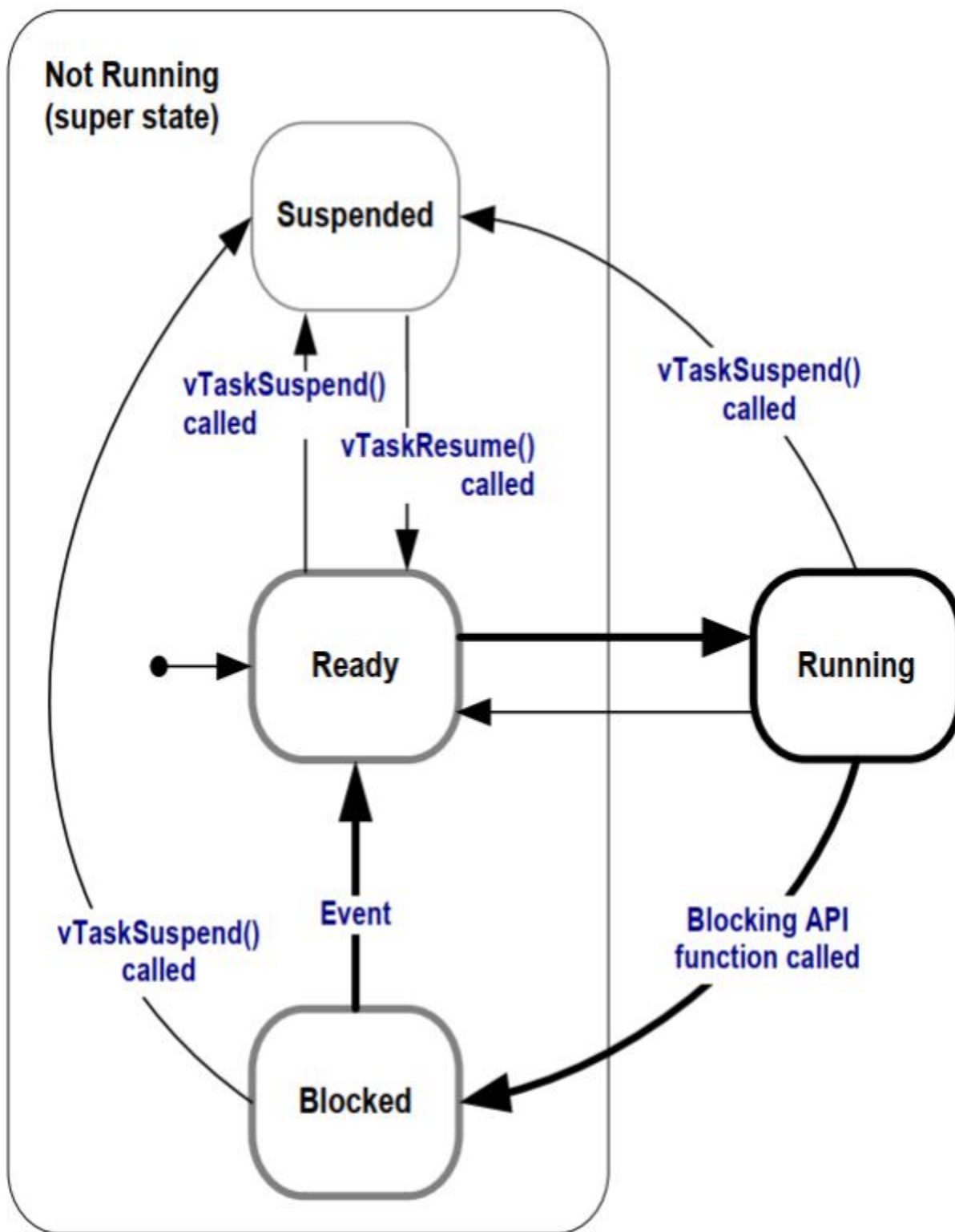


图4.10 粗体线表示示例4.4中的任务所执行的状态转换

4.7.5 vTaskDelayUntil() API函数

vTaskDelayUntil() 类似于 **vTaskDelay()**。如刚才演示的，**vTaskDelay()** 参数指定了调用 **vTaskDelay()** 的任务与再次退出阻塞状态之间应该发生的滴答中断次数。指定任务保持处于阻塞状态的时间长度

但是任务离开延迟状态的时间，与相对于vTaskDelay（）被调用的时间有关。

vTaskDelayUntil（）指定调用任务从阻塞状态移动到就绪状态的确切勾选计数值。直到（）是API函数时使用一个固定的执行期间（你希望你的任务定期执行固定频率），调用任务的时间是绝对的，而不是相对于函数被调用（与vTaskDelay（））。

```
void vTaskDelayUntil( TickType_t * pxPreviousWakeTime,
                    TickType_t xTimeIncrement );
```

vTaskDelayUntil（）API函数的原型

vTaskDelayUntil（）参数

•pxPreviousWakeTime

这个参数的命名是基于这样一个假设，vTaskDelayUntil（）被用于执行一个以固定频率定期执行的任务。在这种情况下，pxPreviousWakeTime保持任务最后一次离开阻塞状态（被“唤醒”）的时间。此时间被用作参考点，以计算任务下一次离开阻塞状态的时间。

pxPreviousWakeTime所指向的变量会在函数中自动更新；它通常不会被应用程序代码修改，但必须在第一次使用之前初始化为当前的滴答计数。清单4.15演示了如何初始化该变量。

•xTimeIncrement

这个参数的命名也是基于这样一个假设，函数被用来实现一个周期性执行的任务，其固定频率由xTimeIncrement设置。

xTimeIncrement在“滴答”中指定。宏pdMS_TO_TICKS（）可用于将以毫秒为单位指定的时间转换为在刻度中指定的时间。

Example 4.5 将示例任务转换为使用vTaskDelayUntil（）

在示例4.4中创建的两个任务是周期性任务，但是使用vTaskDelay（）并不能保证它们运行的频率是固定的，因为任务离开阻塞状态的时间相对于它们调用vTaskDelay（）的时间。将任务转换为vTaskDelayUntil（）而不是vTaskDelay（）解决了这个潜在的问题。

```
void vTaskFunction( void * pvParameters )
{
```

```

char * pcTaskName;
TickType_t xLastWakeTime;

/*
 *要打印出来的字符串将通过该参数传入。将其转换为
 *字符指针。*/
pcTaskName = ( char * ) pvParameters;

/*
 *xLastWakeTime变量需要用当前的标记进行初始化
 *计数。注意，这是唯一一次写入变量
 *明确。在此之后，在vTaskDelayUntil() 自动更
 *新。*/
xLastWakeTime = xTaskGetTickCount();

/*与大多数任务一样，这个任务是在一个无限循环中实现的。
*/for ( ; ; ) {
    /*请打印出此任务的名称。*/
    vPrintLine ( pcTaskName );

    /*
     *这个任务应该恰好每250毫秒执行一次。按
     *vTaskDelay ( ) 函数，时间以滴答为单位测量，和
     * pdMS_TO_TICKS ( ) 宏用于将毫秒转换为刻度。
     *唤醒时间自动更新，所以
     *没有被任务明确地更新。*/
    vTaskDelayUntil( &xLastWakeTime, pdMS_TO_TICKS( 250 ) );
}
}

```

清单4.15 使用 vTaskDelayUntil() 实现示例任务

示例4.5 所产生的输出与图4.8中的示例4.4所显示的输出完全相同

示例4.6 结合阻塞和非阻塞任务

前面的示例单独检查了轮询和阻塞任务的行为。这个示例重申了我们已经说过的关于预期的系统行为的内容，并演示了当两种方案组合时的执行顺序，如下所示：

1. 在优先级为1处创建了两个任务。这些东西只是不断地打印出一个字符串。

这些任务不会进行可能导致它们进入“阻塞”状态的API函数调用，因此始终处于“就绪”或“正在运行”状态。这种性质的任务被称为“连续处理”任务，因为它们

总是有工作要做（尽管在这种情况下，这是相当琐碎的工作）。清单4.16显示了连续处理任务的源代码。

2. 然后在优先级2处创建第三个任务，它高于其他两个任务的优先级。第三个任务也只是打印出一个字符串，但这一次是周期性的，所以它使用`vTaskDelayUntil()` API函数在每次打印迭代之间将自身置于阻塞状态。

清单4.17 显示了定期任务的源代码。

```
void vContinuousProcessingTask( void * pvParameters )
{
    char * pcTaskName;;

    /*
     *要打印出来的字符串将通过该参数传入。将其转换为
     *字符指针。*/
    pcTaskName = ( char * ) pvParameters;

    /*与大多数任务一样，这个任务是在一个无限循环中实现的。
    */for ( ; ; ) {
        /*
         *打印出此任务的名称。这个任务只是重复地这样做
         *从来没有任何阻塞或延时。*/
        vPrintLine ( pcTaskName );
    }
}
```

清单4.16 在例4.6中使用的连续处理任务

```
void vPeriodicTask( void * pvParameters )
{
    TickType_t xLastWakeTime;

    const TickType_t xDelay3ms = pdMS_TO_TICKS ( 3 );

    /*
     *xLastWakeTime变量需要用当前的标记进行初始化
     *计数。注意，这是唯一一次写入变量
     *明确。在此之后，在vTaskDelayUntil() 自动更
     *新。*/
    xLastWakeTime = xTaskGetTickCount();

    /*与大多数任务一样，这个任务是在一个无限循环中实现的。*/
```

```

for ( ; ; )
{
    /*请打印出此任务的名称。*/
    vPrintLine( "Periodic task is running" );

    /*
     *任务应该每3毫秒执行一次
     *在此函数中声明xDelay3ms。*/
    vTaskDelayUntil ( &xLastWakeTime, xDelay3ms );
}
}

```

清单4.17 示例4.6中使用的定期任务

图4.11显示了示例4.6所产生的输出，并解释了由图4.12中所示的执行序列所给出的观察到的行为

```

Continuous task 2 running
Continuous task 2 running
Periodic task is running
Continuous task 1 is running
Continuous task 1 is running
Continuous task 1 is running
Continuous task 1 is running
Continuous task 1 is running
Continuous task 2 is running
Continuous task 2 is running
Continuous task 2 is running
Continuous task 2 is running
Continuous task 2 is running
Continuous task 1 is running
Continuous task 1 is running
Continuous task 1 is running
Continuous task 1 is running
Continuous task 1 is running
Continuous task 1 is running
Continuous task 1 is running
Continuous task 1 is running
Continuous task 1 is running
Continuous task 1 is running
Periodic task is running
Continuous task 2 running
Continuous task 2 running

```

图4.11在执行示例4.6时产生的输出

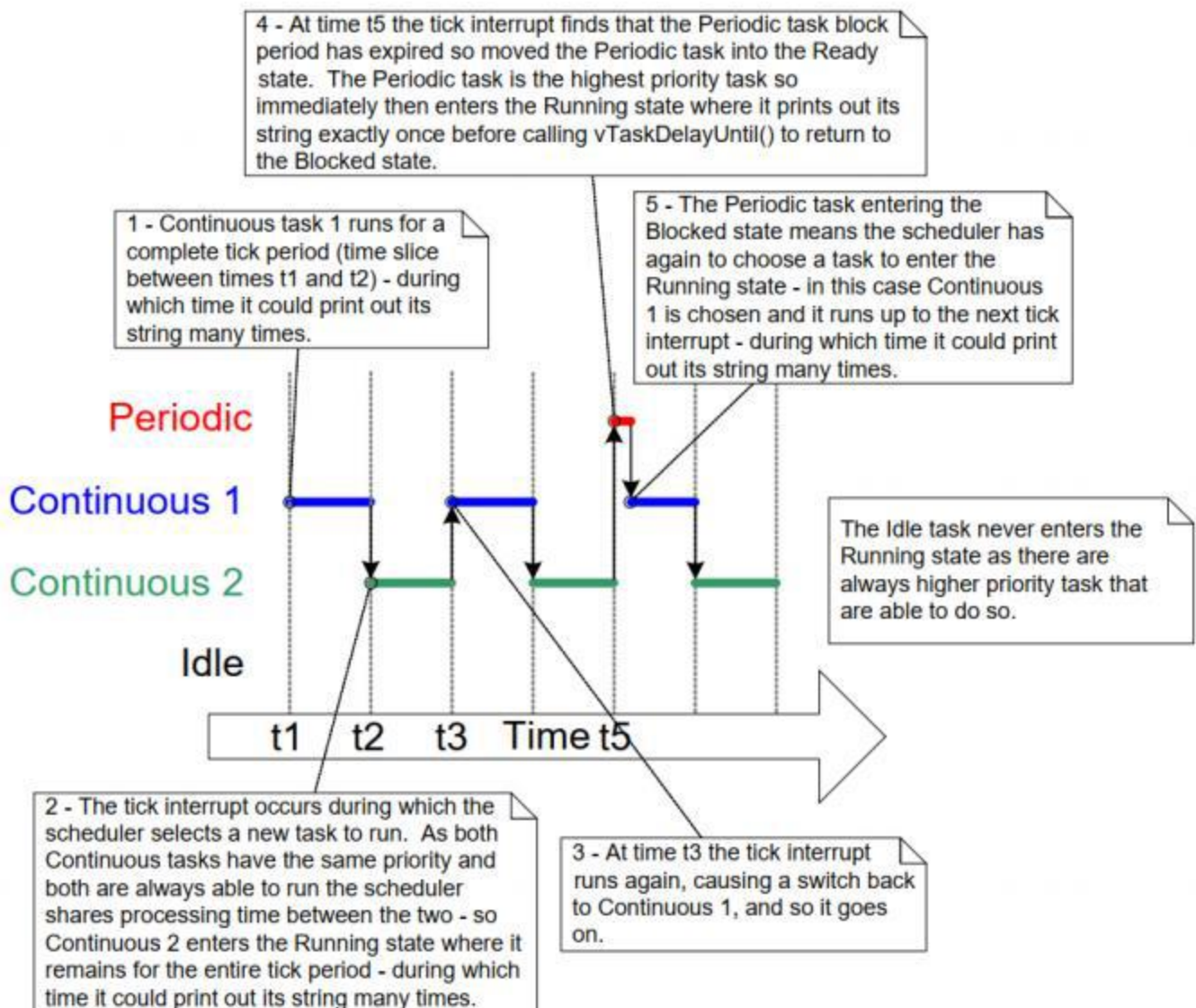


图4.12 示例4.6的执行模式

4.8 空闲任务和空闲任务挂钩

示例4.4中创建的任务大部分时间都处于阻塞状态。在这种状态下，它们无法运行，因此调度程序无法选择它们。

必须始终有一个任务可以进入运行状态^[6]。为了确保出现这种情况，当调用`vTaskStartScheduler()`时，调度程序会自动创建一个空闲任务。空闲任务只是坐在一个循环中执行什么，因此，就像第一个示例中的任务一样，它总是能够运行。

^[6]：即使使用FreeRTOS的特殊低功耗特性也是这种情况，在这种情况下，如果应用程序创建的任务都不能执行，那么执行FreeRTOS的微控制器将被置于低功耗模式。

空闲任务具有最低的优先级（优先级为零），以确保它不会阻塞具有更高优先级的应用程序任务进入正在运行状态。但是，如果需要，没有什么可以阻塞应用程序设计人员在空闲任务上，或者在空闲任务优先级上创建任务。FreeRTOSConfig.h中的configIDLE_SHOULD_YIELD编译时配置常数可以用来防止空闲任务消耗处理时间，这些处理时间将更有效地分配给优先级也为0的应用程序任务。第4.12节，调度算法，描述了configIDLE_SHOULD_YIELD。

以最低优先级运行可确保一旦较高优先级任务进入已就绪状态，空闲任务就会转换出“正在运行”状态。这可以在图4.9中的时间 t_n 中看到，其中空闲任务立即被交换出来，以允许任务2在即时任务2离开阻塞状态时执行。任务2抢占了空闲任务。抢占将自动发生，并且不知道任务将被抢占。

注意：如果一个任务使用vTaskDelete（）API函数来删除自己，那么空闲任务必须不能一直饥饿。这是因为空闲任务负责清理那些删除了自己的任务所使用的内核资源。

4.8.1 空闲任务挂钩功能

通过使用空闲钩子（或空闲回调）函数，可以将应用程序特定的功能直接添加到空闲任务中，该函数是在每次空闲任务循环迭代时由空闲任务自动调用一次的函数。

空闲任务钩子的常用用途包括：

执行低优先级、后台或连续处理功能，而不需要为此目的创建应用程序任务的RAM开销

测量备用处理能力的量。（只有在所有高优先级应用程序任务没有工作可执行时，空闲任务才会运行；因此，测量分配给空闲任务的处理时间可以明确显示空闲处理时间。）

将处理器置于低功耗模式，在不进行应用程序处理的情况下提供一种简单和自动的省电方法（尽管可实现的省电低于无滴答空闲模式实现的省电）。

4.8.2 空闲任务钩功能实现的局限性

空闲任务钩子功能必须遵守以下规则。

一个空闲的任务钩子函数永远不能试图阻塞或挂起它自身。

注意：以任何方式阻塞空闲任务都可能导致没有任务进入运行状态的场景。

如果一个应用程序任务使用vTaskDelete（）API函数来删除自己，那么空闲任务钩子必须总是在合理的时间内返回给调用者。这是因为空闲任务负责

清理分配给删除它们自己的任务的内核资源。如果空闲任务永久保持在空闲钩子功能中，则无法进行此清理。

空闲任务钩子函数必须具有清单4.18中所示的名称和原型。

```
void vApplicationIdleHook( void );
```

清单4.18 空闲任务钩子函数的名称和原型

示例4.7 *定义一个空闲的任务钩子函数

在示例4.4中使用阻塞vTaskDelay（）API调用创建了大量的空闲时间，即空闲任务执行的时间，因为两个应用程序任务都处于阻塞状态。示例4.7通过添加一个空闲钩子函数来利用这个空闲时间，其源代码如清单4.19所示。

```
/*声明一个将由钩子函数增加的变量。*/

volatile unsigned long ulIdleCycleCount = 0UL;

/*空闲钩函数必须称为 vApplicationIdleHook(),
*无参数，无返回。
*/
void vApplicationIdleHook(),
{
    /*这个钩子函数只增加了一个计数器。*/
    ulIdleCycleCount++;
}
```

清单4.19是一个非常简单的空闲钩子函数

configUSE_IDLE_HOOK必须在FreeRTOSConfig.h中设置为1，才能调用空闲钩子函数。

实现已创建任务的函数略有修改，以打印出超周期计数值，如清单4.20所示。

```
void vTaskFunction( void * pvParameters ){

    char * pcTaskName;
    const TickType_t xDelay250ms = pdMS_TO_TICKS( 250 );
    /*

    *要打印出来的字符串将通过该参数传入。将其转换为
    *字符指针。
    */
}
```



```

    */
    pcTaskName = ( char * ) pvParameters;

    /*与大多数任务一样，这个任务是在一个无限循环中实现的。
    */for ( ; ; ) {
        /*
        *打印出此任务的名称和次数
        *ulIdleCycleCount已增加。*/
        vPrintLineAndNumber ( pcTaskName, ulIdleCycleCount );

        /*延迟为250毫秒。*/
        vTaskDelay ( xDelay250ms );
    }
}

```

清单4. 20 示例任务的源代码现在打印出超循环计算值

图4. 13显示了示例4. 7所产生的输出。可以看出，空闲任务钩子函数在应用程序任务的每次迭代之间执行大约400万次（迭代的次数取决于硬件速度）。

```

C:\Temp>rtosdemo
Task 2 is running
ulIdleCycleCount = 0
Task 1 is running
ulIdleCycleCount = 0
Task 2 is running
ulIdleCycleCount = 3869504
Task 1 is running
ulIdleCycleCount = 3869504
Task 2 is running
ulIdleCycleCount = 8564623
Task 1 is running
ulIdleCycleCount = 8564623
Task 2 is running
ulIdleCycleCount = 13181489
Task 1 is running
ulIdleCycleCount = 13181489
Task 2 is running
ulIdleCycleCount = 17838406
Task 1 is running
ulIdleCycleCount = 17838406
Task 2 is running

```

图4. 13在执行示例4. 7时产生的输出

4.9 更改任务的优先级

4.9.1 vTaskPrioritySet() API函数

vTaskPrioritySet() API函数会在调度程序启动后更改任务的优先级。只有在FreeRTOSConfig.h中，当INCLUDE_vTaskPrioritySet被设置为1时，vTaskPrioritySet() API功能才可用。

```
void vTaskPrioritySet( TaskHandle_t pxTask,
                      UBaseType_t uxNewPriority );
```

清单4. 21 vTaskPrioritySet() API函数原型

vTaskPrioritySet() 参数

- pxTask

正在修改其优先级的任务的句柄（主题任务）。有关获取任务句柄的信息，请参见xTaskCreate() API函数的pxCreatedTask参数，或 xTaskCreateStatic() API函数的返回值。

一个任务可以通过传递NULL来代替一个有效的任务句柄来更改它自己的优先级。

- uxNewPriority

要设置的主题任务的优先级。这是自动限制到最大可用值

优先级（configMAX_PRIORITIES-1），其中configMAX_PRIORITIES是在FreeRTOSConfig.h中设置的编译时间常数。

4.9.2 uxTaskPriorityGet() API函数

uxTaskPriorityGet() API函数返回任务的优先级。只有在FreeRTOSConfig.h. 中将INCLUDE_uxTaskPriorityGet 设置为1时才可用。

```
UBaseType_t uxTaskPriorityGet ( TaskHandle_t xTask );
```

清单4. 22 uxTaskPriorityGet() API函数原型

uxTaskPriorityGet() 参数和返回值

- pxTask

正在查询其优先级的任务的句柄（主题任务）。有关如何获取任务句柄的信息，请参见 `xTaskCreate()` API 函数的 `pxCreatedTask` 参数。

任务可以通过传递 NULL 代替有效的任务句柄来查询自己的优先级。

■ 返回值

当前分配给正在被查询的任务的优先级。

示例4.8 更改任务优先级

调度程序总是选择最高的就绪状态任务作为进入“运行”状态的任务。示例4.8通过使用 `vTaskPrioritySet()` API 函数来更改两个任务的优先级来演示这一点。

示例4.8以两个不同的优先级创建了两个任务。这两个任务都没有做出任何可能导致其进入阻塞状态的API函数调用，因此两者都始终处于“已就绪”状态或“正在运行”状态。因此，具有最高相对优先级的任务将始终是调度程序选择的处于运行状态的任务。

示例4.8的行为如下：

1. 任务1（清单4.23）以最高优先级创建，因此保证它先运行。任务1在将任务2（清单4.24）的优先级提高到它自己的优先级以上之前，会打印出几个字符串。
2. 任务2具有最高相对优先级，即开始运行（进入已运行状态）。任意一次只能有一个任务处于“运行”状态，因此当任务2处于“运行”状态时，任务1处于“就绪”状态。
3. 任务2在将自己的优先级设置为任务1的优先级以下之前打印出一条消息。
4. 当任务2将其优先级设置回时，任务1再次成为最高优先级的任务，因此任务1重新进入运行状态，迫使任务2回到就绪状态。

```
void vTask1 (void * pvParameters)
{
    UBaseType_t uxPriority;

    /*
     * 这个任务总是在任务2之前运行，因为它用更高的任务创建的
     * 优先级。任务1和任务2都没有阻塞过，所以两者都会一直在里面
     * 正在运行状态或准备就绪状态。*/

    /*
     * 查询此任务运行的优先级-以NULL表示
     * “返回调用任务的优先级”。
```

```

    */
    uxPriority = uxTaskPriorityGet( NULL );

    for( ;; )
    {
        /*请打印出此任务的名称。*/
        vPrintLine( "Task 1 is running" );

        /*
        *将任务2的优先级设置在任务1的优先级以上将会导致
        *任务2立即开始运行(此时任务2将在
        *两个已创建的任务中优先级更高)。注意到使用
        *任务2句柄 (xTask2句柄) 在调用vTaskPrioritySet()。
        *清单4. 25显示了您是如何将该句柄获取的。*/
        vPrintLine( "About to raise the Task 2 priority" );
        vTaskPrioritySet ( xTask2Handle, (uxPriority + 1) );

        /*
        *任务1只有在其优先级高于任务2时才会运行。
        *因此，要使这个任务达到以下的代码，任务2必须已经
        *执行并将其优先级设置为任务1优先级以下
        */
        //代码，如果有的话
    }
}

```

清单4. 23 示例4. 8中的任务1的实现

```

void vTask2(void * pvParameters)
{
    UBaseType_t uxPriority;

    /*
    *任务1将始终在此任务之前运行，因为任务1被创建为使用
    *更高的优先级。任务1和任务2都不会永远阻塞
    *在运行中或准备就绪的状态下。*
    *查询此任务运行的优先级-以NULL表示
    * “返回调用任务的优先级” 。
    */
    uxPriority = uxTaskPriorityGet( NULL );

    for( ;; )
    {
        /*
        *要使此任务达到此点，任务1必须已经运行和
        *设置此任务的优先级高于其自身任务。

```

```

    */

    /*请打印出此任务的名称。*/
    vPrintLine( "Task 2 is running" );

    /*
    *将此任务的优先级设置回其原始值。
    *将NULL传入为任务句柄意味着“更改调用任务的优先级”。将优先级设置为低于任务1
    *的优先级将导致任务1立即再次开始运行-抢占此任务。*/
    vPrintLine( "About to lower the Task 2 priority" );
    vTaskPrioritySet (NULL, (uxPriority - 2));
}
}

```

清单4. 24 示例4. 8中的任务2的实现

每个任务都可以通过使用NULL代替有效的任务句柄来查询和设置自己的优先级。只有当任务希望引用其自身以外的任务时，例如当任务1更改任务2的优先级时，才需要一个任务句柄。为了允许Task 1这样做，将在创建Task 2时获取并保存Task 2句柄，如清单4. 25中的注释中突出显示的那样。

```

/*声明一个变量，用于保存任务2的句柄。*/
TaskHandle_t xTask2Handle = NULL;

int main (void)
{
    /*
    *创建第一个任务, 优先级为2。未使用该任务参数
    *, 并设置为NULL。任务句柄也不被使用，因此也被设置为
    *NULL。
    */
    xTaskCreate (vTask1, "Task1", 1000, NULL, 2, NULL);
    /*该任务在优先级为2。*/

    /*
    *创建第二个任务，优先级为1. 低于任务1的优先级。
    *同样，不使用任务参数，因此被设置为NULL-
    *但这次的任务句柄是必需的，所以xTask2句柄的地址
    *在最后一个参数中传递。*/
    xTaskCreate (vTask2, "Task2", 1000, NULL, 1, &xTask2Handle);
    /*任务句柄是最后一个参数*/

    /*启动调度程序，以便任务开始执行。*/
    vTaskStartScheduler();
}

```

```

/*
 *如果一切都好，主函数将不会到达这里，因为调度程序现在将会
 *运行已创建的任务。如果主函数确实到达了这里，那么就说明
 *没有足够的堆内存来创建空闲或计时器任务
 *（在本书后面描述）。第三章提供了更多的信息
 */
for(;;)
{
}
}

```

清单4.25 示例4.8的main()的实现

图4.14演示了示例4.8中的任务执行的顺序，所生成的输出如图4.15所示。

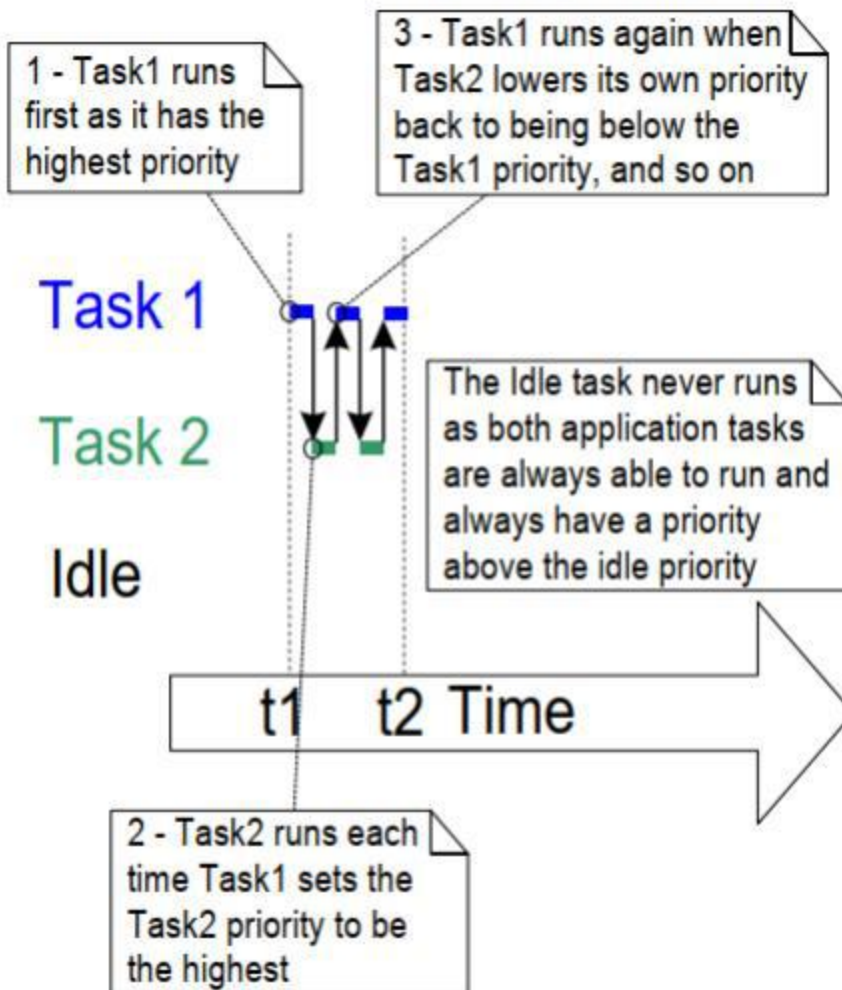


图4.14 运行示例4.8时的任务执行顺序

```
任务1 是 跑步
关于 到 提高跑步 任务2 优先
任务2 了 速度
关于 到 降低运行 任务2 优先
任务1 了
关于 到 提高跑步 任务2 优先
任务2 了 速度
关于 到 降低运行 任务2 优先
任务1 了
关于 到 提高跑步 任务2 优先
任务2 了 速度
关于 到 降低运行 任务2 优先
任务1 了
```

图4. 15在执行示例4. 8时产生的输出

4. 10 删除任务

4. 10. 1 vTaskDelete () API函数

vTaskDelete () API函数将删除一个任务。只有在FreeRTOSConfig.h中将INCLUDE_vTaskDelete设置为1时，vTaskDelete () API函数才可用。

在运行时不断创建和删除任务不是一种好的做法，所以如果您发现自己需要这个功能，请考虑其他设计选项，比如重用任务。

已删除的任务不再存在，并且无法再次进入正在运行的状态。

如果使用动态内存分配创建的任务后来会删除自己，则空闲任务将负责释放分配给使用的内存，例如已删除任务的数据结构和堆栈。因此，在这种情况下，应用程序不要完全耗尽空闲任务是很重要的。

注意：当任务被删除时，只有内核本身分配给任务的内存才会自动释放。如果不再需要，则必须显式地释放在任务实现期间分配的任何内存或其他资源。

```
void vTaskDelete (TaskHandle_t xTaskToDelete);
```

清单4. 26 vTaskDelete () API函数原型

vTaskDelete () 参数

- pxTaskToDelete

要删除的任务（主题任务）的句柄。有关获取任务句柄的信息，请参见xTaskCreate（）API函数的pxCreatedTask和xTaskCreateStatic（）API函数的返回值。

任务可以通过传递NULL来代替有效的任务句柄来删除自身。

示例4.9 删除任务

这是一个非常简单的例子，其行为如下。

1. 任务1设置优先级为1由main（）创建。当它运行时，它将在优先级为2处创建任务2。任务2现在是优先级最高的任务，因此它将立即开始执行。清单4.27显示了main（）的源代码。清单4.28给出了任务1的源代码。
2. 任务2只做一些可以删除自己的事情。它可以通过将NULL传递给vTaskDelete（）来删除自己，但是为了演示目的，它使用自己的任务句柄。清单4.29给出了任务2的源代码。
3. 当任务2被删除时，任务1仍然是一个优先级最高的任务，因此它继续执行——此时它调用vTaskDelay（）进行短时间的阻塞。
4. 空闲任务在任务1处于阻塞状态时执行，并释放分配给现在已删除的任务2的内存。
5. 当任务1离开阻塞状态时，它再次成为最高优先级的就绪状态任务，因此优先于空闲任务。当它进入“运行”状态时，它将再次创建任务2，因此它将继续进行。

```
int main (void)
{
    /*在优先级为1时创建第一个任务。*/
    xTaskCreate (vTask1, "Task1", 1000, NULL, 1, NULL);

    /*启动调度程序，以便任务开始执行。*/
    vTaskStartScheduler();

    /* main（）永远不应该到达这里，因为调度程序已经启动。*/

    for (;;)
    {
    }
}
```

清单4.27 示例4.9的主函数的实现


```

TaskHandle_t xTask2Handle = NULL;

void vTask1 (void * pvParameters)
{
    const TickType_t xDelay100ms = pdMS_TO_TICKS (100UL);

    for(;;)
    {
        /*请打印出此任务的名称。*/
        vPrintLine( "Task 1 is running" );

        /*
         *以更高的优先级创建任务2。
         *将xTask2句柄的地址作为pxCreatedTask参数这样传递
         *将生成的任务句柄写入该变量。*/
        xTaskCreate( vTask2, "Task 2", 1000, NULL, 2, &xTask2Handle );

        /*
         *任务2具有更高的优先级。为了让任务1到达这里，任务2
         *必须已经执行并删除了自己。*/
        vTaskDelay ( xDelay100ms );
    }
}

```

清单4.28 示例4.9中的任务1的实现

```

void vTask2 (void * pvParameters)
{
    /*
     *任务2在启动时立即删除自己。
     *为此，它可以调用vTaskDelete ( )，使用NULL作为参数。
     *为了演示目的，它自己调用vTaskDelete ( )
     *任务处理。*/
    vPrintLine( "Task 2 is running and about to delete itself" );
    vTaskDelete ( xTask2Handle );
}

```

清单4.29示例4.9中的任务2的实现

```

C:\Temp>rtosdemo
Task1 is running

```

[illegible]

图4.16 在执行示例4.9时产生的输出

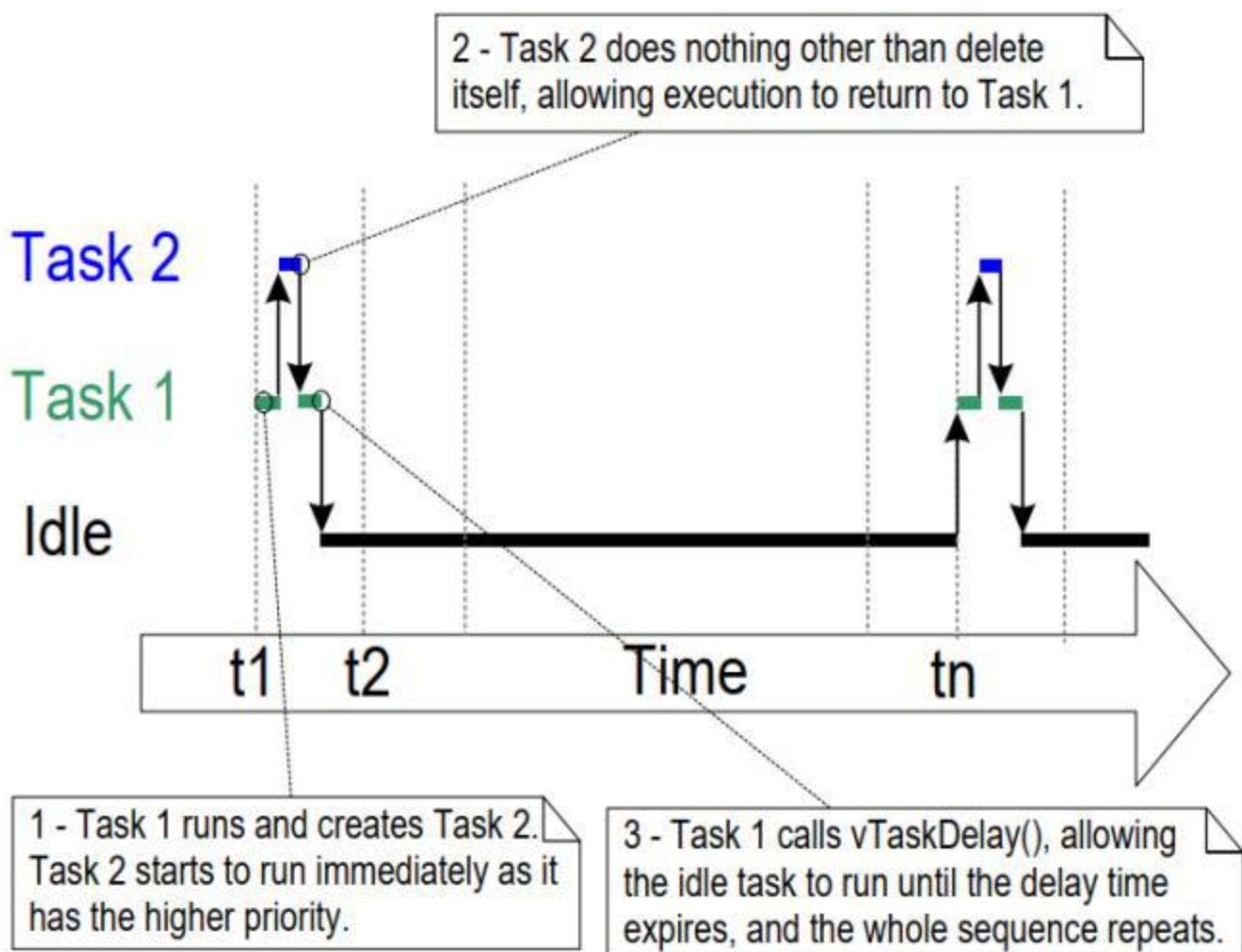


图4.17 示例4.9的执行序列

4.11 线程本地存储和重入性

线程本地存储允许应用程序开发人员在每个任务的任务控制块中存储任意数据。该特性最常用于存储通常由非重入函数存储在全局变量中的数据。

重入函数是一个可以从多个线程安全运行而没有任何副作用的函数。当在没有线程本地存储的多线程环境中使用非重入函数时，必须特别小心从临界部分内检查这些函数调用的带外结果。过度使用临界部分会降低RTOS的性能，因此线程本地存储通常比使用临界部分更可取。

到目前为止，线程本地存储最常用的用法是C标准库和POSIX系统所使用的ISO C标准中使用的errno全局存储。errno全局用于为strtstof和strtol等通用标准库函数提供扩展的结果或错误代码。

4.11.1 C运行时线程本地存储实现

大多数嵌入式C库实现都提供了api，以确保非重入函数可以在多线程环境中正确工作。FreeRTOS包括对两个常用的开源库的重入性api的支持：newlib和picolibc。这些预构建的C运行时线程本地存储实现可以通过在其项目的FreeRTOSConfig.h文件中定义下面列出的相应宏来实现。

```
configUSE_NEWLIB_REENTRANT for newlib
configUSE_PICOLIBC_TLS for picolibc
```

4.11.2 自定义C运行时线程本地存储器

应用程序开发人员可以通过在其FreeRTOSConfig.h中定义以下宏来实现线程本地存储文件：

将configUSE_C_RUNTIME_TLS_SUPPORT定义为1，以启用C运行时线程本地存储支持。

将configTLS_BLOCK_TYPE定义为C类型，它应用于存储C运行时线程本地存储数据。

将configINIT_TLS_BLOCK定义为在初始化C运行时线程本地存储块时应该运行的C代码。

- 将configSET_TLS_BLOCK定义为在转换新任务时应该运行的c代码
- 将configDEINIT_TLS_BLOCK定义为C代码，即在去初始化C运行时线程本地存储块时应该运行的C代码。

4.11.3 应用程序线程本地存储器

除了 C 运行时线程本地存储外，应用程序开发人员还可以定义一组特定于应用程序的指针，将其包含在任务控制块中。通过将

FreeRTOSConfig.h 文件中 configNUM_THREAD_LOCAL_STORAGE_POINTERS 设为非零值。清单 4.30 中定义的 vTaskSetThreadLocalStoragePointer 和 pvTaskGetThreadLocalStoragePointer 函数可分别用于在运行时设置和获取每个线程本地存储指针的值。

```
void * pvTaskGetThreadLocalStoragePointer ( TaskHandle_t xTaskToQuery, BaseType_t xIndex )

void vTaskSetThreadLocalStoragePointer( TaskHandle_t xTaskToSet,
                                         BaseType_t xIndex,
                                         void * pvValue );
```

清单4.30 线程本地存储指针API函数的函数原型

4.12 调度算法

4.12.1 对任务状态和事件的概述

实际正在运行的任务（使用处理时间）处于“正在运行”状态。在单核处理器上，在任何给定时间只能有一个处于运行状态的任务。还可以在多个核心（非对称多处理或AMP）上运行FreeRTOS，或者在多个核心（对称多处理或SMP）上执行FreeRTOS调度任务。这里都没有描述这两种情况。

未实际运行但未处于阻塞状态或挂起状态的任务处于“就绪”状态。调度程序可选择处于“就绪”状态的任务作为进入“正在运行”状态的任务。调度程序将始终选择最高优先级的就绪状态任务以进入运行状态。

任务可以在“阻塞”状态下等待事件，并在事件发生时自动移动到“就绪”状态。时间事件发生在特定的时间，例如，当阻塞时间过期时，通常用于实现周期性或超时行为。当任务或中断服务例程使用任务通知、队列、事件组、消息缓冲区、流缓冲区或许多信号量类型之一发送信息时，就会发生同步事件。它们通常用于表示异步活动，例如到达外围设备的数据。

4.12.2 选择调度算法

调度算法是决定将哪个准备状态任务转换到运行状态的软件例程。

到目前为止，所有的例子都使用了相同的调度算法，但是算法可以使用configUSE_PREEMPTION和configUSE_TIME_SLICING配置常数进行更改。这两个常量都在FreeRTOSConfig.h中定义过。

第三个配置常数configUSE_TICKLESS_IDLE也会影响调度算法，因为它的使用会导致滴答中断在较长时间内被完全关闭。configUSE_TICKLESS_IDLE是一种高级选项，专门用于必须最小化其功耗的应用程序。这个

本节中提供的描述假设configUSE_TICKLESS_IDLE被设置为0，如果常量未定义，这是默认设置。

在所有可能的单核配置中，FreeRTOS调度程序依次选择共享优先级的任务。这种“反过来采取”的政策通常被称为“循环计划”。循环调度算法不能保证在相同优先级的任务之间平均共享时间，只有相同优先级的准备状态任务依次进入运行状态。

调度算法	优先级	configUSE_PREEMPTION	configUSE_TIME_SLICING
有时间切片的抢占制	是	1	1
没有时间切片的抢占制	是	1	0
合作的	不	0	Any

4. 12. 3 优先优先调度与时间切片

下表所示的配置设置FreeRTOS调度程序使用调度算法称为“固定优先优先调度与时间切片”，这是大多数小型RTOS应用程序使用的调度算法，以及本书中提供的所有例子所使用的算法。下一个表提供了在算法的名称中使用的术语的描述。

对用于描述调度策略的术语的说明：

•固定优先级

被描述为“固定优先级”的调度算法并不改变分配给被调度的任务的优先级，但也不阻塞任务本身改变它们自己或其他任务的优先级。

•抢占制

如果优先级高于“运行状态”任务的任务进入就绪状态，优先调度算法将立即“优先”运行状态任务。被抢占意味着不自觉地退出正在运行状态并进入就绪状态（不显式放弃或阻塞），以允许不同的任务进入正在运行状态。任务抢占可以在任何时候发生，而不仅仅是在RTOS滴答中断中。

•时间切片

时间切片用于在同等优先级的任务之间共享处理时间，即使这些任务没有显式地产生或进入阻塞状态。如果有其他就绪状态任务与运行任务的优先级相同，则选择一个新任务，则在每个时间片的末尾进入运行状态。一个时间片等于两个RTOS滴答中断之间的时间。

图4. 18和图4. 19演示了在使用固定优先级优先调度算法时如何调度任务。图4. 18显示了当应用程序中的所有任务都具有独特的优先级时，选择任务进入“正在运行”状态的顺序。图4. 19显示了当一个应用程序中的两个任务具有同一个优先级时，选择哪个任务进入“正在运行”状态的顺序。

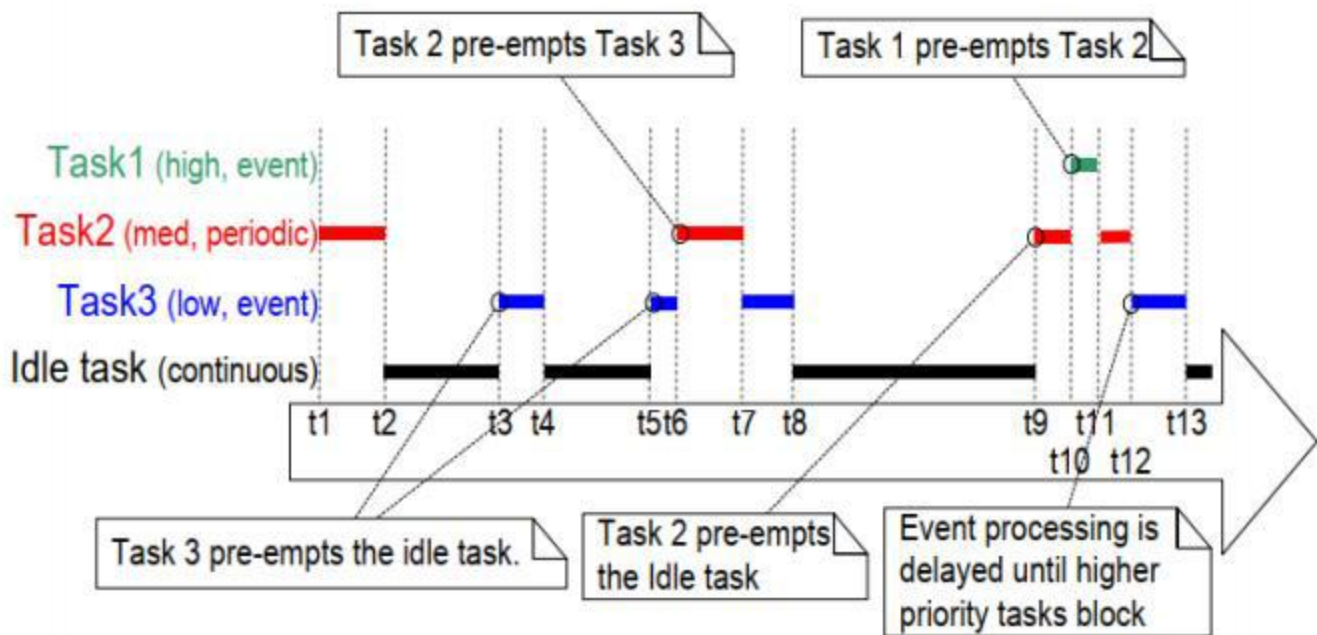


图4. 18 在对每个任务都具有唯一优先级的假设应用程序中，突出显示任务优先级和优先权的执行模式

参见图4. 18:

•空闲任务

空闲任务以最低优先级运行，因此当每次高优先级任务进入就绪状态时，它就会被抢占，例如在t3、t5和t9。

•任务3

Task 3是一个事件驱动的任务，执行时优先级相对较低，但高于空闲优先级。它大部分时间都在阻塞状态等待感兴趣的事件，每次事件发生时都从阻塞状态转换到就绪状态。所有FreeRTOS任务间通信机制（任务通知、队列、信号量、事件组等）可以用于以这种方式通知事件和解除阻塞任务。

事件发生在时间t3和t5之间，也发生在时间t9和t12之间。在时间t3和t5发生的事件会被立即处理，因为在这些时间中，任务3是能够运行的最高优先级的任务。发生在时间t9和t12之间的事件直到时间t12才被处理，因为在此之前，更高优先级的任务任务1和Task 2仍在执行。只有在t12时，任务1和任务2都处于阻塞状态，这使任务3成为最高优先级的就绪状态任务。

•任务2

任务2是一个周期性的任务，其优先级高于任务3的优先级，但低于任务1的优先级。任务的周期间隔意味着任务2希望在时间t1、t6和t9执行。

在时间t6，任务3处于运行状态，但任务2具有较高的相对优先级，因此抢占任务3并立即开始执行。任务2完成其处理，并在时间t7重新进入阻塞状态，此时任务3可以重新进入运行状态以完成其处理。任务3本身在时间t8时阻塞。

•任务1

任务1也是一个事件驱动的任务。它以所有优先级最高的方式执行，因此可以抢占系统中的任何其他任务。唯一显示的Task 1事件发生在时间t10，此时Task 1将抢占了Task 2。任务2只有在任务1在时间t11重新进入阻塞状态后才能完成其处理。

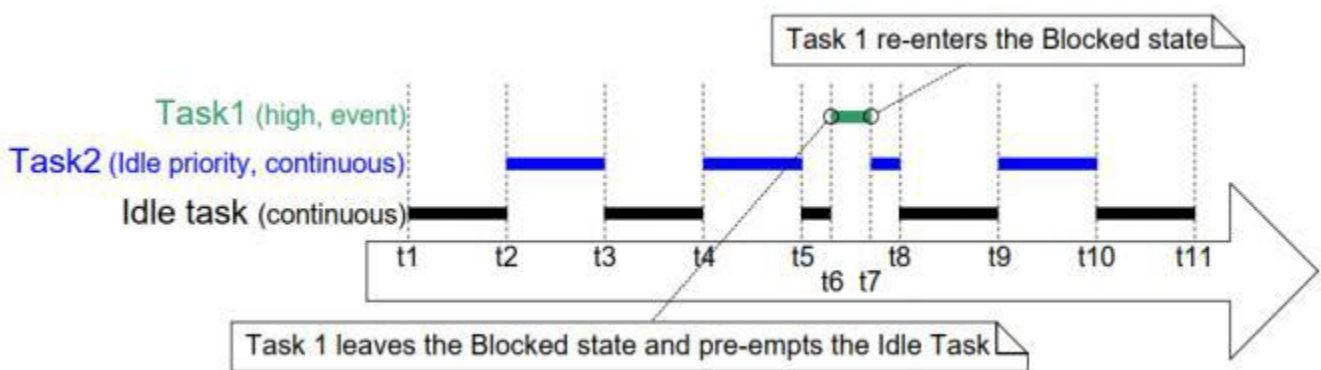


图4.19 在一个有两个任务以相同的优先级运行的假设应用程序中，突出显示任务优先级和时间切片的执行模式

参见图4.19:

•空闲任务和任务2

空闲任务和任务2都是连续处理任务，它们的优先级都为0（可能的最低优先级）。只有当没有更高的优先级的任务能够运行时，调度程序才会为优先级0任务分配处理时间，并通过时间切片共享分配给优先级0任务的时间。每个新的时间切片开始，图4.19中出现在t1、t2、t3、t4、t5、t8、t9、t10和t11。

空闲任务和任务2依次进入运行状态，这可能导致两个任务在同一时间段中部分处于运行状态，就像在时间t5和时间t8之间发生的情况一样。

•任务1

任务1的优先级高于空闲优先级。任务1是一个事件驱动的任务，它将大部分时间花在已阻塞状态中等待感兴趣的事件，从“阻塞”状态转换到“就绪”

每次事件发生时的状态。

感兴趣的事件发生在时间t6。在t6时，Task 1成为能够运行的最高优先级任务，因此任务Task 1通过时间片抢占空闲任务部分。事件的处理在时间t7完成，此时Task 1重新进入阻塞状态。

图4. 19显示了由应用程序编写器创建的任务的空闲任务共享处理时间。如果应用程序编写器创建的空闲优先级任务有工作要做，但空闲任务没有，那么将那么多的处理时间分配给空闲任务可能不可取。

configIDLE_SHOULD_YIELD编译时配置常量可用于更改空闲任务的调度方式：

如果configIDLE_SHOULD_YIELD设置为0，那么空闲任务的整个时间片都处于运行状态，除非它被更高优先级的任务抢占。

如果 configIDLE_SHOULD_YIELD 设置为 1，那么在每次循环迭代时，如果有其他处于就绪状态的闲置优先级任务，闲置任务就会放弃（自愿放弃分配给它的剩余时间片）。

图4. 19所示的执行模式是configIDLE_SHOULD_YIELD设置为0时观察到的。

图4. 20中所示的执行模式是在configIDLE_SHOULD_YIELD设置为1时观察到的情况。

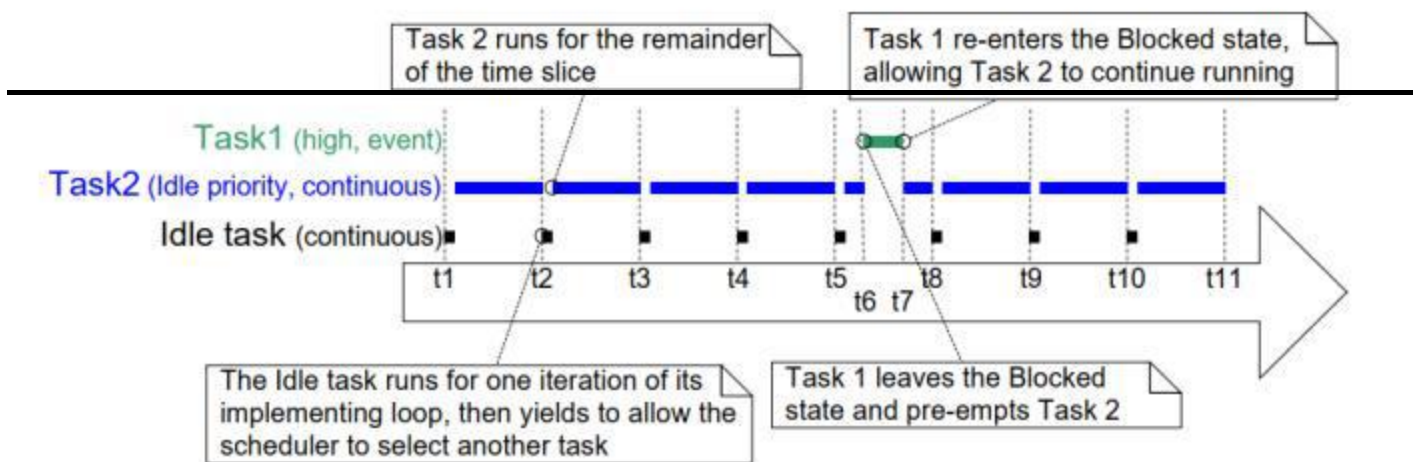


图4. 20对于如图4. 19所示的相同场景的执行模式。但这次的configIDLE_SHOULD_YIELD设置为1

图4. 20还显示，当configIDLE_SHOULD_YIELD设置为1时，选择在空闲任务后进入运行状态的任务不会执行整个时间片，而是对空闲任务产生的时间片的任何剩余部分执行。

4. 12. 4 优先优先调度，没有时间切片

没有时间切片的优先优先调度维护了与上一节中描述的相同的任务选择和抢占算法，但不使用时间切片来共享处理时间

优先级相同的任务。

下表显示了FreeRTOSConfig.h设置，该设置配置FreeRTOS调度程序使用优先优先调度而没有时间切片。

如图4所示。19，如果使用时间切片，并且有多个具有最高优先级的就绪状态任务能够运行，那么调度程序选择一个新任务在每个RTOS提示中断期间进入运行状态（标记中断标记时间切片的结束）。如果不使用时间切片，则调度程序只选择一个新任务以进入运行状态：

- 优先级较高的任务进入已就绪状态。
- 处于“正在运行”状态的任务将进入“已阻塞”或“已挂起”状态。

当不使用时间切片时，任务上下文切换比当使用时间切片时更少。因此，关闭时间切片会减少调度器的处理开销。然而，关闭时间切片也会导致相同优先级的任务接收到非常不同的处理时间，这个场景如图4. 21所示。因此，运行不需要时间切片的调度程序被认为是一种高级技术，只能供经验丰富的用户使用。

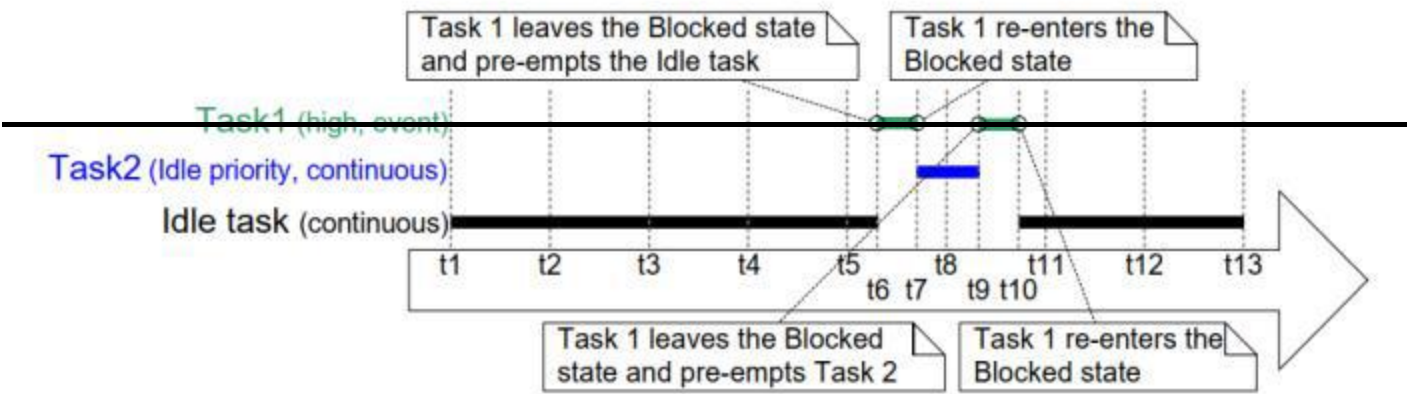


图4. 21执行模式，演示了当不使用时间切片时，同等优先级的任务如何能接收到非常不同的处理时间

参考图4. 21，它假设configIDLE_SHOULD_YIELD被设置为0：

•滴答声中断

滴答中断发生在时间t1、t2、t3、t4、t5、t8、t11、t12和t13时。

•任务1

任务1是一个高优先级的事件驱动任务，它将大部分时间花在阻塞状态中等待其感兴趣的事件。在每次事件发生时，任务1将从阻塞状态转换到“就绪”状态（随后，由于它是最高优先级的就绪状态任务，将转换到“正在运行”状态）。图4. 21显示了任务1在t6和t7之间处理事件，然后在t9和t10之间处理事件。

- 空闲任务和任务2

空闲任务和任务2都是连续处理任务，它们的优先级都为0（空闲优先级）。连续处理任务不会进入“已阻塞”的状态。

没有使用时间切片，因此处于“运行”状态的空闲优先级任务将继续处于“运行”状态，直到它被高优先级任务1抢占。

在图4.21中，空闲任务在时间 t_1 开始运行，并保持运行状态，直到它在时间 t_6 被任务1抢占，即在它进入运行状态后超过4个完整的提示周期。

任务2在时间 t_7 开始运行，这是任务1重新进入阻塞状态以等待另一个事件的时间。任务2仍然处于运行状态，直到它在时间 t_9 也被任务1抢占，这是在它进入运行状态后小于一个提示周期。

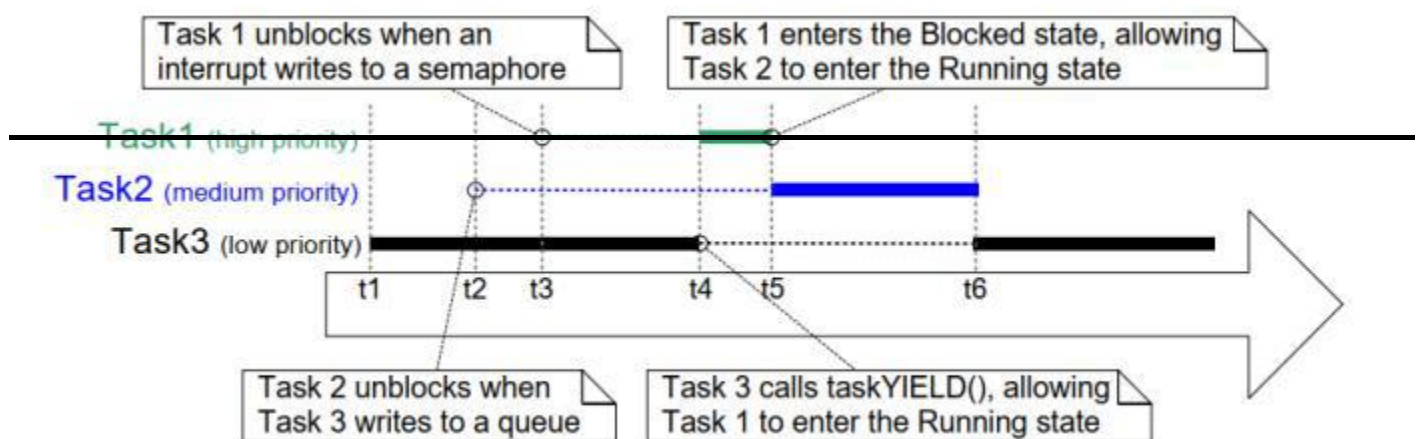
在时间t10，空闲任务重新进入运行状态，尽管它已经接收到的处理时间是任务2的四倍多。

4.12.5 合作调度

这本书主要关注抢占式的调度，但FreeRTOS也可以使用协同调度。下表显示了FreeRTOSConfig.h设置，它将FreeRTOS调度程序配置为使用协作调度。

当使用协作调度程序时（因此假设应用程序提供的中断服务例程不显式地请求上下文切换），只有当运行状态任务进入阻塞状态，或运行状态任务通过调用taskYIELD（）显式地产生（手动请求重新调度）时才会发生上下文切换。任务永远不会被抢占，因此不能使用时间切片。

图4.22演示了协作调度器的行为。图4.22中的水平虚线显示了任务处于“已就绪”状态的时间。



4.22 演示了协作调度器的行为的执行模式

参见图4. 22:

•任务1

任务1的优先级最高。它以阻塞状态开始，等待一个信号量。

在时间t3，一个中断给出信号量，导致任务1离开阻塞状态并进入准备状态（第6章将介绍来自中断的信号量）。

在时间t3时，任务1是最高优先级的准备状态任务，如果使用了抢占式调度器，任务1将成为运行状态任务。但是，由于正在使用协作调度程序，Task 1一直处于就绪状态，直到时间t4，此时运行状态任务调用taskYIELD（）。

•任务2

任务2的优先级介于任务1和任务3的优先级之间。它以阻塞状态开始，等待任务3在时间t2发送给它的消息。

在时间t2，任务2是最高优先级的准备状态任务，如果使用了抢占式调度器，任务2将成为运行状态任务。但是，当使用协作调度程序时，Task 2仍然处于就绪状态，直到运行状态任务进入阻塞状态或调用taskYIELD（）。

运行状态任务在时间t4时调用taskYIELD（），但此时任务1是最高优先级的就绪状态任务，因此任务2在时间t5时任务1实际上不会重新进入阻塞状态时才成为运行状态任务。

在时间t6，任务2重新进入阻塞状态以等待下一个消息，此时任务3再次成为最高优先级的就绪状态任务。

在处理多任务的应用程序中，应用程序编写器必须注意到多个任务不会同时访问资源，因为同时访问可能会破坏资源。例如，请考虑以下场景，其中所访问的资源是一个UART（串行端口）。两个任务向UART写入字符串；任务1写“数据库”，任务2写“123456789”：

1. 任务1处于“正在运行”状态，并开始写入其字符串。它向UART写入“abcdefg”，但在编写任何其他字符之前离开运行状态。
2. 任务2进入运行状态，并在离开运行状态之前向UART写入“123456789”写入UART。
3. 任务1重新进入“正在运行”状态，并将其字符串的其余字符写入UART。

在这种情况下，实际上写给UART的是“被劫持者123456789”。Task 1写入的字符串并没有按照预期的完整顺序写入UART，而是已经损坏，因为Task 2写入UART的字符串出现在其中。

使用协同调度器通常比使用优先制式调度器更容易避免同时访问引起的问题^[7]：

[^7]: 本书后面将介绍在任务之间安全共享资源的方法。FreeRTOS本身提供的资源，如队列和信号量，在任务之间共享总是安全的。

- 当您使用优先调度程序时，运行状态任务可以在任何时候被抢占，包括当它与另一个任务共享的资源处于不一致的状态时。正如UART示例所示，让资源处于不一致的状态可能会导致数据损坏。

当您使用协作调度程序时，您可以控制何时切换到另一个任务。因此，您可以确保在资源处于不一致状态时不会发生切换到另一个任务。

在上面的UART示例中，您可以确保Task 1在写入它后不会离开“运行”状态

整个字符串到UART，通过这样做，消除了字符串被另一个任务的活动损坏的可能性。

如图4.22所示，使用协同调度器使系统比使用优先调度器时的响应更慢：

- 当使用优先调度程序时，调度程序在任务执行时立即开始运行任务成为最高优先级的准备就绪状态任务。这在实时系统中通常是至关重要的，因为它必须在一个确定的时间段内响应高优先级的事件。
- 当使用协作调度程序时，在运行状态任务进入阻塞状态或taskYIELD（）调用之前，不会执行切换到已成为最高优先级的就绪状态任务的任務。

5 队列管理

5.1 简介

“队列”提供了任务到任务、任务到中断和中断到任务的通信机制。

5.1.1 范围

本章包括：

- 如何创建一个队列。
- 队列如何管理其包含的数据。
- 如何将数据发送到队列。
- 如何从队列中接收数据。
- 阻塞一个队列意味着什么。
- 如何阻塞多个队列。
- 如何覆盖队列中的数据。
- 如何清除一个队列。
- 当写入队列和阅读队列时，任务优先级的影响。

本章仅涉及任务到任务之间的通信。第七章介绍了任务到中断和中断到任务的通信。

5.2 一个队列的特征

5.2.1 数据存储

一个队列可以包含有限数量的固定大小的数据项^[8]。一个队列可以保存的最大项数称为它的“长度”。在创建队列时，将设置每个数据项的长度和大小。

[8]：在第(待定)章中描述的FreeRTOS消息缓冲区，为保存可变长度消息的队列提供了一个较轻的替代方案。

队列通常用作先入先出(FIFO)缓冲区，其中数据被写入队列的末端（尾部）并从队列的前端（头）删除。图5.1演示了从正在用作FIFO的队列中写入和读取的数据。也可以写入队列的前面，并覆盖已经在队列前面的数据。

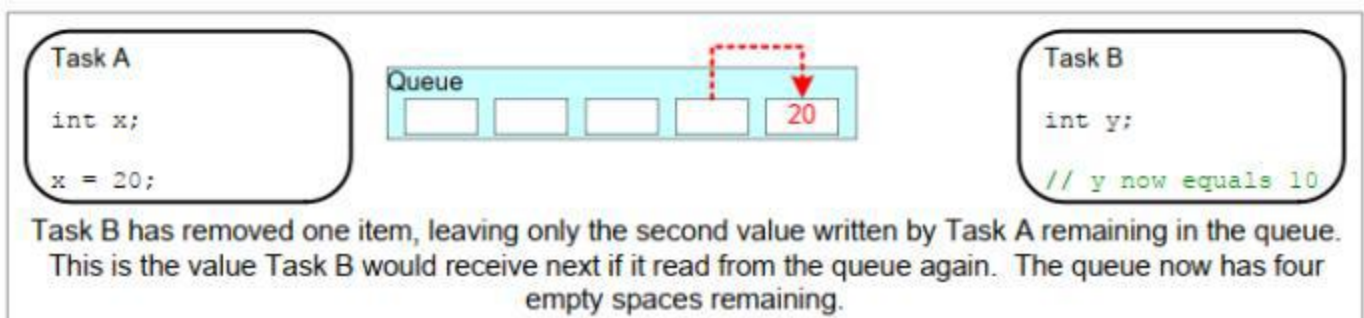
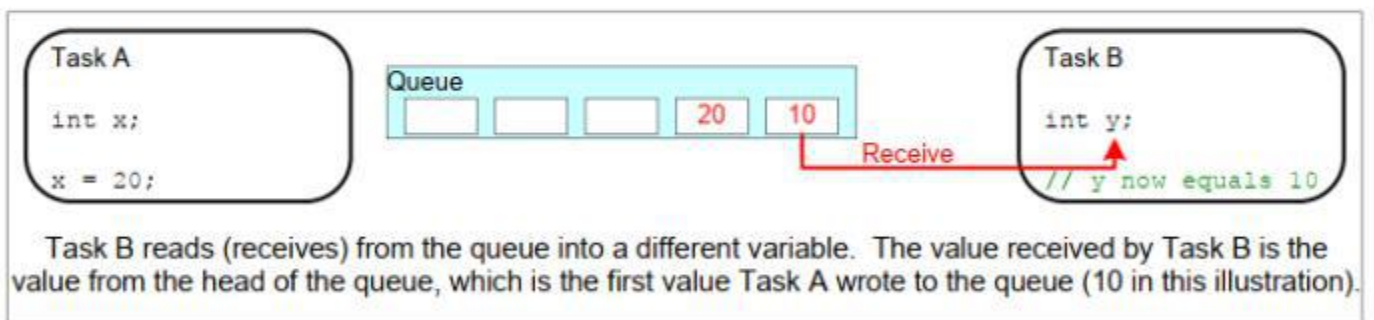
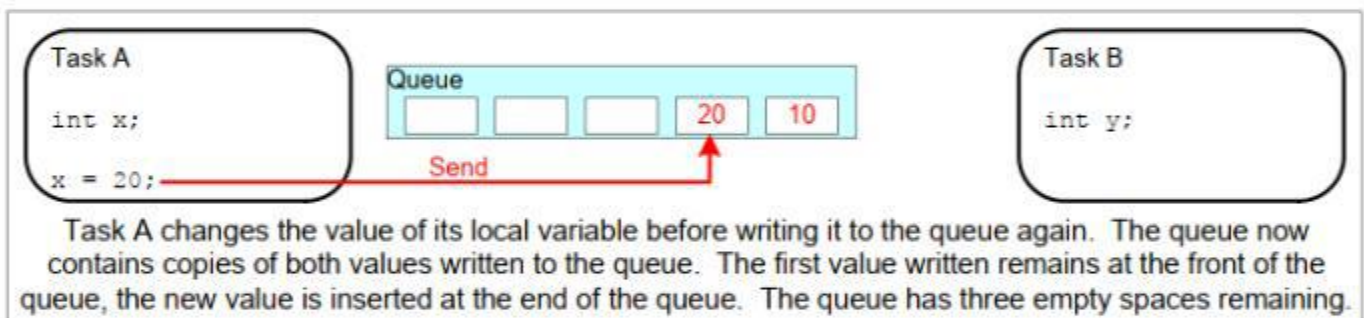
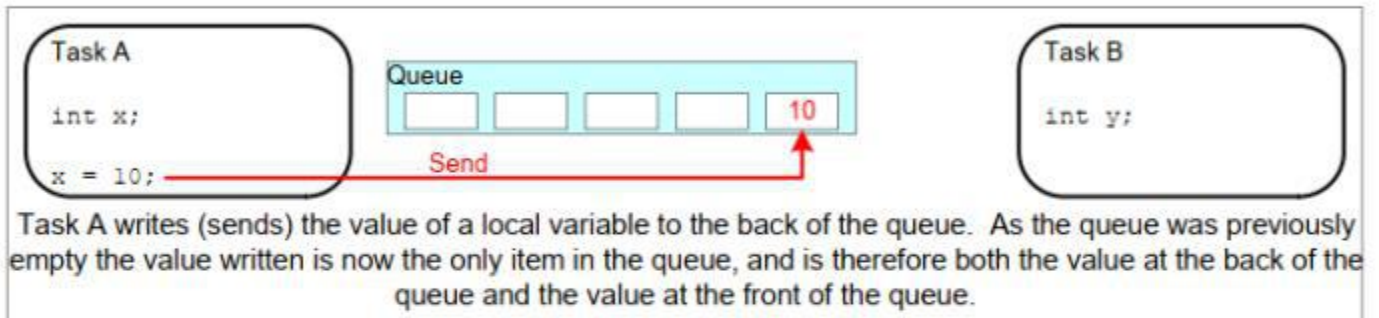
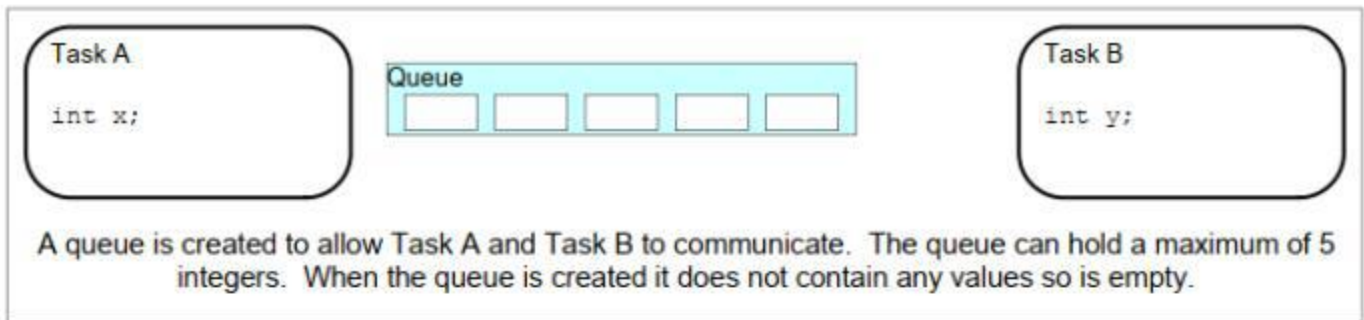


图5.1 从队列中写入和读取的示例序列

有两种方式可以实现队列行为：

1. 队列基于拷贝

拷贝队列意味着发送到队列的数据将字节一个字节复制到队列中。

2. 队列基于引用

引用队列意味着队列只持有指向发送到队列的数据的指针，而不是数据本身。

FreeRTOS使用**拷贝**队列方法，因为它比引用队列更强大，也更容易使用，因为：

- 复制队列并不阻塞队列也被用于引用队列。例如，
 - 当正在排队的数据的大小使将数据复制到队列中变得不现实时，那么就可以将指向数据的指针复制到队列中。
 - 堆栈变量可以直接发送到队列，即使在声明它的函数退出后该变量将不存在。
 - 可以将数据发送到队列，而不需要首先分配缓冲区来保存数据，然后将数据复制到已分配的缓冲区中，并排队生成对缓冲区的引用。
 - 发送任务可以立即重用已发送到队列的变量或缓冲区。
 - 发送任务和接收任务已完全解耦；应用程序设计器不需要关注自己，关注哪个任务“拥有”数据，或者哪个任务负责释放数据。RTOS完全负责分配用于存储数据的内存。
 - 内存保护系统限制对 RAM 的访问，在这种情况下，只有发送和接收任务都可以访问引用的数据时才能完成引用排队。按复制排队允许数据**跨越**内存保护边界。

5.2.2 通过多项任务进行访问

队列本身就是一个对象，可以被任何知道它们存在的任务或ISR访问。任意数量的任务都可以写入同一队列，任意数量的任务都可以从同一队列中读取。在实践中，队列有多个写器很常见，但队列有多个读取器就不常见了。

5.2.3 阻塞在队列读取情况中

当一个任务尝试从队列中读取时，它可以指定一个“块”时间。如果队列已为空，则此时是任务处于“阻塞”状态以等待队列中的数据可用的时间。当另一个任务或中断将数据放置到队列中时，处于阻塞状态的数据可用的任务将自动移动到“就绪”状态。如果指定的阻塞时间在数据可用之前到期，则任务也将自动从“阻塞”状态移动到“就绪”状态。

以有多个读取器，因此单个队列可能有多个任务阻塞在其上等待数据。在这种情况下，当数据可用时，只有一项任务会被解除阻塞。被解除阻塞的任务始终是正在等待数据的最高优先级任务。如果两个或多个阻塞任务具有相同的优先级，则解除阻塞的任务是等待时间最长的任务。

5.2.4 阻塞在队列写入情况下

正如从队列中读取时一样，任务可以在写入队列时选择指定阻塞时间。在这种情况下，如果队列已经满，阻塞时间是任务在阻塞状态下等待队列上的空间可用的最大时间。

队列可以有多个写入器，因此一个已满的队列可能有多个任务被阻塞，等待完成发送操作。在这种情况下，当队列上的空间可用时，只有一项任务会被解除阻塞。被解除阻塞的任务始终是等待空间的最高优先级任务。如果两个或多个阻塞任务具有相同的优先级，则解除阻塞的任务是等待时间最长的任务。

5.2.5 对多个队列的阻塞情况

可以将队列分组为集合，允许任务进入“阻塞”状态，等待数据在集中的任何队列上可用。第5.6节“从多个队列接收”演示了队列集。

5.2.6 创建队列：静态分配和动态分配的队列

两个API函数创建队列：`xQueueCreate()`，`xQueueCreateStatic()`

每个队列需要两个RAM块，第一个块保存其数据结构，第二块块保存队列数据。`xQueueCreate()`从堆中分配所需的RAM（动态地）。`xQueueCreateStatic()`使用传递给该函数的预分配的RAM作为参数。

5.3 使用队列

5.3.1 `xQueueCreate()` API函数

清单5.1显示了 `xQueueCreate()` 函数原型。`xQueueCreateStatic()` 有两个额外参数，分别指向预先分配给保存队列的数据结构和数据存储区域的内存。

```
QueueHandle_t xQueueCreate ( UBaseType_t uxQueueLength, UBaseType_t uxItemSize );
```

清单5.1 `xQueueCreate()` API函数原型

参数和返回值：

- `uxQueueLength`

所创建的队列在任何时候都可以保存的最大的数据项。

- `uxItemSize`

可以存储在队列中的每个数据项的字节大小。

- 返回值

如果返回NULL，则无法创建队列，因为FreeRTOS没有足够的堆内存来分配队列数据结构和存储区域。第2章提供了关于FreeRTOS堆的更多信息。

如果返回非空值，则该队列已成功创建，并且返回的值是已创建队列的句柄。

`xQueueReset()` 是一个API函数，它将以前创建的队列恢复到其原始的空状态。

5.3.2 `xQueueSendToBack()` 和 `xQueueSendToFront()` API函数

正如预期的那样，`xQueueSendToBack()` 将数据发送到队列的后面（尾部），并且 `xQueueSendToFront()` 将数据发送到队列的前端（头）。

`xQueueSend()` 完全等价于 `xQueueSendToBack()`

注意：永远不要从中断服务例程调用 `xQueueSendToFront()` 或 `xQueueSendToBack()`。中断安全版本的 `xQueueSendToFrontFromISR()` 和 `xQueueSendToBackFromISR()` 应该在ISR中使用。这些内容将在第7章中进行描述。

```
BaseType_t xQueueSendToFront ( QueueHandle_t xQueue,
                               const void * pvItemToQueue,
                               TickType_t xTicksToWait );
```

清单5.2 `xQueueSendToFront()` API函数原型

```
BaseType_t xQueueSendToBack ( QueueHandle_t xQueue,
                               const void * pvItemToQueue,
                               TickType_t xTicksToWait );
```

清单5.3 `xQueueSendToBack()` API函数原型

函数参数和返回值

- `xQueue`

数据（写入）的队列的句柄。队列句柄将被返回，从调用`xQueueCreate()` or `xQueueCreateStatic()` 创建队列的函数返回。

- `pvItemToQueue`

指向要复制到队列中的数据指针。

队列可以容纳的每个项目的大小是在创建队列时设置的，以便将许多字节从 `pvItemToQueue` 复制到队列存储区域中。

- `xTicksToWait`

如果队列已满，则任务应保持在“阻塞”状态以等待队列上的空间可用的最长时间。

如果`xTicksToWait`为零且队列已经满，那么`xQueueSendToFront`和`xQueueSendToBack`都将立即返回。

阻塞时间以滴答周期指定，因此它所表示的绝对时间取决于滴答频率。宏`pdMS_TO_TICKS()`可用于将以毫秒为单位指定的时间转换为在刻度中指定的时间。

如果在`FreeRTOSConfig.h`中`INCLUDE_vTaskSuspend`设置为1，则将`INCLUDE_vTaskSuspend`等待设置为`portMAX_DELAY`将导致任务无限期等待（不超时）。

- 返回值

有两个可能的返回值：

- `pdPASS`

当数据被成功地发送到队列时，将返回`pdPASS`。

如果指定了阻塞时间（`xTicksToWait`不是零），则可能将调用任务放入阻塞状态，等待函数返回之前队列中的可用空间，但数据在阻塞时间过期之前成功写入队列。

- `errQUEUE_FULL`（与`pdFAIL`的值相同）

如果由于队列已满，数据无法写入队列，则返回`errQUEUE_FULL`。

如果指定了阻塞时间（`xTicksToWait`不是零），那么调用任务将被放置入阻塞状态，等待另一个任务

或中断在队列中腾出空间，但是指定的阻塞时间在此发生之前已经过期。

5.3.3

`xQueueReceive()` API函数

`xQueueReceive()` 从队列中接收（读取）一个项目。接收到的项目将从队列中删除。

注意：永远不要从中断服务例程调用 `xQueueReceive()`。描述中断安全的 `xQueueReceiveFromISR()` 在第7章。

```
BaseType_t xQueueReceive ( QueueHandle_t xQueue,
                          void *const pvBuffer,
                          TickType_t xTicksToWait );
```

清单5. 4x查询接收（）API函数原型

`xQueueReceive()` 函数参数和返回值

• `xQueue`

正在接收数据的队列的句柄（读取）。队列句柄将从调用`xQueueCreate()` or `xQueueCreateStatic()`创建队列的函数返回。

• `pvBuffer`

指向将接收到的数据复制到的内存的指针。

在创建队列时，将设置队列所持有的每个数据项的大小。`pvBuffer`所指向的内存必须至少大到足以容纳那么多个字节。

• `xTicksToWait`

如果队列已为空，则任务应保持在“阻塞”状态以等待队列中数据可用的最长时间。

如果`xtickso`等待为零，那么如果队列已经为空，`xQueueReceive()`将立即返回。

阻塞时间以滴答周期指定，因此它所表示的绝对时间取决于滴答频率。宏`pdMS_TO_TICKS()`可用于将以毫秒为单位指定的时间转换为在刻度中指定的时间。

如果`FreeRTOSConfig.h`中的`INCLUDE_vTaskSuspend`设置为1，则将`INCLUDE_vTaskSuspend`等待设置为`portMAX_DELAY`将导致任务无限期等待（不超时）。

• 返回值

有两个可能的返回值：

◦ `pdPASS`

当从队列中成功读取数据时，将返回`pdPASS`。

如果指定了阻塞时间（`xTicksToWait`不是零），则可能将调用任务置于阻塞状态，等待数据在队列上可用，但在阻塞时间过期之前成功从队列读取数据。

• `errQUEUE_EMPTY`（与`pdFAIL`的值相同）

如果由于队列已经为空，因此无法从队列中读取数据，则返回`errQUEUE_EMPTY`。

如果指定了阻塞时间（`xTicksToWait`不是零），那么调用任务将被放置入阻塞状态，等待另一个任务

或中断将数据发送到队列，但阻塞时间在此发生之前已经过期。

5.3.4 `uxQueueMessagesWaiting()` API函数

`uxQueueMessagesWaiting()` 查询当前队列中的项目数。

注意：永远不要从中断服务例程调用`uxQueueMessagesWaiting()`。应该使用中断安全的`uxQueueMessagesWaitingFromISR()`来代替它。

```
UBaseType_t uxQueueMessagesWaiting ( QueueHandle_t xQueue );
```

清单5.5e `uxQueueMessagesWaiting()` 函数原型

函数参数和返回值

• `xQueue`

要查询的队列的句柄。队列句柄将从调用`xQueueCreate()` or `xQueueCreateStatic()`创建队列的函数返回。

• 返回值

当前正在进行查询的队列中的数据项数。如果返回0，则该队列为空。

示例5.1 从队列接收时阻塞

本示例演示了创建队列、从多个任务向队列发送数据，以及从队列接收数据。创建的队列用于保存`int32_t`类型的数据项。发送到队列的任务不指定阻塞时间，而从队列接收的任务则指定。

发送到队列的任务的优先级低于从队列接收的任务。这意味着队列不应该包含多个项目，因为一旦数据发送到队列，接收任务就会解除阻塞，抢占发送任务（因为它具有更高的优先级），并删除数据，留下数据队列再次空了。

该示例创建了清单5.6中所示的任务的两个实例。一个连续地将值100写入该队列，另一个连续地将该值200写入同一队列。任务参数用于将这些值传递到每个任务实例中。

```

static void vSenderTask( void *pvParameters )
{

    int32_t lValueToSend;

    BaseType_t xStatus;

    /*创建了这个任务的两个实例，因此发送到队列的值通过任务参数传递进来——这样每个实例都可以使用不同的值。创建该队列是为了保存int32_t类型的值，因此可以将该参数强制转换为所需的类型。*/
    lValueToSend = (int32_t) pvParameters;

    /*像大多数任务一样，该任务是在一个无限循环中实现的。*/
    for ( ; ; )
    {
        /*将值发送到该队列。

        第一个参数是要将数据发送到的队列。队列是在调度程序启动之前创建的，因此在此任务开始执行之前。

        第二个参数是要发送的数据的地址，在这种情况下是lValueToSend的地址。

        第三个参数是阻塞时间——如果队列已满，任务应保持阻塞状态以等待队列中出现可用空间的时间。在这种情况下，没有指定阻塞时间，因为队列永远不应该包含超过一个项目，因此永远不会满。*/

        xStatus = xQueueSendToBack ( xQueue, &lValueToSend, 0 );

        if (xStatus != pdPASS)
        {
            /*发送操作无法完成，因为队列已满。这发生了错误，因为该队列永远不应该包含多个项目！*/
            vPrintString( "Could not send to the queue.\r\n" );
        }
    }
}

```

Listing 5.6 实现了例 5.1 中使用的发送任务

清单5.7显示了从队列接收数据的任务的实现。接收任务指定100毫秒的阻塞时间，之后就会进入阻塞状态，以等待可用数据。当队列中任何一个数据可用或经过100毫秒时仍没有可用数据，它将退出阻塞状态。

在本例中，有两个任务在不断向队列写入数据，因此 100 毫秒的超时时间永远不会达到。

```
static void vReceiverTask( void *pvParameters )
{
    /*声明用于保存从队列中接收的值的变量。*/

    int32_t lReceivedValue;
    BaseType_t xStatus;
    const TickType_t TickType_t ToWait=pdMS_TO_TICKS (100);

    /*这个任务也被定义在一个无限循环中。*/
    for(;;)
    {
        /*此调用应该始终发现队列为空，因为此任务将立即删除写入队列的任何数据。*/
        if (uxQueueMessagesWaiting( xQueue ) != 0)
        {
            vPrintString( "Queue should have been empty!\r\n" );
        }

        /*将从队列中接收数据。

        第一个参数是要从中接收数据的队列。队列在调度程序启动之前创建，因此在此任务首次运行之前创建。

        第二个参数是接收到的数据将被放置在其中的缓冲区。在这种情况下，缓冲区只是一个变量的地址，该变量的大小是保存接收数据所需的。

        最后一个参数是阻塞时间——如果队列已经为空，则任务将保持在阻塞状态以等待数据可用的最大时间。*/
        xStatus = xQueueReceive ( xQueue, &lReceivedValue, xTicksToWait );

        if (xStatus == pdPASS)
        {
            /*数据已成功从队列接收到，并打印出接收到的值。*/
            vPrintStringAndNumber( "Received = ", lReceivedValue );
        }
        else
        {
            /*等待 100 毫秒后仍未从队列中收到数据。 这肯定是个错误，因为发送任务是自由运行的，会不断
            *向队列写入内容
            */
            vPrintString ( “无法从队列接收信息.\r\n” );
        }
    }
}
```

```

    }
}

```

清单5.7 实例5.1的接收方任务的实现

清单5.8包含了主函数的定义。它仅仅在启动调度程序之前创建队列和三个任务。创建的队列最多只包含5个int32_t值，即使任务的相对优先级意味着该队列每次永远不会包含超过一个项目。

```

/*声明一个 QueueHandle_t 类型的变量。 用于存储所有三个任务都要访问的队列句柄。*/
QueueHandle_t xQueue;

int main (void)
{
    /*创建的队列最多包含5个值，每个值为
       大为足以容纳一个int32_t类型的变量。*/
    xQueue = xQueueCreate( 5, sizeof( int32_t ) );

    if (xQueue != NULL)
    {
        /*创建发送到队列的任务的两个实例。任务参数用于将任务将写入的值传递给队列，
           因此一个任务将连续写入100到队列，而另一个任务将连续写入200到队列。这两个
           任务都是在优先级为1的位置上创建的。*/
        xTaskCreate( vSenderTask, "Sender1", 1000, ( void * ) 100, 1, NULL );
        xTaskCreate( vSenderTask, "Sender2", 1000, ( void * ) 200, 1, NULL );

        /*创建从队列中读取的任务。此任务被创建
           的优先级为2，因此高于发送方任务的优先级。*/
        xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 2, NULL );
        /*启动调度程序，以便已创建的任务开始执行。*/
        vTaskStartScheduler();
    }
    else
    {
        /*无法创建该队列。*/
    }

    /*如果一切正常，那么主函数将永远不会到达这里，因为调度程序现在将运行任务。如果主
       函数确实到达这里，那么很可能没有足够的空闲RTOS堆内存用于创建空闲任务。第3章提
       供了关于堆内存管理的更多信息。*/
    for(;;);
}

```

Listing 5.8 示例5.1中的主函数的实现

图5.2显示了示例5.1所产生的输出。

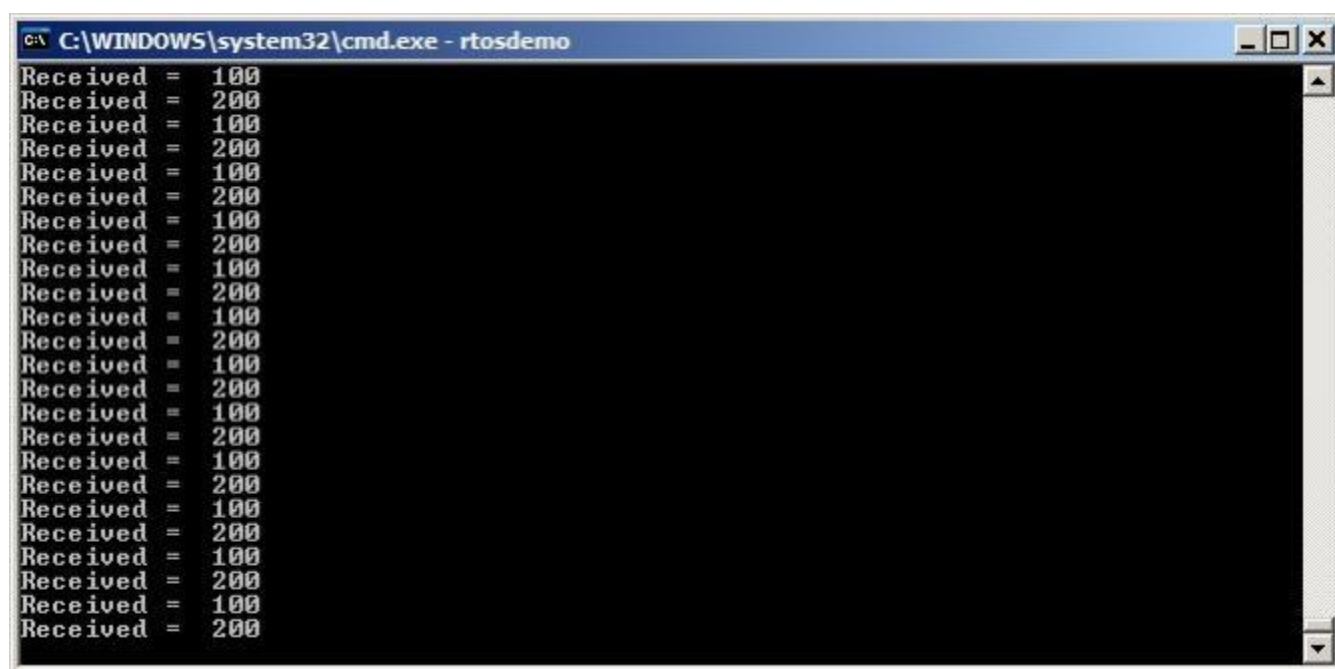


图5.2在执行示例5.1时产生的输出

图5.3演示了执行的顺序。

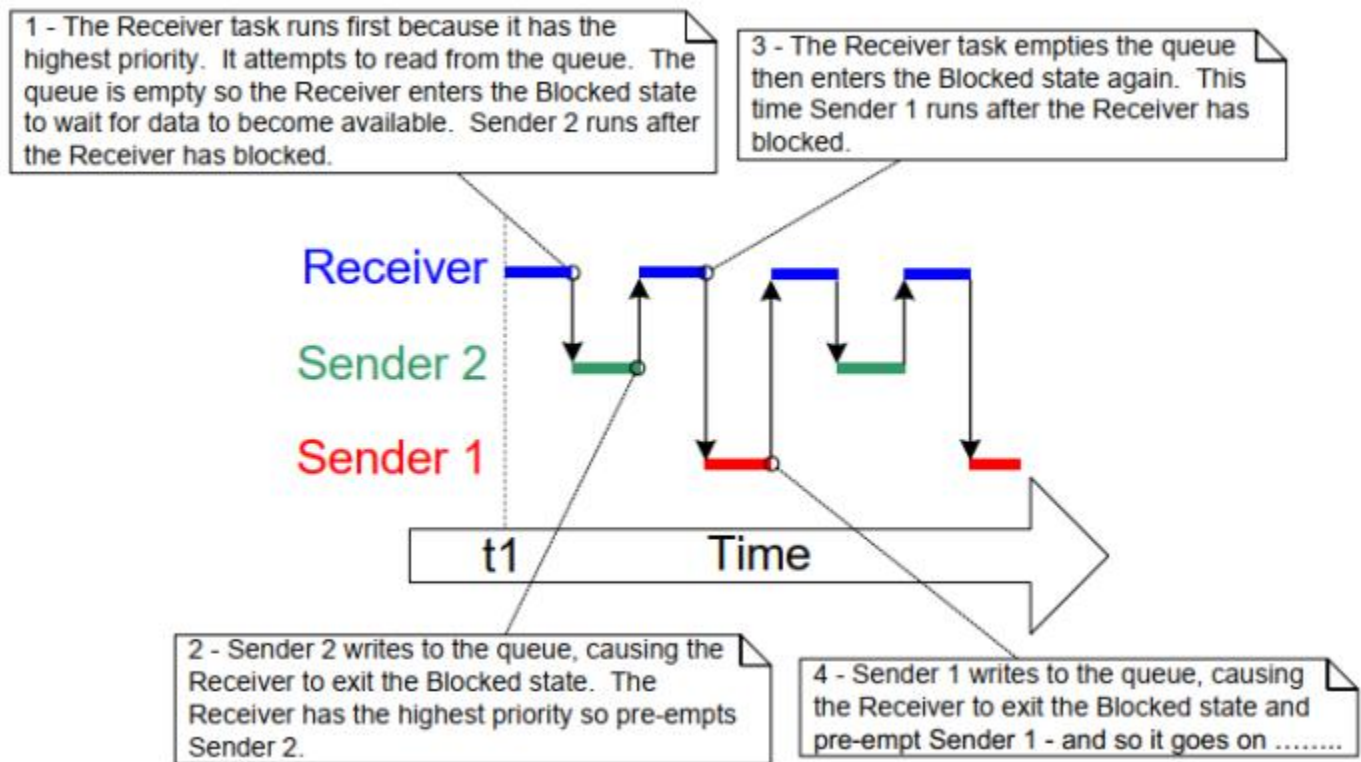


图5.3 由示例5.1生成的执行顺序

5.4 从多个来源接收数据

在FreeRTOS设计中，为一个任务从多个源接收数据是很常见的。接收任务需要知道数据来自哪里，以确定如何处理它。要实现的一种简单的设计模式，它使用单个队列来传输同时包含数据值和数据源的结构，如图5.4所示。

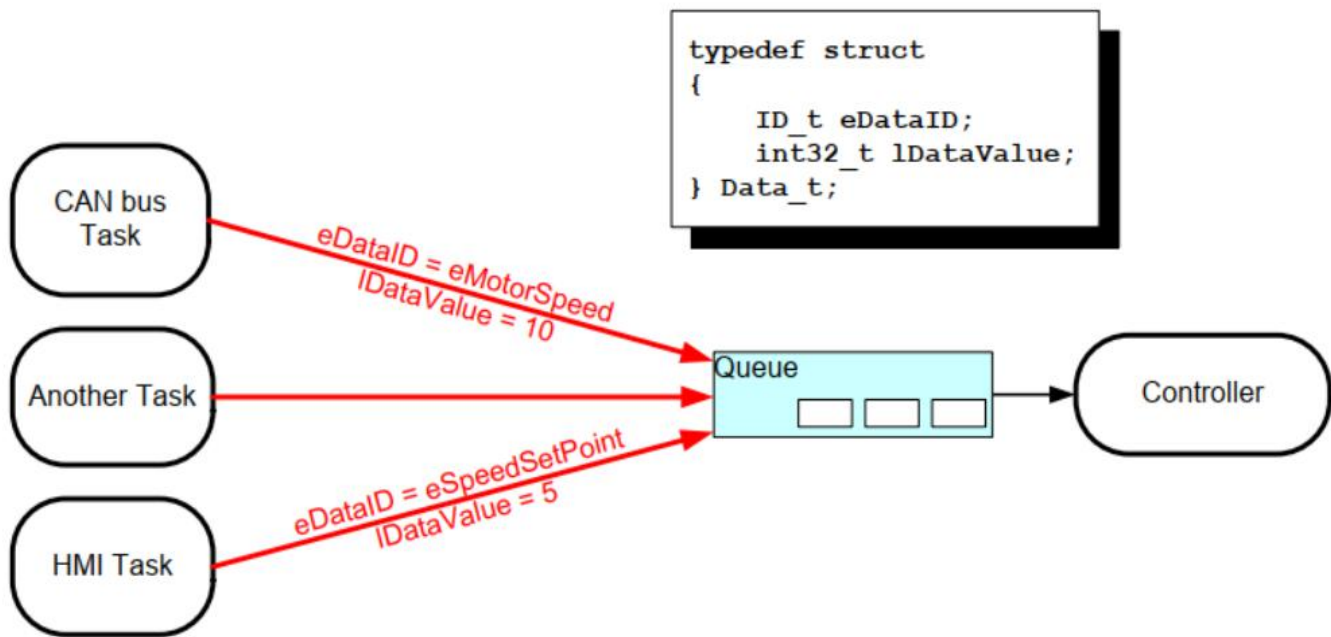


图5.4 在队列中发送结构体的示例场景

参见图5.4:

- 所创建的队列保存了Data_t类型的结构。该结构允许数据值和枚举类型，指示数据在一条消息中发送到队列的含义。
- 中央控制器的任务是执行系统的主要功能。它必须对队列中传达给它的输入和系统状态变化做出反应
- CAN 总线任务用于封装 CAN 总线接口功能。CAN 总线任务接收并解码报文后，会将已解码的报文以 Data_t 结构发送给控制器任务。传输结构中的 eDataID 成员会告诉控制器任务数据是什么。这里显示的是电机速度值。传输结构的 lDataValue 成员将告诉控制器任务实际电机速度值。
- 人机界面（HMI）任务用于封装人机界面的所有功能。机器操作员可以通过多种方式输入命令和查询值，这些方式必须要在人机界面任务中进行检测和解释。当输入新命令时，人机界面任务会以 Data_t 结构向控制器任务发送该命令。传输结构中的 eDataID 成员会告诉控制器任务数据是什么。这里显示的是一个新的设定点值。传输结构中的 lDataValue 成员将告诉控制器任务实际的设定点值。

第（RB-TBD）章介绍了如何扩展这种设计模式，使控制器任务可以直接回复排队等待结构的任务。

实例5.2 在发送到队列和在队列上发送结构时发生阻塞

示例5.2与示例5.1类似，但由于任务优先级发生了反转，因此接收任务的优先级低于发送任务。此外，创建的队列保存的是结构，而不是整数。

清单5.9显示了示例5.2所使用的结构的定义。

```
/*定义一个用于标识数据源的枚举类型。*/
typedef enum
{
    eSender1,
    eSender2
} DataSource_t;

/*定义将在队列上传递的结构类型。*/
typedef struct
{
    uint8_t ucValue;
    DataSource_t eDataSource;
} Data_t ;

/*声明两个将在队列上传递的Data_t类型的变量。*/
static const Data_t xStructsToSend[ 2 ] =
{
    { 100, eSender1 }, /*由发送器1使用。*
    { 200, eSender2 } /*由发送器2使用。*
};
```

清单5.9 要在队列上传递的结构体的定义，以及在示例中使用的两个变量的声明

在示例5.1中，接收任务具有最高的优先级，因此队列从来不包含超过一个项目。这是因为一旦将数据放入队列中，接收任务就会抢占发送任务。在示例5.2中，发送任务具有更高的优先级，因此队列通常将已满。这是因为，一旦接收任务从队列中删除一个项目，它就会被一个发送任务抢占，然后发送任务立即重新填充队列。然后，发送任务重新进入“阻塞”状态，等待队列中的空间再次可用。

清单5.10显示了发送任务的实现。发送任务指定了100毫秒的阻塞时间，因此每次队列变满时，它都进入阻塞状态，等待空间可用。当队列中有一个空间可用，或者100毫秒后没有可用空间时，它将离开“阻塞”状态。在本例中，接收任务不断地在队列中腾出空间，因此100毫秒的超时永远不会达到。

```
static void vSenderTask( void *pvParameters )
{
    BaseType_t xStatus;
```

```

const TickType_t TickType_t ToWait=pdMS_TO_TICKS (100);

/*与大多数任务一样，该任务是在一个无限循环中实现的。*/

for ( ; ; ) {
    /*发送到队列。
    第二个参数是被发送的结构的地址。该地址作为任务参数传入，因此可以直接使用
    pvParameters。
    第三个参数是阻塞时间——如果队列已经满，那么任务应该保持在阻塞状态，等待队
    列上的空间可用的时间。指定阻塞时间，因为发送任务的优先级高于接收任务，因此
    队列有望变满。当两个发送任务都处于“被阻塞”状态时，接收任务将从队列中删除
    项目。*/

    xStatus = xQueueSendToBack ( xQueue, pvParameters, xTicksToWait );
    if (xStatus != pdPASS)
    {
        /*即使等待了 100 毫秒，也无法完成发送操作。这肯定是个错误，因为一旦两个发送任务
        都处于阻塞状态，接收任务就应立即在队列中腾出空间。
        */

        vPrintString( "Could not send to the queue.\r\n" );
    }
}

```

5.10 例 5.2 发送任务的实现

接收任务的优先级最低，因此仅当两个发送任务都处于阻塞状态时才运行。

发送任务只有在队列已满时才会进入阻塞状态，因此接收任务只有在队列已满时才会执行。因此，即使它没有指定阻塞时间，它也应该总是可以接收到数据。

清单5.11 显示了接收任务的实现。

```

static void vReceiverTask( void *pvParameters )
{
    /* Declare the structure that will hold the values received from
    the
    queue. */
    Data_t xReceivedStructure;
    BaseType_t xStatus;
    /* This task is also defined within an infinite loop. */
    for( ;; )

```

```

{
    /*因为它具有最低的优先级，所以这个任务只会在发送任务处于阻塞状态时运行。发
       送任务只有在队列已满时才会进入阻塞状态，因此此任务始终期望队列中的项目数
       量等于队列长度，在此情况下为3。*/
    if (uxQueueMessageWaiting (xQueue) != 3)
    {
        vPrintString( "Queue should have been full!\r\n" );
    }

    /*是从队列中接收到的信息。

    第二个参数是接收到的数据将被放置在其中的缓冲区。在这种情况下，缓冲区只是一个
    变量的地址，该变量具有保存接收到的结构所需的大小。

    最后一个参数是阻塞时间——如果队列已经为空，则任务将保持在阻塞状态以等
    待数
    据可用的最大时间。在这种情况下，不需要阻塞时间，因为此任务只在队列满时
    运
    行。*/
    xStatus = xQueueReceive ( xQueue, &xReceivedStructure, 0 );
    if (xStatus == pdPASS) {
        /*数据已成功地从队列中接收到，并打印出
           已接收到的值和该值的来源。*/
        if (xReceivedStructure.eDataSource ==
            eSender1 ) {
            vPrintStringAndNumber( "From Sender 1 = ",
                                   xReceivedStructure.ucValue );
        }
        else {
            vPrintStringAndNumber( "From Sender 2 = ",
                                   xReceivedStructure.ucValue );
        }
    }
}
else
{
    /*从队列中没有收到任何消息。这是一个错误，因为此任务只能在队列已满
       时运行。*/
    vPrintString( "Could not receive from the queue.\r\n" );
}
}
}

```

清单5.11 示例5.2的接收任务的定义

与前面的例子相比，主函数仅略有变化。创建该队列以保存三个Data_t结构，并反转发送和接收任务的优先级。清单5.12显示了主函数的实现。

```
int main (void)
{
    /*创建的队列最多包含3个Data_t类型的结构。*/
    xQueue=xQueueCreate (3, sizeof (Data t) );
    if (xQueue != NULL)
    {
        /*创建将写入队列的任务的两个实例。该参数用于将任务将写入的结构传递给队列，因此一个任务将连续发送xStructsToSend[0]到队列，而另一个任务将连续发送xStructsToSend[1]。这两个任务都在优先级2上创建，这高于接收器的优先级。*/
        xTaskCreate( vSenderTask, "Sender1", 1000, &( xStructsToSend[ 0 ] ), 2, NULL );
        xTaskCreate( vSenderTask, "Sender2", 1000, &( xStructsToSend[ 1 ] ), 2, NULL );

        /*创建将从队列中读取的任务。此任务创建的优先级为1，因此低于发送器任务的优先级。*/
        xTaskCreate (vReceiverTask, "Receiver", 1000, NULL, 1, NULL);

        /*启动调度程序，以便已创建的任务开始执行。*/ vTaskStartScheduler();
    }
    else
    {
        /*无法创建该队列。*/
    }

    /*如果一切都好，那么主函数将永远不会到达这里，因为调度程序现在将运行任务。如果主函数确实到达了这里，那么很可能没有足够的堆内存来创建空闲任务。第3章提供了关于堆内存管理的更多信息。*/
    for(;;);
}
```

清单5.12 示例5.2的主函数的实现

图5.5显示了示例5.2所产生的输出。

[illegible]

图5.5 示例5.2所产生的输出

图5.6演示了由于发送任务的优先级高于接收任务的优先级而导致的执行顺序。下面是对图5.6的进一步解释，以及对前四条消息来自同一任务的描述。

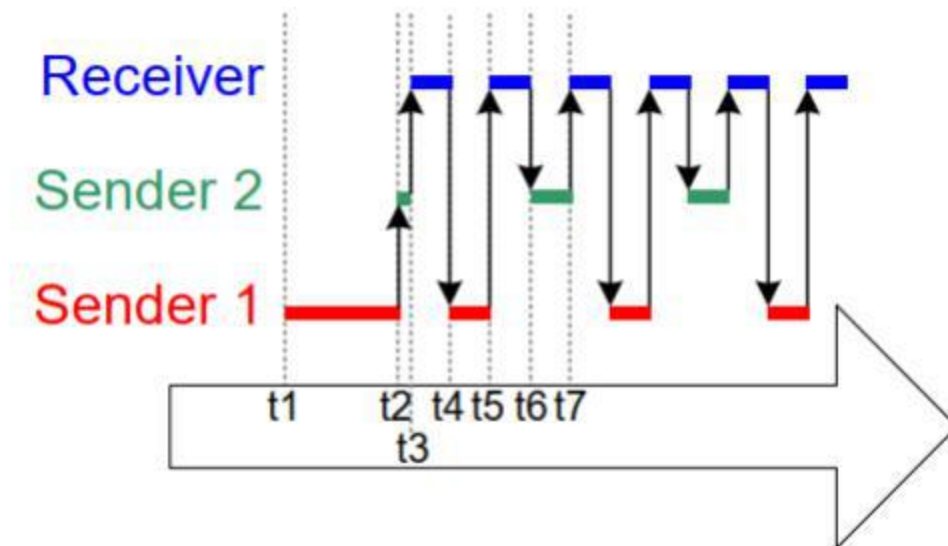


图5.6 由示例5.2生成的执行顺序

图5.6的关键

- t1

任务发送者1执行并发送3个数据项。

- t2

队列已满，因此发送人1进入阻塞状态，等待其下一次发送完成。任务发送者2现在是可以运行的最高优先级任务，因此它进入“运行”状态。

- t3

任务发送者2发现队列已满，因此它进入“阻塞”状态，等待其第一次发送完成。任务接收器现在是可以运行的最高优先级任务，因此它进入“运行”状态。

- t4

两个优先级高于接收任务优先级的任务正在等待队列上的可用空间，导致任务接收器一旦从队列中删除一个项目就被抢占。任务发件人1和发件人2具有相同的优先级，因此调度器选择等待时间最长的任务作为将进入运行状态的任务——在这种情况下是任务发送人1。

- t5

任务发送者1将向该队列发送另一个数据项。队列中只有一个空位，因此任务发送方1进入阻塞状态，等待其下一次发送完成。任务接收器同样是可以运行的最高优先级的任务，以便它进入运行状态。

任务发送方1现在已向该队列发送了四个项，而任务发送方2仍在等待将其第一个项发送到该队列。

- t6

两个优先级高于接收任务优先级的任务正在等待队列中的可用空间，因此一旦任务接收器从队列中删除一个项目，就会被抢占。这次发件人2等待的时间比发件人1长，因此发件人2进入“运行”状态。

- t7

任务发送者2会将一个数据项发送到该队列。队列中只有一个空位，因此发送人2进入阻塞状态，等待下一次发送完成。任务发送器1和发件人2都在等待队列中变为可用空间，因此任务接收器是唯一可以进入“正在运行”状态的任务。

5.5 使用大的或可变大小的数据

5.5.1 队列指针

如果队列中存储的数据大小很大，那么最好使用队列传输指向数据的指针，而不是将数据本身逐字节复制到队列中或从队列中复制出来。传输指针在处理时间和创建队列所需的 RAM 量方面都更加高效。然而，在对指针进行排队时，必须格外小心以确保：

- 被指向的RAM的所有者已被明确定义。

当通过指针在任务之间共享内存时，必须确保两个任务不会同时修改内存内容，或采取任何其他可能导致内存内容无效或不一致的操作。理想情况下，在将指针发送到队列之前，只允许发送任务访问内存，并且在从队列接收到指针后，只允许接收任务访问内存。

- 被指向的RAM仍然有效。

如果被指向的内存是动态分配的，或者是从预先分配的缓冲区池中获得的，那么应该只有一个任务负责释放内存。在释放内存后，任何任务都不应尝试访问该内存。

指针不应被用于访问已分配到任务堆栈上的数据。在堆栈帧发生更改后，该数据将无效。

举个例子，清单5.13、5.14和5.15演示了如何使用队列将一个指向缓冲区的指针从一个任务发送到另一个任务：

清单5.13创建了一个最多可以容纳5个指针的队列。

清单5.14分配一个缓冲区，将缓冲区写入一个字符串，然后将指向缓冲区的指针发送到队列。

清单5.15从队列接收一个指向缓冲区的指针，然后打印缓冲区中包含的字符串。

```
/*声明 QueueHandle_t 类型的变量来保存正在创建的队列的句柄。
*/
QueueHandle_t xPointerQueue;

/*在这种情况下，可以创建一个最多可以容纳5个指针的队列
   这个例子里是字符指针。*/
xPointerQueue = xQueueCreate( 5,
sizeof( char * ) );
```

清单5.13 创建一个包含指针的队列

```
/*一个获得一个缓冲区的任务，它将一个字符串写入该缓冲区，然后
   将缓冲区的地址发送到在清单5.13中创建的队列。*/
void vStringSendingTask( void *pvParameters ) {
    char *pcStringToSend;
    const size_t xMaxStringLength = 50;
    BaseType_t xStringNumber = 0;

    for(;;)
    {
        /*获取一个至少是xMaxStringLength较大的缓冲区。prvGetBuffer（）的实现没有
           显示-它可能获得
```

```

        来自预分配缓冲区池的缓冲区，或者只是动态分配缓冲区。*/
        pcStringToSend = ( char * ) prvGetBuffer( xMaxStringLength )

/*在缓冲区中写入一个字符串。*/
snprintf( pcStringToSend, xMaxStringLength, "String number %d\r\n",
          xStringNumber );
/*增加计数器，使字符串在每次迭代中都有所不同
   */
xStringNumber++;

/*将缓冲区的地址发送到已在5.13中创建的队列
该缓冲区的地址存储在
pcStringToSend中。*/
xQueueSend( xPointerQueue, /* The handle of the queue. */
            &pcStringToSend, /* The address of the pointer that points
                               to the buffer. */
            portMAX_DELAY );

    }
}

```

清单5.14 使用队列发送指向缓冲区的指针

```

/*从已创建的队列接收缓冲区地址的任务
并在清单5.14中写入。该缓冲区包含一个已被打印出来的字符串。*/

void vStringReceivingTask( void *pvParameters )
{
    char * pcReceivedString;

    for(;; )
    {
        /*接收一个缓冲区的地址。*/
        xQueueReceive( xPointerQueue
                      &pcReceivedString,
                      portMAX_DELAY );

        /*缓冲区包含一个字符串，并将其打印出来。*/
        vPrintString ( pcReceivedString );

        /*缓冲区不再需要任何内容——释放它，以便可以释放或重用它。*/
        prvReleaseBuffer ( pcReceivedString );
    }
}

```

清单5.15 使用队列来接收指向缓冲区的指针**5.5.2 使用队列发送不同类型和长度的数据^[9]**

^[9]: FreeRTOS消息缓冲区是保留可变长度数据的队列的较轻替代方案。

本书的前几节展示了两种强大的设计模式：向队列发送结构，以及向队列发送指针。结合这些技术，允许任务使用单个队列来接收来自任何数据源的任何数据类型。FreeRTOS+TCP TCP/IP堆栈的实现提供了一个如何实现这一点的实际示例。

在自身任务中运行的TCP/IP堆栈必须处理来自许多不同源的事件。不同的事件类型与不同的数据类型和长度相关联。IPStackEvent_t结构描述了在TCP/IP任务之外发生的所有事件，并被发送到队列上的TCP/IP任务。清单5.16显示了IPStackEvent_t的结构。IPStackEvent_t结构中的pvData成员是一个指针，可以用来直接保存一个值，或指向一个缓冲区。

```

/* TCP/IP 堆栈中用于识别事件的枚举类型的子集
 */
typedef enum
{
    eNetworkDownEvent = 0,    /*网络接口已丢失，或需要（重新）连接。
    */
    eNetworkRxEvent,          /*已从网络收到一个数据包。*/
    eTCPAcceptEvent,          /*调用 FreeRTOS_accept() 来接受或等待新客户端。*/

    /*其他事件类型出现在这里，但没有显示在此列表中。*/
} eIPEvent_t;

/* 描述事件的结构，并通过队列发送到 TCP/IP 任务。*/

typedef struct IP_TASK_COMMANDS
{
    /*一种标识事件的枚举类型。请参阅eIPEvent_t
    上面的定义。*/
    eIPEvent_t eEventType;

    /*可保存值或指向缓冲区的泛型指针。*/
    void *pvData;
} IPStackEvent_t;

```

清单5.16 在FreeRTOS+TCP中，用于将事件发送到TCP/IP堆栈任务的结构

示例TCP/IP事件及其关联数据包括：

eNetworkRxEvent: 从网络接收到一个数据包。

网络接口使用 `IPStackEvent_t` 类型的结构, 将接收数据事件发送到 TCP/IP 任务。该结构的 `eEventType` 成员设置为 `eNetworkRxEvent`, 该结构的 `pvData` 成员用于指向包含接收到的数据的缓冲区。清单5.17显示了伪代码示例

```
void vSendRxDataToTheTCPTask( NetworkBufferDescriptor_t *pRxedData ){
    IPStackEvent_t xEventStruct;

    /*完成了IPStackEvent_t的结构。接收到的数据存储在pRxedData中。*/
    xEventStruct.eEventType = eNetworkRxEvent;
    xEventStruct.pvData= (void*) pRxedData;

    /*将IPStackEvent_t结构发送到TCP/IP任务。*/
    xSendEventStructToIPTask ( &xEventStruct );
}
```

清单5.17 伪代码，显示如何使用 `IPStackEvent_t` 结构将从网络接收到的数据发送到TCP/IP任务

eTCPAcceptEvent: 套接字是指接受或等待来自客户端的连接。

- 调用 `FreeRTOS_accept()` 的任务使用 `IPStackEvent_t` 类型的结构向 TCP/IP 任务发送接受事件。结构体的 `eEventType` 成员设置为 `eTCPAcceptEvent`, 结构体的 `pvData` 成员设置为接受连接的套接字句柄。清单 5.18 显示了一个伪代码示例

```
void vSendAcceptRequestToTheTCPTask (Socket_t xSocket)
{
    IPStackEvent_t xEventStruct;

    /*完成了IPStackEvent_t的结构。*/
    xEventStruct.eEventType = eTCPAcceptEvent;
    xEventStruct.pvData= (void*) xSocket;

    /*将IPStackEvent_t结构发送到TCP/IP任务。*/
    xSendEventStructToIPTask ( &xEventStruct );
}
```

清单5.18 伪代码，显示了如何使用IPStackEvent_t结构发送正在接受到TCP/IP任务的连接的套接字的句柄

- eNetworkDownEvent：网络需要连接，或重新连接。

网络接口使用 IPStackEvent_t 类型的结构向 TCP/IP 任务发送网络故障事件。结构体的 eEventType 成员被设置为 eNetworkDownEvent。网络瘫痪事件与任何数据都没有关联，因此不会使用结构体的 pvData 成员。清单 5.19 显示了一个伪代码示例

```
void vSendNetworkDownEventToTheTCPTask (Socket_t xSocket)
{
    IPStackEvent_t xEventStruct;

    /*实现了IPStackEvent_t的结构。*/
    xEventStruct.eEventType = eNetworkDownEvent;

    xEventStruct.pvData = NULL;

    /*将IPStackEvent_t结构发送到TCP/IP任务。*/
    xSendEventStructToIPTask ( &xEventStruct );
}
```

Listing 5.19 显示如何使用IPStackEvent_t结构将网络停机事件发送到TCP/IP任务

清单5.20显示了在TCP/IP任务中接收和处理这些事件的代码。可以看出，从队列中接收到的IPStackEvent_t结构中的eEventType成员被用来确定如何解释pvData成员。

```
IPStackEvent_t xReceivedEvent;

/* 阻塞网络事件队列，直到接收到事件，或 xNextIPSleep ticks 过去后仍未接收到事件。eEventType 被设置为 eNoEvent，以防调用 xQueueReceive() 时因超时而返回，而不是因为收到了事件。*/

xReceivedEvent.eEventType = eNoEvent;
xQueueReceive ( xNetworkEventQueue, &xReceivedEvent, xNextIPSleep );

/*如果收到了哪个事件？*/
switch(xReceivedEvent.eEventType )
{

    case eNetworkDownEvent:
        /*尝试（重新）建立一个连接。此事件与任何数据关联。*/
```

```

        prvProcessNetworkDownEvent();
        break;

    case eNetworkRxEvent:
        /*该网络接口已接收到一个新的数据包。指向接收数据的指针存储在接收的
        IPStackEvent_t结构的pvData成员中。处理接收到的数据。*/
        prvHandleEthernetPacket ( (NetworkBufferDescriptor_t *)
                                   ( xReceivedEvent.pvData) );

        break;

    case eTCPAcceptEvent:

        /*调用了 FreeRTOS_accept() API 函数。 接受连接的套接字句柄存储在接收到的 IPStackEvent_t 结构
        的 pvData 成员中。*/
        xSocket = (FreeRTOS_Socket_t *) (xReceivedEvent.pvData);
        xTCPCheckNewClient ( xSocket );
        break;

    /*其他事件类型也以相同的方式处理，但在这里不显示。*/

}

```

清单5.20 显示如何接收和处理IPStackEvent_t结构的伪代码

5.6 从多个队列进行接收

5.6.1 队列集

应用程序设计通常需要一个任务来接收不同大小的数据、不同含义的数据和来自不同来源的数据。上一节演示了如何使用接收结构的单个队列以简洁有效的方式完成这一点。然而，有时应用程序的设计者正在处理限制其设计选择的约束条件，因此需要为某些数据源使用单独的队列。例如，被集成到设计中的第三方代码可能会假定存在一个专用队列。在这种情况下，可以使用“队列集”。

队列集允许任务从多个队列接收数据，而无需使用任务依次轮询每个队列，以确定哪个队列包含数据。

使用队列集接收从多个源接收数据的设计，比使用接收结构的单个队列实现相同功能的设计更整洁和效率更低。因此，建议只在设计约束使其使用绝对必要时使用队列集。

以下部分介绍如何使用由设置的队列：

- 创建队列集。

- 正在向集合中添加队列。

信号量也可以被添加到一个队列集中。信号量在这本书的后面会有描述。从队列集中读取，

- 以确定集合中的哪些队列包含数据。

当作为集合成员的队列接收数据时，接收队列的句柄被发送到队列集，当任务调用从队列集读取的函数时返回。因此，如果从队列集返回一个队列句柄，则知道该句柄引用的队列包含数据，然后任务可以直接从该队列读取。

注意：如果队列是队列集的成员，则每次从队列集接收其句柄时必须从队列读取，并且在从队列集接收其句柄之前不能从队列读取。

通过在FreeRTOSConfig.h中将configUSE_QUEUE_SETS编译时配置常量设置为1来启用队列集功能。

5.6.2 xQueueCreateSet () API函数

队列集必须在明确创建后才能使用。在撰写本文时，还没有 xQueueCreateSetStatic() 的实现。不过，队列集本身也是队列，因此可以通过特制的 xQueueCreateStatic() 调用，使用预分配的内存创建队列集。

队列集由句柄引用，它们是QueueSetHandle_t类型的变量。xQueueCreateSet() 创建队列集，并返回创建的队列集的QueueSetHandle_t。

```
QueueSetHandle_t xQueueCreateSet( const UBaseType_t uxEventQueueLength);
```

清单5.21 xQueueCreateSet()API函数原型

xQueueCreateSet() 参数和返回值

●uxEventQueueLength

当作为队列集成员的队列接收数据时，接收队列的句柄会被发送到队列集。uxEventQueueLength 定义了正在创建的队列集在同一时间可容纳的队列句柄的最大数量。

只有当队列集中的队列接收到数据时，队列句柄才会被发送到队列集。如果队列已满，则无法接收数据，因此，如果队列中的所有队列都已满，则不能将任何队列句柄发送到队列集。因此，队列集一次必须保持的最大项目数量是集合中每个队列的长度之和。

例如，如果集合中有三个空队列，并且每个队列的长度为5，那么集合中的队列总共可以在所有队列之前接收15个项目（每个队列乘以5个项目）

集合中的队列已满。在该示例中，uxEventQueueLength必须设置为15，以保证队列集可以接收到发送到它的每个数据项。

信号量也可以被添加到一个队列集中。信号量在这本书的后面也有讨论。为了计算uxEventQueueLength，二进制信号量的长度为1，互斥锁的长度为1，计数信号量的长度由信号量的最大计数值给出。

另一个例子是，如果一个队列集将包含一个长度为3的队列和一个二进制信号量（其长度为1），则uxEventQueueLength必须设置为4（3加1）。

•返回值

如果返回NULL，则无法创建队列集，因为没有足够的FreeRTOS堆内存来分配队列集数据结构和存储区域。第3章提供了关于FreeRTOS堆的更多信息。

如果返回非NULL值，则已成功创建队列集，并且返回的值是已创建队列集的句柄。

5.6.3 xQueueAddToSet（）API函数

该函数将一个队列或信号量添加到一个队列集中。信号量在这本书的后面会有描述。

```
BaseType_t xQueueAddToSet ( QueueSetMemberHandle_t xQueueOrSemaphore,
                             QueueSetHandle_t xQueueSet );
```

清单5.22 xQueueAddToSet（）函数原型

参数和返回值

•xQueueOrSemaphore

正在添加到队列集的队列或信号量的句柄。

队列句柄和信号量句柄都可以被强制转换为QueueSetMemberHandle_t类型。

•xQueueSet

要向其添加该队列或信号量的队列集的句柄。

返回值

有两个可能的返回值：

1. pdPASS

■

这表示队列集已成功创建。

2. pdFAIL

这表示无法将该队列或信号量添加到该队列集中。

队列和二进制信号量只能在它们为空时被添加到一个集合中。计数信号量只能在其计数为零时添加到集合中。队列和信号量一次只能是一个集合的成员。

5.6.4 xQueueSelectFromSet() API函数

`xQueueSelectFromSet()` 从队列集中读取一个队列句柄。

当作为队列集成员的队列或信号传递器接收到数据时，接收队列或信号传递器的句柄会被发送到队列集，并在任务调用 `xQueueSelectFromSet()` 时返回。如果调用 `xQueueSelectFromSet()` 返回了句柄，则句柄所引用的队列或寄存器中已知包含数据，调用任务必须直接从队列或寄存器中读取数据。

注意：不要从集合成员的队列或信号量读取数据，除非队列或信号量的句柄首先从调用 `xQueueSelectFromSet()` 返回。只能从 `xQueueSelectFromSet()` 返回队列句柄或信号传递句柄队列或信号传递中读取一个数据项

```
QueueSetMemberHandle_t xQueueSelectFromSet( QueueSetHandle_t xQueueSet,
                                             const TickType_t xTicksToWait );
```

清单5.23 `xQueueSelectFromSet()` API函数原型

参数和返回值

• `xQueueSet`

接收（读取）队列句柄或信号句柄的队列集句柄。队列集句柄将从用于创建队列集的 `xQueueCreateSet()` 调用中返回。

• `xTicksToWait`

如果队列集中的所有队列和信号量都为空，则调用任务应保持在“阻塞”状态以等待从队列集中接收队列或信号量句柄的最长时间。

如果 `xTicksToWait` 为零，那么如果该集中的所有队列和信号量都为空，则该函数将立即返回。

阻塞时间以滴答周期指定，因此它所表示的绝对时间取决于滴答频率。宏 `pdMS_TO_TICKS()` 可用于将以毫秒为单位指定的时间转换为在刻度中指定的时间。

如果FreeRTOSConfig.h中的INCLUDE_vTaskSuspend设置为1，则将INCLUDE_vTaskSuspend等待设置为

portMAX_DELAY将导致任务无限期等待（不超时）。

返回值

非NULL的返回值将是已知包含数据的队列或信号量的句柄。如果指定了阻塞时间（xTicksToWait不是零），则可能是调用任务进入了阻塞状态，以等待队列集中的队列或寄存器中的数据可用，但在阻塞时间结束前，从队列集中成功读取了一个句柄。句柄以 QueueSetMemberHandle_t 类型返回，可以将其转换为 QueueHandle_t 类型或 SemaphoreHandle_t 类型。

如果返回值为NULL，则无法从队列集中读取句柄。如果指定了阻塞时间（xTicksToWait不是零），那么调用任务被放置在阻塞状态，等待另一个任务或中断将数据发送到集中的队列或信号量，但阻塞时间在此发生之前已经过期。

示例5.3 *使用队列集

本示例中创建了两个发送任务和一个接收任务。发送任务在两个单独的队列上向接收任务发送数据，每个任务对应一个队列。将这两个队列添加到一个队列集中，然后接收任务从该队列集中读取，以确定这两个队列中哪一个包含数据。

任务、队列和队列集都是在主函数中创建的——其实现请参见清单5.24。

```
/*声明两个QueueHandle_t类型的变量。添加了两个队列
   到相同的队列集。*/
static QueueHandle_t xQueue1 =void, xQueue2 =void;

/*声明一个QueueSetHandle_t类型的变量。这是一个队列集
   添加两个队列。*/
static QueueSetHandle_t xQueueSet =void;

int main (void)
{
    /*创建这两个队列，它们都会发送字符指针。接收任务的优先级高于发送任务的优先级
       ，因此队列在任何时候都不会有超过一个项目*/
    xQueue1 = xQueueCreate( 1, sizeof( char * ) );
    xQueue2 = xQueueCreate( 1, sizeof( char * ) );

    /*创建队列集。集合将添加两个队列，每个队列可以包含1个项目，因此队列集一次必须保
       持的队列处理的最大数量是2（2个队列相乘
       每个队列有1个项目）。*/
    xQueueSet = xQueueCreateSet ( 1 * 2 );
```

```

/*请将这两个队列添加到该集合中。*/
xQueueAddToSet ( xQueue1, xQueueSet );
xQueueAddToSet ( xQueue2, xQueueSet );

/*创建要发送到队列的任务。*/
xTaskCreate( vSenderTask1, "Sender1", 1000, NULL, 1, NULL );
xTaskCreate( vSenderTask2, "Sender2", 1000, NULL, 1, NULL );

/*创建从队列集读取的任务，以确定两个队列中哪个包含数据。*/
xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 2, NULL );

/*启动调度程序，以便已创建的任务开始执行。*/
vTaskStartScheduler();

/*正常地，vTask开始调度程序（）不应该返回，所以如下
行永远不会执行。*/
for( ;; );
return 0;
}

```

Listing 5.24 示例5.3主函数的实现

第一个发送任务使用xQueue1每100毫秒向接收任务发送一个字符指针。第二个发送任务使用xQueue2每200毫秒向接收任务发送一个字符指针。字符指针指向一个标识所发送任务的字符串。清单5.25显示了这两个任务的实现。

```

void vSenderTask1( void *pvParameters )
{
    const TickType_t xBlockTime = pdMS_TO_TICKS( 100 );
    const char * const pcMessage = "Message from vSenderTask1\r\n";

    /*与大多数任务一样，该任务是在一个无限循环中实现的。*/

    for(;;)
    {

        vTaskDelay ( xBlockTime );

        /* 该任务发送字符串到xQueue1。不需要使用阻塞时间，即使队列只有一个数据项。这是因为从队列读取的任务的优先级高于该任务的优先级；一旦该任务写入队列，它将被从队列读取的任务优先使用，因此队列将总是为空在

```

```

        调用xQueueSend ( ) 时。所以数据阻塞时间被设置为0。
        */xQueueSend ( xQueue1, &pcMessage, 0 );
    }
}

/*-----*/

void vSenderTask2( void *pvParameters )
{
    const TickType_t xBlockTime = pdMS_TO_TICKS( 200 );
    const char * const pcMessage = "Message from vSenderTask2\r\n";

    /*与大多数任务一样，该任务是在一个无限循环中实现的。
    */for ( ; ; ) {
        /*200ms的阻塞时间*/
        vTaskDelay ( xBlockTime );

        /*将此任务的字符串发送到xQueue2。不需要使用阻塞时间，即使队列只能使用一个数据项。
        这是因为从队列读取的任务的优先级高于该任务的优先级；一旦该任务写入队列，
        它将被从队列读取的任务优先使用，因此队列将再次为空
        调用xQueueSend ( ) 返回。数据阻塞时间被设置为0。*/
        xQueueSend ( xQueue2, &pcMessage, 0 );
    }
}

```

清单5.25 示例5.3中使用的发送任务

发送任务写入的队列是同一队列集的成员。每次任务向其中一个队列发送时，队列的句柄就会被发送到队列集。接收任务调用 `xQueueSelectFromSet()` 从队列集中读取队列句柄。接收任务从集合中接收到队列句柄后，知道接收句柄引用的队列中包含数据，因此会直接从队列中读取数据。它从队列中读取的数据是一个字符串指针，接收任务会将其打印出来。

如果调用 `xQueueSelectFromSet()` 超时，则返回 `NULL`。在例 5.3 中，`xQueueSelectFromSet()` 的调用具有无限期的阻塞时间，因此永远不会超时并且将返回一个有效的队列句柄。因此，在使用返回值之前，接收任务无需检查 `xQueueSelectFromSet()` 是否返回 `NULL`。

只有当队列句柄所引用的队列包含数据时，`xQueueSelectFromSet()` 才会返回一个队列句柄。所以当从队列中读取时，不需要使用阻塞时间。

清单5.26显示了接收任务的实现。

```

void vReceiverTask( void *pvParameters )
{
    QueueHandle_t xQueueThatContainsData;
    char *pcReceivedString;

    /*与大多数任务一样，该任务是在一个无限循环中实现的。
    */for ( ; ; ) {

        /* 阻塞队列集，等待队列集中的一个队列包含数据。
        将xQueueSelectFromSet() 返回的 QueueSetMemberHandle_t 值转换为 QueueHandle_t
        ，因为已知集合的所有成员都是队列（队列集合不包含任何 Semaphores）。
        */
        xQueueThatContainsData = ( QueueHandle_t ) xQueueSelectFromSet(
                                                    xQueueSet, portMAX_DELAY );

        /*从队列集读取时使用了无限期的阻塞时间，因此除非其中一个队列包含数据，否则不会
        返回，并且队列容器数据不能为NULL。从队列中读取。没有必要指定阻塞时间，因为
        已知该队列包含数据。阻塞时间被设置为0。*/
        xQueueReceive ( xQueueThatContainsData, &pcReceivedString, 0 );
        /*打印从队列中接收到的字符串。*/
        vPrintString ( pcReceivedString );
    }
}

```

清单5.26 示例5.3中使用的接收任务

图5.7显示了示例5.3所产生的输出。可以看出，接收任务从两个发送任务中都接收到字符串。发送任务1使用的阻塞时间是发送任务2使用的阻塞时间的一半，这使得发送任务1发送的字符串打印出来的频率是发送任务2发送的字符串的两倍。

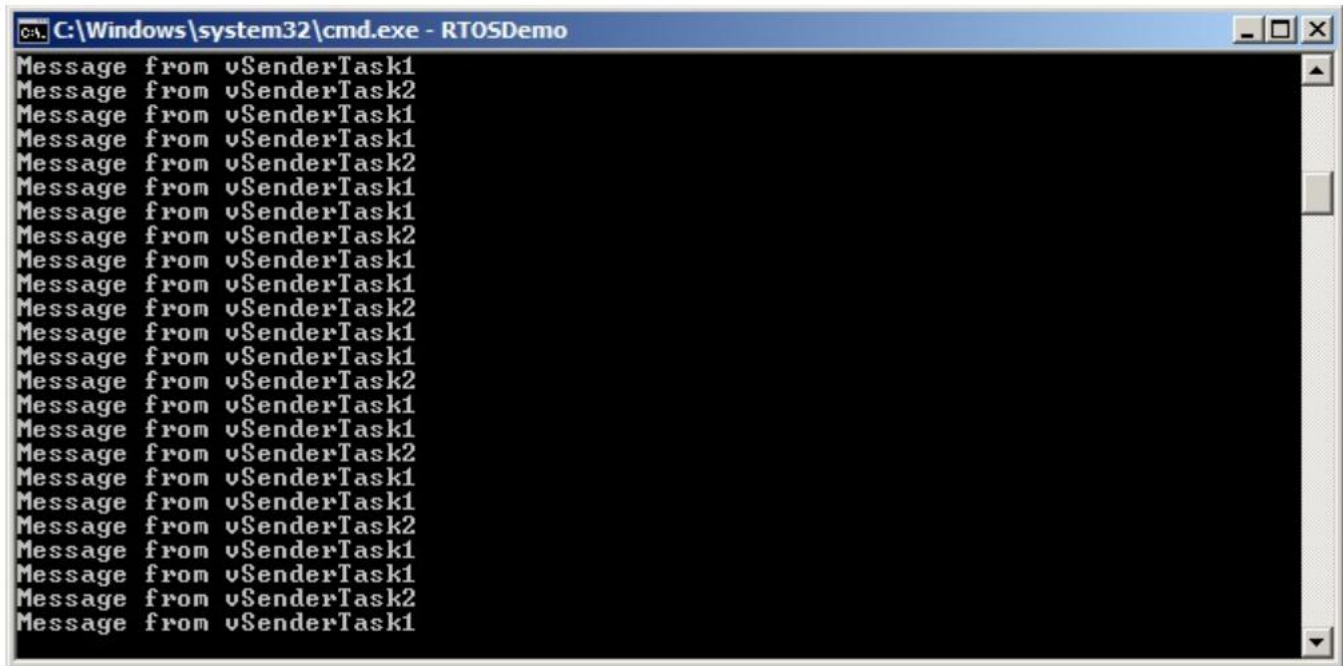


Figure 5.7 The output produced when Example 5.3 is executed

5.6.5 更现实的队列集用例

例子5.3演示了一个非常简单的情况：队列集只包含队列，并且它所包含的两个队列都用于发送一个字符指针。在实际的应用程序中，队列集可能同时包含队列和信号量，而且队列可能不都包含相同的数据类型。在这种情况下，在使用返回的值之前，需要测试 `xQueueSelectFromSet()` 返回的值。清单5.27演示了当集合有以下成员时，如何使用从 `xQueueSelectFromSet()` 返回的值：

二进制信号量。

读取字符指针的队列。

读取 `uint32_t` 值的队列。

清单5.27 假定队列和信号量已经创建并添加到队列集中。

```
/*从中接收字符指针的队列的句柄。*/ QueueHandle_t xCharPointerQueue;

/*从中接收uint32_t值的队列的句柄。*/ QueueHandle_t xUInt32tQueue;

/*二进制信号量的句柄。*/
SemaphoreHandle_t xBinarySemaphore;

/*这两个队列和二进制信号量所属的队列集。*/ QueueSetHandle_t xQueueSet;
```

```

void vAMoreRealisticReceiverTask( void *pvParameters )
{
    QueueSetMemberHandle_t xHandle;
    char *pcReceivedString;
    uint32_t ulRecievedValue;
    const TickType_t xDelay100ms = pdMS_TO_TICKS(100);

    for(;;)
    {
        /* 队列设置的阻塞时间最多100 ms，等待集合中的一个成员包含
        数据。*/
        xHandle = xQueueSelectFromSet ( xQueueSet, xDelay100ms );
        /*测试返回的值。如果返回的值为NULL，则调用xQueueSelectFromSet()超时
        。如果返回的值不是NULL，那么返回的值将是集合的一个成员的句柄。
        QueueSetMemberHandle_t值可以被强制转换为QueueHandle_t或SemaphoreHandle_t
        。是否需要显式强制转换取决于编译器。*/

        if (xHandle == NULL) {

            /*
            调用函数超时。*/
        }
        else if (xHandle == (QueueSetMemberHandle_t) xCharPointerQueue) {
            /*调用返回接收字符指针的队列的句柄。从队列中读取。已知该队列
            包含数据，因此使用的阻塞时间为0。*/
            xQueueReceive ( xCharPointerQueue, &pcReceivedString, 0 );

            /*接收到的字符指针可以在这里进行处理...*/ }
        else if (xHandle == (QueueSetMemberHandle_t) xUInt32tQueue)
        {
            /*调用返回接收uint32_t类型的队列的句柄。从队列中读
            取。这个已知队列包含数据，因此使用的阻塞时间为0。*/
            xQueueReceive (xUInt32tQueue, &ulRecievedValue, 0 );

            /*接收到的值可以在这里进行处理...*/
        }
        else (如果xHandle == (QueueSetMemberHandle_t) xBinarySemaphore) {
            /*返回二进制信号量的句柄。获取信号量。信号量为
            已知可用，因此使用的阻塞时间为0。*/
            xSemaphoreTake ( xBinarySemaphore, 0 );
        }
    }
}

```

```

        /*在采集信号时需要的任何处理都可以在这里执行*/...
    }
}
}

```

清单5. 27 使用包含队列和信号量的队列集

5.7 使用队列来创建邮箱

在嵌入式社区中对术语没有共识，“邮箱”在不同的RTOSes中意味着不同的东西。在本书中，术语“邮箱”用来指代长度为1的队列。一个队列可以被描述为一个邮箱，因为它在应用程序中使用的方式，而不是因为它与一个队列的功能不同：

- 队列用于将数据从一个任务发送到另一个任务，或者从中断服务例程发送到一个任务。
发送方将一个项目放在队列中，而接收方将从队列中删除该项目。数据通过队列从发送方传递到接收方。
- 邮箱用于保存可以由任何任务或任何中断服务例程读取的数据。数据
不通过邮箱，而是保留在邮箱中，直到它被覆盖。发件人将覆盖邮箱中的值。接收方会从邮箱中读取该值，但不会从邮箱中删除该值。

本章介绍了两个可以将队列用作邮箱的两个队列API函数。

清单5. 28显示了如何创建要作为邮箱使用的队列。

```

/*邮箱可以保存固定大小的数据项。在创建邮箱（队列）时，将设置数据项的大小。在本例中，创建邮箱以保存Example_t结构。Example_t包含一个时间戳，以允许邮箱中保存的数据记录上次更新邮箱的时间。本示例中使用的时间戳仅用于演示目的——邮箱可以保存应用程序编写者想要的任何数据，并且该数据不需要包含时间戳。*/
typedef struct xExampleStructure
{
    TickType_t xTimeStamp;
    uint32_t ulValue;
} Example_t ;

/*邮箱是一个队列，因此其句柄存储在QueueHandle_t类型的变量中
*/
QueueHandle_t xMailbox;

void vAFunction( void )
{

```



```
/*创建用作邮箱的队列。 队列的长度为 1，以便与 xQueueOverwrite() 应用程序接口函数配合使用，具体说明如下*/
```

```
    xMailbox = xQueueCreate( 1, sizeof( Example_t ) );
}
```

清单5.28 正在创建一个要用作邮箱的队列

5.7.1 xQueueOverwrite() API 函数

与 xQueueSendToBack() API 函数一样，xQueueOverwrite() API 函数也是向队列发送数据。与 xQueueSendToBack() 不同的是，如果队列已满，xQueueOverwrite() 会覆盖队列中已有的数据。

xQueueOverwrite() 只能用于长度为 1 的队列。覆盖模式将始终写入队列前部并更新队列前部指针，但不会更新等待中的报文。如果定义了 configASSERT，当队列长度大于 1 时，将出现断言

注意：永远不要从中断服务例程中调用xQueueOverwrite()。应该使用中断安全版本的xQueueOverwriteFromISR()来代替它。

```
 BaseType_t xQueueOverwrite( QueueHandle_t xQueue, const void * pvItemToQueue );
```

清单5.29 xQueueOverwrite() API 函数原型

参数和返回值

• xQueue

发送（写入）数据的队列句柄。队列句柄将通过调用 xQueueCreate() 或 xQueueCreateStatic() 返回，用于创建队列。

• pvItemToQueue

一个指向要复制到队列中的数据指针。

在创建队列时设置了队列可以保存的每个数据项的大小，因此这些字节将从pvItemToQueue复制到队列存储区域中。

• 返回值

即使队列已满，xQueueOverwrite() 也会写入队列，所以pdPASS是唯一可能的返回值。

清单5.30显示了如何使用xQueueOverwrite() 写入在清单5.28中创建的邮箱（队列）。

```

void vUpdateMailbox( uint32_t ulNewValue )
{
    在清单5.28中定义了/* Example_t。*/
    Example_t xData;

    /*: 将新的数据写入到Example_t结构中。*/
    xData.ulValue = ulNewValue;

    /*使用RTOS标记计数作为存储在Example_t结构中的时间戳。*/
    xData.xTimeStamp = xTaskGetTickCount();

    /*将结构发送到邮箱——覆盖邮箱中已经存在的任何数据。*/
    xQueueOverwrite ( xMailbox, &xData );
}

```

清单5.30使用xqueue覆盖() API函数

5.7.2 xQueuePeek() API函数

xQueuePeek() 从队列中接收（读取）一个项目，而不需要从队列中删除该项目。xQueuePeek() 从队列的头部接收数据，而不修改存储在队列中的数据或者存储在队列中的数据顺序。

注意：永远不要从中断服务例程调用xQueuePeek()。应该使用中断安全版本的xQueuePeekFromISR()来代替它。

xQueuePeek() 与xxQueueReceive() 具有相同的函数参数和返回值。

```

BaseType_t xQueuePeek ( QueueHandle_t xQueue,
                        void * const pvBuffer,
                        TickType_t xTicksToWait );

```

清单5.31xQueuePeek() API函数原型

清单5.32显示了xQueuePeek() 被用于接收发布到清单5.30中的邮箱（队列）的项目。

```

BaseType_t vReadMailBox (Example_t *pxData)
{
    TickType_t xPreviousTimeStamp;
    BaseType_t xDataUpdated;

    /*此函数使用从邮箱接收到的最新值更新Example_t结构。在它被新数据覆盖之前，
    记录已经包含的时间戳*pxData。*/

```

```
xPreviousTimeStamp = pxData->xTimeStamp;
```

/*使用邮箱中包含的数据更新pxData指向的Example_t结构。如果这里使用xQueueReceive（），那么邮箱将保留为空，任何其他任务都无法读取数据。使用xQueuePeek（）而不是xQueueReceive（）可以确保数据保留在邮箱中。

指定了阻塞时间，因此调用任务将处于“阻塞”状态，如果邮箱为空，则将等待邮箱包含数据。

使用了无期限的阻塞时间，因此没有必要检查从xQueuePeek（）返回的值，因为

xQueuePeek（）只会当数据可用时返回。*/

```
(xQueuePeek( xMailbox, pxData, portMAX_DELAY );
```

/*如果自上次调用此函数后，从邮箱读取的值已更新，则返回pdTRUE。否则返回pdFALSE。*/

```
if (pxData->xTimeStamp > xPreviousTimeStamp)
```

```
{
```

```
    xDataUpdated = pdTRUE;
```

```
}
```

```
else
```

```
{
```

```
    xDataUpdated = pdFALSE;
```

```
}
```

```
return xDataUpdated;
```

```
}
```

清单5.32 使用xQueuePeek（）API函数

6 软件定时器管理

6.1 章节介绍及适用范围

软件定时器用于在未来设定的时间，或以固定的频率定期安排功能的执行。由软件定时器执行的函数称为软件定时器的回调函数。

软件定时器由FreeRTOS内核实现，并在其控制之下。它们不需要硬件支持，也与硬件计时器或硬件计数器无关。

请注意，根据FreeRTOS使用创新设计以确保最大效率的理念，软件定时器不使用任何处理时间，除非软件定时器回调函数实际执行。

软件定时器功能是可选的。要包括软件定时器功能：

1. 构建FreeRTOS源文件FreeRTOS/Source/timers.c作为你的项目的一部分。
2. 定义下面在应用程序的FreeRTOSConfig.h中详细说明的常量：

•configUSE_TIMERS

在FreeRTOSConfig.h中将configUSE_TIMERS设置为1。

•configTIMER_TASK_PRIORITY

设置计时器服务任务的优先级为0到（configMAX_PRIORITIES - 1）。

■ configTIMER_QUEUE_LENGTH

设置定时器命令队列在任何时间都可以保存的最大未处理命令数。

•configTIMER_TASK_STACK_DEPTH

设置分配给计时器服务任务的堆栈的大小（用字(words)表示，而不是字节）。

6.1.1 范围

本章包括：

- 将软件定时器的特性与任务的特性相比较。
- RTOS守护进程任务。
- 计时器命令队列。
- 一次性软件定时器和周期性软件定时器之间的区别。
- 如何创建、启动、重置和更改一个软件定时器的时间周期。

6.2 软件定时器回调函数

时器回调函数以 C 语言函数的形式实现。它们唯一的特殊之处在于其原型必须返回 void，并将软件定时器的句柄作为唯一参数。清单 6.1 演示了回调函数原型。

```
void ATimerCallback( TimerHandle_t xTimer );
```

清单6.1软件定时器回调函数原型

软件定时器回调函数从开始到尾执行，并以正常方式退出。它们应保持简短，并且不能进入阻塞状态。

请注意：正如我们将看到的，软件定时器回调函数是在启动 FreeRTOS 调度器时自动创建的任务上下文中执行的。因此，软件定时器回调函数绝不能调用会导致调用任务进入阻塞状态的 FreeRTOS API 函数。可以调用 xQueueReceive() 等函数，但前提是函数的 xTicksToWait 参数（指定函数的阻塞时间）设置为 0。调用 vTaskDelay() 等函数是不行的，因为调用 vTaskDelay() 总是会将调用任务置入阻塞状态。

6.3 软件定时器的属性和状态

6.3.1 软件定时器的时间周期

软件定时器的“周期”是指从软件定时器被启动到软件定时器的回调函数被执行之间的时间。

6.3.2 一次性和自动重载的计时器

软件定时器有两种类型：

1. 一次性计时器

一旦启动，一次性定时器将只执行一次回调函数。单次定时器可以手动重启，但不会自动重启

2. 自动重载计时器

一旦启动，一个自动重载计时器将在每次过期时重新启动自己，从而导致定期执行其回调函数。

图6.1显示了一次性计时器和自动重新加载计时器之间的行为差异。虚线表示了滴答中断发生的时间。

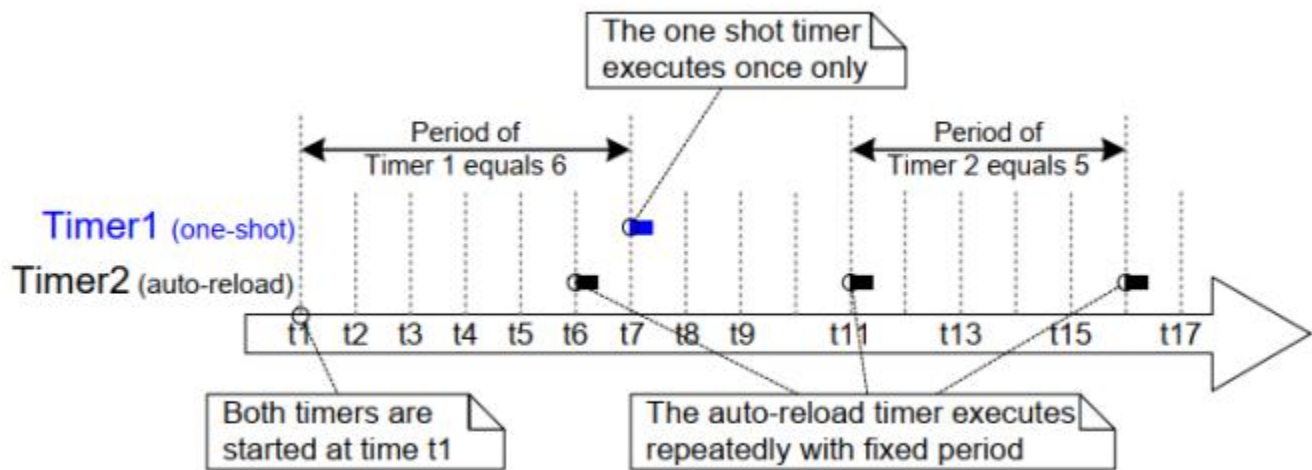


图6.1 一次性计时器和自动重载计时器之间的行为差异

参见图6.1:

•定时器1

计时器1是一个一次性计时器，周期为6个滴答(ticks)。它在时间 t_1 开始，所以它的回调函数在6个ticks后。也就是 t_7 执行。由于计时器1是一次性计时器，因此它的回调函数不会再次执行。

•定时器2

计时器2是一个自动重载的计时器，其周期为5个ticks。它在时间 t_1 开始，所以它的回调函数在时间 t_1 之后每5个ticks后执行一次。在图6.1中，这是在时间 t_6 、 t_{11} 和 t_{16} 。6

6.3.3 软件定时器状态

软件定时器可以处于以下两种状态之一：

•休眠状态

一个休眠的软件定时器，可通过其句柄进行引用，但并未运行，因此其回调函数不会执行

•运行

运行中的软件定时器将在软件定时器进入运行状态后或上次重置软件定时器后，经过与其周期相等的时间后执行其回调函数

图6.2和图6.3分别显示了自动重载计时器和一次计时器的休眠状态和运行状态之间的可能转换。这两个图之间的关键区别是在计时器过期后输入的状态；自动重新加载计时器执行其回调函数，然后重新进入运行状态，一次性计时器执行其回调函数，然后进入休眠状态。

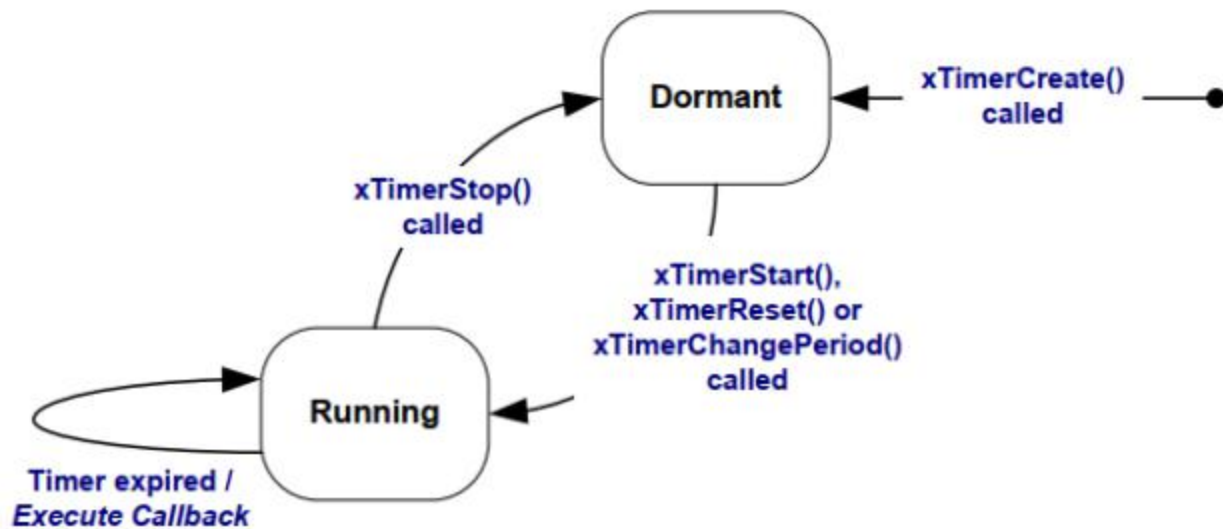


图6.2 自动重载软件定时器的状态和转换

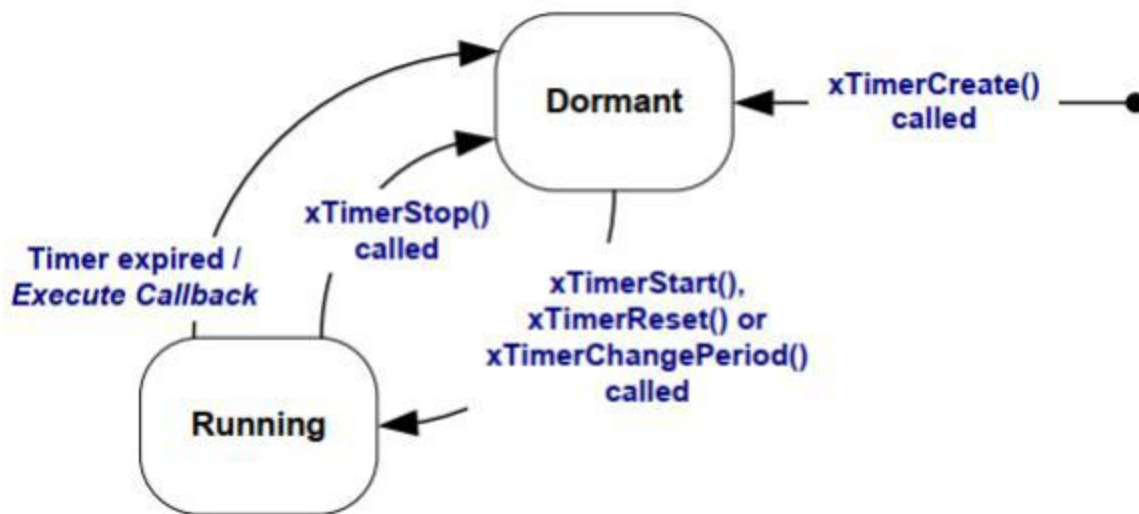


图6.3 一次性软件定时器的状态和转换

`xTimerDelete()` API函数将删除一个计时器。一个定时器可以随时被删除。该函数原型如清单6.2所示。

```
BaseType_t xTimerDelete ( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

清单6.2 `xTimerDelete()` API函数原型

参数和返回值

• `xTimer`

正在被删除的计时器的句柄。

- `xTicksToWait`

指定调用任务应保持在阻塞状态以等待删除命令成功发送到计时器命令队列的时间（以时钟周期为单位）（如果在调用 `xTimerDelete()` 时队列已满）。如果在调度程序启动之前调用 `xTimerDelete()`，则 `xTicksToWait` 将被忽略。

- 返回值

有两个可能的返回值：

- `pdPASS`

如果该命令已成功发送到定时器命令队列，则将返回 `pdPASS`。

- `pdFAIL`

当阻塞时间到达时，如果删除命令无法发送到计时器命令队列，则将返回 `pdFAIL`。

6.4 软件定时器的上下文

6.4.1 RTOS守护进程（计时器服务）任务

所有软件定时器回调函数都在相同的RTOS守护进程（或“定时器服务”）任务的上下文中执行^[^10]。

^[^10]：该任务过去被称为“计时器服务任务”，因为它最初只用于执行软件定时器回调函数。现在，同样的任务也被用于其他目的，因此它具有更通用的名称“RTOS守护进程任务”。

守护进程任务是一个标准的FreeRTOS任务，它在调度程序启动时自动创建。它的优先级和堆栈大小分别由 `configTIMER_TASK_PRIORITY` 和 `configTIMER_TASK_STACK_DEPTH` 编译时配置常量设置。这两个常量都是在 `FreeRTOSConfig.h` 中定义的。

软件定时器回调函数不能调用将导致调用任务进入阻塞状态的FreeRTOS函数API，因为这样做将导致守护进程任务进入阻塞状态。

6.4.2 计时器命令队列

软件定时器API 函数将命令从调用任务发送到称为“计时器命令队列”的队列上，命令由守护程序任务接收。如图 6.4 所示。命令的示例包括“启动计时器”、“停止计时器”和“重置计时器”。

计时器命令队列是一个标准的FreeRTOS队列，它在调度程序启动时自动创建。计时器命令队列的长度由 `FreeRTOSConfig.h` 中的 `configTIMER_QUEUE_LENGTH` 编译时时间配置常量。

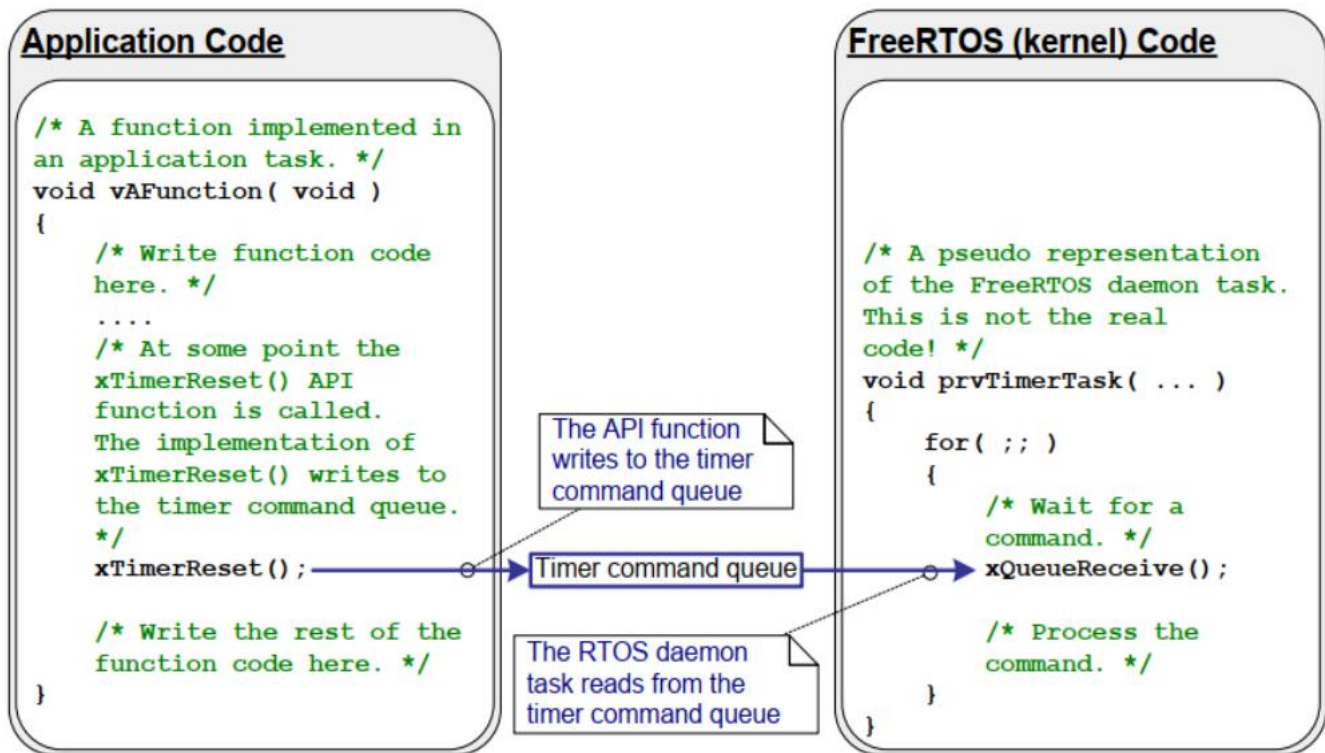


图6.4 软件定时器API函数使用的与RTOS守护进程任务通信的计时器命令队列

6.4.3 守护进程任务调度

守护进程任务像任何其他FreeRTOS任务一样被调度；当它是能够运行的最高优先级任务时，它只会处理命令，或执行计时器回调函数。图6.5和图6.6演示了`configTIMER_TASK_PRIORITY`设置如何影响执行模式。

图6.5显示了当守护进程任务的优先级低于调用`xTimerStart()` API函数的任务的优先级时的执行模式。

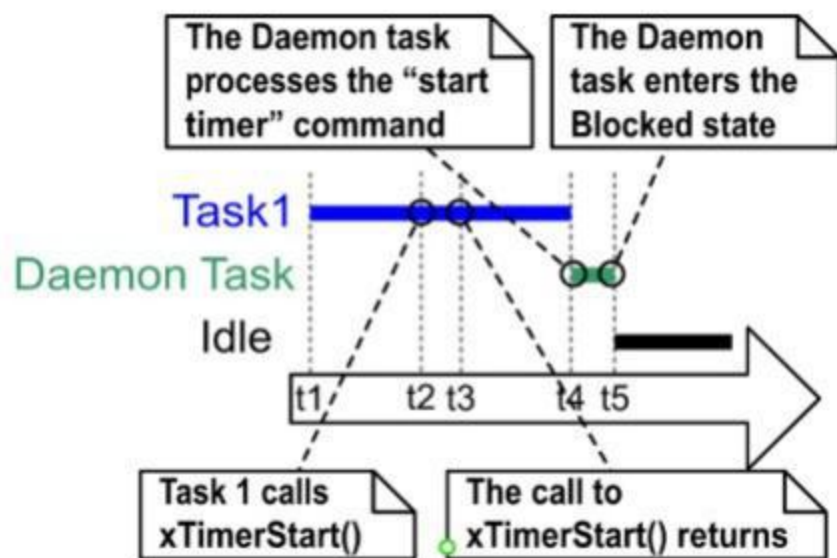


图6.5当调用`xTimerStart()`的任务的优先级高于守护进程任务的优先级时的执行模式

参见图6.5，其中Task 1的优先级高于守护任务的优先级，守护任务的优先级高于空闲任务的优先级：

1. 在时间t1

任务1处于“运行”状态，守护进程任务处于“阻塞”状态。

如果一个命令被发送到计时器命令队列，守护进程任务将离开阻塞状态，在这种情况下，它将处理该命令，或者如果一个软件定时器过期，在这种情况下，它将执行软件定时器的回调函数。

2. 在时间t2

任务1调用`xTimerStart()`。

`xTimerStart()`向定时器命令队列发送命令，导致守护进程任务离开“阻塞”状态。任务1的优先级高于守护进程任务的优先级，因此守护进程任务不会抢占任务1。

任务1仍处于“运行”状态，守护进程任务已离开“阻塞”状态并进入“就绪”状态。

3. 在时间t3

任务1完成`xTimerStart()` API函数的执行。任务1从该函数开始到结束执行了`xTimerStart()`，没有离开Running状态

4. 在时间t4

Task 1 调用一个API函数，导致它进入阻塞状态。守护进程任务现在是处于已就绪状态下的最高优先级任务，因此调度程序选择守护进程任务作为进入正在运行状态的任务。然后，守护进程任务开始处理由任务1发送到计时器命令队列的命令。

注意：启动软件定时器将过期的时间是从“启动计时器”命令发送到计时器命令队列开始计算的——它不是从守护进程任务从计时器命令队列收到“启动计时器”命令开始计算的。

5. 在时间t5

守护进程任务已经完成了由任务1发送给它的命令的处理，并尝试从计时器命令队列接收更多的数据。计时器命令队列为空，因此守护进程任务重新进入“阻塞”状态。如果命令发送到定时器命令队列，或者软件定时器过期，守护进程任务将再次离开“阻塞”状态。

空闲任务现在是处于“就绪”状态下的最高优先级任务，因此调度程序选择空闲任务作为进入“正在运行”状态的任务。

图6.6显示了与图6.5所示的类似的场景，但是这次守护进程任务的优先级高于调用xTimerStart（）的任务的优先级。

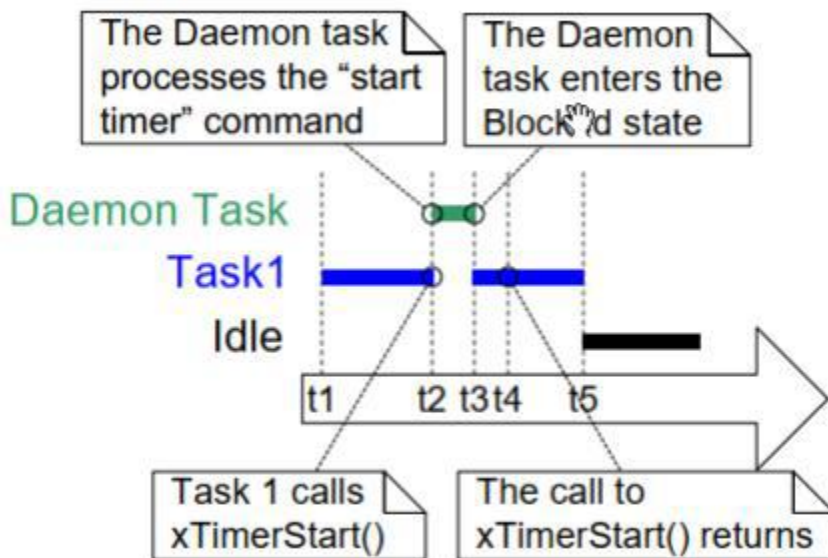


图6.6当调用xTimerStart（）的任务的优先级低于守护进程任务的优先级时的执行模式

参见图6.6，其中守护进程任务的优先级高于任务1的优先级，任务1的优先级高于空闲任务的优先级：

1. 在时间t1

与前面一样，任务1处于“运行”状态，守护进程任务处于“阻塞”状态。

2. 在时间t2

任务1调用xTimerStart（）。

xTimerStart（）向定时器命令队列发送命令，导致守护进程任务离开“阻塞”状态。守护进程任务的优先级高于任务1的优先级，因此调度程序选择守护进程任务作为进入正在运行的状态的任务。

任务1在完成执行xTimerStart（）函数执行之前被守护进程任务抢占，现在处于“就绪”状态。

守护进程任务开始处理由任务1发送到计时器命令队列的命令。

3. 在时间t3

守护进程任务已经完成了由任务1发送给它的命令的处理，并尝试从计时器命令队列接收更多的数据。定时器命令队列为空，因此守护进程任务重新进入“阻塞”状态。

任务1现在是处于“已就绪”状态下的最高优先级任务，因此调度程序选择任务1作为要进入“正在运行”状态的任务。

4. 在时间t4

任务1在完成执行xTimerStart（）函数之前已经被守护程序任务抢占，并且只在重新进入运行状态后退出（返回）xTimerStart（）。

5. 在时间t5

Task 1调用一个API函数，导致它进入阻塞状态。空闲任务现在是处于“就绪”状态下的最高优先级任务，因此调度程序选择空闲任务作为进入“正在运行”状态的任务。

在图6.5所示的场景中，从Task 1向计时器命令队列发送一个命令到守护进程任务接收和处理该命令之间传递的时间。在图6.6所示的场景中，守护进程任务在任务1从发送该命令的函数返回之前，已经收到并处理了任务1发送给它的命令。

发送到计时器命令队列的命令包含一个时间戳。时间戳用于说明在应用程序任务正在发送的命令和守护进程任务正在处理的同一命令之间传递的任何时间。例如，如果发送一个“启动计时器”命令来启动一个周期为10次的计时器，则时间戳用于确保正在启动的计时器在发送命令后过期10次，而不是在守护进程任务处理命令后过期10次。

6.5 创建和启动一个软件定时器

6.5.1 xTimerCreate() API函数

FreeRTOS还包括xxTimerCreateStatic()函数，它分配在编译时静态创建计时器所需的内存：软件定时器必须显式创建才能使用。

软件定时器由TimerHandle_t类型的变量引用。xTimerCreate()用于创建一个软件定时器，并返回一个TimerHandle_t来引用它创建的软件定时器。软件定时器是它在休眠状态创建的。

软件定时器可以在调度程序运行之前创建，也可以在调度程序启动后从任务中创建。

第2.5节：数据类型和编码样式指南描述了所使用的数据类型和命名约定。

```
TimerHandle_t xTimerCreate( const char * const pcTimerName,
                           const TickType_t xTimerPeriodInTicks,
                           const BaseType_t xAutoReload,
                           void * const pvTimerID,
                           TimerCallbackFunction_t pxCallbackFunction );
```

清单6.3 xTimerCreate () API函数原型

参数和返回值

•pcTimerName

计时器的一个描述性名称。FreeRTOS没有以任何方式使用它。它纯粹作为调试辅助工具。用人类可读的名称来识别一个计时器比试图用其句柄来识别它要简单得多。

•xTimerPeriodInTicks

用刻度符指定的计时器的周期。pdMS_TO_TICKS ()宏可用于将以毫秒为单位指定的时间转换为在刻度中指定的时间。不能为0。

•xAutoReload

将x自动重新加载设置为pdTRUE，以创建一个自动重新加载计时器。将xAuto重新加载设置为pdFALSE以创建一个热点计时器。

•pvTimerID

每个软件定时器都有一个ID值。ID是一个空指针，应用程序编写器可以用于任何目的。当相同的回调函数被多个软件定时器使用时，该ID特别有用，因为它可以用于提供特定于计时器的存储。在本章中的一个示例中演示了计时器ID的使用。

pvTimerID为正在创建的任务的ID设置一个初始值。

pxCallbackFunction

软件定时器回调函数只是符合清单 6.1 所示原型的 C 函数。 pxCallbackFunction参数是指向函数（实际

- 上只是函数名）的指针，以用作正在创建的软件定时器的回调函数。

■ 返回值

如果返回NULL，则无法创建软件定时器，因为FreeRTOS没有足够的堆内存来分配必要的数据结构。

如果返回非NULL值，则表示软件定时器已成功创建。返回的值是已创建的计时器的句柄。

第3章提供了关于堆内存管理的更多信息。

6.5.2 xTimerStart () API函数

xTimerStart () 用于启动处于休眠状态的软件定时器，或重置（重新启动）处于正在运行状态的软件定时器。

xTimerStop () 用于停止处于“正在运行”状态的软件定时器。停止软件定时器就等于将计时器转换为休眠状态。

可以在调度器启动之前调用xTimerStart ()，但就算这样做，软件定时器直到调度器启动时才会实际启动。

注意：永远不要从中断服务例程调用xTimerStart ()。应该使用中断安全版本xTimerStartFromISR ()。

```
BaseType_t xTimerStart ( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

清单6.4 xTimerStart () API函数原型

参数和返回值

• xTimer

正在启动或重置的软件定时器的句柄。该句柄从调用xTimerCreate () 函数时返回。

• xTicksToWait

xTimerStart () 使用定时器命令队列向守护进程任务发送“启动定时器”命令。

xTicksToWait 指定了调用任务在定时器命令队列已满的情况下，为等待队列中出现可用空间而保持阻塞状态的最长时间。

如果 xTicksToWait 为零，且定时器命令队列已满，则 xTimerStart () 将立即返回。

阻塞时间以滴答周期指定，因此它所表示的绝对时间取决于滴答频率。宏pdMS_TO_TICKS () 可用于将毫秒为单位指定的时间转换为滴答为单位的时间。

如果INCLUDE_vTaskSuspend在FreeRTOSConfig.h中被设置为1。然后，将xTicksToWait设置为portMAX_DELAY。如果在FreeRTOSConfig.h中将INCLUDE_vTaskSuspend设置为1，那么将xTicksToWait设置为portMAX_DELAY，将导致调用任务无限期地（无超时）停留在阻塞状态，以等待定时器命令队列中出现可用空间。

•返回值

有两个可能的返回值：

◦pdPASS

只有当“启动计时器”命令被成功地发送到计时器命令队列时，才会返回pdPASS。

如果守护进程任务的优先级高于调用xTimerStart()的任务的优先级，那么调度程序将确保在xTimerStart()返回之前处理启动命令。这是因为一旦定时器命令队列中有数据，守护进程任务就会抢先执行调用xTimerStart()的任务。

如果指定了阻塞时间（xTicksToWait不为零），则调用任务有可能进入阻塞状态，以等待定时器命令队列中出现可用空间后再返回函数，但在阻塞时间到期前，数据已成功写入定时器命令队列。

◦pdFAIL

如果“启动计时器”命令无法写入计时器命令队列，因为该队列已满，则将返回pdFAIL。

如果指定了阻塞时间（xTicksToWait不是零），那么调用任务将被放置入阻塞状态，等待守

护进

程任务在计时器命令队列中腾出空间，但指定的阻塞时间在发生之前已经过期。

示例6.1 创建一次性和自动重载计时器

此示例创建并启动一次性计时器和自动重载计时器——如清单6.5所示。

```
/*分配给一次性和自动重载计时器的周期分别是3.33秒和半秒。*/
#define mainONE_SHOT_TIMER_PERIOD pdMS_TO_TICKS( 3333 )
#define mainAUTO_RELOAD_TIMER_PERIOD pdMS_TO_TICKS( 500 )

int main(void)
{
    TimerHandle_t xAutoReloadTimer, xOneShotTimer;
```



```

BaseType_t xTimer1Started, xTimer2Started;

/*创建一个一次性计时器，将句柄存储到已创建的计时器xOneShotTimer中
*/
xOneShotTimer = xTimerCreate(
    /*软件定时器的名称-FreeRTOS并不使用。*/
    "OneShot",
    /*软件定时器的周期。*/
    mainONE_SHOT_TIMER_PERIOD ,
    /*设置uxAutoReload为pdFALSE, 来创建一个一次性软件定时器。*/
    pdFALSE,
    /*这个例子不使用计时器id。*/
    0,
    /*回调功能，供被创建的软件定时器使用。*/ prvOneShotTimerCallback );

/*创建自动重载计时器，并将句柄存储到已创建的计时器中
*/
xAutoReloadTimer = xTimerCreate(
    /*软件定时器的名称-FreeRTOS并不使用。*/
    "AutoReload",
    /*软件定时器的周期。*/
    mainAUTO_RELOAD_TIMER_PERIOD ,
    /*设置uxAutoReload为pdTRUE, 来创建一个自动重新加载计时器。*/ pdTRUE,
    /*这个例子不使用计时器id。*/
    0,
    /*回调功能，供被创建的软件定时器使用。*/ prvAutoReloadTimerCallback );

/*检查软件定时器是否创建。*/
if( ( xOneShotTimer != NULL ) && ( xAutoReloadTimer != NULL ) )
{
    /*使用阻塞时间为0，启动软件定时器。调度程序还没有启动，因此在这里指定
    的任何阻塞时间都将被忽略。*/
    xTimer1Started = xTimerStart ( xOneShotTimer, 0 );
    xTimer2Started = xTimerStart ( xAutoReloadTimer, 0 );

    /*xTimerStart() 的实现使用定时器命令队列，如果定时器命令队列已满，xTimerStart() 将失效。
    在调度程序启动之前，定时器服务任务不会被创建，
    因此所有发送到命令队列的命令都会留在队列中，直到调度程序启动。
    检查对 xTimerStart() 的两次调用是否都通过。。
    */

    if( ( xTimer1Started == pdPASS ) && ( xTimer2Started == pdPASS ) )
    {
        /*启动调度程序。*/
        vTaskStartScheduler();
    }
}

```



```

    }

    /*一如既往，不应该到达这一行。*/
    for(;;)
}

```

清单6.5: 创建和启动示例6.1中使用的计时器

定时器的回调函数每次被调用时都会打印一条信息。单次定时器回调函数的实现如清单 6.6 所示。自动重新加载定时器回调函数的实现如清单 6.7 所示

```

static void prvOneShotTimerCallback( TimerHandle_t xTimer )
{
    TickType_t xTimeNow;

    /*获取当前的滴答计数。*/
    xTimeNow = xTaskGetTickCount();

    /*输出一个字符串来显示执行回调的时间。*/

    vPrintStringAndNumber( "One-shot timer callback executing", xTimeNow );

    /*文件范围变量。*/
}    ulCallCount++;

```

清单6.6 示例6.1中的一次性计时器所使用的回调函数

```

static void prvAutoReloadTimerCallback (TimerHandle_t xTimer)
{
    TickType_t xTimeNow;

    /*获取当前的滴答计数。*/
    xTimeNow = xTaskGetTickCount();

    /*输出一个字符串来显示执行回调的时间。*/

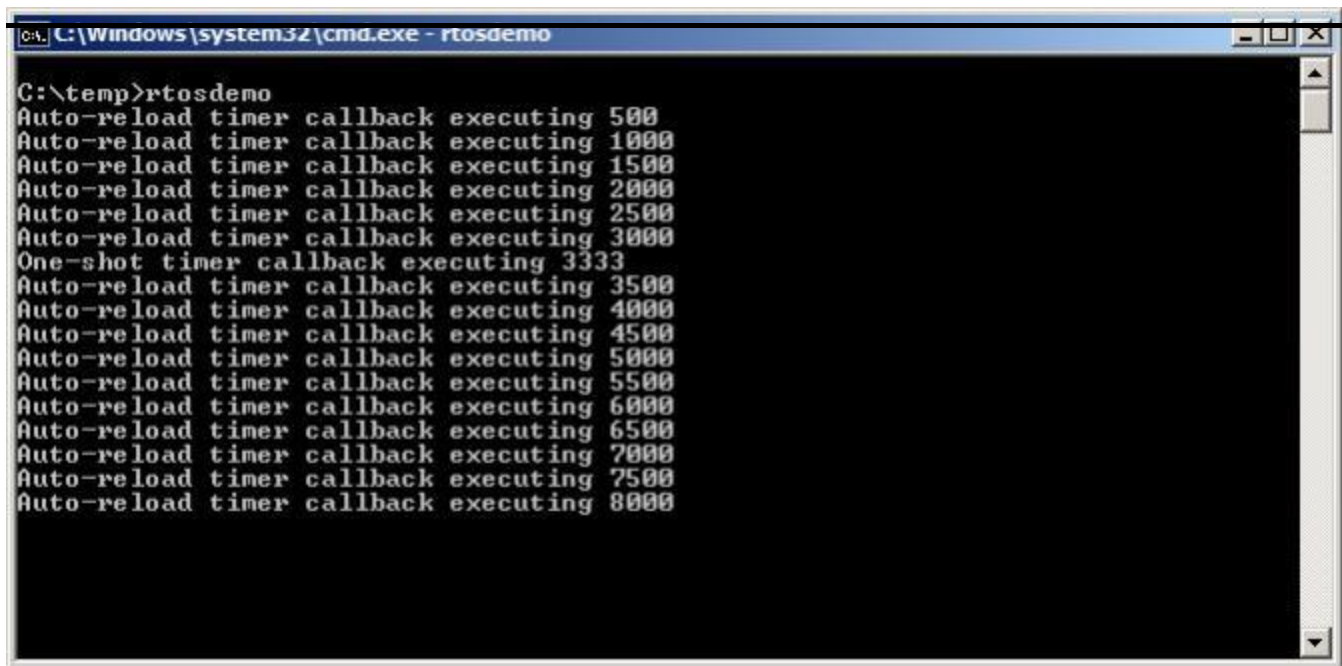
    vPrintStringAndNumber( "Auto-reload timer callback executing", xTimeNow);

    ulCallCount++;
}

```

清单6.7 示例6.1中的自动重载计时器所使用的回调函数

执行此示例所生成的输出如图6.7所示。图6.7显示了自动重载计时器的回调函数，执行周期为500滴答（清单6.5中mainAUTO_RELOAD_TIMER_PERIOD设置为500），而一次性计时器回调函数仅在滴答为3333时执行一次。（清单6.5中mainONE_SHOT_TIMER_PERIOD设置为3333）。



```

C:\temp>rtosdemo
Auto-reload timer callback executing 500
Auto-reload timer callback executing 1000
Auto-reload timer callback executing 1500
Auto-reload timer callback executing 2000
Auto-reload timer callback executing 2500
Auto-reload timer callback executing 3000
One-shot timer callback executing 3333
Auto-reload timer callback executing 3500
Auto-reload timer callback executing 4000
Auto-reload timer callback executing 4500
Auto-reload timer callback executing 5000
Auto-reload timer callback executing 5500
Auto-reload timer callback executing 6000
Auto-reload timer callback executing 6500
Auto-reload timer callback executing 7000
Auto-reload timer callback executing 7500
Auto-reload timer callback executing 8000

```

图6.7 执行示例6.1时产生的输出

6.6 计时器ID

每个软件定时器都有一个ID，它是应用程序编写器可以用于任何目的的标记值。ID存储在一个空指针（void *）中，因此它可以直接存储一个整数值，指向任何其他对象，或用作函数指针。

在创建软件定时器时，为ID分配一个初始值，之后可以使用vTimerSetTimerID（）API函数更新ID，并使用pvTimerGetTimerID（）API函数进行查询。

与其他软件定时器API功能不同，vTimerSetTimerID（）和pvTimerGetTimerID（）直接访问软件定时器——它们不会向计时器命令队列发送命令。

6.6.1 vTimerSetTimerID（）API函数

```
(const TimerHandle_t xTimer, 空白*pvNewID);
```

清单6.8 vTimerSetTimerID（）API函数原型

vTimerSetTimerID（）参数

- xTimer

正在使用新的ID值更新的软件定时器的句柄。该句柄将从调用xTimerCreate（）时返回，用于创建软件定时器。

- pvNewID

将设置软件定时器ID的值。

6.6.2 pvTimerGetTimerID（）API函数

```
void pvTimerGetTimerID (const TimerHandl_t xTimer);
```

清单6.9pvTimerGetTimerID（）API函数原型

参数和返回值

- xTimer

正在查询的软件定时器的句柄。该句柄从调用xTimerCreate（）时返回。

- 返回值

正在查询的软件定时器的ID。

示例6.2 使用回调函数参数和软件定时器

ID

相同的回调函数可以分配给多个软件定时器。完成后，回调函数参数用于确定哪个软件定时器已过期。

例 6.1 使用了两个不同的回调函数； 其中一个回调函数用于单发定时器，另一个回调函数用于自动重载定时器。 例 6.2 创建了与例 6.1 类似的功能，但为两个软件定时器分配了一个回调函数

例 6.2 中使用的 main（） 函数与例 6.1 中使用的 main（） 函数几乎完全相同。 唯一的区别在于软件定时器是在哪里创建的。 清单 6.10 显示了这一区别，其中 prvTimerCallback（） 被用作两个定时器的回调函数

```
/*创建一次性计时器，句柄存储在
   xOneShotTimer.*/
xOneShotTimer = xTimerCreate ( "OneShot",
                                mainONE_SHOT_TIMER_PERIOD ,
                                pdFALSE,
                                /*计时器的ID被初始化为NULL。*/
                                NULL,
```

```

/*两个计时器都使用的计时器回调。*/
prvTimerCallback );

/*创建自动重载软件定时器，并将句柄存储在中
xAutoReloadTimer */
xAutoReloadTimer = xTimerCreate ( "AutoReload",
                                   mainAUTO_RELOAD_TIMER_PERIOD ,
                                   pdTRUE,
                                   /*计时器的ID被初始化为NULL。*/
                                   NULL,
                                   prvTimerCallback );

```

清单6.10 创建示例6.2中使用的计时器

`prvTimerCallback()` 会在任一计时器到期时执行。`prvTimerCallback()` 的实现会使用函数参数来确定调用该函数是因为一次性定时器，还是因为自动重载定时器过期。

`prvTimerCallback()` 还演示了如何使用软件定时器 ID 作为定时器特定存储：每个软件定时器都会在自己的 ID 中对过期次数进行计数，自动重载定时器会在执行到第五次时利用计数停止自己的运行

`prvTimerCallback()` 实现如清单6.9所示。

```

static void prvTimerCallback( TimerHandle_t xTimer )
{
    TickType_t xTimeNow;
    uint32_t ulExecutionCount;

    /*该软件定时器过期次数的计数存储在定时器 ID 中。 获取 ID，将其递增，然后保存为新的 ID 值。
    ID 是一个 void 指针，因此会被转换为 uint32_t。*/
    ulExecutionCount = (uint32_t) pvTimerGetTimerID ( xTimer );
    ulExecutionCount++;
    vTimerSetTimerID ( xTimer, (void*) ulExecutionCount );

    /*获取当前的滴答计数。*/
    xTimeNow = xTaskGetTickCount();

    /*一次性计时器的句柄在创建计时器时存储在xOneShotTimer中。将传递到此函数中的句柄与
    xOneShotTimer进行比较，以确定到期的是一次性的还是自动重载的计时器，然后
    输出一个字符串来显示回调执行的时间。*/
    if (xTimer==xOneShotTimer)
    {
        vPrintStringAndNumber( "One-shot timer callback executing", xTimeNow );
    }
}

```

```

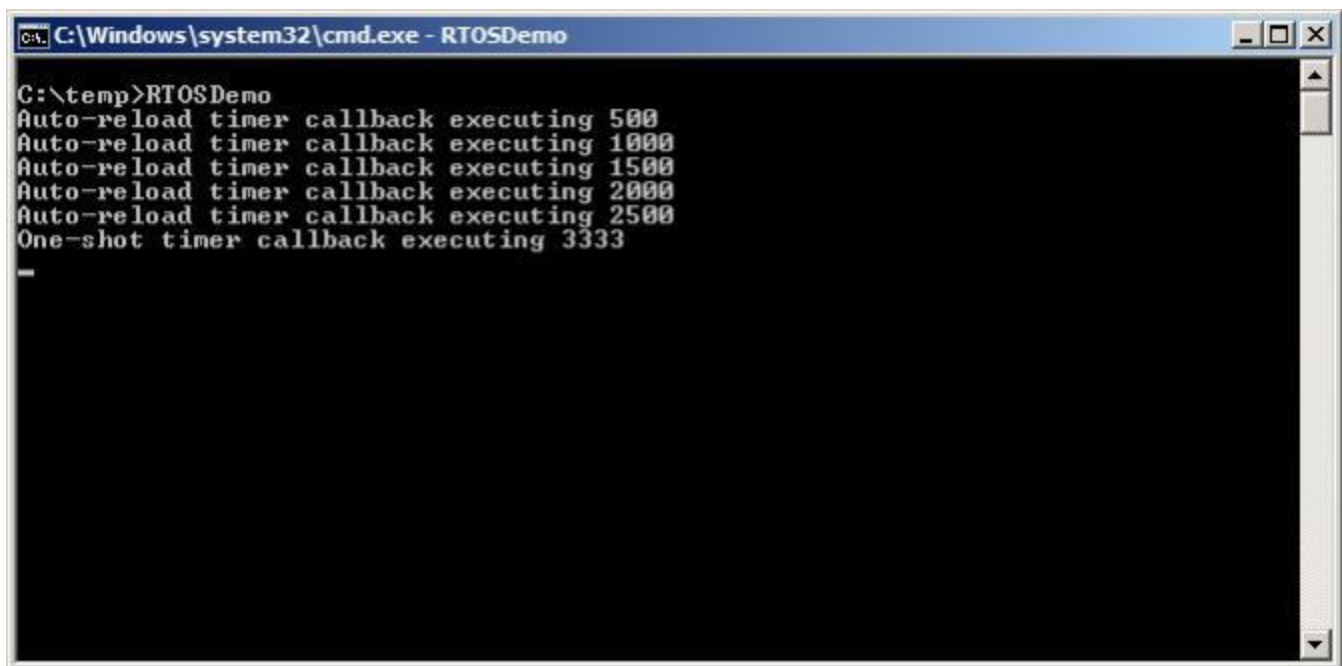
    }
    else
    {
        /* xTimer不等于xOneShotTimer, 所以它一定是到期的自动重载计时器。*/
        vPrintStringAndNumber( "Auto-reload timer callback executing", xTimeNow);

        if( ulExecutionCount == 5 ){
            /*在执行5次后停止自动重载计时器。此回调函数在RTOS守护进程任务的上下文
            文中执行, 因此不能调用任何可能将守护进程任务置于被阻塞状态的函数。因
            此, 使用的阻塞时间为0。*/
            xTimerStop ( xTimer, 0 );
        }
    }
}

```

清单6. 11 示例6. 2中使用的计时器回调函数

示例6. 2所产生的输出如图6. 8所示。可以看出，自动重载计时器只执行5次。



```

C:\Windows\system32\cmd.exe - RTOSDemo
C:\temp>RTOSDemo
Auto-reload timer callback executing 500
Auto-reload timer callback executing 1000
Auto-reload timer callback executing 1500
Auto-reload timer callback executing 2000
Auto-reload timer callback executing 2500
One-shot timer callback executing 3333

```

图6. 8 执行示例6. 2时产生的输出

6. 7 更改计时器的时间周期

每个正式的 FreeRTOS 移植都会提供一个或多个示例项目。大多数示例项目都具有自检功能，并使用 LED 指示灯对项目状态进行可视化反馈；如果自检一直通过，LED 指示灯就会缓慢闪烁；如果自检失败，LED 指示灯就会快速闪烁。

一些示例项目在任务中执行自检，并使用 `vTaskDelay()` 函数控制 LED 的切换速度。其他示例项目在软件定时器回调函数中执行自检，并使用定时器的周期来控制 LED 的切换速度。

6.7.1 xTimerChangePeriod() API 函数

使用 `xTimerChangePeriod()` 更改软件定时器的周期。

如果使用 `xTimerChangePeriod()` 来更改已经运行的计时器的周期，则计时器将使用新的周期值来重新计算其到期时间。重新计算的过期时间相对于调用 `xTimerChangePeriod()` 的时间，而不是相对于计时器最初启动的时间。

如果使用 `xTimerChangePeriod()` 来更改处于休眠状态（未运行的定时器）的定时器周期，则定时器将计算到期时间，并过渡到运行状态（定时器将开始运行）。

*注意：不要从中断服务例程调用 `xTimerChangePeriod()`。
应该使用中断安全版本的 `xTimerChangePeriodFromISR()` 来代替它。*

```
BaseType_t xTimerChangePeriod ( TimerHandle_t xTimer,
                                TickType_t xNewPeriod,
                                TickType_t xTicksToWait );
```

清单 6.12 `xTimerChangePeriod()` API 函数原型

参数和返回值

- `xTimer`

正在用一个新的周期值更新的软件定时器的句柄。该句柄从调用 `xTimerCreate()` 时返回。

- `xTimerPeriodInTicks`

软件定时器的新周期，以滴答指定。`pdMS_TO_TICKS()` 宏可用于将以毫秒为单位指定的时间转换为在滴答中指定的时间。

- `xTicksToWait`

`xTimerChangePeriod()` 使用定时器命令队列向守护进程任务发送“更改周期”命令。`xTicksToWait` 指定了调用任务在定时器命令队列已满的情况下，为等待队列中出现空位而保持阻塞状态的最长时间。

如果xTicksToWait为零且计时器命令队列已满，则函数将立即返回。

宏pdMS_TO_TICKS () 可用于将以毫秒为单位指定的时间转换为在滴答中指定的时间。

如果INCLUDE_vTaskSuspend在FreeRTOSConfig.h中被设置为1。然后将xTicksToWait设置为portMAX_DELAY将导致调用任务无限期地处于阻塞状态（没有超时），等待计时器命令队列中的空间可用。

如果在调度程序启动之前调用该函数，则忽略xTicksToWait，并且xTimerChangePeriod()的行为类似xTicksToWait被设置为零。

•返回值

有两个可能的返回值：

◦pdPASS

只有当数据被成功地发送到计时器命令队列时，才会返回pdPASS。

如果指定了阻塞时间（xTicksToWait 不为零），则调用任务有可能进入阻塞状态，以等待计时器命令队列中出现可用空间后再返回函数，但在阻塞时间到期前，数据已成功写入定时器命令队列。

◦pdFAIL

如果无法将“更改期间”命令写入计时器命令队列，因为该队列已满，则将返回pdFAIL。

如果指定了阻塞时间（xTicksToWait 不为零），则调用任务将进入阻塞状态，等待守护进程任务在队列中腾出空间，但在此之前指定的阻塞时间已过期

清单6.13显示了在软件定时器回调函数中包含自检功能的FreeRTOS示例里，如何使用xTimerChangePeriod () 来提高在自检失败时LED切换的速率。执行自检的软件定时器被称为“检查计时器”。

```
/*检查计时器的创建时间为3000毫秒，导致LED每3秒切换一次。如果自检功能检测到一个意外的状态，那么检查计时器的周期将被更改为只有200毫秒，从而产生更快的切换速率。*/
const TickType_t xHealthyTimerPeriod = pdMS_TO_TICKS( 3000 );
const TickType_t xErrorTimerPeriod = pdMS_TO_TICKS( 200 );

/*检查计时器使用的回调函数。*/
static void prvCheckTimerCallbackFunction (TimerHandle_t xTimer)
```

```

{
    static BaseType_t xErrorDetected = pdFALSE;

    if (xErrorDetected == pdFALSE)
    {
        /*尚未检测到任何错误。再次运行自检函数。该函数要求示例创建的每个任务报告自己
           的状态，并检查所有任务实际上仍在运行（以便能够正确报告它们的状态）。*/
        if( CheckTasksAreRunningWithoutError() == pdFAIL )
        {
            /* 一个或多个任务报告了意外状态。可能已经发生了一个错误。减少检查计时器的
               周期，以增加这个回调函数执行的速率，这样做也会增加了LED切换的速率。此
               回调函数在RTOS守护进程任务的上下文中执行，因此设置阻塞时间为0用来确
               保守护进程任务永远不会进入“阻塞”状态。*/
            xTimerChangePeriod(
                xTimer, /*要更新的定时器*/
                xErrorTimerPeriod,
                0);/*在发送此命令时不要阻塞*/
        }

        /*锁定已检测到的错误。*/
        xErrorDetected = pdTRUE;
    }

    /* 切换LED。LED切换的速率将取决于这个函数被调用的频率，这是由检查计时器的周期决定
       的。如果 CheckTasksAreRunningWithoutError() 返回 pdFAIL，定时器的周期将从
       3000ms 缩短到 200ms。*/
    ToggleLED();
}

```

清单6.13 使用xTimerChangePeriod()

6.8 重置一个软件定时器

重置软件定时器意味着重新启动计时器；计时器的到期时间被重新计算为相对于重置计时器的时间，而不是计时器最初启动的时间。图6.9演示了这一点，它显示了一个启动周期为6的计时器，然后重置两次，然后最终到期并执行其回调函数。

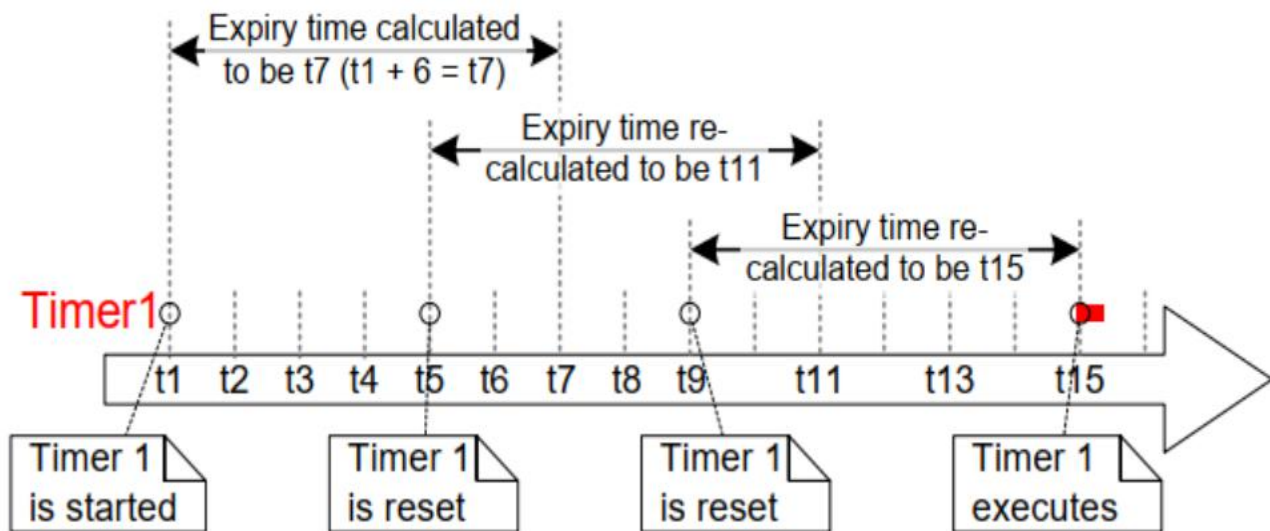


图6.9 启动和重置一个周期为6个滴答的软件定时器

参见图6.9:

- 计时器1在时间t1开始。它的周期为6，所以它执行回调函数的时间最初计算为t7，即它启动后的6个刻度(tick)。
- 定时器 1 在时间 t7 到达之前被重置，因此它还没有过期和执行回调函数。定时器 1 在 t5 时被重置，因此它执行回调函数的时间被重新计算为 t11，即重置后的 6 个滴答
- 定时器 1 在时间 t11 之前再次重置，因此又是在过期和执行回调函数之前。
定时器 1 在 t9 时被重置，因此它执行回调函数的时间被重新计算为 t15，即上次重置后的 6 个刻度。
- 定时器1不会再次重置，所以它在时间t15到期，并相应地执行其回调函数。

6.8.1 xTimerReset () API函数

使用xTimerReset () API功能重置计时器。

xTimerReset () 也可以用来启动一个处于休眠状态的计时器。

*注意：永远不要从中断服务例程调用xTimerReset () 。
应该使用中断安全版本的xTimerResetFromISR () 来代替它。*

```
BaseType_t xTimerReset ( TimerHandle_t xTimer, TickType_t xTicksToWait );
```

清单6.14 xTimerReset () API函数原型

参数和返回值

•xTimer

正在重置或正在启动的软件定时器的句柄。该句柄从调用xTimerCreate（）时返回

- 该函数使用计时器命令队列将“重置”命令发送给守护进程任务。xTicksToWait指定调用任务已满时，保持处于阻塞状态等待计时器命令队列上空间可用的最大时间。

如果xTicksToWait为零，并且计时器命令队列已经满，则函数将立即返回。

如果INCLUDE_vTaskSuspend在FreeRTOSConfig.h中被设置为1。然后，将xTicksToWait设置为portMAX_DELAY将导致调用任务无限期地处于阻塞状态（没有超时），以等待计时器命令队列中的空间可用。

•返回值

有两个可能的返回值：

pdPASS

只有当数据被成功地发送到计时器命令队列时，才会返回pdPASS。

如果指定了阻塞时间（xTicksToWait 不为零），则调用任务有可能进入阻塞状态，以等待计时器命令队列中出现可用空间后再返回函数，但数据已在阻塞时间到期前成功写入定时器命令队列。如果无法将“reset”命令写入计时器命令队列，因为该队列已满，则将返回pdFAIL。

pdFAIL

如果指定了阻塞时间（xTicksToWait不是零），那么调用任务将被放入“阻塞”状态，等待守护进程任务在队列中腾出空间，但指定的阻塞时间在此发生之前已过期。

示例6.3 重置一个软件定时器

本示例模拟了手机上的背光灯的行为。背光：

-
- 当按下一个键时，它将被打开。
 - 在一定的时间段内继续按下按键将保持。
 - 如果在一定时间段内没有按键，则自动关闭。

一个一次性的软件定时器用于实现此行为：

- 当按下一个按键时，[模拟的]背光就会被打开，并在软件定时器的回调功能中被关闭。

每次按下一个键时，软件定时器将被重置。

■

因此，为防止背光灯熄灭而必须按下按键的时间等于软件定时器的周期； 如果在定时器到期前没有按键

- 重置软件定时器，则执行定时器的回调功能，并关闭背光灯

`xxSimulatedBacklightOn` 变量用于保存背光状态。 `xSimulatedBacklightOn` 设置为 `pdTRUE` 表示背光打开，`pdFALSE` 表示背光关闭。

软件定时器回调函数如清单6.15所示。

```
static void prvBacklightTimerCallback( TimerHandle_t xTimer )
{
    TickType_t xTimeNow = xTaskGetTickCount();

    /*背光计时器已过期，关闭背光。*/
    xSimulatedBacklightOn = pdFALSE;

    /*打印背光被关闭的时间。*/
    vPrintStringAndNumber(
        "Timer expired, turning backlight OFF at time\t\t", xTimeNow);}
```

清单6.15 示例6.3中使用的一次性计时器的回调函数

示例6.3创建了一个任务来轮询键盘[¹¹]。该任务如清单6所示。但是由于下一段中描述的原因，清单6.16并不代表一个最优设计。

[¹¹]：打印到窗口控制台，以及从窗口控制台读取键，都会导致执行Windows系统调用。Windows系统调用，包括使用Windows控制台、磁盘或TCP/IP堆栈，可能会对FreeRTOS Windows端口的行为产生不利影响，通常应该避免。*

使用 FreeRTOS 可以让您的应用程序由事件驱动。事件驱动设计能非常有效地利用处理时间，因为只有当事件发生时才会使用处理时间，而不会浪费处理时间去轮询未发生的事件。例 6.3 无法使用事件驱动，因为在使用 FreeRTOS Windows 端口时处理键盘中断是不现实的，因此必须使用效率低得多的轮询技术。如果清单 6.16 是中断服务例程，则应使用 `xTimerResetFromISR()` 代替 `xTimerReset()`。

```
static void vKeyHitTask( void *pvParameters )
{
```

```

const TickType_t xShortDelay = pdMS_TO_TICKS( 50 );
TickType_t xTimeNow;

vPrintString( "Press a key to turn the backlight on.\r\n" );

/*理想情况下，应用程序应该是事件驱动的，并使用中断来处理按键。在使用FreeRTOS
Windows端口时使用键盘中断是不实际的，因此此任务用于轮询按键。*/
for(;;)
{
    /*是否按下了一个键？*/
    if ( _kbhit () != 0 )
    {
        /*已按下一个键。记录时间。*/
        xTimeNow = xTaskGetTickCount();

        if( xSimulatedBacklightOn == pdFALSE )
        {
            /*背光是关闭的，所以打开它，并打印的时间在
            它是打开的。*/
            xSimulatedBacklightOn = pdTRUE;
            vPrintStringAndNumber(
                "Key pressed, turning backlight ON at time\t\t",
                xTimeNow );
        }
        else
        {
            /*背光已经亮了，所以打印一条信息，说计时器即将被重置和它被重置的时间
            。*/
            vPrintStringAndNumber(
                "Key pressed, resetting software timer at time\t\t",
                xTimeNow );
        }

        /* 重置软件定时器。如果背光之前是关闭的，那么这个调用将启动计时器。如果
        背光以前已打开，那么此调用将重新启动计时器。一个真正的应用程序可以在
        一个中断中读取按键。如果这个函数是一个中断服务例程，那么
        必须使用xTimerResetFromISR()来代替xTimerReset()。*/
        xTimerReset ( xBacklightTimer, xShortDelay );

        /*读取并丢弃被按下的键-这个简单的例子是不需要的。*/
        (void) _getch ();
    }
}

```

```
}  
}
```

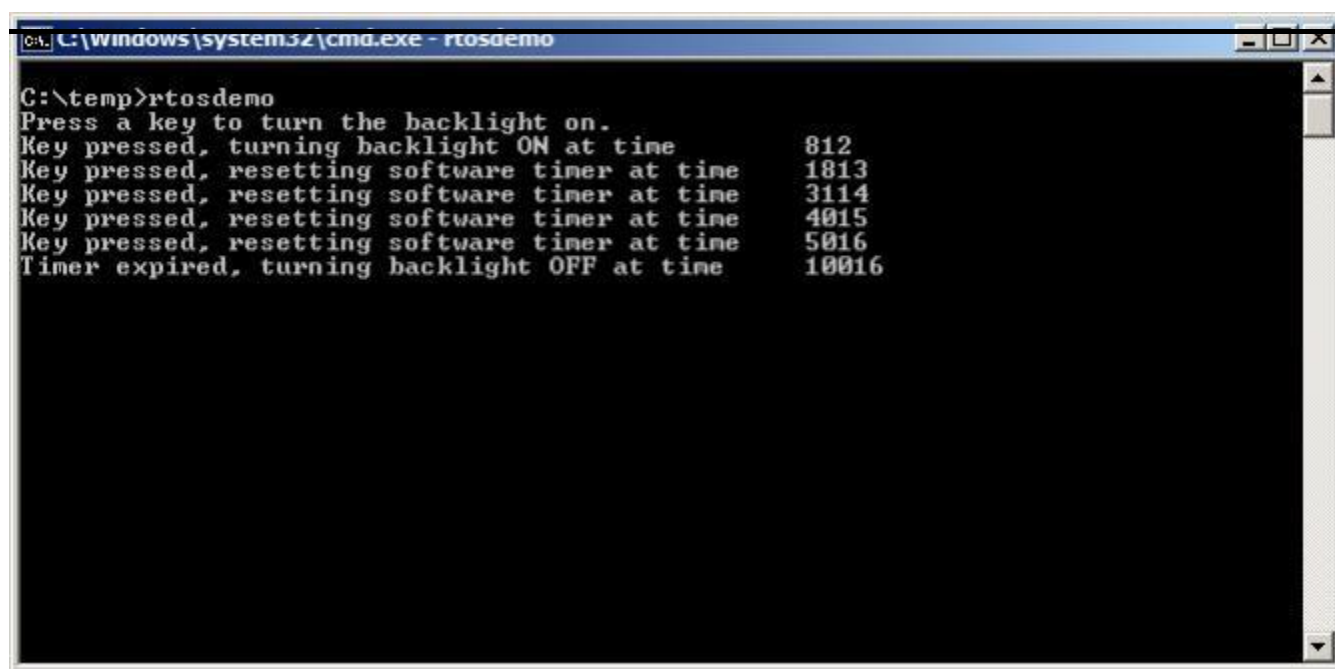
清单6.16 示例6.3中用于重置软件定时器的任务

当执行示例6.3时产生的输出如图6.10所示。参照图6.10:

- 第一次按键发生在滴答计数为812时。那时，背光被打开了，一次性计时器就开始了。
- 当滴答为1813、3114、4015和5016时，进一步按键。所有这些按键都会导致计时器在计时器过期之前被重置。

当标记计数为10016时，计时器已过期。那时，背光被关闭了。

■



```
C:\temp>rtosdemo  
Press a key to turn the backlight on.  
Key pressed, turning backlight ON at time      812  
Key pressed, resetting software timer at time  1813  
Key pressed, resetting software timer at time  3114  
Key pressed, resetting software timer at time  4015  
Key pressed, resetting software timer at time  5016  
Timer expired, turning backlight OFF at time  10016
```

图6.10 在执行示例6.3时产生的输出

从图6.10中可以看出，计时器的周期为5000；在最后一次按键后，背光正好关闭5000，所以在最后一次重置后关闭5000。

7 中断管理

7.1 简介

7.1.1 事件

嵌入式实时系统必须采取行动来响应来自环境的事件。例如，到达以太网外围设备（事件）上的数据包可能需要将其传递给TCP/IP堆栈以进行处理（操作）。非平凡的系统将不得不服务于来自多个来源的事件，所有这些事件都将有不同的处理开销和响应时间要求。在每种情况下，都必须对最佳的事件处理实施策略做出判断：

应该如何检测到该事件？通常使用中断，但输入也可以进行轮询。

当使用中断时，应在中断服务例程（ISR）内部执行多少处理，以及在外部执行多少处理？通常希望保持每个ISR尽可能短。

如何将事件通信到主代码（非ISR），以及如何构造这些代码以最好地适应潜在异步发生的处理？

FreeRTOS并不对应用程序设计器强加任何特定的事件处理策略，但确实提供了允许以一种简单和可维护的方式实现所选策略的特性。

区分任务的优先级和中断的优先级是很重要的：

- 任务是一种与运行FreeRTOS的硬件无关的软件特性。优先级由应用程序编写器在软件中分配一个任务，并由一个软件算法（调度器）决定将使哪个任务处于运行状态。
- 中断服务例程虽然是用软件编写的，但它是一个硬件特性，因为硬件控制中断服务例程的运行，以及何时运行。任务只有在没有运行ISR时才会运行，因此，优先级最低的中断将中断优先级最高的任务，并且任务无法抢占ISR。

所有将运行FreeRTOS的体系结构都能够处理中断，但是与中断输入和中断优先级分配相关的细节在不同的体系结构中是不同的。

7.1.2 范围

本章包括：

- 哪个FreeRTOS API函数可以在一个中断服务例程中使用。
- 将中断处理延迟到任务的方法。
- 如何创建和使用二进制信号量和计数信号量。
- 二进制信号量和计数信号量之间的区别。
- 如何使用队列将数据进出中断服务例程。

- 中断嵌套模型与一些FreeRTOS端口一起可用。

7.2 使用ISR中的FreeRTOS API

7.2.1 中断安全API

通常有必要从中断服务例程（ISR）中使用 FreeRTOS API 函数提供的功能，但许多 FreeRTOS API 函数执行的操作在 ISR 中不合法。其中最值得注意的是那些将调用函数的任务置入阻塞状态的API--如果 API 函数是从 ISR 调用的，那么它就不会从任务中调用，因此没有调用任务可以置入阻塞状态。

FreeRTOS 通过为某些 API 函数提供两个版本来解决这个问题：一个版本用于任务，另一个版本用于 ISR。

注意：不要从ISR中调用没有“FromISR”的FreeRTOS API函数。

7.2.2 使用一个单独的中断安全API的好处

在中断中使用单独的应用程序接口，可以提高任务代码的效率，提高 ISR 代码的效率，简化中断入口。要了解原因，可以考虑另一种解决方案，即为每个 API 函数提供一个单一版本，既可以从任务中调用，也可以从 ISR 中调用。如果任务和 ISR 都能调用相同版本的 API 函数，那么

- API函数需要额外的逻辑来确定它们是从任务还是ISR调用的。额外的逻辑将通过函数引入新的路径，使函数更长、更复杂、更难测试。

当从任务调用函数时，一些API函数参数会不需要，而当从ISR调用函数时其他一些参数会不需要

每个FreeRTOS端口都需要提供一种机制来确定执行上下文（任务或ISR）。

- 在不容易确定执行上下文（任务或 ISR）的体系结构上，需要额外的、浪费的、使用更复杂的和非标准的中断进入代码，以便由软件提供执行上下文。

7.2.3 使用单独的、中断的安全API的缺点

某些 API 函数有两个版本，可以提高任务和 ISR 的效率，但也带来了一个新问题：有时需要从任务和 ISR 中调用一个不属于 FreeRTOS API 但使用了 FreeRTOS API 的函数。

通常只有在集成第三方代码时才会出现这种问题，因为只有在这种情况下，软件的设计才不受应用程序编写者的控制。如果这确实成为一个问题，那么可以使用以下技术之一来解决：

- 将中断处理延迟到任务^[12]，因此API函数只能从任务的上下文中调用。

- 如果您正在使用支持中断嵌套的FreeRTOS端口，那么请使用以“FromISR”结束的API函数版本，因为这个版本可以从任务和ISR中调用。（反之则不成立，不以“FromISR”结尾的API函数不能从ISR中调用。）
- 第三方代码通常包括一个 RTOS 抽象层，可以通过该抽象层来测试调用函数的上下文（任务或中断），然后调用适合该上下文的 API 函数

[^12]：延迟中断处理将在本书的下一节中介绍。

7.2.4 xHigherPriorityTaskWoken参数

本节介绍xHigherPriorityTaskWoken参数的概念。如果您还没有完全理解本节，请不要担心，因为下面的部分将提供实际的例子。

如果上下文切换是由中断执行的，那么当中断退出时运行的任务可能与输入中断时运行的任务不同——中断将中断一个任务，但返回到另一个任务。

一些FreeRTOS API函数可以将任务从阻塞状态移动到就绪状态。这已经在xQueueSendToBack（）等函数中看到了，如果有一个任务在等待数据在主题队列中可用，它将唤醒任务。

如果被 FreeRTOS API 函数解锁的任务的优先级高于处于运行状态的任务的优先级，那么根据 FreeRTOS 调度策略，应该切换到优先级更高的任务。何时实际切换到优先级更高的任务取决于调用 API 函数的上下文：

- 如果从任务中调用了API函数：

如果在 FreeRTOSConfig.h 中将 configUSE_PREEMPTION 设置为 1，则会在 API 函数中自动切换到优先级更高的任务，换句话说，就是在 API 函数退出之前切换到优先级更高的任务。这在图 6.6 中已有所体现，在该图中，写入定时器命令队列的函数还未退出，就已切换到 RTOS 守护进程任务。

- 如果从一个中断中调用了API函数：

在中断中不会自动切换到更高优先级的任务。取而代之的是设置一个变量，通知应用程序编写者应执行上下文切换。中断安全 API 函数（以“FromISR”结尾的函数）有一个名为 pxHigherPriorityTaskWoken 的指针参数，可用于此目的

如果要执行上下文切换，那么中断安全 API 函数将把 *pxHigherPriorityTaskWoken 设置为 pdTRUE。为了能够检测到这种情况，在首次使用 pxHigherPriorityTaskWoken 之前，必须将其指向的变量初始化为 pdFALSE。

如果应用程序编写者选择不请求 ISR 进行上下文切换，那么优先级较高的任务将一直处于就绪状态，直到调度程序下一次运行，最坏的情况是在下一个滴答中断。

FreeRTOS API 函数只能将 `*pxHighPriorityTaskWoken` 设置为 `pdTRUE`。如果一个 ISR 调用了多个 FreeRTOS API 函数，则可以在每个 API 函数调用中将相同的变量作为 `pxHigherPriorityTaskWoken` 参数传递，并且只需在首次使用前将该变量初始化为 `pdFALSE` 即可。

在API函数的中断安全版本中，上下文切换不会自动发生有几个原因：

- 避免不必要的上下文切换

在任务需要执行任何处理之前，中断可以执行不止一次。例如，考虑一个场景，即任务处理中断驱动的UART接收的字符串；对于UART ISR来说，每次接收到字符时都切换到任务是很浪费的，因为任务只有在收到完整的字符串后才能执行处理。

- 控制执行顺序

中断可能偶尔发生，而且发生的时间无法预测。FreeRTOS 专家用户可能希望暂时避免在应用程序的特定时刻不可预测地切换到不同的任务，尽管这也可以通过 FreeRTOS 调度器锁定机制来实现。

可移植性

它是可以在所有FreeRTOS的端口中使用的最简单的机制

- 效率

针对较小的处理器架构的端口只允许在ISR的最末端请求上下文切换，而删除该限制将需要额外的和更复杂的代码。它还允许在同一个ISR中对一个FreeRTOS API函数进行多个调用，而无需在同一个ISR中为一个上下文切换生成多个请求。

- 在RTOS滴答中断中执行

正如本书后面所述，可以在 RTOS tick 中断中添加应用代码。在 tick 中断内尝试上下文切换的结果取决于正在使用的 FreeRTOS 端口。这样做最多只能导致不必要地调用调度程序。

`pxHigherPriorityTaskWoken` 参数的使用是可选的。如果不需要，则将 `pxHigherPriorityTaskWoken` 设为

`NULL`

7.2.5 portYIELD_FROM_ISR () 和portEND_SWITCHING_ISR () 宏

本节将介绍用于从ISR请求上下文切换的宏。如果您还没有完全理解本节，请不要担心，因为下面的部分将提供实际的例子。

`taskYIELD()` 是一个宏，可以在任务中调用它来请求上下文切换。`portYIELD_FROM_ISR()` 和 `portEND_SWITCHING_ISR()` 都是 `taskYIELD()` 的中断安全版本。`portYIELD_FROM_ISR()` 和 `portEND_SWITCHING_ISR()` 都以相同的方式使用，并且做同样的事情^[13]。一些 FreeRTOS 端口只提供这两个宏中的一个。较新的 FreeRTOS 端口提供了这两个宏。本书中的例子使用了 `portYIELD_FROM_ISR()`。

^[13]：以前，`portEND_SWITCHING_ISR()` 用于要求中断处理程序使用汇编代码封装的 FreeRTOS port，而 `portYIELD_FROM_ISR()` 则用于允许使用 C 语言编写整个中断处理程序的 FreeRTOS port。

```
portEND_SWITCHING_ISR( xHigherPriorityTaskWoken );
```

清单7.1 `portEND_SWITCHING_ISR()` 宏

```
portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
```

清单7.2 `portYIELD_FROM_ISR()` 宏

从中断安全 API 函数传出的 `xHigherPriorityTaskWoken` 参数可直接用作 `portYIELD_FROM_ISR()` 调用中的参数。

如果 `portYIELD_FROM_ISR()` `xHigherPriorityTaskWoken` 参数为 `pdFALSE`（零），则不会请求上下文切换，宏也不起作用。如果 `portYIELD_FROM_ISR()` `xHigherPriorityTaskWoken` 参数不是 `pdFALSE`，则会请求进行上下文切换，处于运行状态的任务可能会发生变化。
中断将始终返回到运行状态下的任务，即使在中断执行期间运行状态下的任务发生了变化

大多数 FreeRTOS 端口都允许在 ISR 中的任何地方调用 `portYIELD_FROM_ISR()`。一些 FreeRTOS 端口（主要是针对较小架构的端口），只允许在 ISR 的最末端调用 `portYIELD_FROM_ISR()`。

7.3 延迟中断处理

通常认为保持 ISR 尽可能短是最佳做法。原因包括：

- 即使任务被分配了很高的优先级，它们也只能在硬件没有中断服务的情况下运行。
- ISR 会扰乱（增加“抖动”）任务的启动时间和执行时间。
- 根据运行 FreeRTOS 的体系结构，当 ISR 正在执行时，可能无法接受任何新的中断，或至少无法接受新中断的子集。

- 应用程序编写者需要考虑变量、外设和内存缓冲区等资源同时被任务和 ISR 访问的后果，并加以防范。
- 一些FreeRTOS端口允许中断嵌套，但中断嵌套可以增加复杂性并降低可预测性，中断的时间越短，它嵌套的可能性就越小。

中断服务例程必须记录该中断的原因，并清除该中断。中断所需要的任何其他处理通常都可以在任务中执行，从而允许中断服务例程尽可能快地退出。这被称为“延迟中断处理”，因为中断所需要的处理是从ISR“延迟”到一个任务的。

将中断处理延迟到任务还允许应用程序编写器相对于应用程序中的其他任务确定处理的优先级，并使用所有FreeRTOS API函数。

如果延迟中断处理的任务的优先级高于任何其他任务的优先级，那么处理将立即执行，就像处理是在ISR本身中执行的一样。这个场景如图7所示。1，其中任务1是一个正常的应用程序任务，任务2是延迟中断处理的任務。

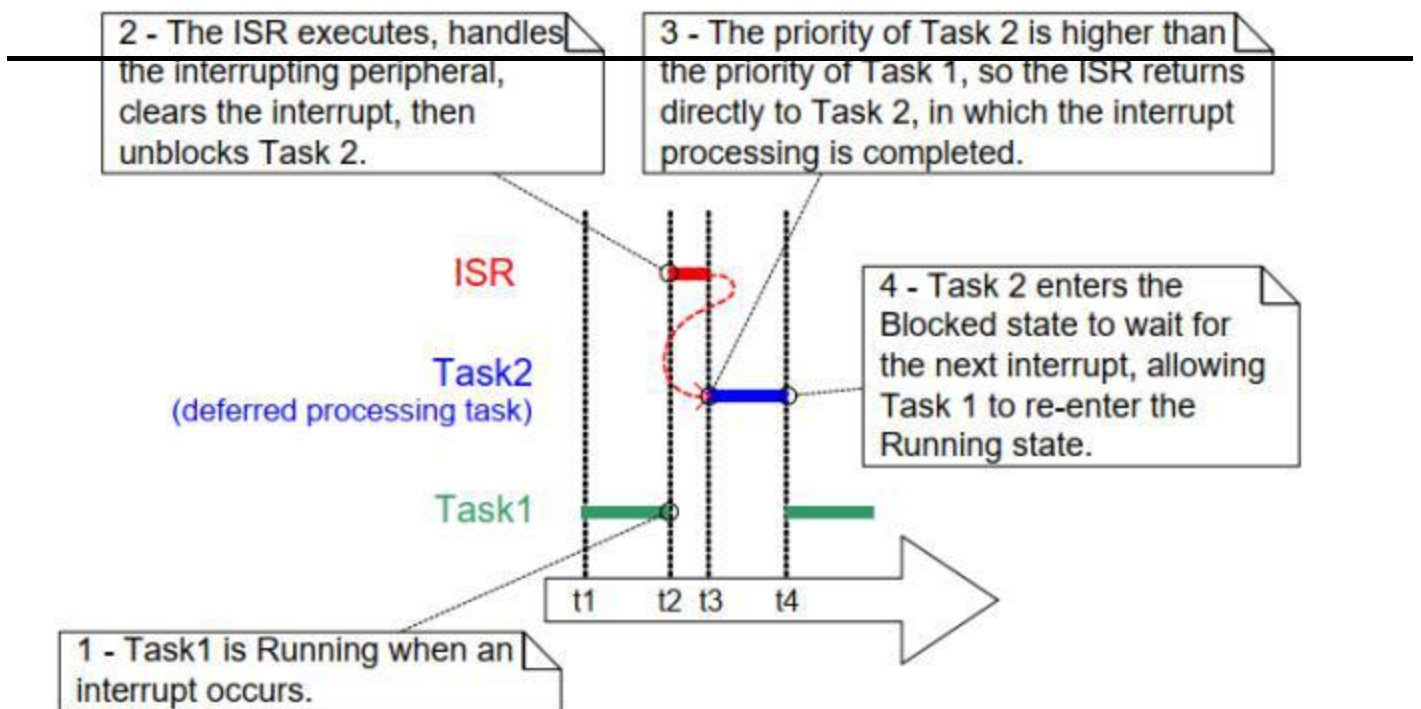


图7.1 在高优先级任务中完成中断处理

在图7.1，中断处理从时间t2开始，有效地结束于时间t4，但在ISR中只花费了时间t2和t3之间的时间。如果没有使用延迟中断处理，那么时间t2和t4之间的整个时间段将会在ISR中花费。

对于何时最好执行ISR中的中断所需要的所有处理，以及何时最好将部分处理推迟到任务，并没有绝对的规则。在以下情况下，延迟处理到任务最有用：

- 中断所需要的处理过程并不简单。例如，如果中断只是存储模拟到数字转换的结果，那么几乎可以肯定这是最好在ISR中执行，但如果转换的结果也必须通过软件过滤器，那么最好是在任务中执行过滤器。

中断处理可以方便地执行不能在ISR中执行的操作，例如写入控制台，或分配内存。

中断处理不是确定性的——这意味着事先不知道处理将需要多长时间。

下面的部分描述并演示了本章到目前为止所介绍的概念，包括可用于实现延迟中断处理的FreeRTOS特性。

7.4 用于同步的二进制信号量

二进制 Semaphore API 的中断安全版本可用于在每次发生特定中断时解除对任务的阻塞，从而有效实现任务与中断的同步。这样，大部分中断事件处理都可以在同步任务中实现，只有**极快**和**极短**的部分直接留在 ISR 中。如上一节所述，二进制信号用于将中断处理“推迟”到一个任务^[14]。

^[14]：使用直接任务通知从中断中解锁任务比使用二进制信号量更有效。在第 10 章“任务通知”中才会涉及直接任务通知。

如图7.1所示，如果中断处理的时间特别紧迫，则可以设置延迟处理任务的优先级，以确保该任务始终优先于系统中的其他任务。然后，可以在执行ISR时调用 `portYIELD_FROM_ISR()`，确保ISR直接返回中断处理被推迟的任务。这样做的效果是确保整个事件处理在时间上连续（无中断）执行，就像所有事件都在ISR本身中执行一样。图7.2重复了图7.1所示的情况，但对文字进行了更新，描述了如何使用信号传递器控制延迟处理任务的执行。

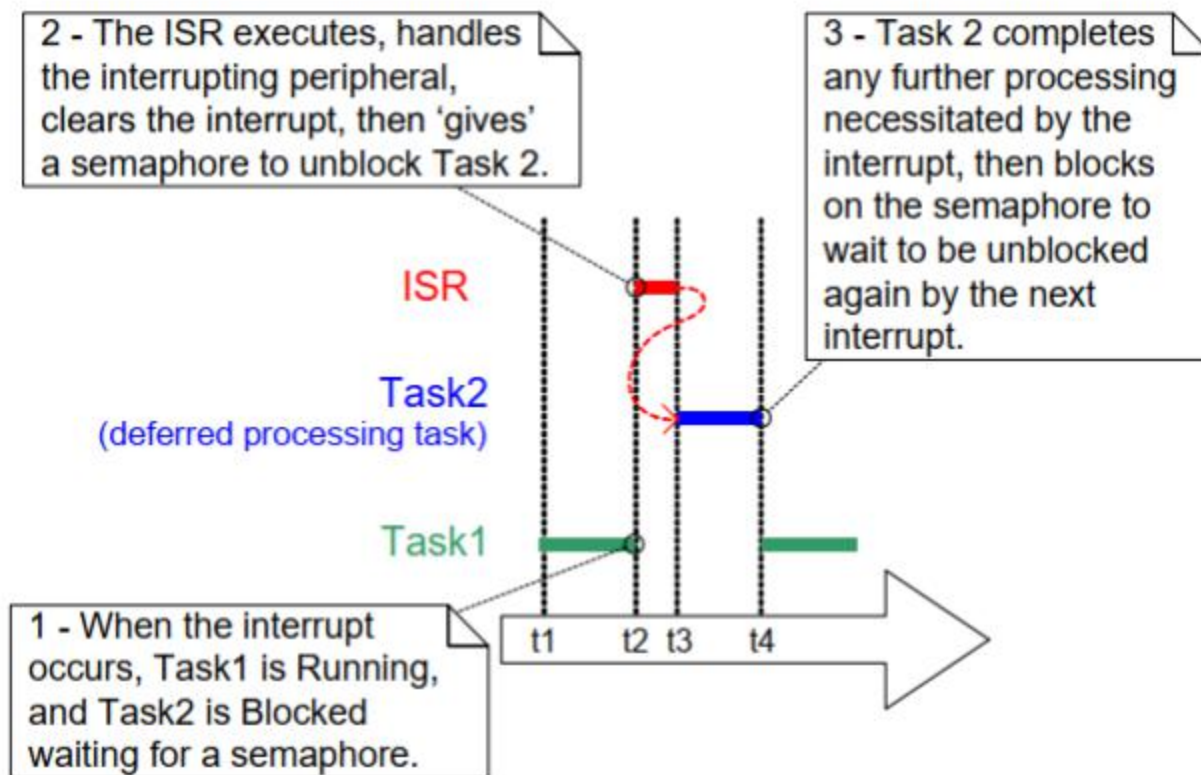


图7.2 使用二进制信号量来实现延迟中断处理

延迟处理任务使用阻塞“take”调用一个 semaphore，以此进入阻塞状态，等待事件发生。当事件发生时，ISR 会在同一个寄存器上执行“give”操作，以解除对任务的阻塞，从而继续进行所需的事件处理。

“take信号”和“give信号”这两个概念根据它们的使用场景有不同的含义。在这个中断同步场景中，二进制信号量在概念上可以看作是一个长度为1的队列。队列在任何时候最多可以包含一个项，因此总是空或满（因此，使用二进制）。通过调用 `xSemaphoreTake()`，中断处理被推迟的任务会有效地尝试从队列中读取有阻塞时间的数据，如果队列为空，则会导致任务进入阻塞状态。当事件发生时，ISR 会使用 `xSemaphoreGiveFromISR()` 函数将一个标记（信号）放入队列，使队列为满。这将导致任务退出“阻塞”状态并移除标记，使队列再次清空。任务完成处理后，会再次尝试从队列中读取数据，如果发现队列为空，就会重新进入“阻塞”状态，等待下一个事件。图 7.3 展示了这一顺序。

通过调用 `xSemaphoreTake()`，中断处理被推迟的任务会有效地尝试从队列中读取有阻塞时间的数据，如果队列为空，则会导致任务进入阻塞状态。当事件发生时，ISR 会使用 `xSemaphoreGiveFromISR()` 函数将一个标记（信号）放入队列，使队列变满。这将导致任务退出“受阻”状态并移除标记，使队列再次清空。任务完成处理后，会再次尝试从队列中读取数据，如果发现队列为空，就会重新进入“阻塞”状态，等待下一个事件。图 7.3 展示了这一顺序

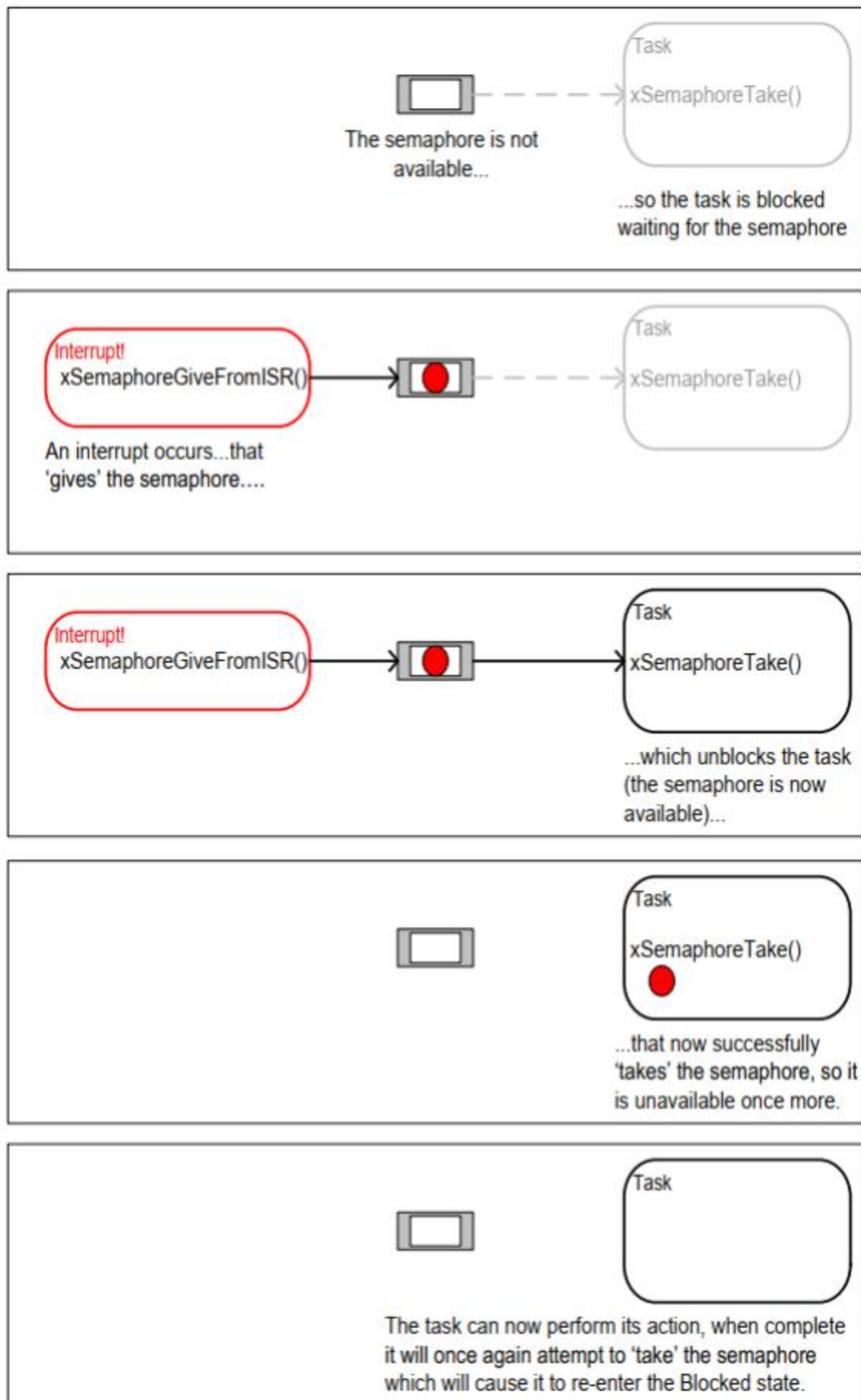


图7.3 使用二进制信号量将任务与中断进行同步

7.4.1 xSemaphoreCreateBinary() API 函数

FreeRTOS 还包括 `xSemaphoreCreateBinaryStatic()` 函数，它分配在编译时静态创建二进制信号量所需的内存：处理所有各种类型的 FreeRTOS 信号量存储在一个 `SemaphoreHandle_t` 类型的变量中。

在使用信号量之前，必须先创建它。若要创建二进制信号量，请使用 `xSemaphoreCreateBinary()` API 函数^[^15]。

[^15]：一些信号量 API 函数实际上是宏，而不是函数。为简单起见，它们在整本书中都被称为函数。

```
SemaphoreHandle_t xSemaphoreCreateBinary( void );
```

清单 7.3 xSemaphoreCreateBinary() API 函数原型

返回值

- 返回值

如果返回 NULL，则无法创建信号量，因为 FreeRTOS 没有足够的堆内存来分配信号量数据结构。

如果返回一个非 null 值，则表示该信号量已创建成功。返回的值应该存储为所创建的信号量的句柄

7.4.2 xSemaphoreTake() API 函数

“take” 信号意味着“获得”或“接收”信号。只有在可用时才能获取。

除了递归互斥之外，所有其他类型的 FreeRTOS 信号都可以使用 `xSemaphoreTake()` 函数来“获取”。

`xSemaphoreTake()` 不能在中断服务例程中使用。

```
BaseType_t xSemaphoreTake ( SemaphoreHandle_t xSemaphore, TickType_t xTicksToWait );
```

清单 7.4 xSemaphoreTake() API 函数原型

参数和返回值

- `xSemaphore`

被“take”的信号量。

Semaphore 由 SemaphoreHandle_t 类型的变量引用。它必须在它之前显式地创建使得

•xTicksToWait

如果任务不可用，应保持在阻塞状态等待信号的最长时间。

如果等待时间为零，那么如果信号量不可用，函数将立即返回。

阻塞时间以滴答周期指定，因此它所表示的绝对时间取决于滴答频率。宏pdMS_TO_TICKS() 可用于将以毫秒为单位指定的时间转换为在刻度中指定的时间。

如果INCLUDE_vTaskSuspend在FreeRTOSConfig.h中被设置为1，那么将xTicksToWait设置为portMAX_DELAY将导致任务无限期等待（没有超时）。

•返回值

有两个可能的返回值：

◦pdPASS

只有当对函数的调用成功获得信号量时，才会返回pdPASS。如果指定了阻塞时间（xTicksToWait不是零），那么调用任务可能被放入阻塞状态以等待信号量可用，但是信号量在阻塞时间过期之前变得可用。

◦pdFALSE

该信号量不可用。

如果指定了阻塞时间（xTicksToWait不是零），那么调用任务将被放入阻塞状态，等待信号量可用，但在此发生之前阻塞时间已到期。

7.4.3 xSemaphoreGiveFromISR() API函数

二进制和计数信号量[¹⁶]可以使用xSemaphoreGiveFromISR()函数“give”。

[¹⁶]：计数信号量在本书的后面部分都有描述。

xSemaphoreGiveFromISR() 是 xSemaphoreGive() 的中断安全版本，因此具有本章开头所述的pxHigherPriorityTaskWoken 参数。


```
BaseType_t xSemaphoreGiveFromISR ( SemaphoreHandle_t xSemaphore, BaseType_t
                                   *pxHigherPriorityTaskWoken );
```

清单7.5 xSemaphoreGiveFromISR() API函数原型

参数和返回值

• xSemaphore

被“give”的信号量。

信号量由SemaphoreHandle_t类型的变量引用，并且必须在使用之前显式地创建。

• pxHigherPriorityTaskWoken

可能会有一个或多个任务阻塞在单个 semaphore 上，等待该 semaphore 可用。调用xSemaphoreGiveFromISR() 会使信号量可用，从而导致一个等待信号传递器的任务离开阻塞状态。如果调用 xSemaphoreGiveFromISR() 导致任务离开阻塞状态，且解除阻塞的任务的优先级高于当前执行的任务（被中断的任务），则 xSemaphoreGiveFromISR() 将在内部把 *pxHigherPriorityTaskWoken 设为 pdTRUE。

如果xSemaphoreGiveFromISR()将这个值设置为pdTRUE，那么通常应该在退出中断之前执行一个上下文切换。这将确保中断直接返回到最高优先级的准备就绪状态任务。

• 返回值

有两个可能的返回值：

◦ pdPASS

只有当调用函数成功时，才会返回pdPASS。

◦ pdFAIL

如果一个信号量已经可用，它就不能被“give”，并且函数将返回pdFAIL。

实例7.1使用二进制信号量将任务与中断进行同步

此示例使用二进制寄存器从中断服务例程中解锁任务，从而有效地使任务与中断同步。

使用一个简单的周期性任务，每 500 毫秒产生一个软件中断。使用软件中断是为了方便，因为在某些目标环境中使用真实中断非常复杂

清单7.6显示了定期任务的实现。注意，任务在生成中断之前和之后打印一个字符串。这允许在执行示例时产生的输出中观察到执行序列。

```

/*本例中使用的软件中断编号。 所示代码来自 Windows 项目，
其中数字 0 至 2 由 FreeRTOS Windows 端口本身使用，因此 3 是应用程序可用的第一个数字
*/

#define mainINTERRUPT_NUMBER 3

static void vPeriodicTask( void *pvParameters )
{
    const TickType_t xDelay500ms = pdMS_TO_TICKS (500UL);

    /*与大多数任务一样，该任务是在一个无限循环中实现的。*/
    for ( ; ; ) {
        /*阻塞，直至再次产生软件中断。*/
        vTaskDelay ( xDelay500ms );

        /*产生中断，在中断产生之前和之后都打印一条信息，以便从输出中看出执行顺序。

        生成软件中断的语法取决于所使用的 FreeRTOS 端口。
        下面使用的语法只能在 FreeRTOS Windows 移植中使用，在 Windows 移植中只能模拟此类中断。
        */

        vPrintString( "Periodic task - About to generate an interrupt.\r\n" );
        vPortGenerateSimulatedInterrupt( mainINTERRUPT_NUMBER );
        vPrintString( "Periodic task - Interrupt generated.\r\n\r\n\r\n" );

    }
}

```

清单7.6 实施例7.1中定期生成软件中断的任务的实现

清单7.7显示了延迟中断处理的任务的实现——通过使用二进制信号量与软件中断同步的任务。同样，在任务的每个迭代中都会打印出一个字符串，因此，执行任务和中断的顺序可以从执行示例时产生的输出中明显看出。

需要注意的是，虽然清单7.7中所示的代码对于示例7.1是足够的。如果中断是由硬件产生的，那么这个案例是不够充分的。下小节描述了如何更改代码的结构，使其适合与硬件生成的中断一起使用

```

static void vHandlerTask( void *pvParameters )
{
    /*与大多数任务一样，该任务是在一个无限循环中实现的。
    */for ( ; ; ) {
        /*使用信号量去等待事件发生。 该信号是在调度程序启动前创建的，因此是在该任务首次运行前创建的。
        任务会无限期阻塞，这意味着只有在成功获取信号后才会返回该函数调用，
        因此无需检查 xSemaphoreTake() 返回的值。
        */
        xSemaphoreTake( xBinarySemaphore, portMAX_DELAY );

        /*若要到达这里，该事件必须已经发生。处理事件（在
        在这个案例中，只要打印出一条信息）。*/
        vPrintString( "Handler task - Processing event.\r\n" );
    }
}

```

清单7.7 实施例7.1中延迟中断处理的任务（与中断同步的任务）的实现

清单7.8显示了ISR。除了“give”信号量来解除延迟中断处理的任务之外，这几乎没有什么作用。

请注意 `xHigherPriorityTaskWoken` 变量的使用方式。在调用 `xSemaphoreGiveFromISR()` 之前，它被设置为 `pdFALSE`，然后在调用 `portYIELD_FROM_ISR()` 时用作参数。如果 `xHigherPriorityTaskWoken` 等于 `pdTRUE`，将在 `portYIELD_FROM_ISR()` 宏内请求进行上下文切换

对于 FreeRTOS Windows 移植而言，ISR 的原型和强制上下文切换所调用的宏都是正确的，但对于其他 FreeRTOS 移植而言可能有所不同。请参阅 FreeRTOS.org 网站上针对特定端口的文档页面，以及 FreeRTOS 下载中提供的示例，以找到您使用的端口所需的语法

与大多数运行 FreeRTOS 的体系结构不同，FreeRTOS Windows 移植需要 ISR 来返回值。

与Windows端口一起提供的`portYIELD_FROM_ISR()`宏的实现包括返回语句，因此清单7.8没有显示明确返回的值。

```

static uint32_t ulExampleInterruptHandler( void )
{
    BaseType_t xHigherPriorityTaskWoken;

    /* xHigherPriorityTaskWoken必须初始化为
    pdFALSE，因为它将被设置为pdTRUE在中断安全函数
    API内，如果需要上下文切换。*/

```

```

xHigherPriorityTaskWoken = pdFALSE;

/* "give"信号量，解锁任务，传入xHigherPriorityTaskWoken地址
   */
xSemaphoreGiveFromISR ( xBinarySemaphore, &xHigherPriorityTaskWoken );

/*将 xHigherPriorityTaskWoken 值传入 portYIELD_FROM_ISR()。
  如果 xHigherPriorityTaskWoken 在 xSemaphoreGiveFromISR() 中被设置为 pdTRUE，
  那么调用 portYIELD_FROM_ISR() 将请求进行上下文切换。如果 xHigherPriorityTaskWoken 仍为 pdFALSE，
  那么调用 portYIELD_FROM_ISR() 将不会有任何效果。
  与大多数 FreeRTOS 移植不同，Windows 移植要求 ISR 返回值
  - 返回语句在 Windows 版本的 portYIELD_FROM_ISR() 中
  */
portYIELD_FROM_ISR( xHigherPriorityTaskWoken );

}

```

清单7.8 示例7.1中使用的软件中断的ISR

主函数创建二进制信号量，创建任务，安装中断处理程序，并启动调度程序。该实现如清单7.9所示。

用于安装中断处理程序的函数的语法特定于FreeRTOS Windows端口，对于其他FreeRTOS端口可能有所不同。请参阅FreeRTOS上的端口特定文档页。来找到您正在使用的端口所需的语法。

int main (void)

```

{
    /*在使用信号量之前，必须显式地创建它。在这个
       创建了一个二进制信号量的示例。*/
    xBinarySemaphore = xSemaphoreCreateBinary();

    /*检查信号量已成功创建。*/if
    ( xBinarySemaphore != NULL )
    {
        /*创建“处理程序”任务，这是延迟中断处理的任务。这是将与中断同步的任务。处理
           程序任务具有高优先级，以确保它在中断退出后立即运行。在这种情况下，所选择的
           优先级为3。*/
        xTaskCreate( vHandlerTask, "Handler", 1000, NULL, 3, NULL );

        /*创建将定期生成一个软件中断的任务。创建它的优先级低于处理程序任务，以确保
           在每次处理程序任务退出阻塞状态时它都会被抢占。*/
        xTaskCreate( vPeriodicTask, "Periodic", 1000, NULL, 1, NULL );
    }
}

```

```

/*安装软件中断的处理程序。这一点所需的语法取决于所使用的FreeRTOS端口。这里显
示的语法只能与FreeRTOS windows端口一起使用，而这些中断只能被模拟。*/
vPortSetInterruptHandler( mainINTERRUPT_NUMBER,
                           ulExampleInterruptHandler );

} /*启动调度程序，以便已创建的任务开始执行。*/ vTaskStartScheduler();

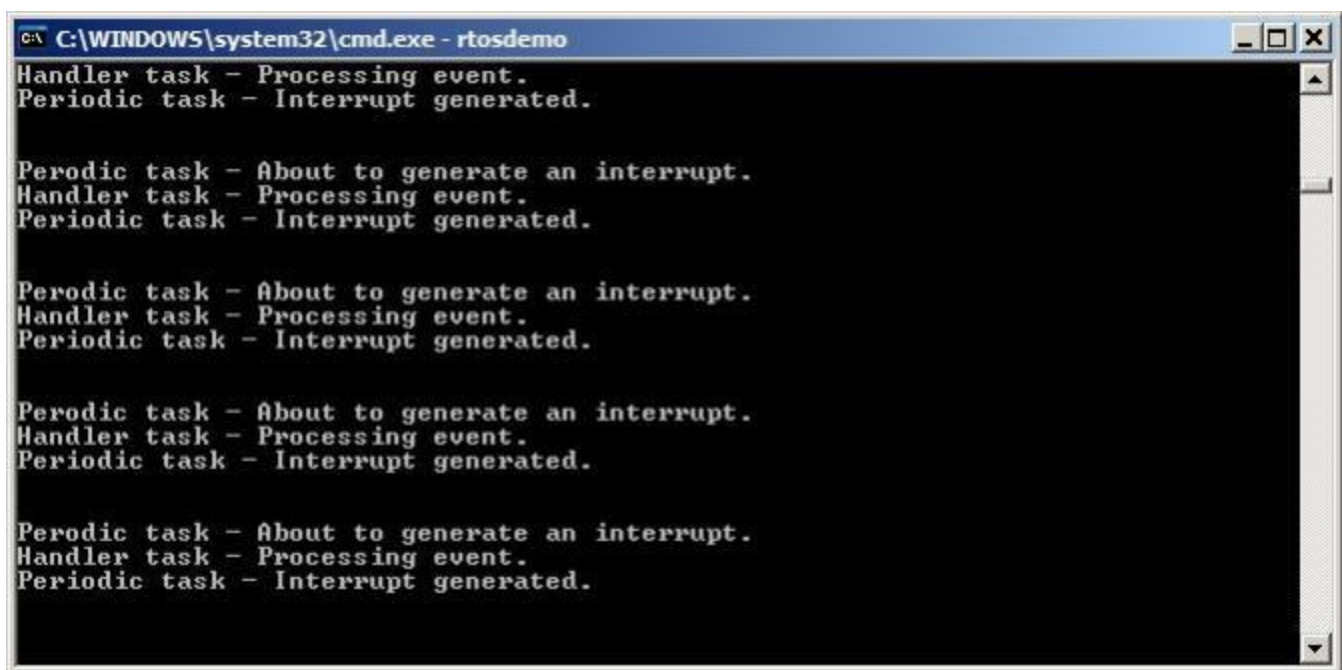
/*通常，永远不能到达以下行。*/
for(;;)
}

```

清单7.9 实例7.1主函数的实现

例 7.1 的输出结果如图 7.4 所示。不出所料，一旦中断发生，vHandlerTask() 就会进入运行状态，因此该任务的输出会与周期任务的输出相分割。

图 7.5 提供了进一步说明



```

C:\WINDOWS\system32\cmd.exe - rtosdemo
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

```

图7.4 执行示例7.1时产生的输出

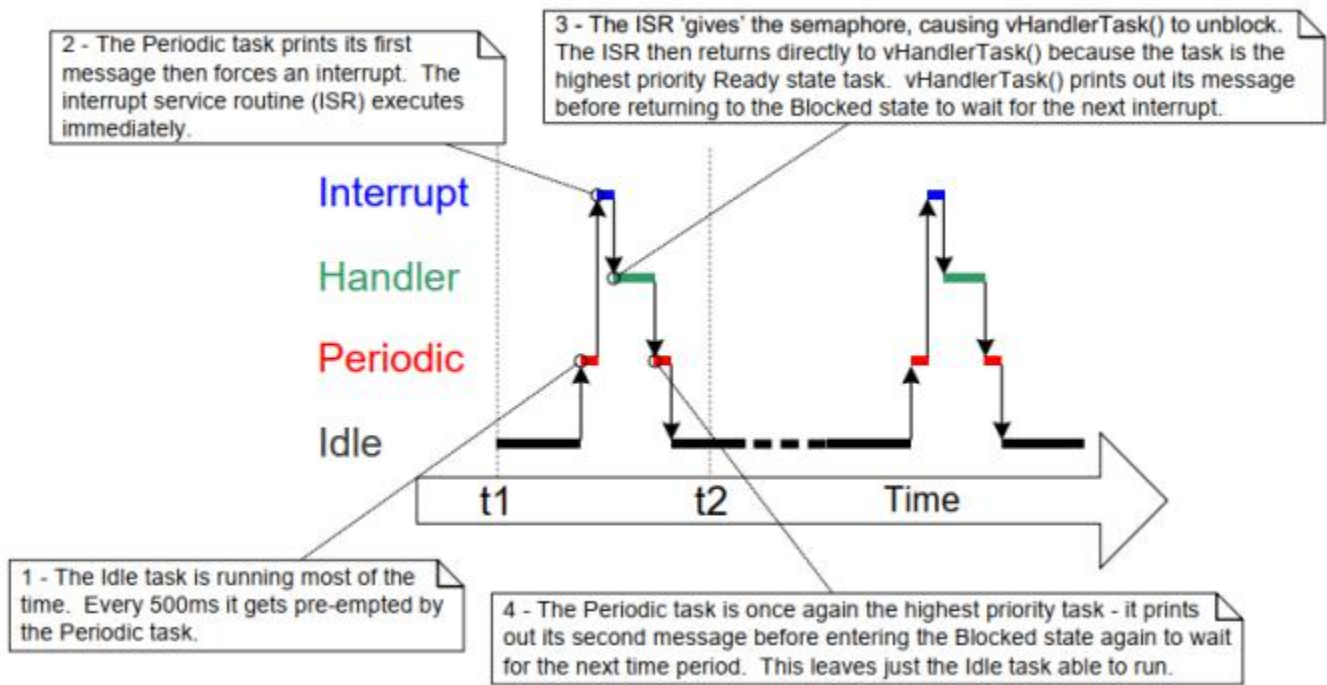


图7.5: 执行示例7.1时的执行顺序

7.4.4 改进了示例7.1中使用的任务的实现。

示例7.1使用二进制信号量来将任务与中断同步。执行顺序如下：

1. 中断发生。
2. ISR执行并“give”信号来解锁任务。
3. 该任务在ISR之后立即执行，并“take”了信号量。
4. 该任务处理了该事件，然后尝试再次“take”该信号量——进入阻塞状态，因为该信号量还不可用（尚未发生另一个中断）。

只有当中断以相对较低的频率发生时，示例7.1中使用的任务的结构才合适。要理解原因，考虑一下如果在任务完成对第一个中断的处理之前，发生了第二个和第三个中断，会发生什么：

- 当第二个ISR执行时，信号量将为空，因此ISR将提供信号量，并且该任务将在完成处理第一个事件后立即处理第二个事件。该场景如图7.6所示。
- 当第三个ISR被执行时，信号量将已经可用，从而防止ISR give 信号量，所以任务不会知道第三个事件发生了。该场景如图7.7所示。

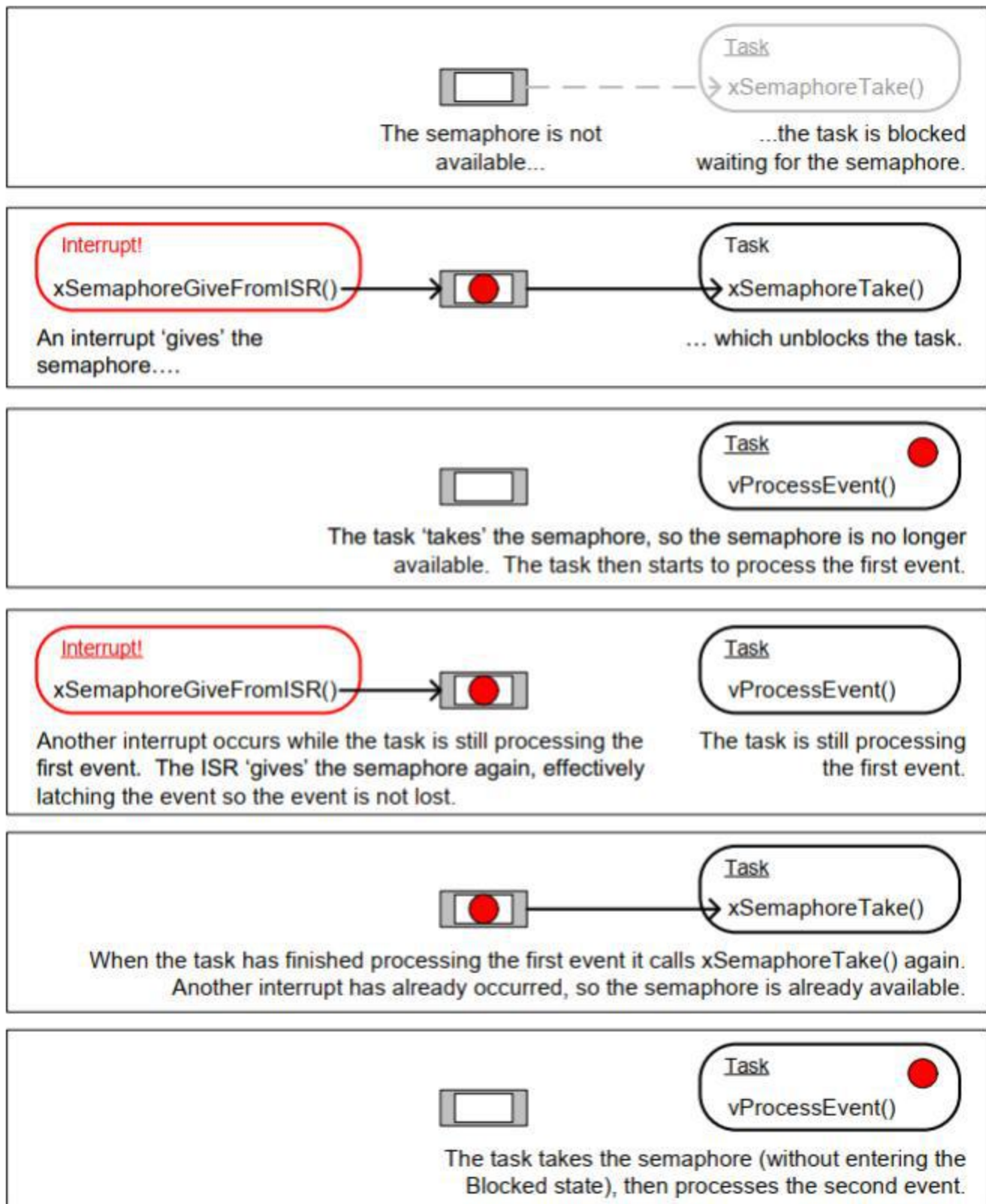


图7.6 在任务完成对第一个事件的处理之前发生一个中断的场景

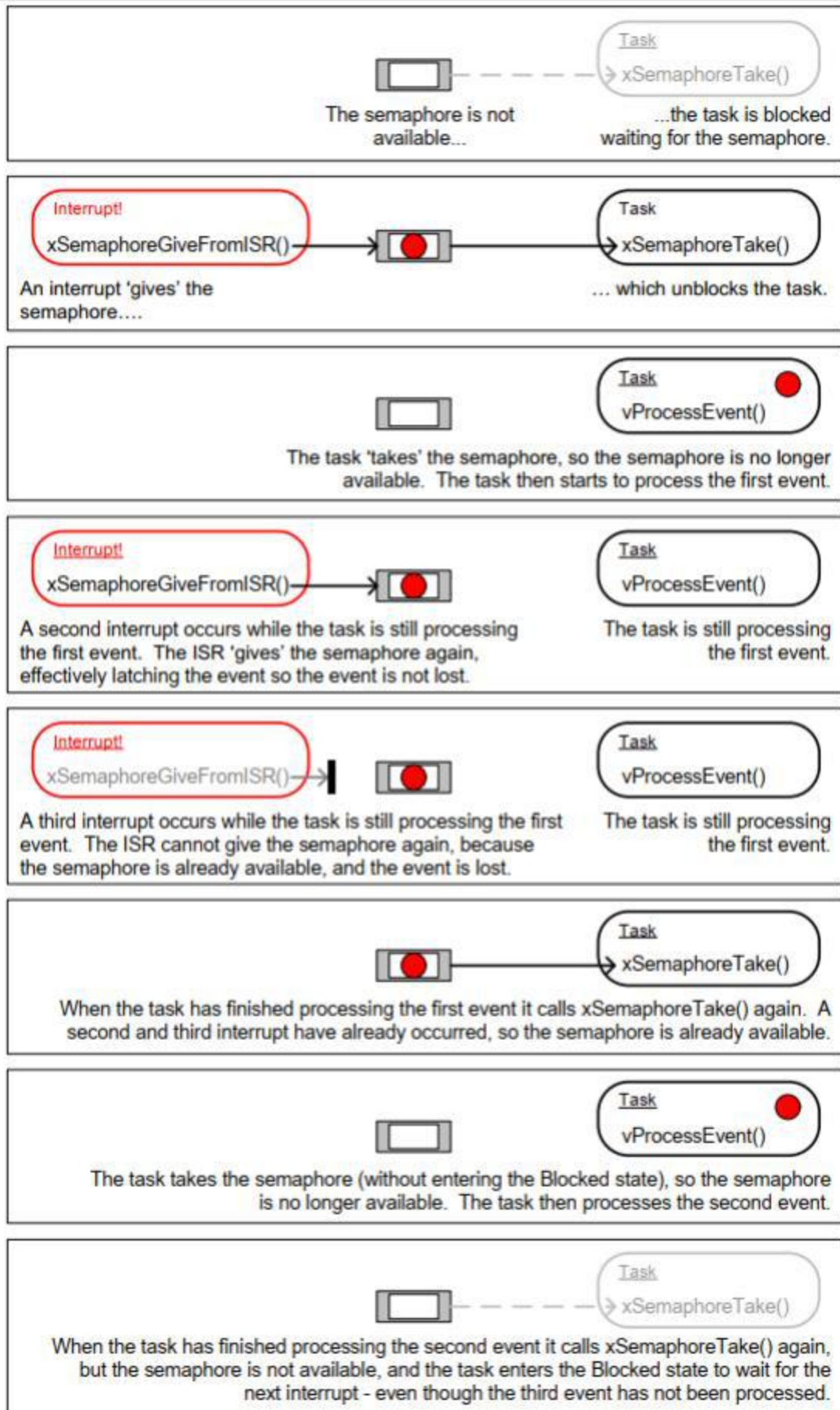


图7.7 任务完成对第一个事件的处理之前发生两次中断的场景

例 7.1 中使用的延迟中断处理任务，如列表 7.7 所示，其结构使其仅在每次调用 `xSemaphoreTake()` 之间处理一个事件。这对于示例 7.1 来说已经足够了，因为生成事件的中断由软件触发，并且发生在可预测的时间。在现实中应用程序，中断由硬件生成，并且发生在不可预测的时间。因此，为了最小化可能会错过中断，则必须构建延迟中断处理任务，以便它能够处理每次调用 `xSemaphoreTake()` 之间已经可用的所有事件^[17]。这可以通过清单 7.10，展示了如何构建 UART 的延迟中断处理程序。在示例 7.10 中，它是假设 UART 在每次接收到字符时都会生成一个接收中断，并且 UART 将将收到的字符放入硬件 FIFO（硬件缓冲区）

^[17]：或者，也可以使用计数信号量或直接任务通知来计数事件。计数符号将在下一节中描述。直接任务通知在第9章，任务通知中描述。直接到任务通知是首选的方法，因为它们在运行时和RAM使用方面都是最有效的。

示例 7.1 中使用的延迟中断处理任务还有另一个弱点；当它调用 `xSemaphoreTake()`。相反，该任务将 `portMAX_DELAY` 作为 `xSemaphoreTake()` `xTicksToWait` 传递参数，这会导致任务无限期等待（无超时）信号量可用。示例代码中经常使用无限期超时，因为它们的使用简化了示例的结构，并且因此，使示例更易于理解。但是，无限期超时在实际应用中通常是不好的做法应用程序，因为它们使从错误中恢复变得困难。例如，考虑任务正在等待中断给出信号量，但硬件中的错误状态阻塞了中断：

- 如果任务是在等待而没有超时，那么它将不知道错误状态，并且将永远等待。
- 如果任务等待超时，则 `xSemaphoreTake()` 会在超时后返回 `pdFAIL`，任务可以在下次执行时检测并清除错误。清单 7.10 演示了这种情况。

```
static void vUARTReceiveHandlerTask( void *pvParameters )
{
    /*xMaxExpectedBlockTime 保存两个中断之间的最长预期时间*/

    const TickType_t xMaxExpectedBlockTime = pdMS_TO_TICKS( 500 );

    /*与大多数任务一样，该任务是在一个无限循环中实现的。*/
    for ( ; ; ) {
        /*信号量由UART的接收（Rx）中断“give”。为下一个中断等待最多 xMaxExpectedBlockTime 周期*/
        if( xSemaphoreTake( xBinarySemaphore, xMaxExpectedBlockTime ) == pdPASS)
        { /*获得了信号量。处理之前的所有挂起的Rx事件
```

```

        再次调用xSemaphoreTake（）。每个Rx事件都将在UART的接收FIFO中放置一个字
        符，并且假定UART_RxCount（）将返回FIFO中的字符数。*/
    else（UART_RxCount（）> 0）
    {
        假设/* UART_ProcessNextRxEvent（）处理一个Rx字符，将FIFO中的字符
        数减少1个。*/
        UART_ProcessNextRxEvent（）；
    }

    /*不再有 Rx 事件挂起（FIFO 中不再有字符），因此循环返回并调用 xSemaphoreTake（）
    等待下一个中断。在代码中的这一点和调用
    xSemaphoreTake（）之间发生的任何中断都将被锁存在信号量中，因此不会丢失

    }
    else
    {
        /*未在预期的时间内接收到一个事件。检查UART中存在可能阻塞UART产生任何中
        断的任何错误条件，并在必要时清除这些错误条件。*/
        UART_ClearErrors（）；
    }
}
}

```

清单7.10 使用UART接收处理程序为例的延迟中断处理任务的推荐结构

7.5 计数信号器

就像二进制信号量可以被认为是长度为1的队列一样，计数信号量也可以被认为是长度超过1的队列。任务对存储在队列中的数据不感兴趣——只关心队列中的项目的数量。在FreeRTOSConfig.h中，configUSE_COUNTING_SEMAPHORES必须设置为1。使计数信号量可用。

每次“give”一个计数信号量时，就会使用其队列中的另一个空位。队列中的项目数是信号量的“count”值。计数信号量通常用于两件事：

1. 计数事件[¹⁸]

在这个场景中，事件处理程序将在每次事件发生时“give”一个信号量，从而导致信号量的计数值在每个“give”上递增。一个任务每次处理一个事件时都会“take”一个信号，导致信号量的计数值在每次“take”时减少。

计数值是已发生的事件数量与已处理的事件数量之间的差异。该机制如图7.8所示。

用于计数事件的计数信号量所创建的初始计数值为零。

[^18]：使用直接到任务的通知来计数事件比使用计数信号量更有效。直接任务通知直到第9章才包括在内。

2. 资源管理。

在此场景中，计数值表示可用资源的数量。要获得对资源的控制，任务必须首先获得一个信号量，它会减少信号量的计数值。当计数值为零时，就没有空闲的资源了。当任务完成资源时，它会“give”信号量，这将增加信号量的计数值。

将创建用于管理资源的计数信号量，从而使它们的初始计数值等于可用资源的数量。第7章介绍了使用信号量来管理资源。

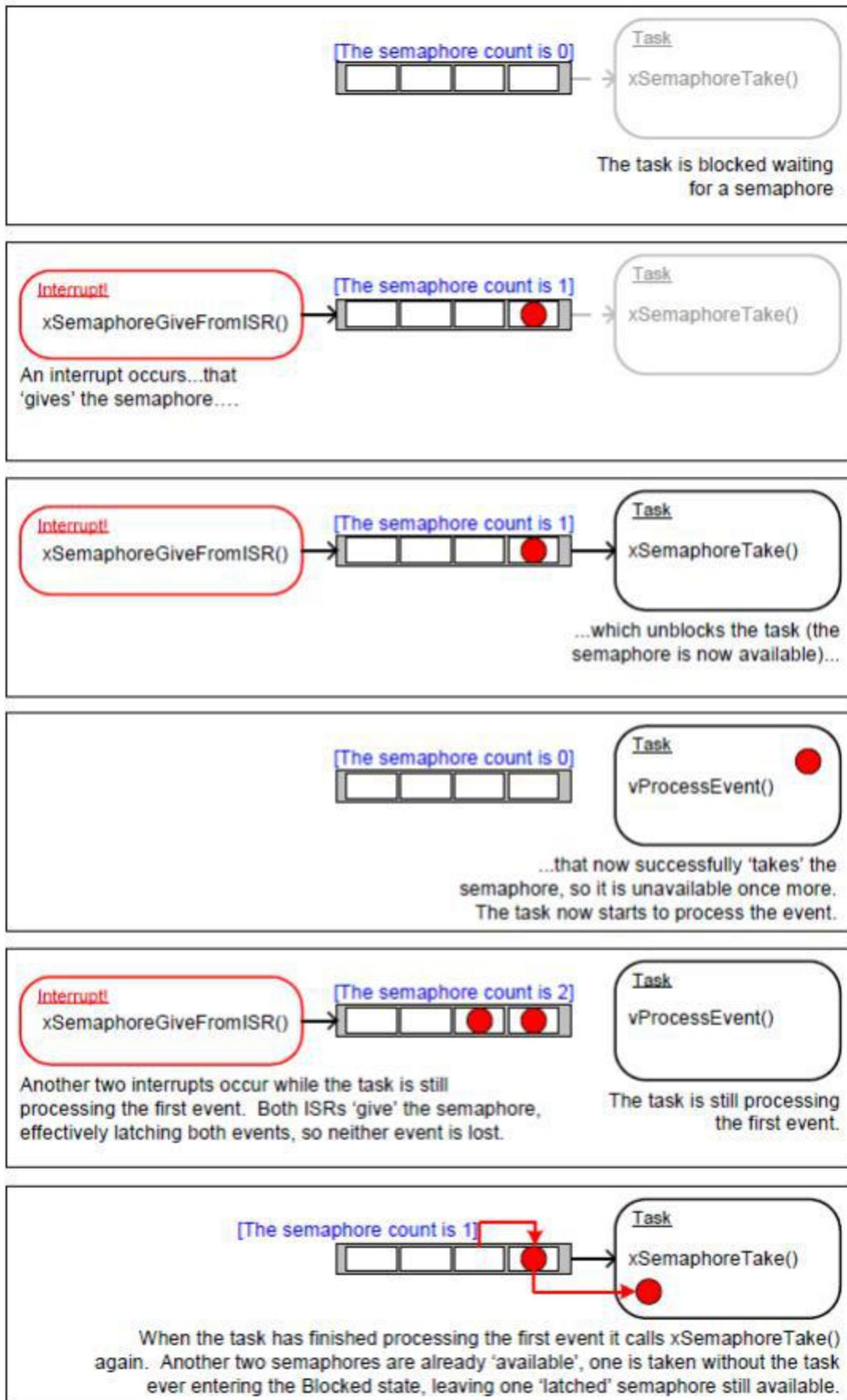


图7.8 使用计数信号量来“计数”事件

7.5.1 xSemaphoreCreateCounting() API 函数

FreeRTOS 还包括 `xSemaphoreCreateCountingStatic()` 函数，该函数在编译时静态分配创建计数信号量所需的内存：所有各种类型的 FreeRTOS 信号量的句柄都存储在 `SemaphoreHandle_t` 类型的变量中。

在使用信号量之前，必须先创建它。要创建计数信号量，请使用 `xSemaphoreCreateCounting()` API 函数。

```
SemaphoreHandle_t xSemaphoreCreateCounting ( UBaseType_t uxMaxCount,
                                              UBaseType_t uxInitialCount );
```

清单7.11 `xSemaphoreCreateCounting()` API 函数原型

参数和返回值

• `uxMaxCount`

信号量将计算到的最大值。继续类比队列的话，`uxMaxCount` 值实际上是队列的长度

当信号量用于计数或锁定事件时，`uxMaxCount` 是可以锁定的最大事件数。

当信号量用于管理对资源集合的访问时，应该将 `uxMaxCount` 设置为可用资源的总数。

• `uxInitialCount`

信号量被创建后的初始计数值。

当信号量用于计数或锁定事件时，`uxInitialCount` 应该设置为零（因为在创建信号量时，我们假设还没有发生事件）。

当信号量用于管理对资源集合的访问时，应将 `uxInitialCount` 设置为等于 `uxMaxCount`（因为在创建信号量时，我们假设所有资源都是可用的）。

• 返回值

如果返回 `NULL`，则无法创建信号量，因为 FreeRTOS 没有足够的堆内存来分配信号量数据结构。第3章提供了关于堆内存管理的更多信息。

如果返回一个非 `null` 值，则表示该信号量已创建成功。返回的值应该存储为所创建的信号量的句柄

。

示例7.2 使用计数信号量使任务与中断同步

例 7.2 通过使用计数信号量代替二进制信号量来改进例 7.1 的实现。main() 已更改为包含对 xSemaphoreCreateCounting() 的调用，而不是对 xSemaphoreCreateBinary() 的调用。新的 API 调用如清单 7.12 所示

```
/*在使用信号量之前，必须显式地创建它。在本例中，将创建了一个计数信号量。创建信号量具有最大值
计数值为10，初始计数值为0。*/
xCountingSemaphore = xSemaphoreCreateCounting ( 10, 0 );
```

清单7.12 例 7.2 中对 xSemaphoreCreateCounting() 的调用用于创建计数信号量

为了模拟高频发生的多个事件，中断服务例程被更改为每个中断多次“给出”信号量。每个事件都被锁存在信号量的计数值中。修改后的中断服务程序如清单 7.13 所示。

```
static uint32_t ulExampleInterruptHandler( void )
{
    BaseType_t xHigherPriorityTaskWoken;

    /*xHigherPriorityTaskWoken 参数必须初始化为 pdFALSE，因为如果需要上下文切换，
    它将在中断安全 API 函数内设置为 pdTRUE*/

    xHigherPriorityTaskWoken = pdFALSE;

    /*多次give “信号量。第一个将解除阻塞延迟的操作
    中断处理任务，下面的“give”是为了演示信号量锁存事件，以允许延迟中断的任务
    依次处理它们，而不会丢失事件。这模拟处理器接收的多个中断，即使在这种情况下事件
    是在单个中断发生中模拟的。*/

    xSemaphoreGiveFromISR ( xCountingSemaphore, &xHigherPriorityTaskWoken );
    xSemaphoreGiveFromISR ( xCountingSemaphore, &xHigherPriorityTaskWoken );
    xSemaphoreGiveFromISR ( xCountingSemaphore, &xHigherPriorityTaskWoken );

    /* 将优先任务唤醒值传递到portYIELD_FROM_ISR ()。如果xHigherPriorityTaskWoken被设置为
    pdTRUE，那么调用portYIELD_FROM_ISR ()将请求一个上下文切换。如果高级任务唤醒仍然是
    pdFALSE，那么调用portYIELD_FROM_ISR ()将没有影响。与大多数FreeRTOS端口不同，Windows
    端口要求ISR返回一个值——返回语句在视窗版本的portYIELD_FROM_ISR ()中。*/
```

```

portYIELD_FROM_ISR (xHigherPriorityTaskWoken);
}

```

清单7.13 实施例7.2所使用的中断服务例程的实现

示例7.1中使用的函数都未修改。

当执行示例7.2时产生的输出如图7.9所示。可以看出，中断处理的任务在每次生成中断时都处理所有三个（模拟）事件。这些事件被锁定在信号量的计数值中，从而允许任务依次处理它们。

图7.9 在执行示例7.2时产生的输出

7.6 将工作延迟到RTOS守护进程任务

到目前为止，所提出的延迟中断处理示例要求应用程序编写者为每个使用延迟处理技术的中断创建一个任务。也可以使用 `xTimerPendFunctionCallFromISR()` [19] API 函数将中断处理推迟到RTOS守护进程任务，这消除了为每个中断创建一个单独的任务。将中断处理延迟到守护进程任务称为“集中式延迟中断处理”。

守护任务最初被称为定时器服务任务，因为它最初只用于执行软件定时器回调函数。因此，`xTimerPendFunctionCall()` 是在 `timers.c` 中实现的，并且根据函数名以实现该函数的文件名作为前缀的约定，函数名以“Timer”为前缀。

第6章描述了与软件定时器相关的FreeRTOS API函数如何向计时器命令队列上的守护进程任务发送命令。调用 `xTimerPendFunctionCall()` 和 `xTimerPendFunctionCallFromISR()`

函数使用相同的定时器命令队列向守护进程任务发送“执行函数”命令。发送到后台守护进程任务的函数随后在后台守护任务的上下文中执行。

集中式延迟中断处理的优点包括：

- 较低的资源使用

它不需要为每个延迟的中断创建一个单独的任务。

- 简化的用户模型

延迟中断处理函数是一个标准的C函数。

集中式延迟中断处理的缺点包括：

- 灵活性较低

不可能分别设置每个延迟中断处理任务的优先级。每个延迟的中断处理函数都在守护进程任务的优先级上执行。如第6章所述，守护进程任务的优先级是由FreeRTOSConfig.h中的configTIMER_TASK_PRIORITY配置的。

- 不确定性

xTimerPendFunctionCallFromISR() 将命令发送到计时器命令队列的后面。
在 xTimerPendFunctionCallFromISR() 将“执行函数”命令发送到队列之前，守护程序任务将处理已存在于计时器命令队列中的命令。

不同的中断有不同的时间约束，因此在同一应用程序中使用这两种延迟中断处理的方法是很常见的。

7.6.1 使用xTimerPendFunctionCallFromISR() 函数

xTimerPendFunctionCallFromISR() 是 xTimerPendFunctionCall() 的中断安全版本。这两个 API 函数都允许应用程序编写者提供的函数由 RTOS 守护程序任务执行，因此在 RTOS 守护程序任务的上下文中执行。要执行的函数以及函数输入参数的值都被发送到计时器命令队列上的守护程序任务。因此，该函数实际执行的时间取决于守护程序任务相对于应用程序中其他任务的优先级。

```

BaseType_t xTimerPendFunctionCallFromISR ( PendedFunction_t
                                           xFunctionToPend,
                                           void* pvParameter1,
                                           uint32_t ulParameter2,
                                           BaseType_t *pxHigherPriorityTaskWoken );

```

清单7.14 函数原型


```
void vPendableFunction( void *pvParameter1, uint32_t ulParameter2 );
```

清单7.15 `xTimerPendFunctionCallFromISR()` 的
`xFunctionToPend` 参数中传递的函数必须符合的原型

参数和返回值

• `xFunctionToPend`

一个指向将在守护进程任务中执行的函数的指针（实际上只是函数名）。该函数的原型必须与清单7.15中所示的原型相同。

• `pvParameter1`

将传递给守护进程任务执行的函数的 `pvParameter1` 参数的值。该参数有一个 `void *` 类型，允许使用它来传递任何数据类型。例如，整数类型可以直接转换为 `void *`，或者可以使用 `void *` 指向一个结构。

• `ulParameter2`

将作为该函数的 `ulParameter2` 参数传递到由守护程序任务执行的函数的值。

• `pxHigherPriorityTaskWoken`

`xTimerPendFunctionCallFromISR()` 写入计时器命令队列。如果 RTOS 守护程序任务处于阻塞状态以等待计时器命令队列上的数据变得可用，则写入计时器命令队列将导致守护程序任务离开阻塞状态。如果守护程序任务的优先级高于当前正在执行的任务（被中断的任务）的优先级，则 `xTimerPendFunctionCallFromISR()` 在内部会将 `*pxHigherPriorityTaskWoken` 设置为 `pdTRUE`。如果 `xTimerPendFunctionCallFromISR()` 将此值设置为 `pdTRUE`，则在退出中断之前必须执行上下文切换。这将确保中断直接返回到守护程序任务，因为守护程序任务将是最高优先级的就绪状态任务。

• 返回值

有两个可能的返回值：

◦ `pdPASS`

如果将“执行函数”命令写入计时器命令队列，则将返回 `pdPASS`。

◦ `pdFAIL`

如果无法将“执行函数”的命令写入计时器命令队列，因为计时器命令队列已满，则将返回 `pdFAIL`。第6章介绍了如何设置计时器命令队列的长度。

示例7.3 集中式延迟中断处理

示例7.3提供了与示例7.1类似的功能。但不使用信号量，也没有专门创建一个任务来执行中断所需的处理。相反，该处理将由RTOS守护进程任务执行。

例 7.3 使用的中断服务程序如清单 7.16 所示。它调用 `xTimerPendFunctionCallFromISR()` 将指向名为 `vDeferredHandlingFunction()` 的函数的指针传递给守护程序任务。延迟中断处理由 `vDeferredHandlingFunction()` 函数执行。

中断服务例程每次执行时都会递增一个名为 `ulParameterValue` 的变量。`ulParameterValue` 在调用 `xTimerPendFunctionCallFromISR()` 时用作 `ulParameter2` 的值，因此当守护任务执行 `vDeferredHandlingFunction()` 时，它也将用作调用 `vDeferredHandlingFunction()` 时的 `ulParameter2` 的值。本示例中未使用该函数的其他参数 `pvParameter1`

```
static uint32_t ulExampleInterruptHandler( void )
{
    static uint32_t ulParameterValue = 0;
    BaseType_t xHigherPriorityTaskWoken;

    /* xHigherPriorityTaskWoken 参数必须初始化为 pdFALSE，
    因为如果需要上下文切换，它将在中断安全 API 函数内设置为 pdTRUE。 */

    xHigherPriorityTaskWoken = pdFALSE;

    /*将指向中断延迟处理函数的指针发送给守护程序任务。延迟处理函数的 pvParameter1
    参数未使用，因此只需设置为 NULL。延迟处理函数的 ulParameter2 参数用于传递一个
    数字，每次执行此中断处理程序时该数字都会加一。 */

    xTimerPendFunctionCallFromISR(
        vDeferredHandlingFunction, /* 处理函数 */
        NULL,
        ulParameterValue,
        &xHigherPriorityTaskWoken );

    ulParameterValue++;

    /* 将 xHigherPriorityTaskWoken 值传递到 portYIELD_FROM_ISR() 中。如果 xHigherP
    riorityTaskWoken 在 xTimerPendFunctionCallFromISR() 内设置为 pdTRUE，则调用 po
    rtYIELD_FROM_ISR() 将控制 FreeRTOS 实时内核请求上下文切换。如果 xHigherPriorit
    yTaskWoken 仍为 pdFALSE，则调用 portYIELD_FROM_ISR() 将无效。与大多数 FreeRTOS
    端口不同，Windows 要求 ISR 返回一个值 - 返回语句位于 Windows 版本的portYIELD
    _FROM_ISR() 内。
    */
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

清单7.16 示例7.3中使用的软件中断处理程序

`vDeferredHandlingFunction()` 的实现如清单 7.17 所示。它打印出一个固定字符串及其 `ulParameter2` 参数的值。

`vDeferredHandlingFunction()` 必须具有清单 7.15 中所示的原型，尽管在本例中实际上只使用了一个参数。

```
static void vDeferredHandlingFunction( void *pvParameter1, uint32_t ulParameter2 ){
    /*处理该事件. 在这种情况下，只需打印出一条消息和ulParameter2. 的值。在本例中不使用pvParameter1
    */
    vPrintStringAndNumber( "Handler function - Processing event ", ulParameter2 );
}
```

清单7.17 执行示例7.3中的中断所需的处理的函数

示例7.3所使用的主函数如清单7.18所示。它比示例7.1中使用的主函数更简单，因为它既不创建信号量，也不创建执行延迟中断处理的任务。

`vPeriodicTask()` 是周期性产生软件中断的任务。它的创建优先级低于守护程序任务的优先级，以确保一旦守护程序任务离开阻塞状态，它就会被守护程序任务抢占。

```
int main( void )
{
    /*生成软件中断的任务的优先级低于守护进程任务的优先级。守护进程任务的优先级由
    FreeRTOSConfig中的configTIMER_TASK_PRIORITY设置。*/
    const UBaseType_t ulPeriodicTaskPriority= configTIMER_TASK_PRIORITY - 1;

    /* 创建将定期生成一个软件中断的任务。*/
    xTaskCreate( vPeriodicTask, "Periodic", 1000, NULL, ulPeriodicTaskPriority,
        NULL );

    /*安装软件中断的处理程序。*/
}
```

所需的语法取决于所使用的 FreeRTOS 端口。此处显示的语法只能与 FreeRTOS Windows 端口一起使用，其中仅模拟此类中断。*/

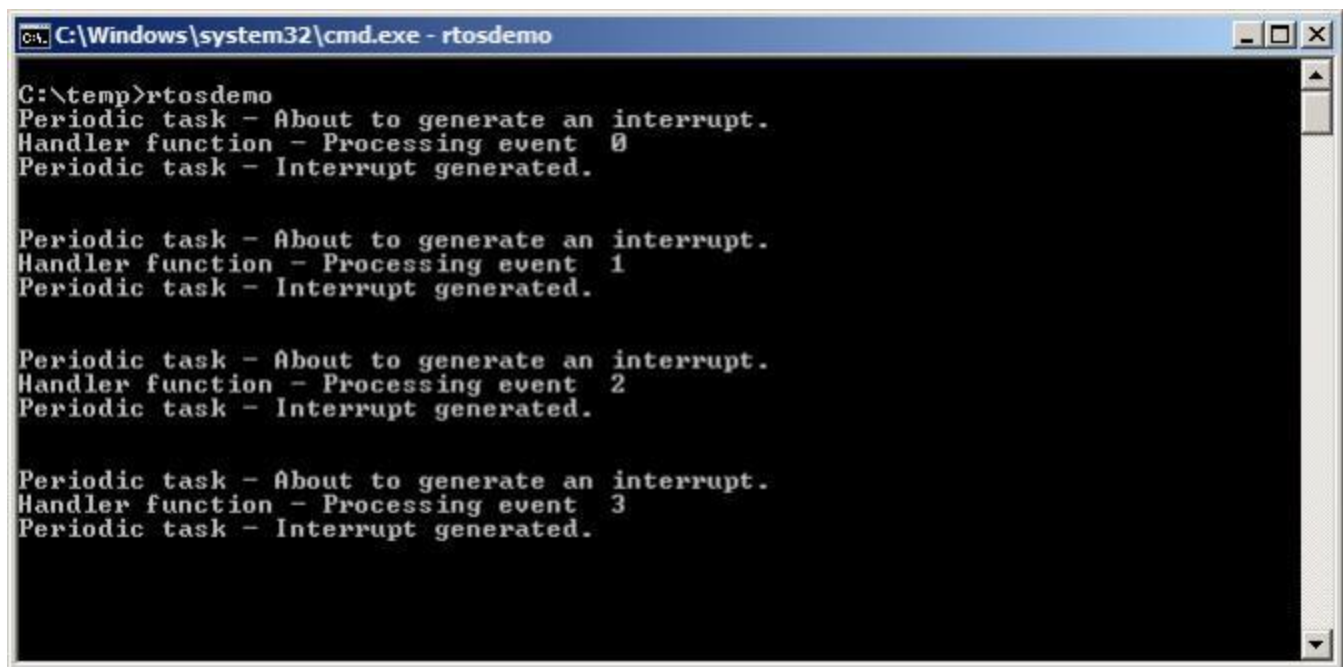
```
vPortSetInterruptHandler( mainINTERRUPT_NUMBER, ulExampleInterruptHandler );

/*启动调度程序，以便已创建的任务开始执行。*/
vTaskStartScheduler();

/*通常，永远不能到达以下行。*/
for(;;)
}
```

清单7.18 示例7.3的主函数的实现

例 7.3 产生如图 7.10 所示的输出。守护任务的优先级高于产生软件中断的任务的优先级，因此一旦产生中断，vDeferredHandlingFunction() 就会被守护任务执行。这导致 vDeferredHandlingFunction() 输出的消息出现在周期性任务输出的两个消息之间，就像使用信号量解锁专用延迟中断处理任务时一样。图 7.11 提供了进一步的解释



```
C:\temp>rtosdemo
Periodic task - About to generate an interrupt.
Handler function - Processing event 0
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler function - Processing event 1
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler function - Processing event 2
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler function - Processing event 3
Periodic task - Interrupt generated.
```

图7.10 执行示例7.3时产生的输出

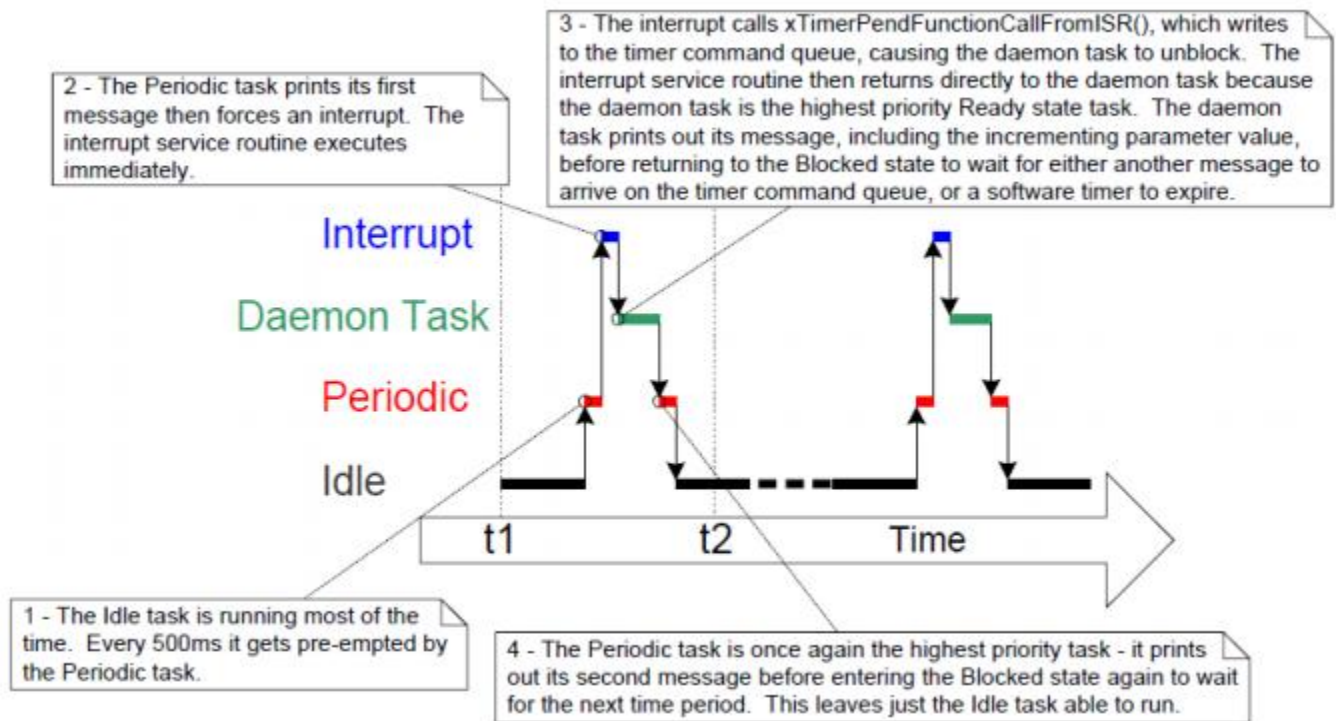


图7.11 执行示例7.3时的执行顺序

7.7 在中断服务例程中使用队列

二进制和计数信号量用于通信事件。队列用于通信事件和传输数据。

`xQueueSendToFrontFromISR()` 是可在中断服务例程中安全使用的 `xQueueSendToFront()` 版本，
`xQueueSendToBackFromISR()` 是可在中断服务例程中安全使用的 `xQueueSendToBack()` 版本，
`xQueueReceiveFromISR()` 是 `xQueueReceive()` 可以安全地在中断服务例程中使用。

7.7.1 `xQueueSendToFrontFromISR()` 和 `xQueueSendToBackFromISR()` API函数

```
BaseType_t xQueueSendToFrontFromISR ( QueueHandle_t xQueue,
                                     const void *pvItemToQueue
                                     BaseType_t *pxHigherPriorityTaskWoken );
```

清单7.19 `xQueueSendToFrontFromISR()` API函数原型

```
BaseType_t xQueueSendToBackFromISR ( QueueHandle_t xQueue,
                                     const void *pvItemToQueue
                                     BaseType_t *pxHigherPriorityTaskWoken );
```

清单7.20 xQueueSendToBackFromISR() API函数原型

xQueueSendFromISR() and xQueueSendToBackFromISR()在功能上是等价的。

参数和返回值

- xQueue

数据发送（写入）到的队列的句柄。队列句柄将从用于创建队列的 xQueueCreate() 调用中返回

- pvItemToQueue

指向要放入队列的项目的指针。

队列将保存的每个项目的大小是在创建队列时定义的，因此这么多字节将从 pvItemToQueue 复制到队列存储区域。

- pxHigherPriorityTaskWoken

单个队列上可能有一个或多个任务被阻塞，等待数据可用。调用xQueueSendToFrontFromISR() 或 xQueueSendToBackFromISR() 可以使数据可用，从而导致这样的任务离开阻塞状态。如果调用API函数导致任务离开阻塞状态，并且未阻塞任务的优先级高于当前执行的任务（被中断的任务），那么，API函数将任务设置为pdTRUE。

如果 xQueueSendToFrontFromISR() 或 xQueueSendToBackFromISR() 将此值设置为 pdTRUE，则应在退出中断之前执行上下文切换。这将确保中断直接返回到最高优先级的就绪态任务

- 返回值

有两个可能的返回值：

- pdPASS

只有当数据已成功发送到队列时，才会返回pdPASS。

- errQUEUE_FULL

如果由于队列已满，数据无法发送到队列，则返回errQUEUE_FULL。

7.7.2 使用ISR队列时的注意事项

队列提供了一种简单而方便的方式来将数据从中断传递到任务，但是如果数据到达的频率很高，那么使用队列并不有效。

FreeRTOS下载中的许多演示应用程序都包括一个简单的UART驱动程序，它使用一个队列从UART的接收ISR中传递字符。在这些演示中，使用队列有两个原因：用来演示队列从ISR中使用，并故意加载系统，以测试FreeRTOS端口

以这种方式使用队列的 ISR 绝对不是为了代表高效的设计，除非数据到达缓慢，否则建议生产代码不要复制此技术。适用于生产代码的更有效的技术包括：

- 使用直接内存访问（DMA）硬件来接收和缓冲字符。该方法实际上没有软件开销。
然后，可以使用直接任务通知 [^20] 来解锁仅在检测到传输中断后才处理缓冲区的任务

[^20]：直接到任务通知提供了解除ISR阻塞的最有效的方法。直接任务通知详见第9章，任务通知。

- 将每个接收到的字符复制到一个线程安全的RAM缓冲区中[^21]。同样的，一个直接到任务的通知可以用于取消阻塞将在接收到完整的消息后或在检测到传输中断后处理缓冲区的任务。

[^21]：作为 FreeRTOS+TCP (<https://www.FreeRTOS.org/tcp>) 的一部分提供的“流缓冲区”可用于此目的。

在ISR中直接处理接收到的字符，然后使用队列将处理数据（而不是原始数据）的结果发送给任务。之前的图5.4已经证明了这一点。

实例7.4 从中断内的队列中进行发送和接收

此示例演示了在同一中断中使用 `xQueueSendToBackFromISR()` 和 `xQueueReceiveFromISR()`。和以前一样，为了方便起见，中断是由软件生成的。

将创建一个定期任务，每200毫秒向一个队列发送5个数字。它只在发送了所有五个值之后才会生成一个软件中断。该任务的实现如清单7.21所示。

```
static void vIntegerGenerator( void *pvParameters )
{
    TickType_t xLastExecutionTime;
    uint32_t ulValueToSend = 0;
    int i;

    /*初始化调用所使用的变量。*/
    xLastExecutionTime =xTaskGetTickCount();

    for(;;)
    {
        /*这是一项定期执行的任务。阻塞直到是时候再次运行。该任务将每200个ms执行一次。*/
        vTaskDelayUntil( &xLastExecutionTime, pdMS_TO_TICKS( 200 ) );

        /*向队列发送5个数字，每个值比前一个值高一个。通过中断的方式从队列中读取这些数。*/
    }
}
```



```

        服务程度中断服务例程总是会排空队列，因此此任务保证能够写入所有五个值，而不
        需要指定阻塞时间。*/
    for ( i = 0; i < 5; i++)
    {
        xQueueSendToBack ( xIntegerQueue, &ulValueToSend, 0 );
        ulValueToSend++;
    }

    /* 生成中断，以便中断服务例程可以从队列中读取这些值。用于生成软件中断的语法依
       赖于正在使用的FreeRTOS端口。下面使用的语法只能与FreeRTOS Windows端口一起使
       用，在该端口中只能模拟这些中断。*/
    vPrintString( "Generator task - About to generate an interrupt.\r\n" );
    vPortGenerateSimulatedInterrupt( mainINTERRUPT_NUMBER );
    vPrintString( "Generator task - Interrupt generated.\r\n\r\n\r\n" );
}
}

```

清单7.21 示例7.4中写入队列的任务的实现

中断服务例程重复调用xQueueReceiveFromISR()，直到周期性任务写入队列的所有值都被读出，并且队列为空。每个接收值的最后两位用作字符串数组的索引。然后，通过调用 xQueueSendFromISR() 将指向相应索引位置处的字符串的指针发送到不同的队列。中断服务程序的实现如清单7.22所示

```

static uint32_t ulExampleInterruptHandler( void )
{
    BaseType_t xHigherPriorityTaskWoken;
    uint32_t ulReceivedNumber;

    /*这些字符串被声明为静态常量，以确保它们不被分配到中断服务例程的堆栈上，因此即使
       在中断服务例程没有执行时也会存在。*/

    static const char *pcStrings[] =
    {
        "String 0\r\n",
        "String 1\r\n",
        "String 2\r\n",
        "String 3\r\n"
    };

    /*与往常一样，xHigherPriorityTaskWoken 初始化为 pdFALSE，
       以便能够检测到它在中断安全 API 内设置为 pdTRUE
    */
}

```


请注意，由于中断安全 API 函数只能将 `xHigherPriorityTaskWoken` 设置为 `pdTRUE`，因此在对 `xQueueReceiveFromISR()` 的调用和对 `xQueueSendToBackFromISR()` 的调用中使用相同的 `xHigherPriorityTaskWoken` 变量是安全的

```
xHigherPriorityTaskWoken = pdFALSE;

/*从队列中读取，直到队列为空。*/
while( xQueueReceiveFromISR( xIntegerQueue, &ulReceivedNumber,
                             &xHigherPriorityTaskWoken ) != errQUEUE_EMPTY )
{
    /*将接收到的值截断到最后两位（包括值0到3），然后使用截断的值作为pcStrings[]
       数组的索引，以选择要发送给另一个队列的字符串（char *）。*/
    ulReceivedNumber &= 0x03;
    xQueueSendToBackFromISR ( xStringQueue,
                             &pcStrings[ ulReceivedNumber ],
                             &xHigherPriorityTaskWoken );
}

/*如果从xIntegerQueue队列接收，导致任务离开阻塞
   状态，如果离开阻塞状态的任务的优先级高于运行状态下的任务的优先级，那么在
   xQueueReceiveFromISR中将被设置为pdTRUE。

   如果发送到xStringQueue导致任务离开阻塞状态，并且离开阻塞状态的的任务的优先
   级，那么xHigherPriorityTaskWoken将在xQueueSendToBackFromISR中设置为pdTRUE。

   xHigherPriorityTaskWoken 用作 portYIELD FROM ISR() 的参数。
   如果 xHigherPriorityTaskWoken 等于 pdTRUE，则调用 portYIELD_FROM_ISR() 将请求上
   下文切换。如果 xHigherPriorityTaskWoken 仍为 pdFALSE，则调用 portYIELD_FROM_ISR()
   将无效。

   Windows端口使用的portYIELD_FROM_ISR（）的实现包括一个返回语句，这就是为什
   么这个函数不显式地返回一个值的原因。*/
portYIELD_FROM_ISR (xHigherPriorityTaskWoken);
}
```

清单7.22 实施例7.4所使用的中断服务例程的实现

从队列上的中断服务例程块接收字符指针的任务，直到消息到达，并在收到每个字符串时打印出来。其实现如清单7.23所示。

```

static void vStringPrinter( void *pvParameters )
{
    char *pcString;

    for(;;){

        /*阻塞在队列上等待数据到达。*/
        xQueueReceive( xStringQueue, &pcString, portMAX_
        DELAY );
        /*打印出收到的字符串。*/
        vPrintString ( pcString );
    }
}

```

清单7.23 打印出示例7.4中从中断服务例程接收到的字符串的任务

通常，main（）会在启动调度程序之前创建所需的队列和任务。其实现如清单7.24所示。

```

int main( void )
{
    /*在使用一个队列之前，必须先创建该队列。创建本示例中使用的两个队列。一个队列可以保
    存uint32_t类型的变量，另一个队列可以保存char*类型的变量。这两个队列最多可以容纳
    10个项目。一个真正的应用程序应该检查返回的值
    以确保已成功创建队列。*/
    xIntegerQueue = xQueueCreate( 10, sizeof( uint32_t ) );
    xStringQueue = xQueueCreate( 10, sizeof( char * ) );

    /*创建使用队列将整数传递给中断的任务
    创建该任务的优先级为1。*/
    xTaskCreate( vIntegerGenerator, "IntGen", 1000, NULL, 1, NULL );

    /*创建将打印出从中断服务程序中发送到它的字符串的任务
    此任务是在更高的优先级2 创建的。*/
    xTaskCreate( vStringPrinter, "String", 1000, NULL, 2, NULL );

    /*安装软件中断的处理程序。这一点所需的语法取决于所使用的FreeRTOS端口。这里显示的
    语法只能与FreeRTOS Windows端口一起使用，而这些中断只是模拟的。*/
    vPortSetInterruptHandler( mainINTERRUPT_NUMBER, ulExampleInterruptHandler );

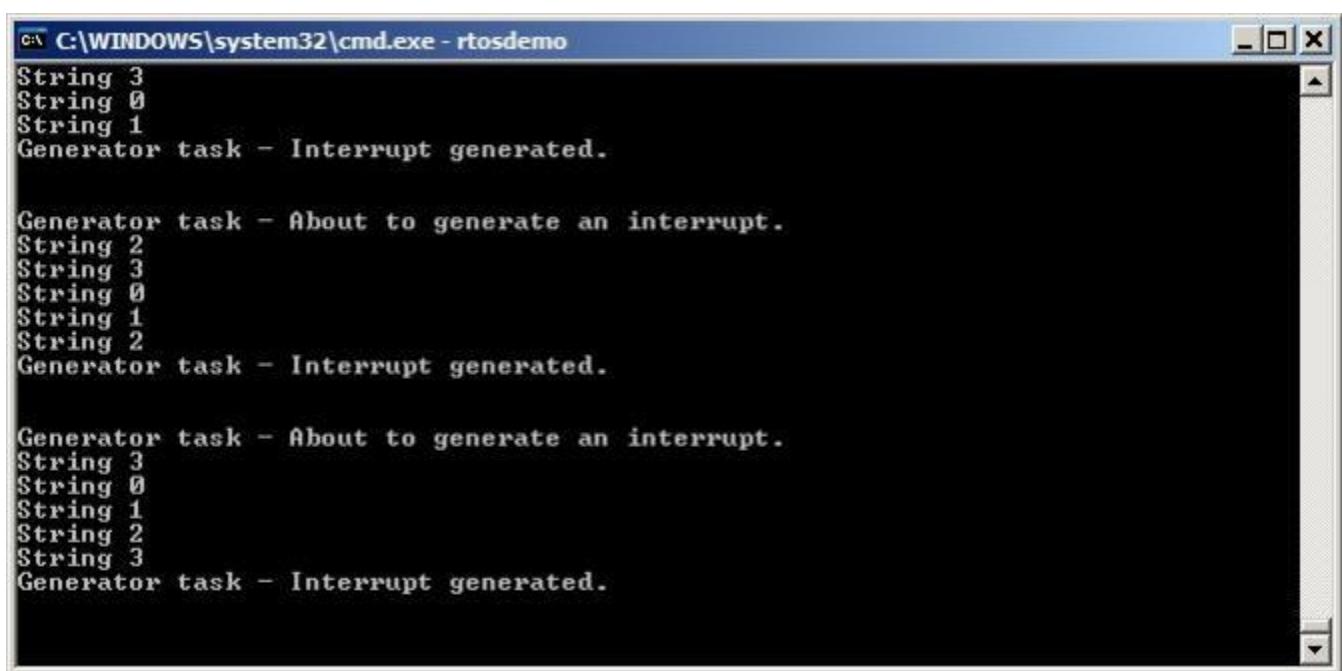
    /*启动调度程序，以便已创建的任务开始执行。*/
    vTaskStartScheduler();
}

```

```
/*如果一切都好，那么主函数将永远不会到达这里，因为调度程序现在将运行任务。如果主  
函数确实到达了这里，那么很可能没有足够的堆内存来创建空闲任务。第2章提供了关  
于堆内存管理的更多信息。*/  
for( ; ; )  
}
```

清单7.24 示例7.4中的主函数

当执行示例7.4时产生的输出如图7.12所示。可以看出，中断接收所有的五个整数，并产生五个字符串作为响应。更多的解释如图7.13所示。



```
C:\WINDOWS\system32\cmd.exe - rtosdemo  
String 3  
String 0  
String 1  
Generator task - Interrupt generated.  
  
Generator task - About to generate an interrupt.  
String 2  
String 3  
String 0  
String 1  
String 2  
Generator task - Interrupt generated.  
  
Generator task - About to generate an interrupt.  
String 3  
String 0  
String 1  
String 2  
String 3  
Generator task - Interrupt generated.
```

图7.12 执行示例7.4时产生的输出

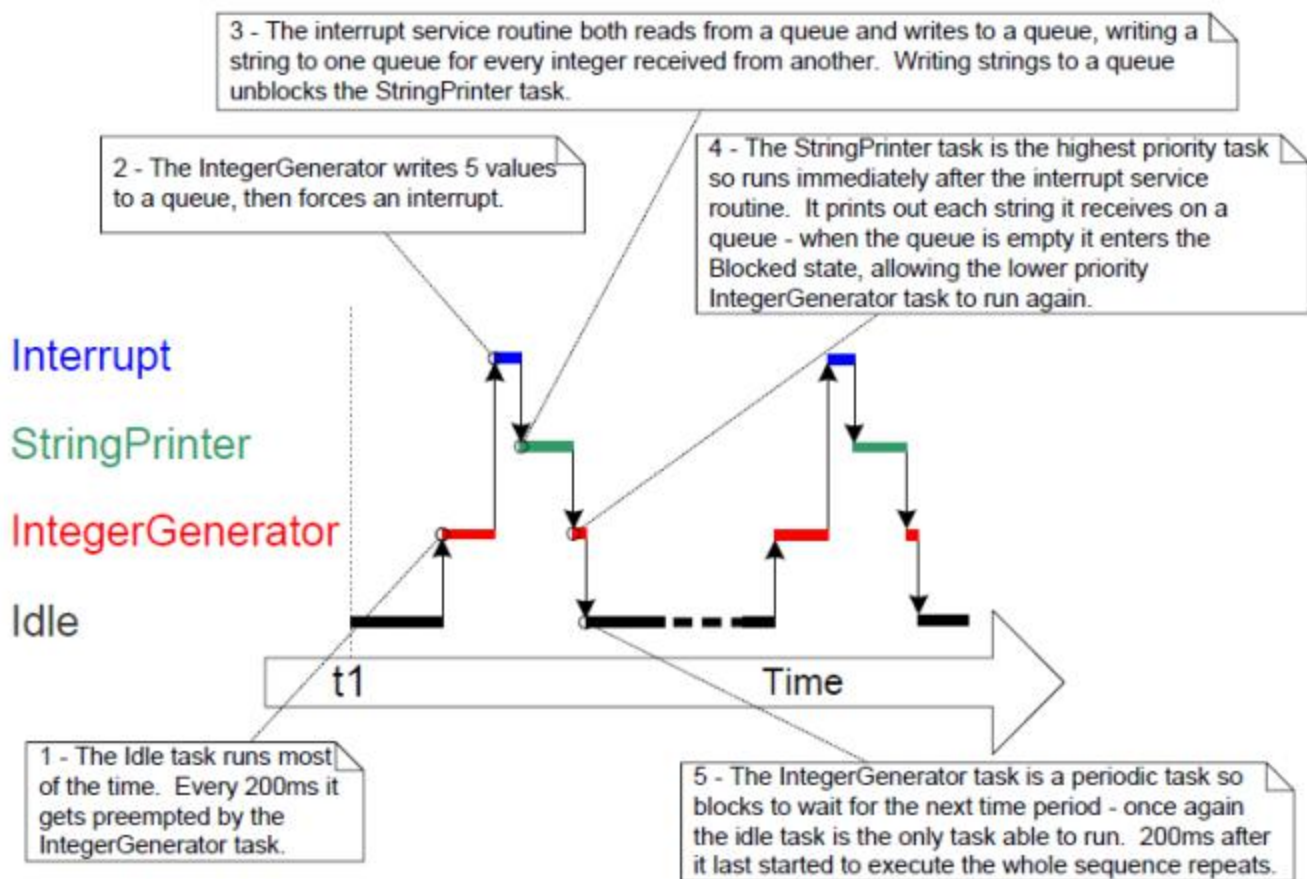


图7.13 示例7.4所产生的执行顺序

7.8 中断嵌套

任务优先级和中断优先级之间出现混淆是很常见的。本节讨论中断优先级，即中断服务例程（ISR）相对于彼此执行的优先级。分配给任务的优先级与分配给中断的优先级没有任何关系。硬件决定 ISR 何时执行，而软件决定任务何时执行。响应硬件中断而执行的 ISR 将中断任务，但任务不能抢占 ISR。

支持中断嵌套的端口需要在FreeRTOSConfig.h中定义下面详细介绍的一个或两个常量。

`configMAX_SYSCALL_INTERRUPT_PRIORITY`和`configMAX_API_CALL_INTERRUPT_PRIORITY`都定义了相同的属性。旧的FreeRTOS端口使用`configMAX_SYSCALL_INTERRUPT_PRIORITY`，而更新的FreeRTOS端口使用`configMAX_API_CALL_INTERRUPT_PRIORITY`。

控制中断嵌套的常量

- `configMAX_SYSCALL_INTERRUPT_PRIORITY`或`configMAX_API_CALL_INTERRUPT_PRIORITY`

设置中断安全的可以调用的FreeRTOSAPI函数的最高中断优先级。

- `configKERNEL_INTERRUPT_PRIORITY`

设置提示中断所使用的中断优先级，并且必须始终设置为尽可能低的中断优先级。

如果正在使用的FreeRTOS端口也不使用`configMAX_SYSCALL_INTERRUPT_PRIORITY`常量，那么任何使用中断安全的FreeRTOS API函数的中断也必须在`configKERNEL_INTERRUPT_PRIORITY`定义的优先级上执行。

每个中断源都具有一个数字优先级和一个逻辑优先级：

- 数字优先级

数字优先级只是分配给中断优先级的数字。例如，如果分配一个中断的优先级为7，那么它的数字优先级为7。同样地，如果一个中断的优先级为200，那么它的数字优先级为200。

- 逻辑优先级

中断的逻辑优先级描述了该中断的优先级高于其他中断的优先级。

如果两个优先级不同的中断同时发生，那么处理器将对两个中断中逻辑优先级较高的中断执行ISR，然后对两个中断中逻辑优先级较低的中断执行ISR。

中断可以中断（嵌套）任何具有较低逻辑优先级的中断，但中断不能中断（嵌套）任何具有相同或更高逻辑优先级的中断。

中断的数字优先级和逻辑优先级之间的关系取决于处理器架构；在某些处理器上，分配给中断的数字优先级越高，中断的逻辑优先级就越高，而在其他处理器架构上，分配给中断的数字优先级越高，中断的逻辑优先级就越低。

通过将 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 设置为比 `configKERNEL_INTERRUPT_PRIORITY` 更高的逻辑中断优先级，可以创建完整的中断嵌套模型。图 7.14 对此进行了演示，其中显示了一个场景：

-
- 该处理器有七个独特的中断优先级。
 - 分配的数字优先级为7的中断比分配的数字优先级为1的中断具有更高的逻辑优先级。
 - `configKERNEL_INTERRUPT_PRIORITY`被设置为1。
 - `configMAX_SYSCALL_INTERRUPT_PRIORITY`被设置为3。

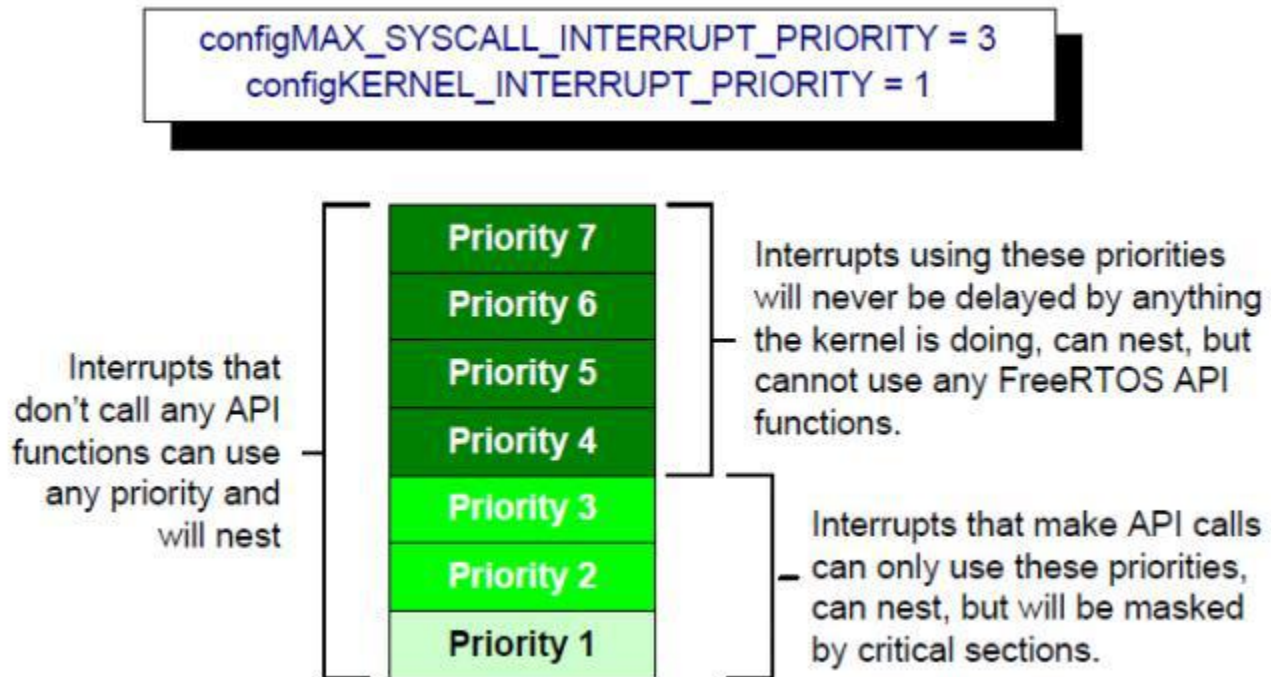


图7.14 影响中断嵌套行为的常量

参见图7.14:

- 当内核或应用程序位于临界区时，使用优先级 1 到 3（含）的中断将被阻塞执行。以这些优先级运行的 ISR 可以使用中断安全的 FreeRTOS API 函数。关键部分在第 7 章中描述
- 使用优先级 4 或更高优先级的中断不受临界区的影响，因此调度程序所做的任何操作都不会阻塞这些中断立即执行（在硬件本身的限制范围内）。以这些优先级执行的 ISR 无法使用任何 FreeRTOS API 函数
- 通常，需要非常严格的计时精度的功能（例如电机控制）将使用高于 configMAX_SYSCALL_INTERRUPT_PRIORITY 的优先级，以确保调度程序不会在中断响应时间中引入抖动

7.8.1 给ARM Cortex-M^[22]和ARM GIC用户的一个说明

^[22]：本节仅部分适用于Cortex-M0和Cortex-M0+核心。

Cortex-M处理器上的中断配置令人困惑，而且容易出错。为了帮助您的开发，FreeRTOS Cortex-M端口会自动检查中断配置，但仅在定义了configASSERT（）时才会检查。configASSERT（），详见第11.2节。

ARM Cortex内核和ARM通用中断控制器（GICs）使用数字上的低优先级数字来表示逻辑上的高优先级中断。这似乎是违反了直觉的，而且很容易被忘记。如果你愿意为中断分配一个逻辑上的低优先级，然后必须为它分配一个高数值值。

如果您希望为中断分配一个逻辑上的高优先级，那么必须为它分配一个低数值值。

Cortex-M中断控制器允许使用最多8位来指定每个中断优先级，使255成为可能的最低优先级。零是最高的优先级。然而，Cortex-M微控制器通常只实现8个可能位中的一个子集。实际实现的位数取决于微控制器族。

当仅实现了八个可能位的子集时，只能使用字节的最高有效位，而最低有效位则未实现。未实现的位可以取任何值，但通常将它们设置为 1。图 7.15 对此进行了演示，该图显示了二进制 101 的优先级如何存储在实现四个优先级位的 Cortex-M 微控制器中



图7.15 实现四个优先级位的优先级Cortex-M微控制器如何存储二进制101的优先级

在图 7.15 中，二进制值 101 已移至最高有效四位，因为最低有效四位未实现。未实现的位已设置为 1

某些库函数期望在将优先级值向上移至已实现的（最高有效）位后指定优先级值。当使用这样的函数时，图 7.15 所示的优先级可以指定为十进制 95。十进制 95 是二进制 101 上移 4 位，得到二进制 101nnnn（其中“n”是未实现的位），并且未实现的位设置为1 生成二进制 1011111。

某些库函数期望在将优先级值向上移至已实现的（最高有效）位之前指定优先级值。当使用这样的函数时，图 7.15 中所示的优先级必须指定为十进制 5。十进制 5 是没有任何移位的二进制 101

`configMAX_SYSCALL_INTERRUPT_PRIORITY`和`configKERNEL_INTERRUPT_PRIORITY`必须以允许它们直接写入Cortex-M寄存器的方式指定，因此在优先级值上移到实现的位之后。

`configKERNEL_INTERRUPT_PRIORITY`必须始终设置为尽可能低的中断优先级。未实现的优先级位可以设置为1，因此无论实际实现了多少优先级位，该常量都可以始终设置为255。

Cortex-M中断将默认为零-可能的最高优先级。Cortex-M硬件的实现不允许

Cortex-M 中断的默认优先级为零——可能的最高优先级。 Cortex-M 硬件的实现不允许将 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 设置为 0，因此使用 FreeRTOS API 的中断的优先级绝不能保留为默认值。

8 资源管理

8.1 本章简介及适用范围

在多任务处理系统中，如果一个任务开始访问一个资源，但在转换出运行状态之前没有完成其访问，则可能会出现错误。如果任务使资源处于不一致的状态，那么通过任何其他任务或中断访问同一资源都可能导致数据损坏或其他类似问题。

以下是一些示例：

- 访问外围设备

考虑以下场景，其中两个任务试图写入一个液晶显示器（LCD）。

1. 任务A执行并开始将字符串“Helloworld”写入LCD。
2. 任务A在输出字符串“Hello”后被任务B抢占。
3. 任务B将“Abort, Retry, Fail?”在进入阻塞状态之前写入LCD。
4. 任务A从它被抢占的点开始，并完成其字符串的其余字符“orld”的输出。

液晶显示器现在显示损坏的字符串“Hello wAbort, Retry, Fail?orld”。

读取、修改、写入操作

- 清单8.1显示了一行C代码，以及一个关于C代码通常如何转换为汇编代码的示例。可以看出，PORTA的值首先从内存中读取到一个寄存器中，在寄存器中进行修改，然后再写回内存中。这被称为读、修改、写操作。

```
/*正在编译的C代码。*/
portA |= 0x01;

/*在编译C代码时产生的汇编代码。*/
LOAD R1, [#PORTA] ; Read a value from PORTA into R1
MOVE R2, #0x01 ; Move the absolute constant 1 into R2
OR R1, R2 ; Bitwise OR R1 (PORTA) with R2 (constant 1)
STORE R1, [#PORTA] ; Store the new value back to PORTA
```

清单8.1 一个读、修改、写序列的示例

这是一个“非原子”操作，因为它需要多个指令来完成，并且可以被中断。考虑以下场景，其中有两个任务试图更新内存映射寄存器称为PORTA。

1. 任务A将PORTA的值加载到一个寄存器中——操作的读取部分。
2. 任务A在任务B完成同一操作的修改和写入部分之前被抢占。
3. 任务B更新PORTA的值，然后进入已阻塞的状态。
4. 任务A从它被抢占的点开始继续。它在将更新后的值写入PORTA之前，修改它在寄存器中已经持有的PORTA值的副本。

在这种情况下，任务 A 更新并写回 PORTA 的过期值。在任务 A 获取 PORTA 值的副本之后、任务 A 将其修改后的值写回 PORTA 寄存器之前，任务 B 修改 PORTA。当任务 A 写入 PORTA 时，它会覆盖任务 B 已执行的修改，从而有效地破坏 PORTA 寄存器值。

本示例使用外围寄存器，但在对变量执行读、修改、写操作时也适用同样的原理。

•对变量的非原子访问

更新结构体的多个成员，或更新大于体系结构自然字大小的变量（例如，更新 16 位机器上的 32 位变量），都是非原子操作的示例。如果它们被中断，可能会导致数据丢失或损坏

•函数重入

如果一个函数可以从多个任务或两个任务和中断中调用该函数，则该函数就是“重入”。重入函数被称为“线程安全的”，因为它们可以从多个执行线程中访问，而不会有数据或逻辑操作被损坏的风险。

每个任务都维护自己的堆栈和自己的一组处理器（硬件）寄存器值。如果一个函数不访问除存储在堆栈上或保存在寄存器中的数据以外的任何数据，那么该函数是可重入的，并且线程是安全的。清单8.2是一个可重入式函数的示例。清单8.3是一个不可重入的函数的示例。

如果应用程序使用newlib C库，它必须在FreeRTOSConfig.h中将configUSE_NEWLIB_REENTRANT设置为1，以确保正确分配newlib所需的线程本地存储。

如果应用程序使用C库，它必须在FreeRTOSConfig.h中将configUSE_PICOLIBC_TLS设置为1，以确保configUSE_PICOLIBC_TLS所需的线程本地存储正确分配。

如果应用程序使用任何其他C库并且需要线程本地存储（TLS），它必须在FreeRTOSConfig.h中将configUSE_C_RUNTIME_TLS_SUPPORT设置为1，并且必须实现以下操作宏

- configTLS_BLOCK_TYPE -每个任务TLS块的类型。
- configINIT_TLS_BLOCK -初始化每个任务TLS块。

- `configSET_TLS_BLOCK` -更新当前的TLS块。在上下文切换期间被调用，以确保使用了正确的TLS块。
- `configDEINIT_TLS_BLOCK` -释放TLS块。

```
/*一个参数被传递到该函数中。这将在堆栈上传递，或者在处理器寄存器中传递。这两种方式都是安全的，因为调用函数的每个任务或中断都维护自己的堆栈和自己的寄存器值集，因此调用函数的每个任务或中断都将有自己的lVar1副本。*/
long lAddOneHundred( long lVar1 )
{
    /*这个函数范围变量也将被分配给堆栈或寄存器，这取决于编译器或优化级别。调用此函数的每个任务或中断都将有它自己的lVar2副本。*/

    /*
        long lVar2;

        lVar2 = lVar1 + 100;
        return lVar2;
    */
}
```

清单8.2: 一个可重入式函数的示例

```
/*在这种情况下，lVar1是一个全局变量，所以每个调用的任务
    l非感官函数将访问变量的同一单一副本。*/const Var1;

long lNonsenseFunction( void )
{
    /* lState是静态的，因此没有在堆栈上分配。调用此函数的每个任务都将访问该变量的同一单一副本。*/
    static long lState = 0;
    long lReturn;

    if (lState)
    {
        case 0: lReturn = lVar1 + 10;
                lState = 1;
                break;

        case 1: lReturn = lVar1 + 20;
                lState = 0;
                break;
    }
}
```

清单8.3 一个不可重入的函数示例

8.1.1 互斥

为了确保始终保持数据一致性，必须使用“互斥”技术来管理对在任务之间共享或在任务和中断之间共享的资源的访问。其目标是确保一旦任务开始访问不可重入且非线程安全的共享资源，同一任务就对该资源具有独占访问权限，直到该资源返回到一致状态。

FreeRTOS提供了几个可以用来实现互斥的特性，但最好的互斥方法是（只要可能，因为它通常不实用）设计应用程序，使资源不共享，每个资源只能从一个任务访问。

8.1.2 范围

本章包括：

何时以及为什么需要进行资源管理和控制。

什么是一个关键的部分。

- 相互排斥意味着什么。
- 挂起调度程序意味着什么。
- 如何使用互斥锁。
- 如何创建和使用一个守门人的任务。
- 什么是优先级反转，以及优先级继承如何可以减少（但不是消除）其影响。

8.2 临界区和暂停调度

8.2.1 基础临界区

基础临界区是分别被对宏taskENTER_CRITICAL（）和taskEXIT_CRITICAL（）的调用所包围的代码区域。临界部分也被称为临界区。

taskENTER_CRITICAL（）和taskEXIT_CRITICAL（）不接受任何参数，或返回一个值[²³]。清单8.4展示了它们的使用情况。

[²³]：类似于函数的宏并不像实际的函数那样“返回一个值”。当宏“返回值”时，本书将其视为函数。

```
/*通过将 PORTA 寄存器放置在临界区中，确保对 PORTA 寄存器的访问不会被中断。进入临界区。
*/
```

```
taskENTER_CRITICAL() ;
```

```
/*在调用taskENTER_CRITICAL（）和调用taskEXIT_CRITICAL（）之间不能切换到另一个任务
。中断仍然可以在允许中断嵌套的FreeRTOS端口上执行，但只有逻辑优先级高于分配给
configMAX_SYSCALL_INTERRUPT_PRIORITY常量的值的中断——而这些中断是
```

```

    不允许调用FreeRTOS API函数。*/
    portA |= 0x01;

    /*对PORTA的访问已经完成，因此退出临界区是安全的。*/ taskEXIT_CRITICAL() ;

```

清单8.4 使用临界区来保护对寄存器的访问

本书随附的示例项目使用名为 `vPrintString()` 的函数将字符串写入标准输出，即使用 FreeRTOS Windows 端口时的终端窗口。 `vPrintString()` 从许多不同的任务中调用；因此，理论上，它的实现可以使用关键部分来保护对标准输出的访问，如清单 8.5 所示

```

void vPrintString( const char *pcString )
{
    /*将字符串写到标准输出，使用一个临界区作为一个粗糙的互斥
    */
    taskENTER_CRITICAL() ;
    {
        printf ( "%s" 、 pcString) ;
        fflush (stdout) ;
    }
    taskEXIT_CRITICAL() ;
}

```

清单8.5 `vPrintString()` 一个可能的实现

以这种方式实现的临界区是提供互斥的一种非常粗糙的方法。它们的工作原理是完全禁用中断，或达到 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 设置的中断优先级，这取决于所使用的FreeRTOS端口。抢占的上下文切换只能在中断中发生，因此，只要中断仍然处于禁用状态，调用 `taskENTER_CRITICAL()` 的任务就可以保证保持在运行状态，直到退出临界区。

基本的临界部分必须保持非常短，否则它们将对中断响应时间产生不利影响。每个对 `taskENTER_CRITICAL()` 的每个调用都必须与对 `taskEXIT_CRITICAL()` 的调用紧密配对。因此，标准输出（输出或计算机写入输出数据的流）不应该使用临界区（如清单8.5所示）进行保护，因为写入终端可能是一个相对较长的操作。本章中的示例探讨了替代的解决方案。

临界部分被嵌套是安全的，因为内核会计算嵌套的深度。只有当嵌套深度返回到零时，关键段才会退出，即对之前对 `taskENTER_CRITICAL()` 的每次调用都执行了一次对 `taskEXIT_CRITICAL()` 的调用。

调用 `taskENTER_CRITICAL()` 和 `taskEXIT_CRITICAL()` 是任务更改运行FreeRTOS 的处理器中断启用状态的唯一合法方法。通过任何其他方式更改中断启用状态都会使宏的嵌套计数无效

`taskENTER_CRITICAL()` 和 `taskEXIT_CRITICAL()` 不会以 “FromISR” 结尾，因此不能从中断服务例程中调用。
`taskENTER_CRITICAL_FROM_ISR()` 是 `taskENTER_CRITICAL()` 的中断安全版本，而
`taskEXIT_CRITICAL_FROM_ISR()` 是 `taskEXIT_CRITICAL()` 的中断安全版本。中断安全版本只提供给允许中断嵌套的FreeRTOS端口——它们在不允许中断嵌套的端口中将会弃用。

`taskENTER_CRITICAL_FROM_ISR()` 返回一个必须传递到对 `taskEXIT_CRITICAL_FROM_ISR()` 的匹配调用中的值。清单8.6说明了这一点。

```
void vAnInterruptServiceRoutine( void )
{
    /*声明一个变量，其中将保存 taskENTER_CRITICAL_FROM_ISR() 的返回值。
    */
    UBaseType_t uxSavedInterruptStatus;

    /*这部分 ISR 可以被任何更高优先级的中断中断
    */

    /*使用taskENTER_CRITICAL_FROM_ISR() 来保护此ISR的一个区域。保存从
    taskENTER_CRITICAL_FROM_ISR() 返回的值，以便它可以保存
    被传递到对taskEXIT_CRITICAL_FROM_ISR() 的匹配调用中。*/
    uxSavedInterruptStatus = taskENTER_CRITICAL_FROM_ISR();

    /*ISR 的这一部分位于对 taskENTER_CRITICAL_FROM_ISR() 和 taskEXIT_CRITICAL_FROM_ISR()
    的调用之间，因此只能由优先级高于 configMAX_SYSCALL_INTERRUPT_PRIORITY 常量设置的中断来中断
    */

    /*通过调用taskEXIT_CRITICAL_FROM_ISR() 再次退出临界区，传递
    taskEXIT_CRITICAL_FROM_ISR() 调用返回的值*/
    taskEXIT_CRITICAL_FROM_ISR(uxSavedInterruptStatus);

}
```

清单8.6 中断服务例程中使用临界区

与执行实际受临界区保护的代码相比，使用更多的处理时间来执行进入临界区然后随后退出临界区的代码是浪费的。基本临界区的进入和退出速度非常快，并且始终具有确定性，因此当受保护的代码区域非常短时，它们非常适合使用。

8.2.2 暂停（或锁定）调度程序

还可以通过暂停调度程序来创建关键部分。挂起调度程序有时也称为“锁定”调度程序。

基本临界区保护代码区域不被其他任务和中断访问，但是通过挂起调度程序实现的临界区仅保护代码区域不被其他任务访问，因为中断保持启用状态。

对于太长而无法通过简单地禁用中断来实现的关键部分，可以通过挂起调度程序来实现。然而，调度程序挂起时的中断活动可能会使恢复（或“取消挂起”）调度程序的操作时间相对较长，因此必须考虑在每种情况下使用哪种方法是最佳方法。

8.2.3 vTaskSuspendAll() API函数

```
void vTaskSuspendAll( void );
```

清单8.7 vTaskSuspendAll() API函数原型

通过调用 vTaskSuspendAll() 来暂停调度程序。挂起调度程序可防止发生上下文切换，但会启用中断。如果在调度程序挂起时中断请求上下文切换，则该请求将保持挂起状态，并且仅在调度程序恢复（未挂起）时执行。

调度程序挂起时不得调用 FreeRTOS API 函数

8.2.4 xTaskResumeAll() API函数

```
BaseType_t xTaskResumeAll( void );
```

清单8.8 xTaskResumeAll() API函数原型

通过调用xTaskResumeAll（）来恢复调度程序（未暂停）。

xTaskResumeAll（）返回值

- 返回值

调度程序挂起时请求的上下文切换将保持挂起状态，并且仅在调度程序恢复时执行。如果在 xTaskResumeAll() 返回之前执行挂起的上下文切换，则返回 pdTRUE。否则返回 pdFALSE。

对 vTaskSuspendAll() 和 xTaskResumeAll() 的调用嵌套是安全的，因为内核会保留嵌套深度的计数。仅当嵌套深度返回到零时，调度程序才会恢复，即之前每次调用 vTaskSuspendAll() 时都执行了一次对 xTaskResumeAll() 的调用。

清单8.9显示了vPrintString（）的实际实现，它会挂起调度器以保护对终端输出的访问。

```
void vPrintString( const char *
pcString ) {
    /*将字符串写入 stdout，作为互斥方法挂起调度程序。
    */

    vTaskSuspendScheduler();
    {
        printf ( "%s" 、 pcString );
        fflush ( stdout );
    }
    xTaskResumeScheduler();
}
```

清单8.9 vPrintString()的实现

8.3 互斥锁（和二进制信号量）

互斥锁是一种特殊类型的二进制信号量，用于控制对两个或多个任务之间共享的资源的访问。MUTEX 这个词源自“MUTual EXclusion”。必须在 FreeRTOSConfig.h 中将 configUSE_MUTEXES 设置为 1 才能使互斥锁可用。

当在互斥场景中使用时，互斥锁可以被视为与共享资源关联的令牌。为了使任务合法地访问资源，它必须首先成功“获取”令牌（成为令牌持有者）。当令牌持有者使用完资源后，它必须“归还”令牌。只有当令牌返回后，另一个任务才能成功获取令牌，然后安全地访问同一共享资源。除非持有令牌，否则不允许任务访问共享资源。该机制如图8.1所示

尽管互斥锁和二进制信号量有许多共同的特征，但图8.1中所示的场景（其中互斥锁用于互斥）与图7.6中所示的场景（其中二进制信号量用于同步）完全不同。主要的区别是信号量获得后会发生什么：

- 必须始终返回用于互斥的信号量。
- 用于同步的信号量通常被丢弃而不返回。

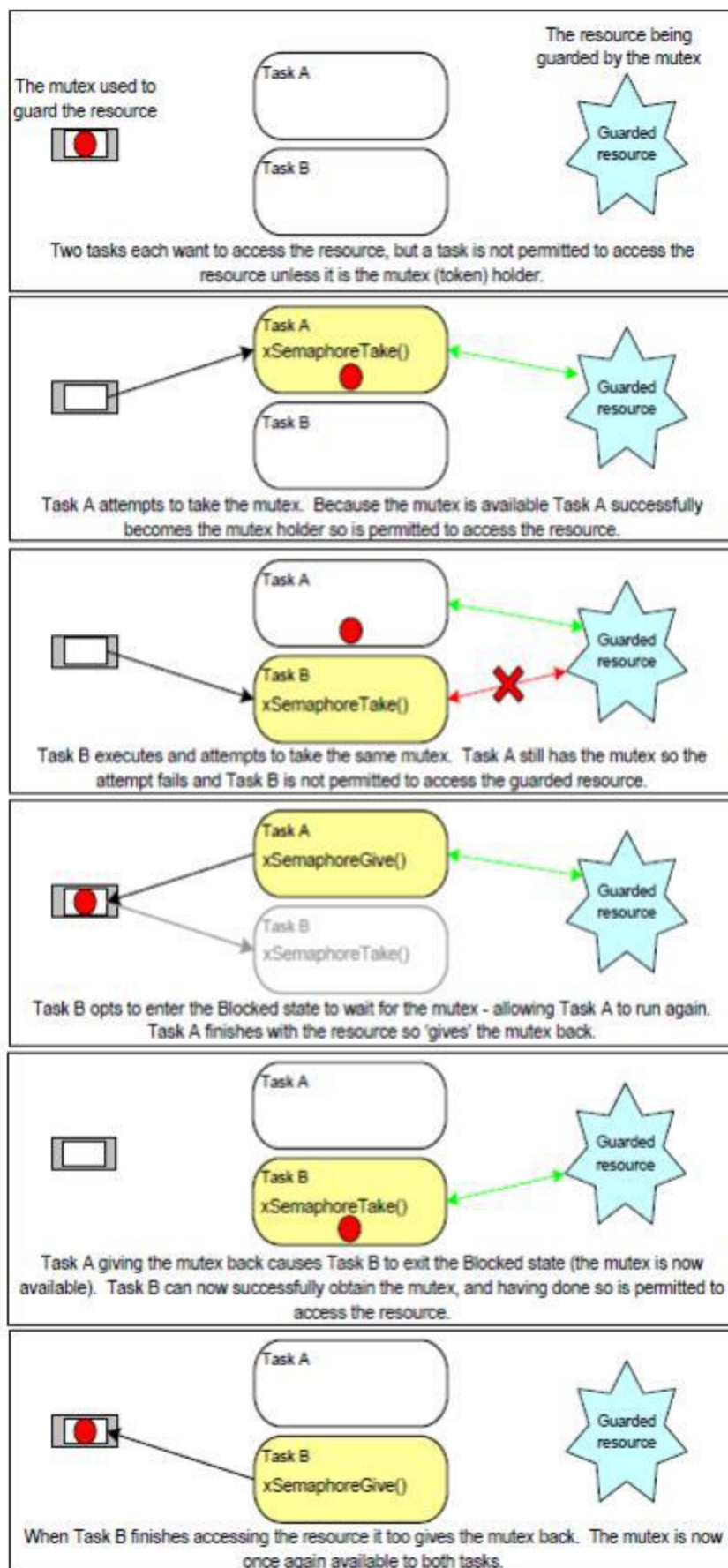


图8.1 使用互斥锁实现的互斥功能

该机制纯粹通过应用程序编写者的原则来工作。任务没有理由不能在任何时候访问资源，但每个任务“同意”不这样做，除非它能够成为互斥锁持有者。

8.3.1 xSemaphoreCreateMutex() API 函数

FreeRTOS 还包括 xSemaphoreCreateMutexStatic() 函数，该函数在编译时静态分配创建互斥锁所需的内存：互斥锁是信号量的一种类型。所有各种类型的 FreeRTOS 信号量的句柄都存储在 SemaphoreHandle_t 类型的变量中

在使用互斥锁之前，必须先创建它。要创建互斥类型信号量，请使用 xSemaphoreCreateMutex() 函数

```
SemaphoreHandle_t xSemaphoreCreateMutex( void );
```

清单8.10x符号库创建Mutex() API函数原型

xSemaphoreCreateMutex() 返回值

- 返回值

如果返回NULL，则无法创建互斥数据，因为FreeRTOS没有足够的堆内存来分配互斥数据结构。第3章提供了关于堆内存管理的更多信息。

非NULL返回值表示互斥锁已成功创建。返回的值应该存储为所创建的互斥锁的句柄。

示例8.1 重写vPrintString() 使用信号量

此示例创建 vPrintString() 的新版本（称为 prvNewPrintString()），然后从多个任务调用新函数。

prvNewPrintString() 在功能上与 vPrintString() 相同，但使用互斥锁控制对标准输出的访问，而不是通过锁定调度程序。prvNewPrintString() 的实现如清单所示

```
static void prvNewPrintString( const char *pcString )
{
    /*互斥锁是在调度程序启动之前创建的，因此在该任务执行时已经存在。
    尝试获取互斥锁，如果互斥锁不能立即可用，则无限期地阻塞以等待互斥锁。
    对 xSemaphoreTake() 的调用只有在成功获取互斥锁时才会返回，因此无需检查函数返回值。
    如果使用任何其他延迟周期，则代码必须在访问共享资源（在本例中为标准输出）
    之前检查 xSemaphoreTake() 返回 pdTRUE 。
    正如本书前面提到的，不建议对生产代码使用无限期超时。
    */
```

```

xSemaphoreTake( xMutex, portMAX_DELAY );
{
    /*仅当成功获取互斥锁后，才会执行以下行。现在可以自由访问标准输出，
    因为任何时候只有一个任务可以拥有互斥锁。*/

    printf ( "%s" 、 pcString );
    fflush ( stdout );

    /*互斥锁必须归还！*/
}
xSemaphoreGive ( xMutex );
}

```

清单8.11 prvNewPrintString()的实现

prvNewPrintString() 由 prvPrintTask() 实现的两个实例重复调用。每次调用之间使用随机延迟时间。任务参数用于将唯一的字符串传递到任务的每个实例中。 prvPrintTask() 的实现如清单 8.12 所示

```

static void prvPrintTask( void *pvParameters )
{
    char *pcStringToPrint;
    const TickType_t xMaxBlockTimeTicks = 0x20;

    /*创建此任务的两个实例。任务打印的字符串使用任务的参数传递到任务中。参数被转换为所需的类型
    */

    pcStringToPrint = ( char * ) pvParameters;

    for(;;)
    {
        /*使用新定义的函数打印出字符串。*/
        prvNewPrintString ( pcStringToPrint );

        /* 等待伪随机时间。请注意，rand() 不一定是可重入的，但在这种情况下，它并不重要，
        因为代码并不关心返回什么值。在更安全的应用程序中，应该使用已知可重入的 rand() 版本
        - 或者应该使用关键部分来保护对 rand() 的调用*/

        vTaskDelay( ( rand() % xMaxBlockTimeTicks ) );
    }
}

```

清单8.12 prvPrintTask()的实现，示例8.1

和往常一样，主函数只是创建互斥锁，创建任务，然后启动调度程序。该实现如清单8.13所示。

`prvPrintTask()` 的两个实例是在不同的优先级上创建的，因此低优先级任务有时会被高优先级任务抢占。由于互斥锁用于确保每个任务获得对终端的互斥访问，即使发生抢占，显示的字符串也将是正确的，而且不会损坏。抢占的频率可以通过减少任务在阻塞状态下花费的最大时间来增加，该时间是由 `xMaxBlockTimeTicks` 设置的。

使用FreeRTOS Windows端口使用示例8.1的注意事项：

调用 `printf()` 将生成一个视窗系统调用。Windows系统调用不受FreeRTOS控制，可能引入不稳定性。

Windows系统调用的执行方式意味着，即使不使用互斥锁，也很少看到损坏的字符串。

```
int main( void )
{
    /*在使用信号量之前，必须显式地创建它。在这里
       创建了一个互斥锁类型信号量的示例。*/
    xMutex = xSemaphoreCreateMutex();

    /* 创建任务前检查信号量是否创建成功 */

    if (xMutex != NULL)
    {
        /*创建两个写入标准输出的任务实例。它们写入的字符串将作为任务的参数传递给任
           务。这些任务是在不同的优先级下创建的，因此将会出现一些抢占。*/
        xTaskCreate ( prvPrintTask, "Print1", 1000,
                     "Task 1 *****\r\n", 1, NULL );

        xTaskCreate ( prvPrintTask, "Print2", 1000,
                     "Task 2 -----\r\n", 2, NULL);

        /*启动调度程序，以便已创建的任务开始执行。*/
        vTaskStartScheduler();
    }

    /*如果一切都好，那么主函数将永远不会到达这里，因为调度程序现在将运行任务。如果主
       函数确实到达了这里，那么很可能没有足够的堆内存来创建空闲任务。第3章提供了关
       于堆内存管理的更多信息。*/
}
```

```
    for( ; ;)
}
```

清单8.13 示例8.1中的主函数的实现

当执行示例8.1时产生的输出如图8.2所示。在图8.3中描述了一个可能的执行序列。

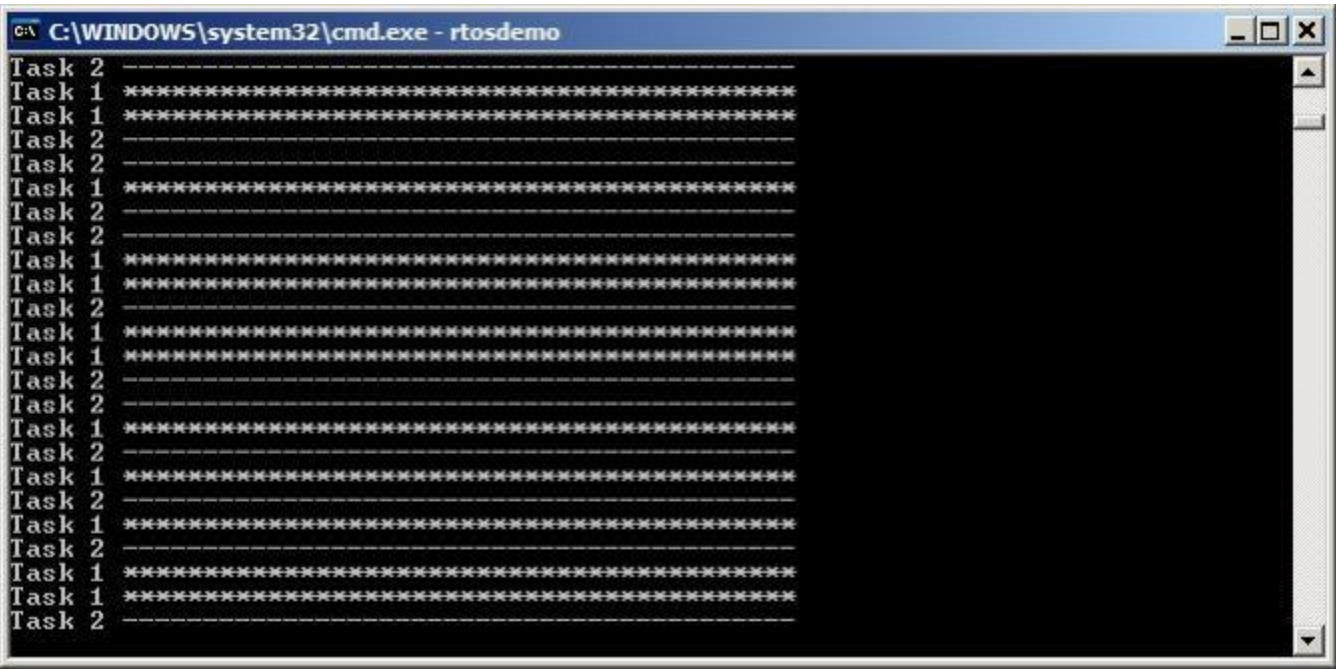


图8.2 在执行示例8.1时产生的输出

图8.2显示，正如预期的那样，在终端上显示的字符串中没有损坏。随机排序是任务所使用的随机延迟周期的结果。

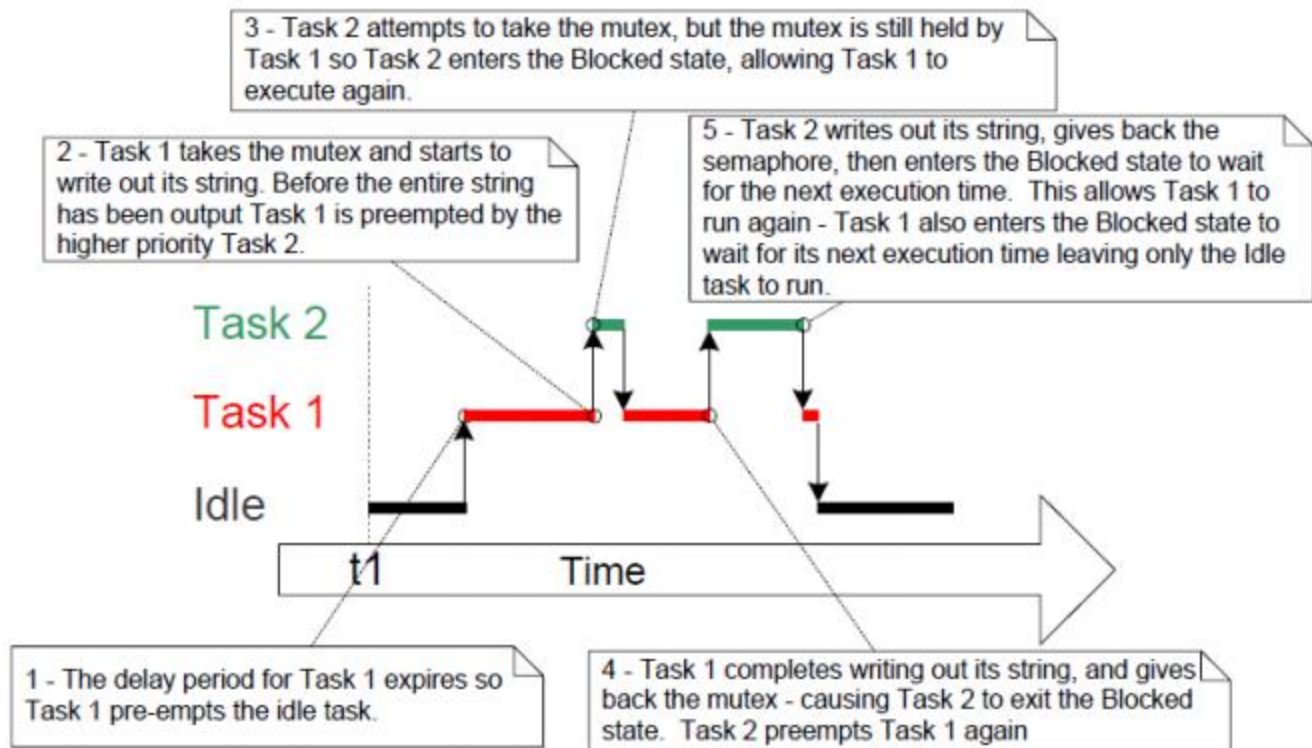


图8.3 示例8.1中可能的执行顺序

8.3.2 优先级反转

图 8.3 演示了使用互斥体提供互斥的潜在缺陷之一。所描述的执行顺序显示较高优先级的任务 2 必须等待较低优先级的任务 1 放弃对互斥体的控制。较高优先级任务以这种方式被较低优先级任务延迟称为“优先级反转”。如果在高优先级任务等待信号量时中优先级任务开始执行，这种不良行为将会进一步加剧（结果将是高优先级任务等待低优先级任务），而更低优先级的任务甚至还没执行执行。最坏的情况如图 8.4 所示

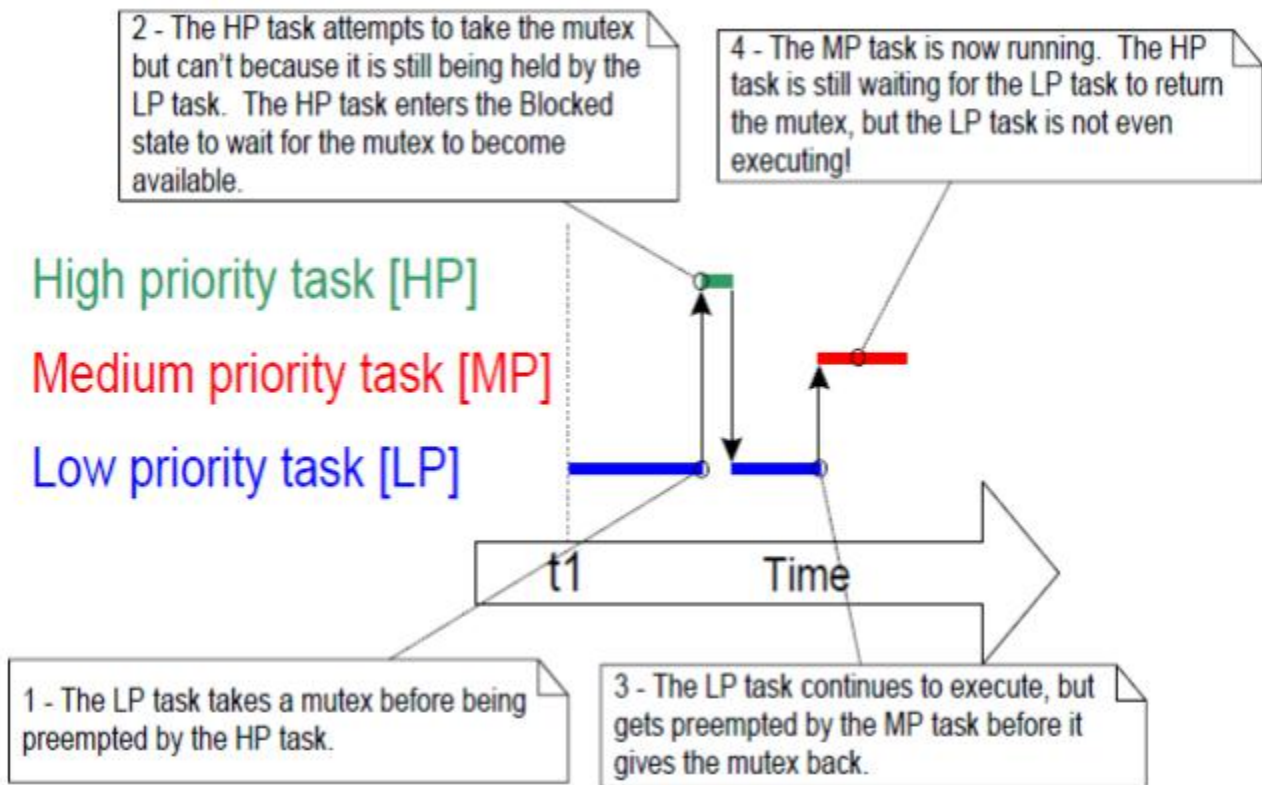


图8.4 一种最坏情况下的优先级倒置情况

优先级反转可能是一个重大问题，但在小型嵌入式系统中，通常可以在系统设计时通过考虑资源的访问方式来避免。

8.3.3 优先级继承

FreeRTOS 互斥锁和二进制信号量非常相似，区别在于互斥锁包含基本的“优先级继承”机制，而二进制信号量则不包含。优先级继承是一种最小化优先级反转负面影响的方案。它不会“修复”优先级反转，而只是通过确保反转始终受时间限制来减轻其影响。然而，优先级继承使系统时序分析变得复杂，并且依靠它来实现正确的系统操作并不是一个好的做法。

优先级继承的工作原理是暂时将互斥锁持有者的优先级提高到尝试获取相同互斥锁的最高优先级任务的优先级。持有互斥锁的低优先级任务“继承”等待互斥锁的任务的优先级。图 8.5 展示了这一点。当互斥锁归还时，互斥锁持有者的优先级会自动重置为其原始值

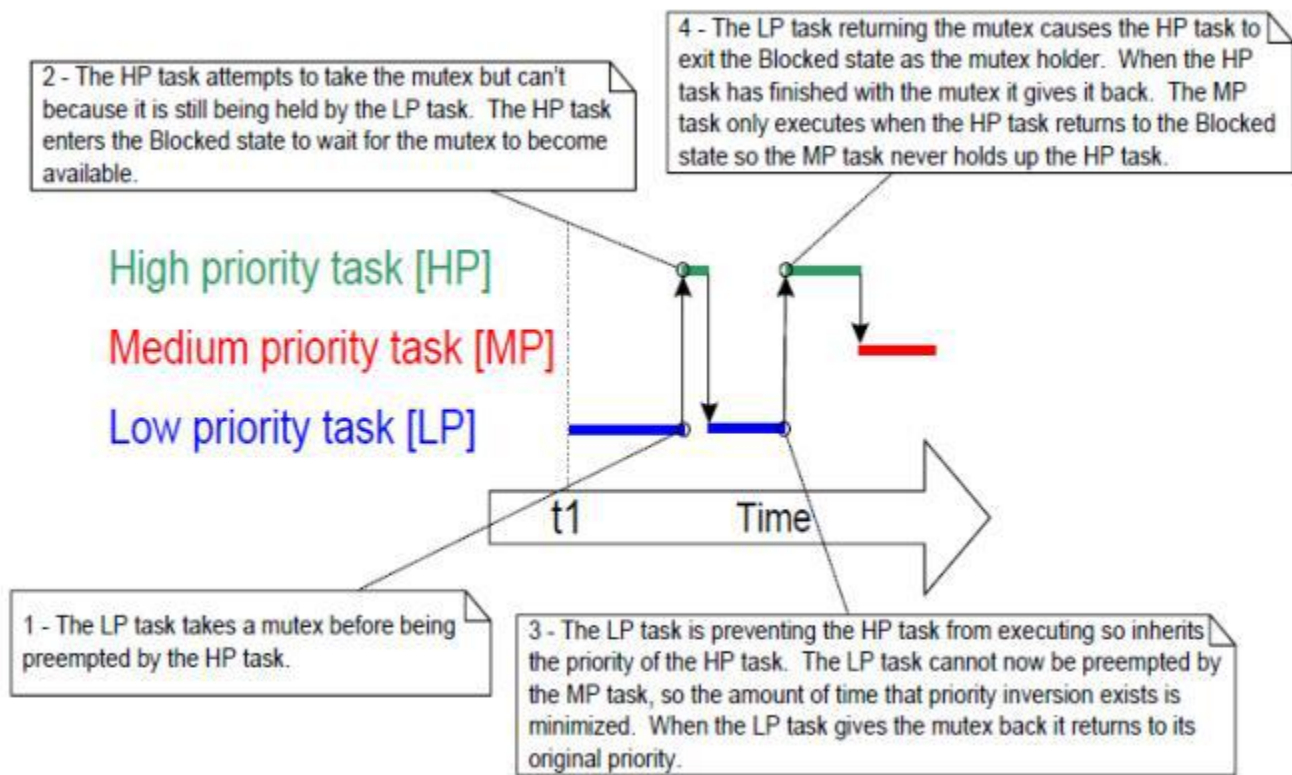


图8.5 优先级继承最小化了优先级反转的影响

如刚才所述，优先级继承功能会影响正在使用互斥锁的任务的优先级。因此，不能从中断服务例程中使用互斥关系。

8.3.4 死锁 (or Deadly Embrace)

“死锁”是使用互斥语言进行互斥的另一个潜在陷阱。“死锁”有时也被称为更具戏剧性的名字“Deadly Embrace”。

当两个任务都无法继续等待对方持有的资源时，就会发生死锁。考虑以下场景，任务A和任务B都需要获得互斥X和互斥Y来执行一个动作：

1. 任务A执行并成功获取互斥锁X。
2. 任务A被任务B优先执行。
3. 任务B在尝试也使用互斥X之前成功地接受了互斥Y——但是互斥X被任务A持有，所以对任务B不可用。任务B选择进入阻塞状态，等待互斥锁X被释放。
4. 任务A继续执行。它试图获取互斥锁Y，但互斥锁Y被任务B控制，所以对任务A不可用。任务A选择进入阻塞状态，等待互斥锁Y被释放。

在这个场景的最后，任务A等待任务B持有的互斥锁，而任务B等待任务a持有的互斥锁。发生死锁，因为两个任务都不能继续。

与优先级反转一样，避免死锁的最佳方法是在设计时考虑其潜力，并设计系统以确保不会发生死锁。特别是，正如本书前面所述，任务无限期地等待（没有超时）来获取互斥体通常是不好的做法。相反，使用比预期必须等待互斥体的最大时间稍长的超时，然后在该时间内未能获取互斥体将是设计错误的症状，这可能是死锁。

在实践中，死锁在小型嵌入式系统中并不是一个大问题，因为系统设计者可以对整个应用程序有很好的理解，因此可以识别并删除可能发生死锁的区域。

8.3.5 递归互斥锁

一个任务也有可能与自身发生死锁。如果一个任务多次尝试使用相同的互斥锁，而没有首先返回互斥锁，就会发生这种情况。请考虑以下场景：

1. 任务已成功获得互斥锁。
2. 当保持互斥锁时，该任务调用一个库函数。
3. 库函数的实现尝试使用相同的互斥锁，并进入阻塞状态，等待互斥锁成为可用状态。

在此场景结束时，任务处于“阻塞”状态，等待互斥锁返回，但任务已经是互斥锁保持者。发生死锁，因为任务处于阻塞状态，等待自身。

通过使用递归互斥锁代替标准互斥体可以避免这种类型的死锁。同一任务可以多次“获取”递归互斥锁，并且只有在之前每次“获取”递归互斥锁的调用都执行了一次“给予”递归互斥锁后，才会返回该互斥锁。

标准的互斥锁和递归互斥锁也以类似的方式创建和使用：

- 标准互斥锁是使用 `xSemaphoreCreateMutex()` 创建的。
递归互斥锁是使用 `xSemaphoreCreateRecursiveMutex()` 创建的。两个API函数具有相同的原型
- 标准互斥锁是使用 `xSemaphoreTake()` “获取”的。
使用 `xSemaphoreTakeRecursive()` “获取”递归互斥锁。这两个API函数具有相同的原型。
- 标准互斥锁是使用 `xSemaphoreGive()` “给出”的。
递归互斥锁是使用 `xSemaphoreGiveRecursive()` “给出”的。这两个API函数具有相同的原型。

清单8.14演示了如何创建和使用递归互斥锁。

```
/*递归互斥锁是 SemaphoreHandle_t 类型的变量。*/
SemaphoreHandle_t xRecursiveMutex;

/*创建和使用递归互斥锁的任务的实现。*/
void vTaskFunction( void *pvParameters )
{
    const TickType_t xMaxBlock20ms = pdMS_TO_TICKS( 20 );
```

```

/*在使用递归互斥锁之前，必须显式地创建它。*/
xRecursiveMutex =xSemaphoreCreateRecursiveMutex();

/*检查信号量是否创建成功。 configASSERT() 在 11.2 节中描述。*/

configASSERT ( xRecursiveMutex );

/*与大多数任务一样，该任务被实现为一个无限循环。*/
for ( ; ; ) {
    /* ...*/

    /*获取递归互斥锁。*/
    if( xSemaphoreTakeRecursive( xRecursiveMutex, xMaxBlock20ms ) == pdPASS )
    {
        /*成功获得递归互斥体。该任务现在可以访问互斥锁保护的资源。此时，递归
        调用计数（即对 xSemaphoreTakeRecursive() 的嵌套调用数）为 1，因为递
        归互斥体仅被获取一次。*/
        /*虽然它已经持有递归互斥锁，但该任务再次接受互斥锁。在实际的应用程序中，
        这只可能发生在这个任务调用的子函数中，因为没有实际的理由有意地使用相同
        的互斥锁。调用任务已经是互斥锁持有者，因此对函数递归的第二次调用只会
        将递归调用计数增加到2。*/
        xSemaphoreTakeRecursive ( xRecursiveMutex, xMaxBlock20ms );

        /* ...*/

        /*任务在完成访问互斥体所保护的资源后返回互斥体。此时递归调用计数为 2，
        因此第一次调用 xSemaphoreGiveRecursive() 不会返回互斥锁。
        相反，它只是将递归调用计数减回到 1。
        */
        xSemaphoreGiveRecursive ( xRecursiveMutex );

        /*对递归的下一个调用将递归调用计数减少为0，所以这次返回递归互斥锁。
        */
        xSemaphoreGiveRecursive ( xRecursiveMutex );

        /*现在，每次对 xSemaphoreTakeRecursive() 的调用都会执行一次对
        xSemaphoreGiveRecursive() 的调用，因此该任务不再是互斥锁持有者。*/
    }
}

```

```

    }
}

```

清单8.14 创建和使用递归互斥锁

8.3.6 互斥锁和任务调度

如果两个不同优先级的任务使用相同的互斥锁，那么FreeRTOS的调度策略会明确任务的执行顺序：能够运行的最高优先级任务将被选择为进入运行状态的任务。例如，如果高优先级任务处于阻塞状态，等待低优先级任务持有的互斥锁，那么一旦低优先级任务返回互斥锁，高优先级任务就会抢占低优先级任务。高优先级任务将成为互斥锁持有者。这种情况已在图 8.5 中看到。

然而，当任务具有相同优先级时，通常会错误地假设任务的执行顺序。如果任务 1 和任务 2 具有相同的优先级，并且任务 1 处于阻塞状态以等待任务 2 持有的互斥体，则当任务 2 “给出”互斥体时，任务 1 不会抢占任务 2。相反，任务 2 将保持运行状态，而任务 1 将简单地从阻塞状态转移到就绪状态。这种情况如图8.6所示，其中垂直线标记了tick中断发生的时间

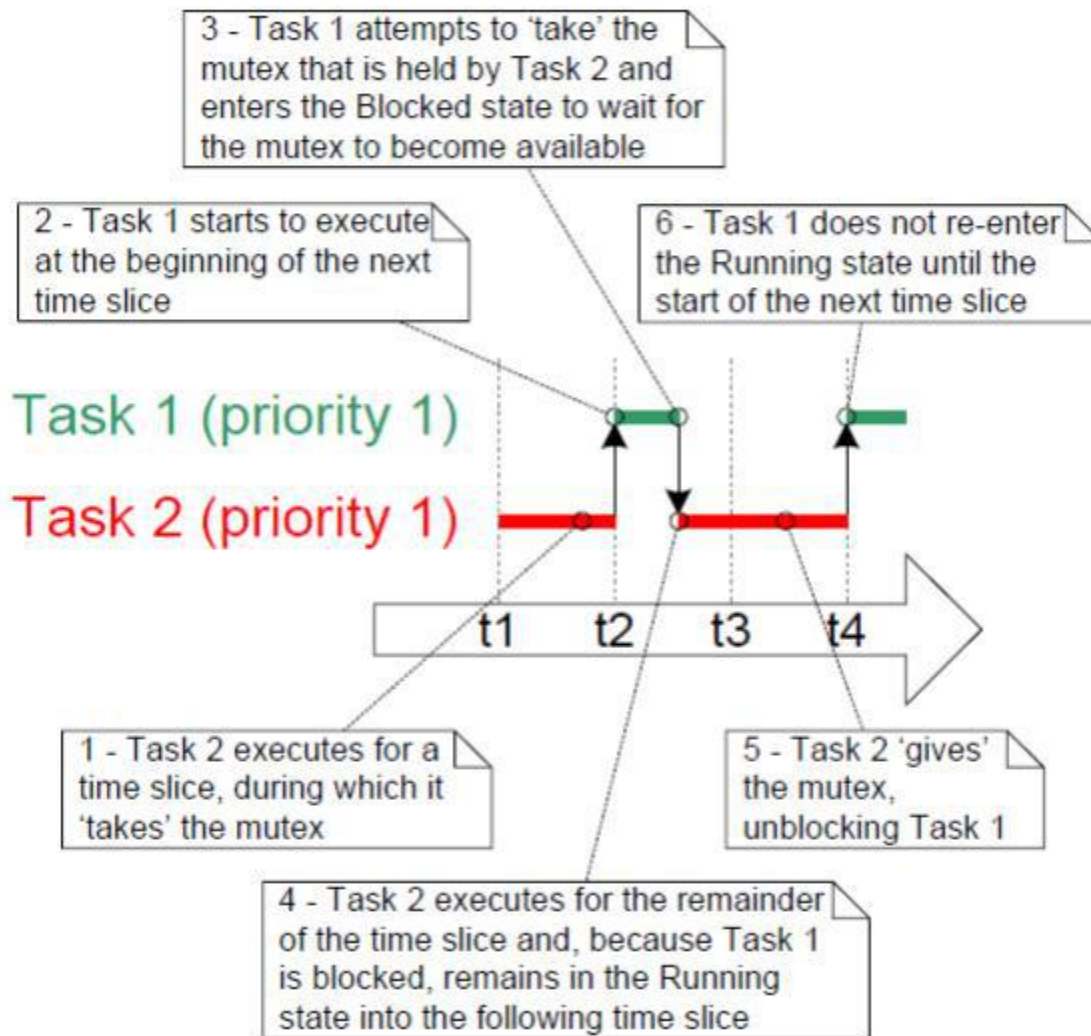


图8.6 当具有相同优先级的任务使用相同的互斥锁时，可能的执行序列

在图8.6所示的场景中，FreeRTOS调度程序不会使任务1成为运行状态任务，因为：

- 任务 1 和任务 2 具有相同的优先级，因此除非任务 2 进入阻塞状态，否则在下一个时钟周期中断之前不会切换到任务 1（假设 FreeRTOSConfig.h 中的 configUSE_TIME_SLICING 设置为 1）。
- 如果任务在紧密循环中使用互斥体，并且每次任务“give”互斥体时都会发生上下文切换，则该任务只会在短时间内保持运行状态。如果两个或多个任务在紧密循环中使用相同的互斥体，则任务之间的快速切换会浪费处理时间

如果多个任务在紧密循环中使用互斥体，并且使用该互斥体的任务具有相同的优先级，则必须注意确保这些任务获得大致相等的处理时间。图 8.7 演示了任务可能无法获得相同处理时间的原因，其中清单 8.15 显示了如果以相同优先级创建所示任务的两个实例时可能发生的执行序列

/*在紧密循环中使用互斥体的任务的实现。该任务在本地缓冲区中创建一个文本字符串，然后将该字符串写入显示器。对显示器的访问受到互斥锁的保护*/

```
void vATask( void *pvParameter )
{
    extern SemaphoreHandle_t xMutex;
    char cTextBuffer[ 128 ];
    for( ;; ) {

        /*生成文本字符串——这是一个快速的操作。*/
        vGenerateTextInALocalBuffer ( cTextBuffer );

        /*获取互斥锁保护访问显示器。*/

        xSemaphoreTake( xMutex, portMAX_DELAY );

        /*将生成的文本写入到显示器中-这是一个缓慢的操作。*/

        vCopyTextToFrameBuffer (cTextBuffer );

        /*文本已写入显示器，因此返回互斥锁。*/xSemaphoreGive ( xMutex );

    }
}
```

清单8.15 一个在紧密循环中使用互斥锁的任务

清单8.15中的注释注意到，创建字符串是一个快速的操作，而更新显示则是一个缓慢的操作。因此，当在更新显示时保持互斥锁时，该任务将在其大部分运行时间内保持互斥锁。

在图8.7中，垂直线标记了滴答中断发生的时间。

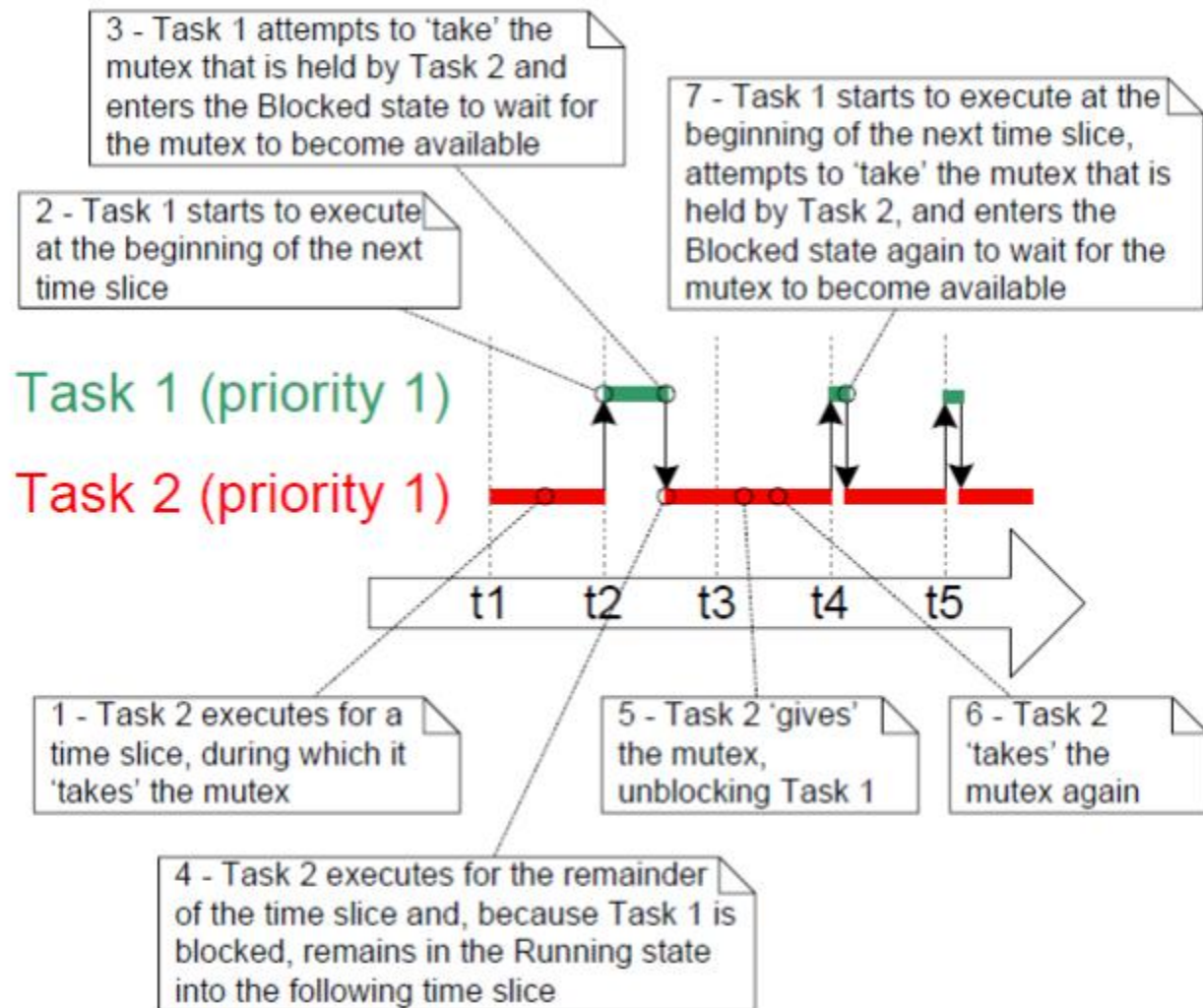


图8.7 如果清单8.15所示的任务的两个实例在相同的优先级下创建，则可能发生的一个执行序列

图 8.7 中的步骤 7 显示任务 1 重新进入阻塞状态——这发生在 `xSemaphoreTake()` API 函数内部。

图 8.7 演示了任务 1 将被阻塞获取互斥锁，直到时间片的开始与任务 2 不是互斥体所有者的一小段时间之一重合。

通过在调用 `xSemaphoreGive()` 之后添加对 `taskYIELD()` 的调用可以避免图 8.7 中所示的情况。清单 8.16 对此进行了演示，其中如果在任务持有互斥体时时钟计数发生变化，则调用 `taskYIELD()`。

```
void vFunction( void *pvParameter )
{
    extern SemaphoreHandle_t xMutex;
    char cTextBuffer[ 128 ];
```

```

TickType_t xTimeAtWhichMutexWasTaken;

for( ; ;)
{
    /*生成文本字符串——这是一个快速的操作。*/
    vGenerateTextInALocalBuffer ( cTextBuffer );

    /*获取正在保护访问显示器的互斥锁。*/
    xSemaphoreTake( xMutex, portMAX_DELAY );

    /*记录互斥锁发生的时间。*/
    xTimeAtWhichMutexWasTaken = xTaskGetTickCount();
    /*将生成的文本写入到显示器中-这是一个缓慢的操作。*/
    vCopyTextToFrameBuffer (cTextBuffer );
    /*文本已写入显示器，因此返回互斥锁。*/
    xSemaphoreGive ( xMutex );

    /*如果每次迭代都调用taskYIELD()，那么该任务只会在短时间内保持运行状态，并且任务之间的快速切换会浪费处理时间。因此，只有在持有互斥体时时钟计数发生变化时才调用taskYIELD()*/

    if( xTaskGetTickCount() != xTimeAtWhichMutexWasTaken )
    {
        taskYIELD();
    }
}
}

```

清单8.16 确保在循环中使用互斥锁的任务接收更多的处理时间，同时确保处理时间不会因为在任务之间太快切换而浪费

8.4 看门人任务

看门人任务提供了一种实现互斥的干净方法，而没有优先级反转或死锁的风险。

看门人任务是拥有资源唯一所有权的任务。仅允许看门人任务直接访问资源，任何其他需要访问资源的任务只能通过使用看门人的服务间接访问资源

8.4.1 重写 vPrintString() 以使用看门人任务

示例8.2为vPrintString()提供了另一种替代实现。这一次，一个看门人任务被用于管理对标准输出的访问。当任务想要将消息写入标准输出时，它不会直接调用打印函数，而是将消息发送给看门人。

“看门人”任务使用FreeRTOS队列来序列化对标准输出的访问。任务的内部实现不必考虑互斥，因为它是唯一允许直接访问标准的任务。

看门人任务大部分时间处于阻塞状态，等待消息到达队列。当消息到达时，看门人只需将消息写入标准输出，然后返回到阻塞状态等待下一个消息。该看门人任务的实现见清单8.18。

中断可以发送到队列，因此中断服务例程也可以安全地使用看门人的服务向终端写入消息。在本例中，使用钩子函数每200个滴答写出一条消息。

滴答钩子tick hook（或滴答回调tick callback）是内核在每次滴答中断期间调用的函数。要使用该功能：

1. 在FreeRTOSConfig.h中将configUSE_TICK_HOOK设置为1。
2. 使用清单8.17中所示的确切函数名和原型，提供钩子函数的实现。

```
void vApplicationTickHook( void );
```

清单8.17 滴答钩子函数的名称和原型

滴答钩子函数在滴答中断的上下文中执行，因此必须保持非常短，必须只使用适量的堆栈空间，并且不能调用任何不以“FromISR（）”结尾的FreeRTOS API函数。

调度程序将始终在tick hook函数之后立即执行，因此从tick hook调用的中断安全FreeRTOS API函数不需要使用它们的pxHigherPriorityTaskWoken参数，并且该参数可以设置为NULL。

```
static void prvStdioGatekeeperTask( void *pvParameters )
{
    char *pcMessageToPrint;

    /*这是唯一允许写入标准输出的任务。任何其他想要向输出写入字符串的任务都不会直接访问标准输出，而是将该字符串发送给此任务。由于只有这个任务访问标准，因此在任务本身的实现中不需要考虑互斥或序列化问题。*/
    for( ; ;)
    {
        /*等待消息到达。指定了一个无限的阻塞时间，因此不需要检查返回值-函数将
```



```

        仅在成功收到消息时才返回。*/
        xQueueReceive( xPrintQueue, &pcMessageToPrint, portMAX_DELAY );

        /* 输出接收到的字符串。*/
        printf( "%s", pcMessageToPrint );
        fflush( stdout );

        /*不断循环以等待下一条消息。*/
    }
}

```

清单8.18: 看门人任务

实例8.2 打印任务的替代实现

写入该队列的任务如清单8.19所示。与前面一样，将创建任务的两个独立实例，并使用任务参数将任务写入队列的字符串传递到任务中。

```

static void prvPrintTask( void *pvParameters )
{
    int iIndexToString;
    const TickType_t xMaxBlockTimeTicks= 0x20;

    /*已创建了此任务的两个实例。任务参数用于将索引传递到任务的字符串数组中。将其转换
    为所需的类型。*/
    iIndexToString = (int) pvParameters;

    for( ; ;)
    {
        /* 打印出字符串，不是直接打印出来，而是通过队列将指向字符串的指针传递给看门人
        任务。该队列是在调度程序启动之前创建的，因此在此任务第一次执行时就已经存在
        了。不指定阻塞时间，是因为队列中应该始终有空间
        xQueueSendToBack ( xPrintQueue, &(amp; pcStringsToPrint[ iIndexToString ]), 0 );

        /* 等待一个伪随机时间。注意，rand ( ) 不一定是重入的，但在这种情况下，这并不
        重要，因为代码不关心返回什么值。在一个更安全的应用程序中，应该使用一个已
        知的rand ( ) 版本-或者对rand ( ) 的调用应该使用关键部分进行保护。*/
        vTaskDelay( ( rand() % xMaxBlockTimeTicks ) );
    }
}

```

清单8.19 示例8.2中的打印任务实现

滴答钩子函数计算它被调用的次数，每次计数达到200时，将其消息发送给看门人任务。仅用于演示目的，钩子写入队列的前面，任务写入队列的后面。钩子实现如清单8.20所示。

```
void vApplicationTickHook( void )
{
    static int iCount = 0;

    /*每200次打印一条信息。该消息并未被写出
      直接发送，但发送给看门人任务。*/
    iCount++;

    if (iCount >= 200)
    {
        /*由于 xQueueSendToFrontFromISR() 是通过 tick hook调用的，
          因此无需使用 xHigherPriorityTaskWoken 参数（第三个参数），该参数被设置为 NULL
          */
        xQueueSendToFrontFromISR ( xPrintQueue,
                                    &( pcStringsToPrint[ 2 ] ),
                                    NULL );

        /*重置计数，准备再次打印出字符串在200滴答
          时刻*/
        iCount = 0;
    }
}
```

清单8.20 滴答钩子实现

通常，主函数创建运行示例所需的队列和任务，然后启动调度程序。主函数的实现如清单8.21所示。

```
/*定义任务和中断将通过看门人任务打印输出的字符串。

static char *pcStringsToPrint[] =
{
    "Task 1 *****\r\n",
    "Task 2 ----- \r\n",
    "Message printed from the tick hook interrupt #####\r\n"
};

/*-----*/

/*声明一个QueueHandle_t类型的变量。队列用于打印任务和滴答中断向看门人任务发送消息。
*/
```

```

QueueHandle_t xPrintQueue;

/*-----*/

int main(void)
{
    /*在使用队列之前，必须显式地创建该队列。队列是
      创建以保存最多5个字符的指针。*/
    xPrintQueue = xQueueCreate( 5, sizeof( char * ) );

    /*检查队列已成功创建。*/
    if( xPrintQueue != NULL )
    {
        /*创建两个向看门人发送消息的任务实例。任务使用的字符串的索引通过任务参数（xTaskCreate（）的第4个参数）传递给任务。这些任务是在不同的优先级下创建的，因此优先级更高任务偶尔会抢断较低优先级的任务。*/
        xTaskCreate( prvPrintString, "Print1", 1000, (void *) 0, 1, NULL );
        xTaskCreate( prvPrintTask, "Print2", 1000, ( void * ) 1, 2, NULL );

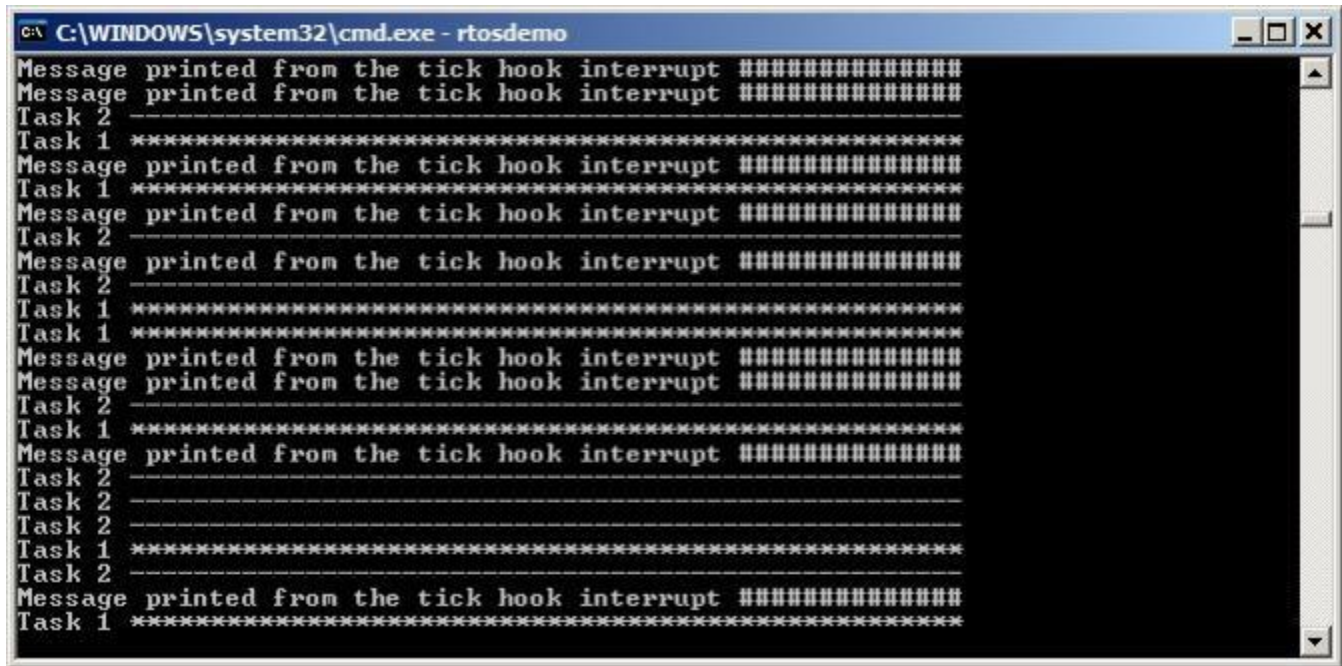
        /*创建一个守门人的任务。这是唯一允许直接访问标准输出的任务。*/
        xTaskCreate( prvStdioGatekeeperTask, "Gatekeeper", 1000, NULL,
            0, NULL );
        /*启动调度程序，以便已创建的任务开始执行。*/vTaskStartScheduler();
    }

    /*如果一切都好，那么主函数将永远不会到达这里，因为调度程序现在将运行任务。如果主函数确实到达了这里，那么很可能没有足够的堆内存来创建空闲任务。第3章提供了关于堆内存管理的更多信息。*/
    for( ; ; )
}

```

清单8.21 示例8.2中的主函数的实现

当执行示例8.2时产生的输出如图8.8所示。可以看出，来自任务的字符串和来自中断的字符串都能正确地打印出来，没有损坏。



```
C:\WINDOWS\system32\cmd.exe - rtosdemo
Message printed from the tick hook interrupt #####
Message printed from the tick hook interrupt #####
Task 2
Task 1 *****
Message printed from the tick hook interrupt #####
Task 1 *****
Message printed from the tick hook interrupt #####
Task 2
Message printed from the tick hook interrupt #####
Task 2
Task 1 *****
Task 1 *****
Message printed from the tick hook interrupt #####
Message printed from the tick hook interrupt #####
Task 2
Task 1 *****
Message printed from the tick hook interrupt #####
Task 2
Task 2
Task 2
Task 1 *****
Task 2
Message printed from the tick hook interrupt #####
Task 1 *****
```

图8.8 在执行示例8.2时产生的输出

看门人任务的优先级低于打印任务的优先级，因此发送到看门人的信息会留在队列中，直到两个打印任务都处于阻塞状态。在某些情况下，给看门人分配更高的优先级是合适的，这样信息就能立即得到处理，但这样做的代价是，看门人会推迟优先级较低的任务，直到它完成对受保护资源的访问。

9 事件组

9.1 本章简介及适用范围

人们已经注意到，实时嵌入式系统必须对事件采取相应的行动。前几章描述了FreeRTOS允许事件通信给任务的特性。这些特性的示例包括信号量和队列，这两者都具有以下属性：

- 它们允许一个任务在已被阻塞的状态下等待单个事件的发生。
- 当事件发生时，它们会取消阻塞单个任务。未解除阻塞的任务是等待该事件的最高优先级任务。

事件组是FreeRTOS的另一个特性，它允许事件通信给任务。与队列和信号量不同：

事件组允许任务在“阻塞”状态下等待其中一个事件的组合发生。

当事件发生时，事件组取消阻塞等待同一事件或事件组合的所有任务。

事件组的这些独特属性使它们在以下方面非常有用：同步多个任务、向多个任务广播事件、允许任务在阻塞状态下等待一组事件中的任何一个发生，以及允许任务在阻塞状态下等待多个操作完成。事件组还能减少应用程序使用的 RAM，因为通常可以用一个事件组取代多个二进制信号量

事件组功能是可选的。要包含事件组功能，请将 FreeRTOS 源文件 `event_groups.c` 作为项目的一部分来构建

9.1.1 范围

本章旨在让读者能够很好地理解：

- 活动组的实际用途。
- 事件组相对于其他FreeRTOS特性的优点和缺点。
- 如何在事件组中设置位。
- 如何在阻塞状态中等待位在事件组中设置。
- 如何使用一个事件组来同步一组任务。

9.2 一个事件组的特征

9.2.1 事件组、事件标志和事件位

事件“标志”是一个布尔（1或0）值，用于指示是否发生了事件。事件“组”是一组事件标志。

事件标志只能为1或0，允许事件标志的状态存储在单个位中，并且事件组中所有事件标志的状态存储在单个变量中；事件组中每个事件标志的状态由类型为EventBits_t的变量中的单个位表示。因此，事件标志也被称为事件“位”。如果EventBits_t变量中的一个位被设置为1，则由该位所表示的事件已经发生。如果EventBits_t变量中的一个位设置为0，则该位表示的事件没有发生。

图9.1显示了如何将各个事件标记映射到EventBits_t类型的变量中的各个位。

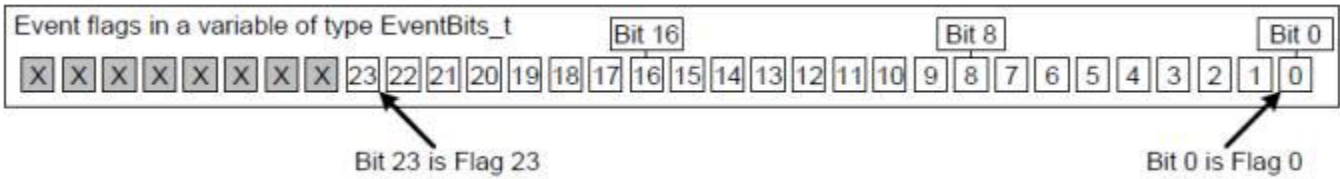


图9.1 事件标志到EventBits_t类型的变量中的位数映射

例如，如果事件组的值为0x92（二进制1001001010），则只设置事件位1、4和7，因此只发生了由位1、4和7表示的事件。图9.2显示了一个类型为EventBits_t的变量，它设置了事件位1、4和7，并且所有其他事件位都被清除，使事件组的值为0x92。

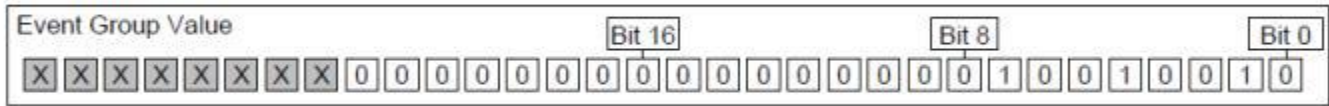


图9.2 一个事件组，其中只设置了位1、4和7，并且所有其他事件标志都是清晰的，使事件组的值为0x92

由应用程序编写者为事件组中的单个位分配意义。例如，应用程序编写器可能会创建一个事件组，然后：

在事件组中定义位0，表示“已从网络收到消息”。在事件组中定义第1位，表示“准备发送到网络的消息”。在事件组中定义位2，表示“中止当前的网络连接”。

9.2.2 关于EventBit_t数据类型的详细信息

事件组中的事件位数取决于FreeRTOSConfig.h中的configTICK_TYPE_WIDTH_IN_BITS编译时配置常数。[²⁴]:

[^24]: configTICK_TYPE_WIDTH_IN_BITS 配置用于保存 RTOS tick 计数的类型，因此似乎与事件组功能无关。虽然将 configTICK_TYPE_WIDTH_IN_BITS 设置为 TICK_TYPE_WIDTH_16_BITS 是可取的，但只有当 FreeRTOS 在能够比 32 位类型更有效地处理 16 位类型的体系结构上执行时，才应该这样做。

- 如果configTICK_TYPE_WIDTH_IN_BITS为TICK_TYPE_WIDTH_16_BITS，则每个事件组包含8个可用事件位。
- 如果configTICK_TYPE_WIDTH_IN_BITS为TICK_TYPE_WIDTH_32_BITS，则每个事件组包含24个可用事件位。
- 如果configTICK_TYPE_WIDTH_IN_BITS为TICK_TYPE_WIDTH_64_BITS，则每个事件组包含56个可用事件位。

9.2.3 通过多个任务进行的访问

事件组本身是任何知道它们存在的任务或ISR都可以访问的对象。任意数量的任务都可以在同一事件组中设置位，任意数量的任务都可以从同一事件组中读取位。

9.2.4 使用事件组的一个实际例子

FreeRTOS+TCP TCP/IP堆栈的实现提供了一个实际示例，说明如何使用事件组来同时简化设计和最小化资源使用。

TCP 套接字必须对许多不同的事件做出响应。事件的例子包括接受事件、绑定事件、读取事件和关闭事件。套接字在任何给定时间所能预期的事件都取决于套接字的状态。例如，如果一个套接字已经创建，但尚未绑定到一个地址，那么它可以接收绑定事件，但不会接收读取事件（如果没有地址，就无法读取数据）。

FreeRTOS+TCP 套接字的状态保存在名为 FreeRTOS_Socket_t 的结构中。该结构包含一个事件组，该事件组为套接字必须处理的每个事件定义了一个事件位。FreeRTOS+TCP API 调用会阻塞等待一个事件或一组事件，其实仅仅阻塞了事件组

事件组还包含一个“中止”位，允许中止TCP连接，无论套接字当时正在等待哪个事件。

9.3 使用事件组的事件管理

9.3.1 xEventGroupCreate() 函数

FreeRTOS还包括xEventGroupCreate() 函数，该函数分配在编译时静态创建事件组所需的内存：必须显式创建事件组才能使用。

使用EventGroupHandle_t类型的变量来引用事件组。上述函数用于创建一个事件组，并返回一个EventGroupHandle_t来引用该事件组

创造

```
EventGroupHandle_t xEventGroupCreate( void );
```

清单 9.1 `xEventGroupCreate()` API 函数原型

函数返回值

- 返回值

如果返回NULL，则无法创建事件组，因为FreeRTOS没有足够的堆内存来分配事件组数据结构。第3章提供了关于堆内存管理的更多信息。

返回的非NULL值表示事件组已成功创建。这个返回的值应该存储为已创建的事件组的句柄。

9.3.2 `xEventGroupSetBits()` 函数

`xEventGroupSetBits()` 函数在事件组中设置一个或多个位，通常用于通知任务，由位或位表示的事件已经发生。

注意：永远不要从中断服务例程中调用`xEventGroupSetBits()`。应该使用中断安全版本的`FromISR()`。

```
EventBits_t xEventGroupSetBits( EventGroupHandle_t xEventGroup,
                                const EventBits_t uxBitsToSet );
```

清单9.2 `xEventGroupSetBits()` 函数原型

参数和返回值

- `xEventGroup`

正在设置位的事件组的句柄。事件组句柄将从用于创建事件组的 `xEventGroupCreate()` 调用中返回

- `uxBitsToSet`

指定事件组中要设置为 1 的一个或多个事件位的位掩码。通过将事件组的现有值与 `uxBitsToSet` 中传递的值按位或运算来更新事件组的值。例如，将 `uxBitsToSet` 设置为 0x04（二进制 0100）将导致事件组中的事件位 3 被设置（如果尚未设置），同时事件组中的所有其他事件位保持不变

- 返回值

返回调用 `xEventGroupSetBits()` 时事件组的值。请注意，返回的值不一定具有由 `uxBitsToSet` 设置的指定位，因为这些位可能已被不同的任务再次清除

9.3.3 xEventGroupSetBitsFromISR() API函数

`xEventGroupSetBitsFromISR()` 是 `xEventGroupSetBits()` 的中断安全版本

give信号量是一种确定性操作，因为预先知道给出信号量最多可以导致一个任务离开阻塞状态。当在事件组中设置位时，事先不知道有多少任务将离开阻塞状态，因此在事件组中设置位不是确定性操作。

FreeRTOS 设计和实现标准不允许在中断服务例程内或禁用中断时执行非确定性操作。因此，`xEventGroupSetBitsFromISR()` 不会直接在中断服务例程内设置事件位，而是将操作推迟到 RTOS 守护程序任务。

```
BaseType_t xEventGroupSetBitsFromISR( EventGroupHandle_t xEventGroup,
                                       const EventBits_t uxBitsToSet,
                                       BaseType_t *pxHigherPriorityTaskWoken );
```

清单9.3 `xEventGroupSetBitsFromISR()` API函数原型

参数和返回值

- `xEventGroup`

正在设置位的事件组的句柄。事件组句柄将从用于创建事件组的 `xEventGroupCreate()` 调用中返回

- `uxBitsToSet`

指定在事件组中设置为1的事件位或事件位的位掩码。事件组的值通过使用`uxBitsToSet`来更新事件组的现有值。

例如，将`uxBitsToSet`设置为0x05（二进制0101）将导致事件组中的事件位2和事件位0被设置（如果它们尚未设置），而使事件组中的所有其他事件位保持不变。

- `pxHigherPriorityTaskWoken`

`xEventGroupSetBitsFromISR()` 不会直接在中断服务例程内设置事件位，而是通过在定时器命令队列上发送命令来将操作推迟到 RTOS 守护程序任务。如果守护程序任务处于阻塞状态以等待计时器命令队列上的数据变得可用

那么写入定时器命令队列将导致守护任务离开阻塞状态。如果守护程序任务的优先级高于当前正在执行的任务（被中断的任务）的优先级，则 `xEventGroupSetBitsFromISR()` 在内部会将 `*pxHigherPriorityTaskWoken` 设置为 `pdTRUE`

如果函数将此值设置为 `pdTRUE`，则应该在退出中断之前执行上下文切换。这将确保中断直接返回到守护进程任务，因为守护进程任务将是最高优先级的准备就绪状态任务。

•返回值

有两个可能的返回值：

- 只有当数据被成功地发送到计时器命令队列时，才会返回 `pdPASS`。
- 如果“设置位”命令无法写入计时器命令，则将返回 `pdFALSE` 因为队列已经满。

9.3.4 xEventGroupWaitBits() API函数

该函数允许任务读取事件组的值，如果事件位尚未设置，则可以选择在阻塞状态下等待事件组中的一个或多个事件位被设置。

```
EventBits_t xEventGroupWaitBits( EventGroupHandle_t xEventGroup,
                                const EventBits_t uxBitsToWaitFor,
                                const BaseType_t xClearOnExit,
                                const BaseType_t xWaitForAllBits,
                                TickType_t xTicksToWait );
```

清单9.4 xEventGroupWaitBits()函数原型

调度程序用来确定任务是否进入阻塞状态以及任务何时离开阻塞状态的条件称为“解除阻塞条件”。解锁条件由 `uxBitsToWaitFor` 和 `xWaitForAllBits` 参数值的组合指定：

- `uxBitsToWaitFor` 指定要测试事件组中的哪些事件位
- `xWaitForAllBits` 指定是使用按位或测试还是按位与测试

如果调用 `xEventGroupWaitBits()` 时满足解锁条件，任务将不会进入阻塞状态。

表 5 中提供了导致任务进入阻塞状态或退出阻塞状态的条件示例。表 5 仅显示事件组的最低有效的四个二进制位和 `uxBitsToWaitFor` 值 - 这些的其他位假设两个值为零

现有事件组值	uxBitsToWaitFor	xWaitForAllBits	结果行为
0000	0101	pdFALSE	由于事件组中的位 0 或位 2 均未设置，调用任务将进入阻塞状态，并且当事件组中的位 0 或位 2 被设置时，调用任务将离开阻塞状态
0100	0101	pdTRUE	调用任务将进入阻塞状态，因为事件组中的位 0 和位 2 未同时设置，并且当事件组中的位 0 和位 2 均设置时，调用任务将离开阻塞状态
0100	0110	pdFALSE	调用任务将不会进入阻塞状态，因为xWaitForAllBit是pdFALSE，并且uxBitsTowait指定的两位之一已经在事件组中设置
0100	0110	pdTRUE	调用任务将进入阻塞状态，因为 xWaitForAllBits 为pdTRUE，并且事件组中仅设置了 uxBitsToWaitFor 指定的两个位之一。当事件组中的位 1 和位 2 均被设置时，任务将离开阻塞状态。

表5 uxBitsToWaitFor 和 xWaitForAllBits 参数的作用

调用任务使用 uxBitsToWaitFor 参数指定要测试的位，并且调用任务可能需要在满足其解锁条件后将这些位清零。可以使用 xEventGroupClearBits() API 函数清除事件位，但使用该函数手动清除事件位将导致应用程序代码中出现竞争条件，如果：

- 有多个任务在使用同一事件组。
- 位由不同的任务或中断服务例程在事件组中设置。

提供 xClearOnExit 参数是为了避免这些潜在的竞争条件。如果 xClearOnExit 设置为 pdTRUE，则事件位的测试和清除对于调用任务来说是一个原子操作（不能被其他任务或中断中断）

xEventGroupWaitBits() 参数和返回值

- xEventGroup

包含正在被读取的事件位的事件组的句柄。

•uxBitsToWaitFor

在事件组中指定事件位或事件位的位掩码。

例如，如果调用任务希望等待事件位0和/或事件位2在事件组中被设置，则将uxBitsToWaitFor设置为0x05（二进制0101）。有关更多的示例，请参见表5。

•xClearOnExit

如果满足调用任务的解锁条件，并且 xClearOnExit 设置为 pdTRUE，则在调用任务退出

xEventGroupWaitBits() API 函数之前，事件组中由 uxBitsToWaitFor 指定的事件位将被清除回 0

如果 xClearOnExit 设置为 pdFALSE，则 xEventGroupWaitBits() API 函数不会修改事件组中事件位的状态

•xWaitForAllBits

uxBitsToWaitFor 参数指定事件组中要测试的事件位。 xWaitForAllBits 指定当 uxBitsToWaitFor 参数指定的一个或多个事件位被设置时，或者仅当 uxBitsToWaitFor 参数指定的所有事件位被设置时，是否应将调用任务从阻塞状态中删除。

如果 xWaitForAllBits 设置为 pdFALSE，则进入阻塞状态以等待满足其解除阻塞条件的任务将在 uxBitsToWaitFor 指定的任何位设置时（或 xTicksToWait 参数指定的超时到期）离开阻塞状态。

如果 xWaitForAllBits 设置为 pdTRUE，则进入阻塞状态以等待满足其解除阻塞条件的任务只有在 uxBitsToWaitFor 指定的所有位均已设置（或者 xTicksToWait 参数指定的超时到期）时才会离开阻塞状态）。

具体示例见表5。

•xTicksToWait

任务应保持阻塞状态以等待满足其解除阻塞条件的最长时间。

如果 xTicksToWait 为零，或者调用 xEventGroupWaitBits() 时满足解锁条件，xEventGroupWaitBits() 将立即返回。

区块时间以滴答周期为单位指定，因此它表示的绝对时间取决于滴答频率。宏 pdMS_TO_TICKS() 可用于将以毫秒为单位的时间转换为以刻度为单位的时间。

如果 FreeRTOSConfig.h 中的 INCLUDE_vTaskSuspend 设置为 1，则将 xTicksToWait 设置为 portMAX_DELAY 将导致任务无限期等待（不会超时）

•返回值

如果 `xEventGroupWaitBits()` 由于满足调用任务的解锁条件而返回，则返回值是满足调用任务的解锁条件时事件组的值（如果 `xClearOnExit` 为 `pdTRUE`，则在自动清除任何位之前）。在这种情况下，返回值也将满足解锁条件。

如果 `xEventGroupWaitBits()` 因为 `xTicksToWait` 参数指定的区块时间到期而返回，则返回值是区块时间到期时事件组的值。这种情况下返回值将不满足解锁条件

9.3.5 xEventGroupGetStaticBuffer() API函数

`xEventGroupGetStaticBuffer()` API 函数提供了一种检索指向静态创建的事件组缓冲区的指针的方法。它与创建事件组时提供的缓冲区相同

***注意：**永远不要从中断服务例程中调用 `xEventGroupGetStaticBuffer()`。

```
BaseType_t xEventGroupGetStaticBuffer ( EventGroupHandle_t xEventGroup,
                                         StaticEventGroup_t ** ppxEventGroupBuffer );
```

清单9.5 `xEventGroupGetStaticBuffer()`函数原型

参数和返回值

- `xEventGroup`

要检索缓冲区的事件组。该事件组必须由

`xEventGroupCreateStatic()` 创建

- `ppxEventGroupBuffer`

用于返回一个指向事件组的数据结构缓冲区的指针。它与创建时提供的缓冲区相同。

- 返回值

有两个可能的返回值：

- 如果成功检索缓冲区，将返回 `pdTRUE`。
- 如果缓冲区检索不成功，则将返回 `pdFALSE`。

示例9.1 试验利用事件组

本示例演示了如何：

- 创建事件组。

- 从中断服务例程中设置事件组中的位。
- 从任务中设置事件组中的位。
- 在事件组上阻塞。

`xEventGroupWaitBits()` `xWaitForAllBits` 参数的效果通过首先执行 `xWaitForAllBits` 设置为 `pdFALSE` 的示例，然后执行 `xWaitForAllBits` 设置为 `pdTRUE` 的示例来演示。

事件位 0 和事件位 1 由任务设置。事件位 2 由中断服务程序设置。使用清单 9.6 中所示的 `#define` 语句为这三位提供了描述性名称

```
/*对事件组中的事件位的定义。*/

#define mainFIRST_TASK_BIT ( 1UL << 0UL ) /* Event bit 0, set by a task */
#define mainSECOND_TASK_BIT ( 1UL << 1UL ) /* Event bit 1, set by a task */
#define mainISR_BIT ( 1UL << 2UL ) /* Event bit 2, set by an ISR */
```

清单9.6 示例9.1中使用的事件位定义

清单 9.7 显示了设置事件位 0 和事件位 1 的任务的实现。它位于一个循环中，重复设置一个位，然后设置另一个位，每次调用 `xEventGroupSetBits()` 之间有 200 毫秒的延迟。在设置每个位之前会打印出一个字符串，以便在控制台中看到执行顺序。

```
static void vEventBitSettingTask( void *pvParameters )
{
    const TickType_t xDelay200ms = pdMS_TO_TICKS( 200UL ), xDontBlock = 0;

    for( ; ; )
    {
        /*在开始下一个循环之前会延迟一段时间。*/
        vTaskDelay ( xDelay200ms );

        /*打印一条消息，说事件位0即将由任务设置，然后设置事件位0。*/
        vPrintString( "Bit setting task -\t about to set bit 0.\r\n" );
        xEventGroupSetBits( xEventGroup, mainFIRST_TASK_BIT );

        /*在设置另一位之前会延迟一段时间。*/
        vTaskDelay ( xDelay200ms );

        /*打印出一条消息，说事件位1即将由任务设置，然后设置事件位1。*/
        vPrintString( "Bit setting task -\t about to set bit 1.\r\n" );
        xEventGroupSetBits( xEventGroup, mainSECOND_TASK_BIT );
    }
}
```

清单9.7 示例9.1中的事件组中设置两个位的任务

清单 9.8 显示了在事件组中设置位 2 的中断服务例程的实现。同样，在设置该位之前会打印出一个字符串，以允许在控制台中看到执行顺序。然而，在这种情况下，由于不应在中断服务例程中直接执行控制台输出，因此使用 `xTimerPendFunctionCallFromISR()` 在 RTOS 守护程序任务的上下文中执行输出

与前面的示例一样，中断服务例程由一个简单的周期性任务触发，该任务迫使软件中断。在本例中，中断每500毫秒生成一次。

```
static uint32_t ulEventBitSettingISR( void )
{
    /*该字符串不会在中断服务例程中打印，而是发送到 RTOS 守护程序任务进行打印。
    因此将其声明为静态以确保编译器不会在 ISR 的堆栈上分配字符串，因为当从守护
    程序任务打印字符串时，ISR 的堆栈帧将不存在*/

    static const char *pcString = "Bit setting ISR -\t about to set bit 2.\r\n";
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    /*打印一条消息，表明即将设置位 2。无法从 ISR 打印消息，因此通过挂起函数调用以在
    RTOS 守护程序任务的上下文中运行来将实际输出推迟到 RTOS 守护程序任务*/

    xTimerPendFunctionCallFromISR ( vPrintStringFromDaemonTask,
                                     (void *) pcString,
                                     0,
                                     &xHigherPriorityTaskWoken );

    /*在事件组中设置第2位。*/
    xEventGroupSetBitsFromISR ( xEventGroup,
                                mainISR_BIT ,
                                &xHigherPriorityTaskWoken );

    /*xTimerPendFunctionCallFromISR() 和 xEventGroupSetBitsFromISR
    () 都写入计时器命令队列，并且都使用相同的
    xHigherPriorityTaskWoken 变量。如果写入定时器命令队列导致 RTOS
    守护程序任务离开阻塞状态，并且 RTOS 守护程序任务的优先级高于当前正在执行的任务（此中断中断的任务）的优先级，则
    xHigherPriorityTaskWoken 将被设置为 pdTRUE。

    xHigherPriorityTaskWoken 用作 portYIELD_FROM_ISR() 的参数。如果
    xHigherPriorityTaskWoken 等于 pdTRUE，则调用 portYIELD_FROM_
    ISR() 将请求上下文切换。如果 xHigherPriorityTaskWoken 仍为
    pdFALSE，则调用 portYIELD_FROM_ISR() 将不起作用。

    Windows 端口使用的 portYIELD_FROM_ISR() 的实现包含一个 return
    语句，这就是该函数没有显式返回值的原因*/
}
```

```
portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}
```

清单9.8 在示例9.1中的事件组中设置第2位的ISR

清单 9.9 显示了调用 `xEventGroupWaitBits()` 来阻塞事件组的任务的实现。该任务为事件组中设置的每个位打印出一个字符串。

`xEventGroupWaitBits()` `xClearOnExit` 参数设置为 `pdTRUE`，因此导致调用 `xEventGroupWaitBits()` 返回的事件位将在 `xEventGroupWaitBits()` 返回之前自动清除。

```
static void vEventBitReadingTask( void *pvParameters )
{
    EventBits_t xEventGroupValue;
    const EventBits_t xBitsToWaitFor = ( mainFIRST_TASK_BIT |
                                          mainSECOND_TASK_BIT |
                                          mainISR_BIT );

    for ( ; ; )
    {
        /*阻塞等待事件组内的事件位被设置。*/
        xEventGroupValue = xEventGroupWaitBits(
                                                    xEventGroup,

                                                    /*测试位*/
                                                    xBitsToWaitFor,

                                                    /*如果满足解除阻塞条件，则清除位
                                                    在退出时*/
                                                    pdTRUE,

                                                    /*不要等待所有位。第二次执行时该
                                                    参数设置为pdTRUE*/
                                                    pdFALSE,

                                                    /*不要超时。*/
                                                    portMAX_DELAY );

        /*为设置的每个位打印一条消息。*/
        if( ( xEventGroupValue & mainFIRST_TASK_BIT ) != 0 )
        {
```



```

        vPrintString( "Bit reading task -\t Event bit 0 was set\r\n" );
    }
    if( ( xEventGroupValue & mainSECOND_TASK_BIT ) != 0 )
    {
        vPrintString( "Bit reading task -\t Event bit 1 was set\r\n" );
    }
    if( ( xEventGroupValue & mainISR_BIT ) != 0 )
    {
        vPrintString( "Bit reading task -\t Event bit 2 was set\r\n" );
    }
}

}

```

清单9.9 示例9.1中阻塞等待事件位被设置的任务

main() 函数在启动调度程序之前创建事件组和任务。其实现请参见清单 9.10。从事件组读取任务的优先级高于向事件组写入任务的优先级，确保每次满足读取任务的解锁条件时，读取任务都会抢占写入任务

```

int main(void)
{
    /*在使用事件组之前，必须先创建它。*/
    xEventGroup = xEventGroupCreate();

    /*创建在事件组中设置事件位的任务*/

    xTaskCreate( vEventBitSettingTask, "Bit Setter", 1000, NULL, 1, NULL );

    /*创建等待在事件组中设置事件位的任务。*/
    xTaskCreate( vEventBitReadingTask, "Bit Reader", 1000, NULL, 2, NULL );

    /*创建用于定期生成软件中断的任务。*/
    xTaskCreate( vInterruptGenerator, "Int Gen", 1000, NULL, 3, NULL );
    /*安装软件中断的处理程序。执行此操作所需的语法取决于所使用的 FreeRTOS 端口。此处
    显示的语法只能与 FreeRTOS Windows 端口一起使用，其中仅模拟此类中断。*/
    vPortSetInterruptHandler( mainINTERRUPT_NUMBER, ulEventBitSettingISR );

    /*启动调度程序，以便已创建的任务开始执行。*/
    vTaskStartScheduler();

    /*永远不应该到达下面一行。*/
}

```

```

    for( ; ; );
    return 0;
}

```

清单9.10 在示例9.1中创建事件组和任务

当 `xEventGroupWaitBits()` `xWaitForAllBits` 参数设置为 `pdFALSE` 时执行例 9.1 时产生的输出如图 9.3 所示。从图 9.3 中可以看出，由于调用 `xEventGroupWaitBits()` 时的 `xWaitForAllBits` 参数设置为 `pdFALSE`，因此每次设置任何事件位时，从事件组读取的任务都会离开阻塞状态并立即执行

```

C:\Windows\system32\cmd.exe - rtosdemo
Bit setting task -      about to set bit 1.
Bit reading task -      event bit 1 was set
Bit setting task -      about to set bit 0.
Bit reading task -      event bit 0 was set
Bit setting task -      about to set bit 1.
Bit reading task -      event bit 1 was set
Bit setting ISR -       about to set bit 2.
Bit reading task -      event bit 2 was set
Bit setting task -      about to set bit 0.
Bit reading task -      event bit 0 was set
Bit setting task -      about to set bit 1.
Bit reading task -      event bit 1 was set
Bit setting ISR -       about to set bit 2.
Bit reading task -      event bit 2 was set
Bit setting task -      about to set bit 0.
Bit reading task -      event bit 0 was set

```

图9.3 将`xWaitForAllBits`设置为`pdFALSE`时，执行示例9.1时产生的输出

当 `xEventGroupWaitBits()` `xWaitForAllBits` 参数设置为 `pdTRUE` 时执行例 9.1 时产生的输出如图 9.4 所示。从图 9.4 中可以看出，由于 `xWaitForAllBits` 参数设置为 `pdTRUE`，因此从事件组读取的任务仅在所有三个事件位设置后才离开阻塞状态。

```

C:\Windows\system32\cmd.exe - rtsdemo
Bit setting task -      about to set bit 1.
Bit setting task -      about to set bit 0.
Bit setting ISR -       about to set bit 2.
Bit reading task -      event bit 0 was set
Bit reading task -      event bit 1 was set
Bit reading task -      event bit 2 was set

Bit setting task -      about to set bit 1.
Bit setting task -      about to set bit 0.
Bit setting ISR -       about to set bit 2.
Bit reading task -      event bit 0 was set
Bit reading task -      event bit 1 was set
Bit reading task -      event bit 2 was set

Bit setting task -      about to set bit 1.
Bit setting task -      about to set bit 0.
Bit setting task -      about to set bit 1.
Bit setting ISR -       about to set bit 2.
Bit reading task -      event bit 0 was set
Bit reading task -      event bit 1 was set
Bit reading task -      event bit 2 was set

Bit setting task -      about to set bit 0.
Bit setting task -      about to set bit 1.

```

图9.4 当将`xWaitForAllBits`设置为`pdTRUE`时，执行示例9.1时产生的输出

9.4 使用事件组进行任务同步

有时，应用程序的设计需要两个或多个任务来相互同步。例如，考虑一个任务A接收到一个事件的设计，然后将该事件所需要的一些处理委托给其他三个任务：任务B、任务C和任务D。如果任务A在任务B、C和D全部完成前一个事件的处理之前无法接收到另一个事件，那么所有四个任务都需要相互同步。每个任务的同步点将在该任务完成其处理之后进行，在其他每个任务完成相同的任务之前不能继续进行。任务A只能在所有四个任务到达同步点后接收另一个事件。

在一个FreeRTOS+TCP演示项目中，可以找到一个不那么抽象的关于需要这种类型的任务同步的例子。该演示在两个任务之间共享一个TCP套接字；一个任务将数据发送到该套接字，而另一个任务从同一个套接字接收数据^[^25]。在确定其他任务不会再次尝试访问该套接字之前，对任何一个任务关闭TCP套接字都是不安全的。如果两个任务中的任何一个希望关闭套接字，那么它必须通知另一个任务其意图，然后等待其他任务停止使用套接字，然后再继续。清单9.10中所示的伪代码演示了向希望关闭套接字的套接字发送数据的任务的场景。

^[^25]：在撰写本文时，这是可以在任务之间共享一个FreeRTOS+TCP套接字的唯一方法。

清单9.10所示的场景非常简单，因为只有两个任务需要相互同步，但很容易看出场景将变得更复杂，并且需要更多任务加入同步，如果其他任务执行依赖于套接字打开的处理。

```

void SocketTxTask( void *pvParameters )
{

```

```

xSocket_t xSocket;
uint32_t ulTxCount = 0UL;

for(;;)
{
    /*创建一个新的套接字。此任务将发送到这个套接字和另一个
    任务将从这个套接字接收。*/
    xSocket = FreeRTOS_socket( ... );

    /*连接套接字。*/
    FreeRTOS_connect( xSocket, ... );

    /*使用队列将套接字发送到接收数据的任务。*/
    xQueueSend( xSocketPassingQueue, &xSocket, portMAX_DELAY );

    /*在关闭套接字之前，请向套接字发送1000个消息。*/
    for( ulTxCount = 0; ulTxCount < 1000; ulTxCount++ )
    {
        if (FreeRTOS_send( xSocket, ... ) < 0)
        {
            /*意外错误 - 退出循环，之后套接字将被关闭。*/

            break;
        }
    }

    /*让 Rx 任务知道 Tx 任务想要关闭套接字*/
    TxTaskWantsToCloseSocket();

    /*这是 Tx 任务的同步点。Tx 任务在此等待 Rx 任务到达其同步点。Rx 任务只有在不再
    使用套接字时才会到达其同步点，并且可以安全地关闭套接字。*/

    xEventGroupSync ( ... );

    /*两个任务都不使用套接字。然后，关闭连接
    关闭套接字。*/
    FreeRTOS_shutdown( xSocket, ... );
    WaitForSocketToDisconnect();
    FreeRTOS_closesocket (xSocket);
}

}

/*-----*/

void SocketRxTask( void *pvParameters )
{
    xSocket_t xSocket;

    for(;;)
    {

```

```

/*等待接收由 Tx 任务创建并连接的套接字。*/
xQueueReceive( xSocketPassingQueue, &xSocket, portMAX_DELAY );

/*继续从套接字接收，直到Tx任务想要关闭套接字。*/
while( TxTaskWantsToCloseSocket() == pdFALSE )
{
    /* 接收数据，然后处理数据。*/
    FreeRTOS_recv( xSocket ); ...
    ProcessReceivedData();
}
/*这是 Rx 任务的同步点 - 仅当不再使用套接字时才到达此处，因此 Tx 任务关闭套
接字是安全的。*/

    xEventGroupSync ( ... );
}
}

```

清单9.11 两个相互同步的任务的伪代码，以确保在套接字关闭之前不再被共享的TCP套接字

事件组可用于创建同步点

每个必须参与同步的任务都会在事件组内分配一个唯一的事件位

- 每个任务在到达同步点时都会设置自己的事件位。
- 设置自己的事件位后，事件组上的每个任务都会阻塞，等待代表所有其他同步任务的事件位也被设置

但是，在此场景中无法使用 `xEventGroupSetBits()` 和 `xEventGroupWaitBits()` API 函数。如果使用它们，则位的设置（以指示任务已到达其同步点）和位的测试（以确定其他同步任务是否已到达其同步点）将作为两个单独的操作来执行。要了解为什么会出现问题，请考虑任务 A、任务 B 和任务 C 尝试使用事件组进行同步的场景：

1. 任务A和任务B已经到达同步点，因此它们的事件位在事件组中被设置，并且它们处于阻塞状态以等待任务C的事件位也被设置。
2. 任务 C 到达同步点，并使用 `xEventGroupSetBits()` 设置其在事件组中的位。一旦任务 C 的位被设置，任务 A 和任务 B 就会离开阻塞状态，并清除所有三个事件位。
3. 然后，任务 C 调用 `xEventGroupWaitBits()` 等待所有三个事件位都被设置，但此时，所有三个事件位都已被清除，任务 A 和任务 B 已离开各自的同步点，因此同步失败

要成功地使用事件组来创建同步点，必须将事件位的设置和随后的事件位测试作为单一的不间断操作来执行。为此目的，提供了 `xEventGroupSync()` API 函数。

9.4.1 xEventGroupSync() API 函数

提供 `xEventGroupSync()` 是为了允许两个或多个任务使用事件组来相互同步。该函数允许任务设置事件组中的一个或多个事件位，然后等待同一事件组中的事件位组合被设置，作为单个不间断操作。

`xEventGroupSync()` `uxBitsToWaitFor` 参数指定调用任务的解锁条件。如果 `xEventGroupSync()` 由于满足解锁条件而返回，则 `uxBitsToWaitFor` 指定的事件位将在 `xEventGroupSync()` 返回之前清回零

```
EventBits_t xEventGroupSync ( EventGroupHandle_t xEventGroup,
                              const EventBit_t uxBitsToSet,
                              const EventBit_t, uxBitsToWaitFor,
                              TickType_t xTicksToWait );
```

清单9.12 xEventGroupSync API函数原型

参数和返回值

•xEventGroup

要在其中设置事件位然后进行测试的事件组的句柄。事件组句柄将从用于创建事件组的 `xEventGroupCreate()` 调用中返回

•uxBitsToSet

指定事件组中要设置为 1 的一个或多个事件位的位掩码。通过将事件组的现有值与 `uxBitsToSet` 中传递的值按位或运算来更新事件组的值。

例如，将 `uxBitsToSet` 设置为 0x04（二进制 0100）将导致事件位 2 被设置（如果尚未设置），同时事件组中的所有其他事件位保持不变。

uxBitsToWaitFor

•

指定要在事件组中测试的一个或多个事件位的位掩码。

例如，如果调用任务想要等待事件组中的事件位 0、1 和 2 被设置，则将 `uxBitsToWaitFor` 设置为 0x07（二进制 111）

•xTicksToWait

任务应保持在“阻塞”状态等待其取消阻塞条件的最长时间。

如果 `xTicksToWait` 为零，或者调用 `xEventGroupSync()` 时满足解锁条件，`xEventGroupSync()` 将立即返回。

阻塞时间以滴答周期为单位指定，因此它表示的绝对时间取决于滴答频率。宏 `pdMS_TO_TICKS()` 可用于将以毫秒为单位的时间转换为以刻度为单位的时间。

如果 `FreeRTOSConfig.h` 中的 `INCLUDE_vTaskSuspend` 设置为 1，则将 `xTicksToWait` 设置为 `portMAX_DELAY` 将导致任务无限期等待（不会超时）

•返回值

如果 `xEventGroupSync()` 由于满足调用任务的解锁条件而返回，则返回值是满足调用任务的解锁条件时事件组的值（在任何位自动清零之前）。在这种情况下，返回值也将满足调用任务的解锁条件。

如果 `xEventGroupSync()` 由于 `xTicksToWait` 参数指定的阻塞时间到期而返回，则返回值为阻塞时间到期时事件组的值。在这种情况下，返回值将不满足调用任务的解锁条件

示例9.2 同步任务

例 9.2 使用 `xEventGroupSync()` 来同步单个任务实现的三个实例。任务参数用于将任务在调用 `xEventGroupSync()` 时设置的事件位传递到每个实例。

该任务在调用 `xEventGroupSync()` 之前打印一条消息，并在对 `xEventGroupSync()` 的调用返回后再次打印一条消息。每条消息都包含一个时间戳。这允许在产生的输出中观察执行顺序。使用伪随机延迟来防止所有任务同时到达同步点。

任务的实现参见清单 9.12

```
static void vSyncingTask( void *pvParameters )
{
    const TickType_t xMaxDelay = pdMS_TO_TICKS( 4000UL );
    const TickType_t xMinDelay = pdMS_TO_TICKS( 200UL );
    TickType_t xDelayTime;
    EventBits_t uxThisTasksSyncBit;
    const EventBits_t uxAllSyncBits = (    mainFIRS_TAS_BIT |
                                           mainSECOND_TASK_BIT |
                                           mainTHIRD_TASK_BIT );

    /*创建该任务的三个实例 - 每个任务在同步中使用不同的事件位。
    使用任务参数将要使用的事件位传递到每个任务实例中。将其存储在 uxThisTasksSyncBit 变量中
    */
}
```

```

uxThisTasksSyncBit = (EventBits_t) pvParameters;

for( ; ; )
{
    /*通过延迟一个伪随机时间来模拟这个任务，这需要一些时间来执行一个动作。这可以
       防止此任务的所有三个实例同时到达同步点，因此可以更容易地观察到该示例的行为
       。*/
    xDelayTime = ( rand() % xMaxDelay ) + xMinDelay;
    vTaskDelay( xDelayTime );

    /*打印一条消息以表明该任务已达到其同步点。 pcTaskGetTaskName() 是一个 API 函数，它返
       回创建任务时分配给任务的名称*/
    vPrintTwoStrings( pcTaskGetTaskName( NULL ), "reached sync point" );

    /*等待所有任务到达各自的任务的
       同步点。*/
    xEventGroupSync(
        xEventGroup,

        /*此任务设置的表示它已到达同步点的位。*/
        uxThisTasksSyncBit,

        /*要等待的位，参与同步的每个任务对应一个位。*/
        uxAllSyncBits,

        /*无限地等待所有三个任务到达
           同步点*/
        portMAX_DELAY );

    /*打印一条消息以表明该任务已通过其同步点。由于使用了无限期延迟，
       因此只有在所有任务到达各自的同步点后才会执行以下行。
       */

    vPrintTwoStrings( pcTaskGetTaskName( NULL ), "exited sync point" );
}

```

清单9.13 示例9.2中使用的任务的实现

主函数创建事件组，创建所有三个任务，然后启动调度程序。其实现情况见清单9.14。

```

/*对事件组中的事件位的定义。*/

```



```

#define mainFIRST_TASK_BIT ( 1UL << 0UL ) /* Event bit 0, set by the 1st task */
#define mainSECOND_TASK_BIT( 1UL << 1UL ) /* Event bit 1, set by the 2nd task */
#define mainTHIRD_TASK_BIT ( 1UL << 2UL ) /* Event bit 2, set by the 3rd task */

/*声明用于同步这三个任务的事件组。*/
EventGroupHandle_t xEventGroup;

int void(main)
{
    /*在使用事件组之前，必须先创建它。*/
    xEventGroup = xEventGroupCreate();

    /*创建任务的三个实例。每个任务都有一个不同的名称，稍后会打印出来以直观地指示正在执行的任务。
    使用任务参数将任务到达其同步点时要使用的事件位传递到任务中*/

    xTaskCreate( vSyncingTask, "Task 1", 1000, mainFIRST_TASK_BIT, 1, NULL );
    xTaskCreate( vSyncingTask, "Task 2", 1000, mainSECOND_TASK_BIT, 1, NULL );
    xTaskCreate( vSyncingTask, "Task 3", 1000, mainTHIRD_TASK_BIT, 1, NULL );

    /*启动调度程序，以便已创建的任务开始执行。*/
    vTaskStartScheduler();

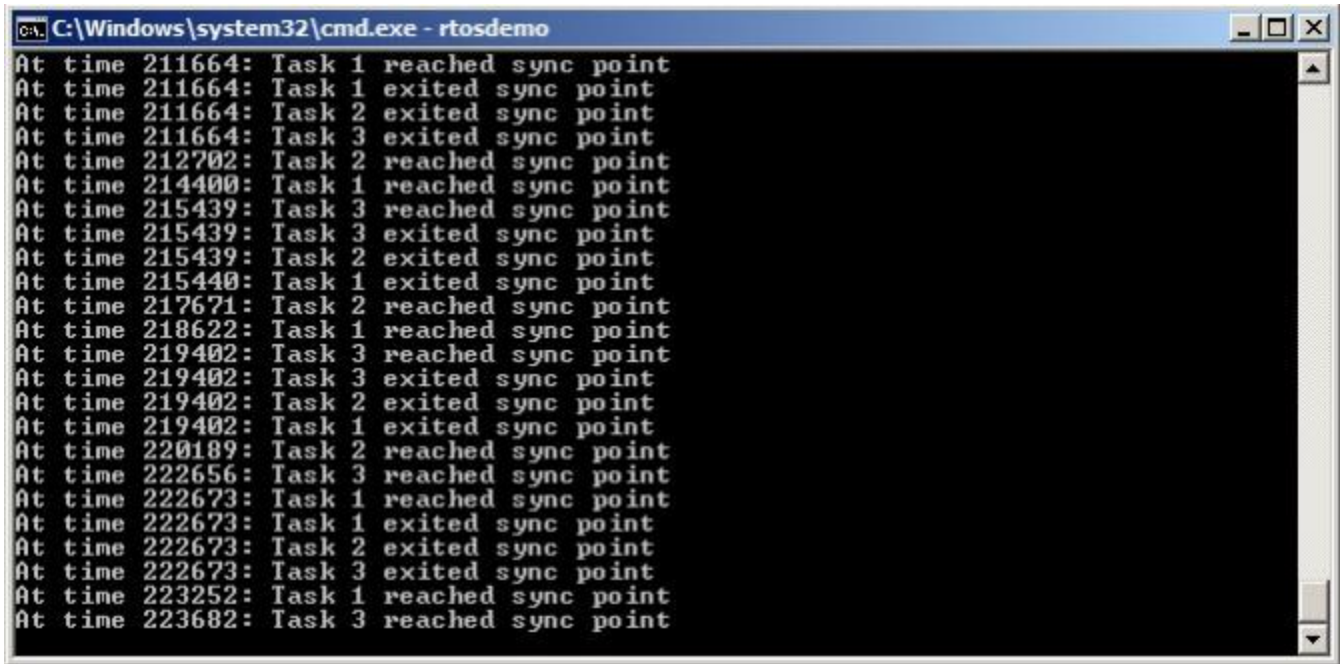
    /*一如既往，永远不能到达以下一行。*/
    for( ; );
    return 0;
}

```

清单9.14 示例9.2中使用的主函数

当执行示例9.2时产生的输出如图9.5所示。可以看出，即使每个任务在不同的（伪随机）时间到达同步点，每个任务也在同一时间退出同步点^[26]（这是最后一个任务到达同步点的时间）。

^[26]：图9.5显示了在FreeRTOS Windows端口中运行的示例，它不提供真正的实时行为（特别是当使用Windows系统调用打印到控制台时），因此将显示一些时间变化。



```
C:\Windows\system32\cmd.exe - rtosdemo
At time 211664: Task 1 reached sync point
At time 211664: Task 1 exited sync point
At time 211664: Task 2 exited sync point
At time 211664: Task 3 exited sync point
At time 212702: Task 2 reached sync point
At time 214400: Task 1 reached sync point
At time 215439: Task 3 reached sync point
At time 215439: Task 3 exited sync point
At time 215439: Task 2 exited sync point
At time 215440: Task 1 exited sync point
At time 217671: Task 2 reached sync point
At time 218622: Task 1 reached sync point
At time 219402: Task 3 reached sync point
At time 219402: Task 3 exited sync point
At time 219402: Task 2 exited sync point
At time 219402: Task 1 exited sync point
At time 220189: Task 2 reached sync point
At time 222656: Task 3 reached sync point
At time 222673: Task 1 reached sync point
At time 222673: Task 1 exited sync point
At time 222673: Task 2 exited sync point
At time 222673: Task 3 exited sync point
At time 223252: Task 1 reached sync point
At time 223682: Task 3 reached sync point
```

图9.5 在执行示例9.2时产生的输出

10 任务通知

10.1 简介

FreeRTOS应用程序通常是一系列独立的任务，这些任务相互通信，以共同提供系统功能。任务通知是一种有效的机制，允许一个任务直接通知另一个任务。

10.1.1 通过中介对象进行通信

这本书已经描述了任务之间可以相互交流的各种方式。到目前为止，所描述的方法都需要创建一个通信对象。通信对象的示例包括队列、事件组和各种不同类型的信号量。

当使用一个通信对象时，事件和数据不会被直接发送到一个接收任务，或一个接收ISR，而是被发送到该通信对象。同样地，任务和ISR从通信对象接收事件和数据，而不是直接从发送事件或数据的任务或ISR接收事件和数据。这一点如图10.1所示。

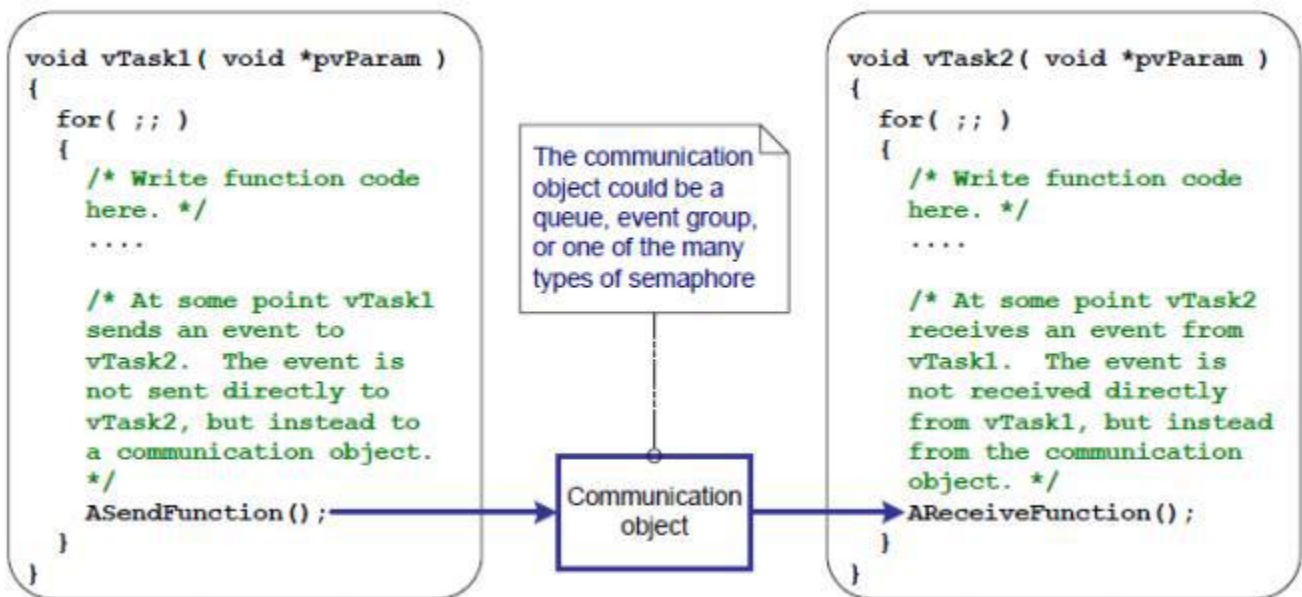


图10.1 用于将事件从一个任务发送到另一个任务的通信对象

10.1.2 任务通知-直接发送到任务通信

“任务通知”允许任务与其他任务交互，并与ISR同步，而不需要一个单独的通信对象。通过使用任务通知，任务或ISR可以将事件直接发送到接收任务。这一点如图10.2所示。

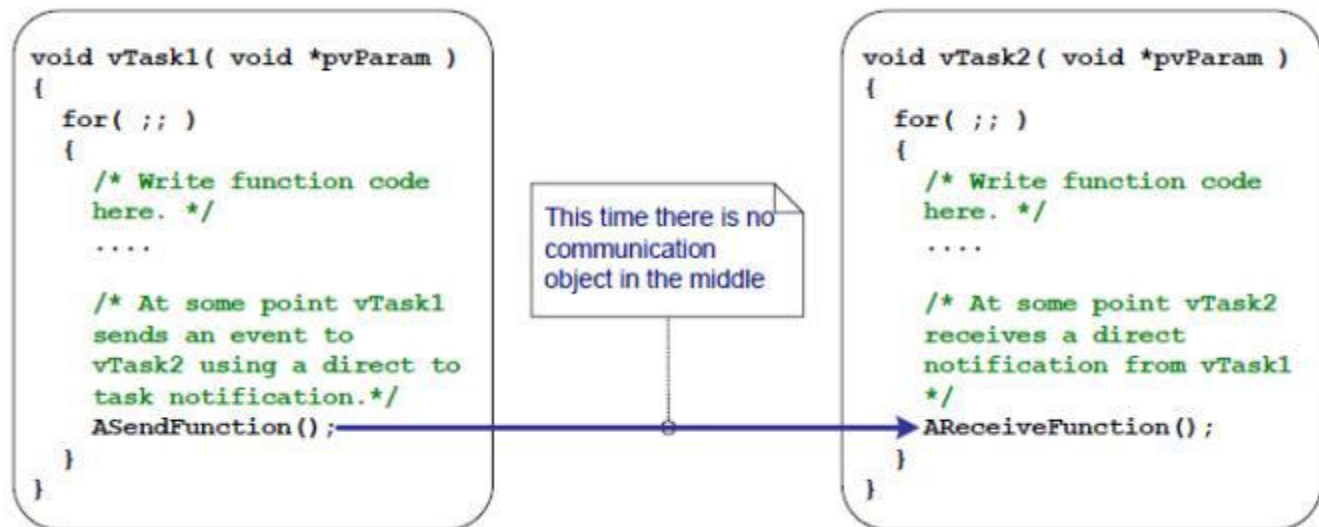


图10.2 一个用于将事件直接从一个任务发送到另一个任务的任务通知

任务通知功能是可选的。

要包含任务通知功能，请在 FreeRTOSConfig.h 中将 configUSE_TASK_NOTIFICATIONS 设置为 1。

当 configUSE_TASK_NOTIFICATIONS 设置为 1 时，每个任务至少有一个“通知状态”（可以是“待处理(挂起) Pending”或“未挂起Not-Pending”）和一个“通知值”（32 位无符号整数）。当任务收到通知时，其通知状态将设置为挂起。当任务读取其通知值时，其通知状态将设置为未挂起。如果 configTASK_NOTIFICATION_ARRAY_ENTRIES 设置为值 > 1，则存在由索引标识的多个通知状态和值。

任务可以在阻塞状态下等待，并有一个可选的超时时间，以使其通知状态变为待处理。

10.1.3 范围

本章讨论：

- 任务的通知状态和通知值。
- 如何以及何时可以使用任务通知来代替通信对象，例如信号量。
- 关于使用任务通知来代替通信对象的优点。

10.2 任务通知;好处和限制

10.2.1 任务通知的性能好处

使用任务通知来向任务发送事件或数据比使用队列、信号量或事件组来执行等效的操作要快得多。

10.2.2 任务通知的 RAM 占用优势

同样，使用任务通知向任务发送事件或数据所需的RAM比使用队列、信号量或事件组执行同等操作的RAM要少得多。这是因为必须创建每个通信对象（队列、信号量或事件组）才能使用，而启用任务通知功能有固定的开销。任务通知的RAM成本是每个任务的`configTASK_NOTIFICATION_ARRAY_ENTRIES * 8`字节。`configTASK_NOTIFICATION_ARRAY_ENTRIES`的默认值为1，因此任务通知的默认大小为每个任务8个字节。

10.2.3 任务通知的局限性

任务通知比通信对象更快，使用更少的RAM，但任务通知不能在所有场景中使用。本节记录了不能使用任务通知的场景：

- 向ISR发送事件或数据

通信对象可用于将事件和数据从ISR发送到任务，并从任务发送到ISR。

任务通知可用于将事件和数据从ISR发送到任务，但它们不能用于将事件或数据从任务发送到ISR。

- 启用多个接收任务（单接收者，不能多接收者）

通信对象可以被任何知道其句柄（可能是队列句柄、信号量句柄或事件组句柄）的任务或ISR访问。任意数量的任务和ISR都可以处理发送到任何给定通信对象的事件或数据。

任务通知被直接发送到接收任务，因此它们只能由被发送通知的任务进行处理。然而，这在实际情况很少是一个限制，因为虽然有多个任务和ISR发送到同一通信对象是常见的，但多个任务和ISR从同一个通信对象接收的情况很少见。

- 缓冲多个数据项

队列是一次可以保存多个数据项的通信对象。已发送到队列但尚未从队列接收的数据将在队列对象中缓冲。

任务通知通过更新接收任务的通知值来向任务发送数据。一个任务的通知值一次只能保存一个值。

- 广播到多个任务

事件组是一种通信对象，可用于一次向多个任务发送一个事件。

任务通知被直接发送到接收任务，因此只能由接收任务进行处理。

在被阻塞的状态下等待发送完成

如果通信对象暂时处于无法向其写入更多数据或事件的状态（例如，当队列已满时，无法向队列发送更多数据），则尝试写入该对象的任务可以选择进入阻塞状态以等待其写入操作完成。

如果任务尝试向已经有待处理通知的任务发送任务通知，则发送任务不可能在阻塞状态下等待接收任务重置其通知状态。

正如将要看到的，这在使用任务通知的实际情况中很少是限制

10.3 使用任务通知

10.3.1 任务通知API选项

任务通知是一个非常强大的功能，通常可以用来代替二进制信号量、计数信号量、事件组，有时甚至可以代替队列。这种广泛的使用场景可以通过使用 `xTaskNotify()` API 函数发送任务通知和 `xTaskNotifyWait()` API 函数接收任务通知来实现。

但是，在大多数情况下，不需要 `xTaskNotify()` 和 `xTaskNotifyWait()` API 函数提供的完全灵活性，更简单的函数就足够了。因此，`xTaskNotifyGive()` API 函数作为 `xTaskNotify()` 的更简单但灵活性较差的替代方案提供，而 `ulTaskNotifyTake()` API 函数作为 `xTaskNotifyWait()` 的更简单但灵活性较差的替代方案提供。

任务通知系统不限于单个通知事件。配置参数 `configTASK_NOTIFICATION_ARRAY_ENTRIES` 默认设置为 1。如果将其设置为大于 1 的值，则会在每个任务内创建一组通知。这允许通过索引管理通知。每个任务通知 api 函数都有一个索引版本。使用非索引版本将导致访问 `notification[0]`（数组中的第一个）。每个 API 函数的索引版本由后缀 `Indexed` 标识，因此函数 `xTaskNotify` 变为 `xTaskNotifyIndexed`。为了简单起见，本书中仅使用每个函数的非索引版本。

任务通知 API 以宏的形式实现，调用每个 API 函数类型的底层通用版本。为简单起见，本书中的 API 宏将被称为函数

10.3.1.1 API函数的完整列表²⁷

- `xTaskNotifyGive`
- `xTaskNotifyGiveIndexed`
- `vTaskNotifyGiveFromISR`
- `vTaskNotifyGiveIndexedFromISR`
- `vTaskNotifyTake`
- `vTaskNotifyTakeIndexed`
- `xTaskNotify`
- `xTaskNotifyIndexed`
- `xTaskNotifyWait`
- `xTaskNotifyWaitIndexed`

- `xTaskNotifyStateClear`
- `xTaskNotifyStateClearIndexed`
- `ulTaskNotifyValueClear`
- `ulTaskNotifyValueClearIndexed`
- `xTaskNotifyAndQueryIndexedFromISR`
- `xTaskNotifyAndQueryFromISR`
- `xTaskNotifyFromISR`
- `xTaskNotifyIndexedFromISR`
- `xTaskNotifyAndQuery`
- `xTaskNotifyAndQueryIndexed`

(27): 这些函数实际上是作为宏实现的。

注意：接收通知的FromISR功能不存在，因为通知总是被发送到一个任务，而中断不与任何任务关联。

10.3.2 xTaskNotifyGive () API函数

`xTaskNotifyGive()` 直接向任务发送通知，并递增（加一）接收任务的通知值。如果接收任务的通知状态尚未挂起，则调用 `xTaskNotifyGive()` 会将其设置为挂起。

提供 `xTaskNotifyGive()` API 函数以允许将任务通知用作二进制或计数信号量的更轻量级且更快的替代方案

```

 BaseType_t xTaskNotifyGive ( TaskHandle_t xTaskToNotify );
 BaseType_t xTaskNotifyGiveIndexed ( TaskHandle_t xTaskToNotify, UBaseType_t
 uxIndexToNotify );

```

清单10.1 `xTaskNotifyGive()` API函数原型

参数和返回值

- `xTaskToNotify`

通知发送到的任务的句柄 – 有关获取任务句柄的信息，请参阅 `xTaskCreate()` API 函数的 `pxCreatedTask` 参数。

- `uxIndexToNotify`

输入到数组中的索引

返回值

- `xTaskNotifyGive()` 是一个调用 `xTaskNotify()` 的宏。由宏传递到 `xTaskNotify()` 的参数被设置为 `pdPASS` 是唯一可能的返回值。 `xTaskNotify()` 稍后描述

10.3.3 vTaskNotifyGiveFromISR() API函数

vTaskNotifyGiveFromISR() 是 xTaskNotifyGive() 的一个版本，可在中断服务例程中使用。

```
( TaskHandle_t xTaskNotifyGiveFromISR ( TaskHandle_t xTaskToNotify,
                                         BaseType_t *pxHigherPriorityTaskWoken );
```

清单10.2 函数原型

参数和返回值

•xTaskToNotify

正在发送通知的任务的句柄

•pxHigherPriorityTaskWoken

如果要向其发送通知的任务正在阻塞状态中等待接收通知，则发送通知将导致该任务离开阻塞状态。

如果调用 vTaskNotifyGiveFromISR() 导致任务离开阻塞状态，并且未阻塞任务的优先级高于当前正在执行的任务（被中断的任务）的优先级，则 vTaskNotifyGiveFromISR() 在内部会将 *pxHigherPriorityTaskWoken 设置为pdTRUE。

如果 vTaskNotifyGiveFromISR() 将此值设置为 pdTRUE，则应在退出中断之前执行上下文切换。这将确保中断直接返回到最高优先级的就绪状态任务。

与所有中断安全 API 函数一样，pxHigherPriorityTaskWoken 参数在使用前必须设置为 pdFALSE

10.3.4 ulTaskNotifyTake() API函数

ulTaskNotifyTake() 允许任务在阻塞状态下等待其通知值大于零，并在返回之前递减（减一）或清除任务的通知值。

提供 ulTaskNotifyTake() API 函数以允许将任务通知用作二进制或计数信号量的更轻量级且更快的替代方案。

```
uint32_t ulTaskNotifyTake ( BaseType_t xClearCountOnExit, TickType_t
                             xTicksToWait );
```


清单10.3 `ulTaskNotifyTake()` API函数原型

参数和返回值

• `xClearCountOnExit`

如果 `xClearCountOnExit` 设置为 `pdTRUE`，则在调用 `ulTaskNotifyTake()` 返回之前，调用任务的通知值将被清除为零。

如果 `xClearCountOnExit` 设置为 `pdFALSE`，并且调用任务的通知值大于零，则调用任务的通知值将在调用 `ulTaskNotifyTake()` 返回之前递减

• `xTicksToWait`

调用任务应保持阻塞状态以等待其通知值大于零的最长时间。

阻塞时间以滴答周期为单位指定，因此它表示的绝对时间取决于滴答频率。宏 `pdMS_TO_TICKS()` 可用于将以毫秒为单位的时间转换为以刻度为单位的时间。

将 `xTicksToWait` 设置为 `portMAX_DELAY` 将导致任务无限期待（不会超时），前提是 `FreeRTOSConfig` 中的 `INCLUDE_vTaskSuspend` 设置为 1

• 返回值

返回的值是调用任务在清除为零或递减之前的通知值，如 `xClearCountOnExit` 参数的值所指定。

如果指定了阻塞时间（`xTicksToWait` 不为零），并且返回值不为零，则调用任务可能被置于阻塞状态以等待其通知值大于零，并且其通知值在区块时间到期之前更新。

如果指定了阻塞时间（`xTicksToWait` 不为零），并且返回值为零，则调用任务将被置于阻塞状态以等待其通知值大于零，但指定的阻塞时间已经过期了且没有收到通知

示例10.1 使用任务通知代替信号量，方法 1

例 7.1 使用二进制信号量从中断服务例程中解锁任务——有效地将任务与中断同步。此示例复制了示例 7.1 的功能，但使用直接任务通知来代替二进制信号量。

清单 10.4 显示了与中断同步的任务的实现。例 7.1 中使用的对 `xSemaphoreTake()` 的调用已被对 `ulTaskNotifyTake()` 的调用所取代。

`ulTaskNotifyTake()` 的 `xClearCountOnExit` 参数设置为 `pdTRUE`，这会导致接收任务的通知值在 `ulTaskNotifyTake()` 返回之前被清除为零。因此需要处理

每次调用 `ulTaskNotifyTake()` 之间已经可用的所有事件。在示例 7.1 中，由于使用了二进制信号量，因此必须从硬件确定待处理事件的数量，这并不总是实用的。在例 10.1 中，待处理事件的数量从 `ulTaskNotifyTake()` 返回。

对 `ulTaskNotifyTake` 的调用之间发生的中断事件被锁存在任务的通知值中，并且如果调用任务已经有待处理的通知，则对 `ulTaskNotifyTake()` 的调用将立即返回。

```
/*周期性任务产生软件中断的速率*/
const TickType_t xInterruptFrequency = pdMS_TO_TICKS( 500UL );

static void vHandlerTask( void *pvParameters )
{
    /* xMaxExpectedBlockTime 设置为比事件之间的最大预期时间稍长一些*/
    const TickType_t xMaxExpectedBlockTime = xInterruptFrequency +
                                              pdMS_TO_TICKS( 10 );

    uint32_t ulEventsToProcess;

    /*与大多数任务一样，该任务是在一个无限循环中实现的。*/
    for ( ; ; ) {
        /*等待接收从中断服务程序直接发送到该任务的通知。*/
        ulEventsToProcess = ulTaskNotifyTake( pdTRUE, xMaxExpectedBlockTime );
        if( ulEventsToProcess != 0 )
            /*若要到达这里，必须至少发生了一个事件。在这里循环，直到处理了所有待处理
              的事件（在本例中，只需为每个事件打印一条消息）。*/
            while( ulEventsToProcess > 0 )
            {
                vPrintString( "Handler task - Processing event.\r\n" );
                ulEventsToProcess--;
            }
        else
        {
            /*如果达到了这部分函数，那么中断没有在预期时间内到达，
              并且（在实际应用中）可能需要执行一些错误恢复操作。*/
        }
    }
}
```

清单10.4 实施例10.1中延迟中断处理的任务（与中断同步的任务）的实现

用于生成软件中断的周期性任务在生成中断之前打印一条消息，并在生成中断后再次打印一条消息。这允许在产生的输出中观察执行顺序。

清单 10.5 显示了中断处理程序。除了直接向中断处理延迟的任务发送通知之外，这几乎没有什么作用

```
static uint32_t ulExampleInterruptHandler( void )
{
    BaseType_t xHigherPriorityTaskWoken;

    /*xHigherPriorityTaskWoken 参数必须初始化为 pdFALSE，因为如果需要上下文切换，
    它将在中断安全 API 函数内设置为 pdTRUE*/

    xHigherPriorityTaskWoken = pdFALSE;

    /*直接向中断处理被推迟的任务发送通知*/
    vTaskNotifyGiveFromISR( /*创建任务时保存了该句柄。*/
                           xHandlerTask,

                           &xHigherPriorityTaskWoken );

    /*将 xHigherPriorityTaskWoken 值传递到 portYIELD FROM ISR()。如果
    xHigherPriorityTaskWoken在 vTaskNotifyGiveFromISR() 中设置为 pdTRUE，则调用 portYIELD_
    FROM_ISR() 将请求上下文切换。如果 xHigherPriorityTaskWoken 仍为 pdFALSE，则调用
    portYIELD FROM_ISR() 将无效。Windows 端口使用的 portYIELD_FROM_ISR() 的实现包含 return
    语句，这就是该函数没有显式返回值的原因*/

    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
}
```

清单10.5 示例10.1中使用的中断服务例程的实现

执行例 10.1 时产生的输出如图 10.3 所示。正如预期的那样，它与执行例 7.1 时产生的结果相同。一旦中断产生，vHandlerTask()就进入运行状态，因此任务的输出会分割周期性任务产生的输出。图 10.4 提供了进一步的解释

```

C:\WINDOWS\system32\cmd.exe - rtosdemo
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

Periodic task - About to generate an interrupt.
Handler task - Processing event.
Periodic task - Interrupt generated.

```

图10.3在执行示例7.1时产生的输出

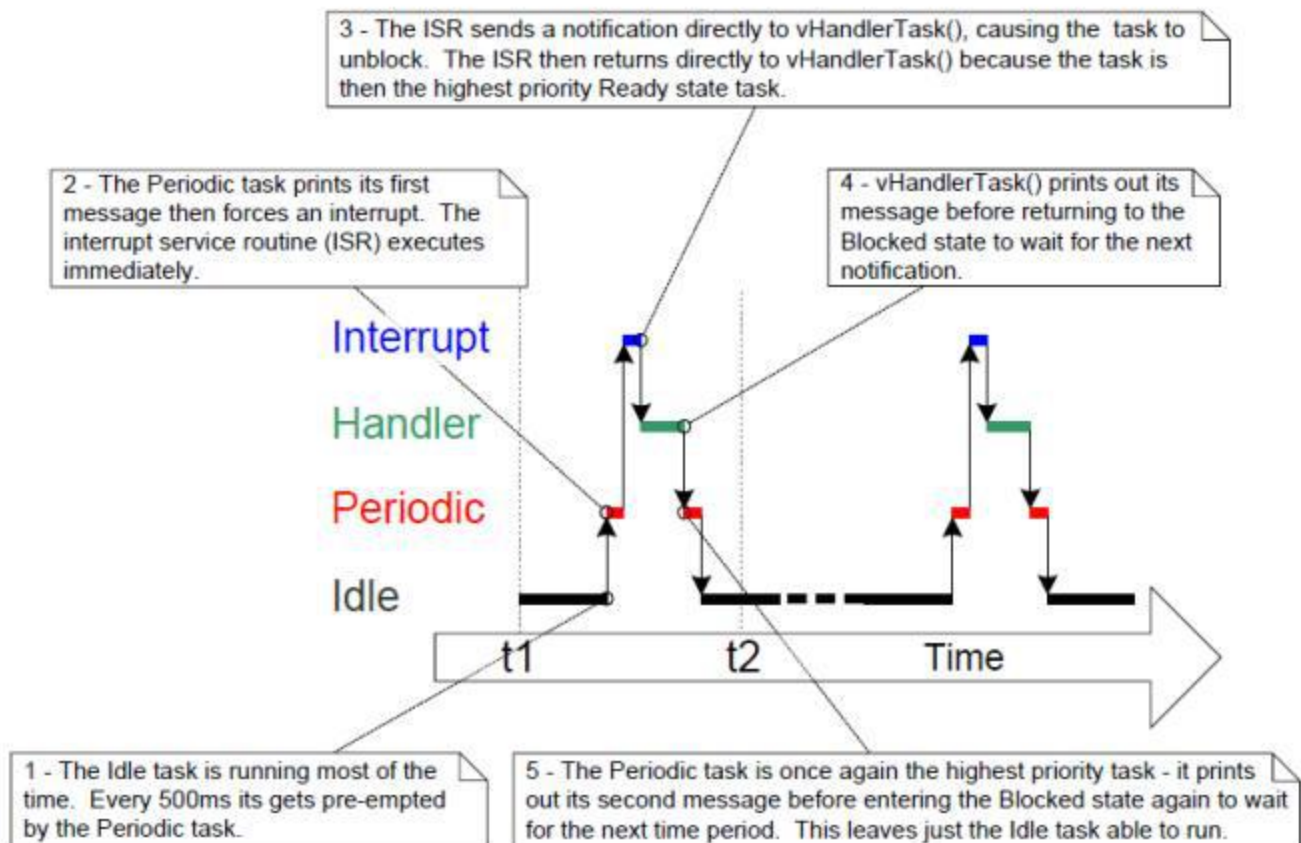


图10.4: 执行示例10.1时的执行顺序

示例10.2 使用任务通知来代替信号量，方法2

在示例 10.1 中，ulTaskNotifyTake() xClearOnExit 参数设置为 pdTRUE。示例 10.1 稍微修改了示例 10.1，以演示当 ulTaskNotifyTake() xClearOnExit 参数设置为 pdFALSE 时的行为。

当 xClearOnExit 为 pdFALSE 时，调用 ulTaskNotifyTake() 只会减少（减一）调用任务的通知值，而不是将其清除为零。因此，通知计数是已发生的事件数与已处理的事件数之间的差。这使得 vHandlerTask() 的结构可以通过两种方式简化：

1. 等待被处理的事件数被保存在通知值中，因此它不需要被保存在本地变量中。
2. 每次调用 ulTaskNotifyTake() 之间只需要处理一个事件

示例10.2中使用的vHandlerTask（）的实现如清单10.6所示。

```
static void vHandlerTask( void *pvParameters )
{
    /*xMaxExpectedBlockTime 设置为比事件之间的最大预期时间稍长一些*/
    const TickType_t xMaxExpectedBlockTime = xInterruptFrequency +
                                                pdMS_TO_TICKS( 10 );

    /*与大多数任务一样，该任务是在一个无限循环中实现的。*/
    for ( ; ; ) {

        if( ulTaskNotifyTake( pdFALSE, xMaxExpectedBlockTime ) != 0 )
        {
            /*要到达这里，必须发生了一个事件。处理该事件（在此情况下，只需打印出一
            条消息）。*/
            vPrintString( "Handler task - Processing event.\r\n" );
        }
        else
        {
            /*如果达到函数的这一部分，那么中断不会在预期的时间内到达，并且（在实际应
            用程序中）
            可能需要执行一些错误恢复操作。*/ }
    }
}
```

清单10.6 实施例10.2中延迟中断处理的任务（与中断同步的任务）的实现

为了演示目的，中断服务例程也被修改为每个中断发送多个任务通知，这样做，模拟以高频发生的多个中断。示例10.2中使用的中断服务例程的实现如清单10.7所示。

```
static uint32_t ulExampleInterruptHandler( void )
{
    BaseType_t xHigherPriorityTaskWoken;

    xHigherPriorityTaskWoken = pdFALSE;
    /*多次向处理程序任务发送通知。第一个“give”将解锁任务，接下来的“give”将演示
    接收任务的通知值用于计数（锁存）事件 - 允许任务依次处理每个事件
    */

    vTaskNotifyGiveFromISR ( xHandlerTask, &xHigherPriorityTaskWoken );
    vTaskNotifyGiveFromISR ( xHandlerTask,&xHigherPriorityTaskWoken );
    vTaskNotifyGiveFromISR ( xHandlerTask,&xHigherPriorityTaskWoken );

    portYIELD_FROM_ISR (xHigherPriorityTaskWoken);
}
```

清单10.7 示例10.2中使用的中断服务例程的实现

执行例 10.2 时产生的输出如图 10.5 所示。可以看出，vHandlerTask()每次产生中断时都会处理所有三个事件

```
C:\WINDOWS\system32\cmd.exe - rtosdemo
Handler task - Processing event.
Handler task - Processing event.
Handler task - Processing event.
Periodic task - Interrupt generated.

Perodic task - About to generate an interrupt.
Handler task - Processing event.
Handler task - Processing event.
Handler task - Processing event.
Periodic task - Interrupt generated.

Perodic task - About to generate an interrupt.
Handler task - Processing event.
Handler task - Processing event.
Handler task - Processing event.
Periodic task - Interrupt generated.

Perodic task - About to generate an interrupt.
Handler task - Processing event.
Handler task - Processing event.
Handler task - Processing event.
```

图10.5 在执行示例10.2时产生的输出

10.3.5 xTaskNotify()和xTaskNotifyFromISR() API函数

xTaskNotify() 是一个更强大的版本，可以通过以下任何一种方式更新接收任务的通知值：

- 增加（加一）接收任务的通知值，在这种情况下 xTaskNotify() 相当于 xTaskNotifyGive()。
- 设置接收任务的通知值中的一位或多位。这允许任务的通知值用作事件组的更轻量级和更快的替代方案
- 将一个全新的数字写入接收任务的通知值，但前提是接收任务自上次更新以来已读取其通知值。这允许任务的通知值提供与长度为 1 的队列提供的功能类似的功能。
- 将一个全新的数字写入接收任务的通知值，即使接收任务自上次更新以来尚未读取其通知值。这允许任务的通知值提供与 xQueueOverwrite() API 函数提供的功能类似的功能。由此产生的行为有时称为“邮箱”。

xTaskNotify() 比 xTaskNotifyGive() 更灵活、更强大，并且由于额外的灵活性和强大功能，它的使用也稍微复杂一些。

xTaskNotifyFromISR() 是 xTaskNotify() 的一个版本，可在中断服务例程中使用，因此具有附加的 pxHigherPriorityTaskWoken 参数。

调用 xTaskNotify() 将始终将接收任务的通知状态设置为挂起（如果尚未挂起）

```

 BaseType_t xTaskNotify ( TaskHandle_t xTaskToNotify,
                          uint32_t ulValue,
                          eNotifyAction eAction );

 BaseType_t xTaskNotifyFromISR ( TaskHandle_t xTaskToNotify,
                                uint32_t ulValue,
                                eNotifyAction eAction,
                                BaseType_t *pxHigherPriorityTaskWoken );

```

列出10.8 xTaskNotify()和xTaskNotifyFromISR() API函数的原型

参数和返回值

- xTaskToNotify

正在发送通知的任务的句柄——请参见xTaskCreate() API函数

- `ulValue`

`ulValue` 的使用方式取决于 `eNotifyAction` 值。见下文。

- `eNotifyAction`

一个枚举类型，指定如何更新接收任务的通知值。见下文。

- 返回值

`xTaskNotify()` 将返回 `pdPASS`，但下面提到的一种情况除外。

- `eNoAction`

接收任务的通知状态被设置为挂起，而不需要更新它的通知值。未使用 `xTaskNotify()` `ulValue`。
`eNoAction` 允许任务通知作为二进制信号量更快更轻的替代品。

有效的 `xTaskNotify()` `eNotifyAction` 参数值及其对接收任务通知值的最终影响

- `eSetBits`

接收任务的通知值是通过在 `xTaskNotify()` `ulValue` 中传递的值进行按位排序的。例如，如果将 `ulValue` 设置为 `0x01`，则将在接收任务的通知值中设置为0位。另一个例子是，如果 `ulValue` 是 `0x06`（二进制 `0110`），则将在接收任务的通知值中设置位1和位2。

`eSetBits` 操作允许任务通知作为事件组更轻的替代。

- `eIncrement`

接收任务的通知值将会递增。未使用 `xTaskNotify()` `ulValue` 参数。

`eIncrement` 操作允许将任务通知用作二进制或计数信号量的更快、更轻量级的替代方案，并且相当于更简单的 `xTaskNotifyGive()` API 函数

- `eSetValueWithoutOverwrite`

如果接收任务在调用 `xTaskNotify()` 之前有待处理的通知，则不执行任何操作，并且 `xTaskNotify()` 将返回 `pdFAIL`。

如果在调用 `xTaskNotify()` 之前接收任务没有待处理的通知，则接收任务的通知值将设置为 `xTaskNotify()` `ulValue` 参数中传递的值

- `eSetValueWithOverwrite`

接收任务的通知值设置为 `xTaskNotify()` `ulValue` 参数中传递的值，无论接收任务在调用 `xTaskNotify()` 之前是否有待处理的通知

10.3.6 e `xTaskNotifyWait()` API函数

`xTaskNotifyWait()` 是 `ulTaskNotifyTake()` 的功能更强大的版本。它允许任务以可选的超时等待调用任务的通知状态变为待处理（如果它尚未处于待处理状态）。

`xTaskNotifyWait()` 提供了在进入函数和退出函数时清除调用任务的通知值中的位的选项。

```
BaseType_t xTaskNotifyWait ( uint32_t      ulBitsToClearOnEntry,
                             uint32_t      ulBitsToClearOnExit,
                             uint32_t      *pulNotificationValue,
                             TickType_t    xTicksToWait );
```

清单10.9 `xTaskNotifyWait()` API函数原型

参数和返回值

•`ulBitsToClearOnEntry`

如果调用任务在调用 `xTaskNotifyWait()` 之前没有待处理的通知，则在进入该函数时，将在任务的通知值中清除 `ulBitsToClearOnEntry` 中设置的任何位。

例如，如果 `ulBitsToClearOnEntry` 为 `0x01`，则任务通知值的位 0 将被清除。

作为另一个示例，将 `ulBitsToClearOnEntry` 设置为 `0xffffffff (ULONG_MAX)` 将清除任务通知值中的所有位，从而有效地将值清除为 0。

•`ulBitsToClearOnExit`

如果调用任务因为收到通知而退出 `xTaskNotifyWait()`，或者因为在调用 `xTaskNotifyWait()` 时已经有待处理的通知，则在任务退出 `xTaskNotifyWait()` 功能。

任务的通知值保存在 `*pulNotificationValue` 中后，这些位将被清除（请参阅下面的 `pulNotificationValue` 的描述）。

例如，如果 `ulBitsToClearOnExit` 为 `0x03`，则任务通知值的位0和位1将在函数退出之前被清除

将 `ulBitsToClearOnExit` 设置为 `0xffffffff (ULONG_MAX)` 将清除任务通知值中的所有位，从而有效地将值清除为 0

•`pulNotificationValue`

用于传递任务的通知值。复制到 `*pulNotificationValue` 的值是任务的通知值，因为它是由于 `ulBitsToClearOnExit` 设置而清除任何位之前的值。

`*pulNotificationValue` 是一个可选参数，如果不需要可以设置为 `NULL`

- `xTicksToWait`

调用任务应保持阻塞状态以等待其通知状态变为挂起状态的最长时间。

区块时间以滴答周期为单位指定，因此它表示的绝对时间取决于滴答频率。宏 `pdMS_TO_TICKS()` 可用于将以毫秒为单位的时间转换为以刻度为单位的时间。

如果 `FreeRTOSConfig.h` 中的 `INCLUDE_vTaskSuspend` 设置为 1，则将 `xTicksToWait` 设置为 `portMAX_DELAY` 将导致任务无限期等待（不会超时）。

- 返回值

有两个可能的返回值：

- `pdTRUE`

这表明 `xTaskNotifyWait()` 返回是因为收到通知，或者因为调用 `xTaskNotifyWait()` 时调用任务已经有待处理的通知。

如果指定了阻塞时间（`xTicksToWait` 不为零），则调用任务可能被置于阻塞状态，以等待其通知状态变为挂起，但其通知状态在阻塞时间之前设置为挂起已到期。

- `pdFALSE`

这表明 `xTaskNotifyWait()` 返回，但调用任务未收到任务通知。

如果 `xTicksToWait` 不为零，则调用任务将保持在阻塞状态以等待其通知状态变为待处理，但在此之前指定的阻塞时间已过期。

10.3.7 外围设备驱动程序中使用的任务通知：UART 示例

外设驱动程序库提供在硬件接口上执行常见操作的函数。通常提供此类库的外设示例包括通用异步接收器和发送器（UART）、串行外设接口（SPI）端口、模数转换器（ADC）和以太网端口。此类库通常提供的函数的示例包括初始化外围设备、向外围设备发送数据以及从外围设备接收数据的函数。

部分外设操作需要较长时间才能完成。此类操作的示例包括高精度 ADC 转换以及 UART 上大数据包的传输。在这些情况下，驱动器库函数可以被实现为轮询（重复读取）外围设备的状态寄存器以确定操作何时完成。

然而，这种方式的轮询几乎总是浪费，因为它使用了 100% 的处理器时间，而没有执行任何生产性处理。这种浪费在多任务系统中尤其昂贵，其中轮询外设的任务可能会阻塞执行具有生产性处理的较低优先级任务。

为了避免浪费处理时间的可能性，高效的 RTOS 感知设备驱动程序应该是中断驱动的，并为长时间操作的任务提供在阻塞状态下等待操作完成的选项。这样，当执行长时间操作的任务处于阻塞状态时，较低优先级的任务可以执行。

RTOS 感知驱动程序库的常见做法是使用二进制信号量将任务置于阻塞状态。清单 10.10 中的伪代码演示了该技术，它提供了在 UART 端口上传输数据的 RTOS 感知库函数的轮廓。在清单 10.10 中：

- xUART 是描述 UART 外设并保存状态信息的结构。该结构体的 xTxSemaphore 成员是 SemaphoreHandle_t 类型的变量。假设信号量已经创建。

xUART_Send() 函数不包含任何互斥逻辑。如果多个任务要使用 xUART_Send() 函数，那么应用程序编写者将必须管理应用程序本身内的互斥。例如，任务可能需要在调用 xUART_Send() 之前获取互斥体。

xSemaphoreTake() API 函数用于在 UART 传输启动后将调用任务置于阻塞状态。

xSemaphoreGiveFromISR() API 函数用于在传输完成后将任务从阻塞状态中移除，此时 UART 外设的发送结束中断服务程序执行。

```
/*驱动程序库功能，将数据发送到UART。*/
```

```
BaseType_t xUART_Send( xUART *pxUARTInstance,
                        uint8_t *pucDataSource,
                        size_t uxLength )
{
    BaseType_t xReturn;

    /*通过尝试在没有超时的情况下获取信号量，确保 UART 的发送信号
    量尚不可用*/

    xSemaphoreTake ( pxUARTInstance->xTxSemaphore, 0 );

    /*开始传输。*/
    UART_low_level_send( pxUARTInstance, pucDataSource, uxLength );

    /*阻塞信号量以等待传输完成。如果获得信号量，则 xReturn 将设置为 pdPASS。如果信号量获取操
    作超时，则 xReturn 将设置为 pdFAIL。 请注意，如果中断发生在调用 UART_low_level_send()
    和调用 xSemaphoreTake() 之间，则该事件将被锁存在二进制信号量中，并且对 xSemaphoreTake()
    的调用将立即返回*/
```

```

    xReturn = xSemaphoreTake ( pxUARTInstance->TxSemaphore,
                               pxUARTInstance->TxTimeout );

    return xReturn;
}
/*-----*/
/*UART 发送结束中断的服务例程，在最后一个字节发送到 UART 后执行
*/

void xUART_TransmitEndISR( xUART *pxUARTInstance )
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    /*清除中断。*/
    UART_low_level_interrupt_clear( pxUARTInstance );

    /*向 Tx 信号量发出传输结束信号。如果任务在等待信号量时处于阻塞状态，
    则该任务将从阻塞状态中移除*/

    xSemaphoreGiveFromISR ( pxUARTInstance->TxSemaphore,
                            &xHigherPriorityTaskWoken );
    portYIELD_FROM_ISR (xHigherPriorityTaskWoken);
}

```

清单10.10 演示如何在驱动程序库传输函数中使用二进制信号量的伪代码

清单10.10中演示的技术是完全可行的，实际上是常见的做法，但它有一些缺点：

- 该库使用多个信号量，这增加了它的RAM占用空间。
- 信号量在创建之前不能使用，因此使用信号量的库在显式初始化之前不能使用。
- 信号量是适用于各种用例的通用对象；它们包含的逻辑允许任意数量的任务在阻塞状态下等待信号量变得可用，并选择（以确定性方式）当信号量变得可用时从阻塞状态中删除哪个任务。执行该逻辑需要花费有限的时间，并且在清单 10.10 所示的场景中，处理开销是不必要的，其中在任何给定时间都不能有多个任务在等待信号量

清单10.11演示了如何通过使用任务通知来代替二进制信号量来避免这些缺点。

注意：如果库使用任务通知，那么库的文档必须清楚地说明调用库函数可以更改调用任务的通知状态和通知值。

在清单10.11中：

- xUART 结构的 xTxSemaphore 成员已被 xTaskToNotify 成员替换。
xTaskToNotify 是一个TaskHandle_t 类型的变量，用于保存等待UART 操作完成的任务的句柄
- FreeRTOS API 函数 xTaskGetCurrentTaskHandle() 用于获取处于运行状态的任务的句柄
- 该库不创建任何FreeRTOS对象，因此它不会产生RAM开销，也不需要显式初始化。
- 任务通知被直接发送到等待UART操作完成的任务，因此不会执行不必要的逻辑。

xUART 结构的 xTaskToNotify 成员可从任务和中断服务例程访问，需要考虑处理器如何更新其值：

- 如果 xTaskToNotify 通过单个内存写入操作进行更新，那么它可以在临界区之外进行更新，正如清单 10.11 所示。如果 xTaskToNotify 是 32 位变量（TaskHandle_t 是 32 位类型），并且运行 FreeRTOS 的处理器是 32 位处理器，则会出现这种情况
- 如果需要多个内存写操作来更新 xTaskToNotify，则 xTaskToNotify 必须只能从临界区内更新，否则中断服务例程可能会在 xTaskToNotify 处于不一致状态时访问它。如果 xTaskToNotify 是 32 位变量，并且运行 FreeRTOS 的处理器是 16 位处理器，就会出现这种情况，因为它需要两次 16 位内存写入操作来更新所有 32 位*/

在内部，在FreeRTOS实现中，TaskHandle_t是一个指针，因此大小（TaskHandle_t）总是等于大小（void *）。

```
/*驱动程序库功能，将数据发送到UART。*/
BaseType_t xUART_Send( xUART *pxUARTInstance,
                        uint8_t *pucDataSource,
                        size_t uxLength)
{
    BaseType_t xReturn;

    /*保存调用此函数的任务的句柄。书中的文本包含了关于以下几行是否需要被临界区
       所保护的注释。*/
    pxUARTInstance->xTaskToNotify = xTaskGetCurrentTaskHandle();

    /*确保调用任务没有尚未等待的通知
```

```

        通过调用 ulTaskNotifyTake() 并将 xClearCountOnExit 参数设置为 pdTRUE, 并将阻塞
        时间设置为 0 (不阻塞)。*/
        ulTaskNotifyTake ( pdTRUE, 0 );

    /*开始传输。*/
    UART_low_level_send( pxUARTInstance, pucDataSource, uxLength );
    /*
    阻塞直到通知传输完成。如果收到通知, 则 xReturn 将设置为 1, 因为 ISR 会将此任务的通知值增加到
    1 (pdTRUE)。如果操作超时, 则 xReturn 将为 0 (pdFALSE), 因为自上面清除为 0 以来, 该任务的通
    知值不会更改。请注意, 如果 ISR 在对 UART_low_level_send() 的调用和对 ulTaskNotifyTake() 的调
    用之间执行, 则该事件将被锁存在任务的通知值中, 并且对 ulTaskNotifyTake() 的调用将立即返回。
    */

    xReturn = ( BaseType_t ) ulTaskNotifyTake ( pdTRUE,
                                                pxUARTInstance->TxTimeout );

    return xReturn;
}
/*-----*/

/*在最后一个字节被发送到UART之后执行的ISR。*/
void xUART_TransmitEndISR( xUART *pxUARTInstance )
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;
    /*除非有任务等待通知, 否则不应执行此函数。使用断言测试此条件。此步骤并非绝对必要,
    但有助于调试。 configASSERT() 在 12.2 节中描述*/

    configASSERT( pxUARTInstance->xTaskToNotify != NULL );

    /*清除中断。*/
    UART_low_level_interrupt_clear( pxUARTInstance );

    /*直接向调用 xUART_Send() 的任务发送通知。
    如果任务处于阻塞等待通知状态, 则该任务将从阻塞状态中移除*/

    vTaskNotifyGiveFromISR ( pxUARTInstance->xTaskToNotify,
                             &xHigherPriorityTaskWoken );

    /*现在没有等待通知的任务。将xUART结构中的xTaskToNotify成员设置为NULL。这一步不
    是严格必要的, 但将有助于调试。*/
    pxUARTInstance->xTaskToNotify = NULL;
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}

```

清单10.11 演示如何在驱动程序库传输功能中使用任务通知的伪代码

任务通知还可以替换接收函数中的信号量，如伪代码清单 10.12 所示，它提供了在 UART 端口上接收数据的 RTOS 感知库函数的轮廓。参见清单 10.12：

- `xUART_Receive()` 函数不包含任何互斥逻辑。如果多个任务要使用 `xUART_Receive()` 函数，那么应用程序编写者将必须管理应用程序本身内的互斥。例如，任务可能需要在调用 `xUART_Receive()` 之前获取互斥体
- UART 的接收中断服务程序将 UART 接收到的字符放入 RAM 缓冲区中。
`xUART_Receive()` 函数从 RAM 缓冲区返回字符
- `xUART_Receive()` `uxWantedBytes` 参数用于指定要接收的字符数。如果 RAM 缓冲区尚未包含所请求的字符数，则调用任务将进入阻塞状态，等待缓冲区中的字符数已增加的通知。`while()` 循环用于重复此序列，直到接收缓冲区包含请求的字符数或发生超时
- 调用任务可能多次进入阻塞状态。因此，阻塞时间会根据调用 `xUART_Receive()` 以来已经过去的时间量进行调整。这些调整可确保 `xUART_Receive()` 内花费的总时间不超过 `xUART` 结构的 `xRxTimeout` 成员指定的阻塞时间。使用 FreeRTOS `vTaskSetTimeOutState()` 和 `xTaskCheckForTimeOut()` 辅助函数来调整阻塞时间。

```
/*驱动程序库函数，以从UART接收数据。*/

size_t xUART_Receive(xUART*pxUART实例,
                    uint8_t *pucBuffer,
                    size_t uxWanted字节)
{
    size_t uxReceived = 0;
    TickType_t xTicksToWait;
    TimeOut_t xTimeOut

    /*记录输入此函数的时间。*/
    vTaskSetTimeOutState ( &xTimeOut );

    /*xTicksToWait 是超时值 - 它最初设置为该 UART 实例的最大接收超时*/

    xTicksToWait = pxUARTInstance->xRxTimeout;

    /*保存调用此函数的任务的句柄。书中的文本包含了关于以下几行是否需要被临界区
       所保护的注释。*/
    pxUARTInstance->xTaskToNotify = xTaskGetCurrentTaskHandle();
```

```

/*循环，直到缓冲区包含所需的字节数；或者
超时发生。*/
while( UART_bytes_in_rx_buffer( pxUARTInstance ) < uxWantedBytes )
{
    /*等待超时，调整 xTicksToWait 以考虑到目前为止在此函数中花费的时间*/

    if( xTaskCheckForTimeOut( &xTimeOut, &xTicksToWait ) != pdFALSE ){
        /*在所需的字节数之前超时，退出循环。*/
        break;
    }

    /*接收缓冲区尚未包含所需的字节数。等待最多 xTicksToWait 时钟周期以通知接收
    中断服务例程已将更多数据放入缓冲区。调用任务在调用此函数时是否已经有待处理
    的通知并不重要，如果有，它只会围绕此 while 循环额外迭代一次*/

    ulTaskNotifyTake ( pdTRUE, xTicksToWait );
}

/*没有任务正在等待接收通知，因此将 xTaskToNotify 设置回 NULL。
书中包含有关以下行是否需要受关键部分保护的注释。*/

pxUARTInstance->xTaskToNotify = NULL;

/*尝试将 uxWantedBytes 从接收缓冲区读入 pucBuffer。
返回实际读取的字节数（可能小于 uxWantedBytes）*/
uxReceived = UART_read_from_receive_buffer( pxUARTInstance,
                                              pucBuffer,
                                              uxWantedBytes );

return uxReceived;
}

/*-----*/

/*UART的接收中断的中断服务例程*/
void xUART_ReceiveISR( xUART *pxUARTInstance )
{
    BaseType_t xHigherPriorityTaskWoken = pdFALSE;

    /*将接收到的数据复制到该 UART 的接收缓冲区并清除中断*/

    UART_low_level_receive( pxUARTInstance );

    /*如果任务正在等待新数据的通知，请立即通知它*/
    if( pxUARTInstance->xTaskToNotify != NULL )

```



```

{
    vTaskNotifyGiveFromISR ( pxUARTInstance->xTaskToNotify,
                             &xHigherPriorityTaskWoken );
    portYIELD_FROM_ISR( xHigherPriorityTaskWoken );
}

```

清单10.12 伪代码演示如何在驱动程序库接收函数中使用任务通知

10.3.8 外围设备驱动程序中使用的任务通知：ADC 示例

上一节演示了如何使用 `vTaskNotifyGiveFromISR()` 将任务通知从中断发送到任务。`vTaskNotifyGiveFromISR()` 是一个使用简单的函数，但其功能有限：它只能将任务通知作为无价值事件发送，不能发送数据。本节演示如何使用 `xTaskNotifyFromISR()` 通过任务通知事件发送数据。清单 10.13 中的伪代码演示了该技术，它提供了模数转换器（ADC）的 RTOS 感知中断服务例程的概要。在清单 10.13 中：

假设至少每 50 毫秒启动一次 ADC 转换。

- `ADC_ConversionEndISR()` 是 ADC 转换结束中断的中断服务程序，每次有新的 ADC 值可用时都会执行该中断。
- `vADCTask()` 实现的任务处理 ADC 生成的每个值。假设创建任务时任务的句柄存储在 `xADCTaskToNotify` 中
- `ADC_ConversionEndISR()` 使用 `eAction` 参数设置为 `eSetValueWithoutOverwrite` 的 `xTaskNotifyFromISR()` 向 `vADCTask()` 任务发送任务通知，并将 ADC 转换结果写入任务的通知值中*/
- `vADCTask()` 任务使用 `xTaskNotifyWait()` 等待新 ADC 值可用的通知，并从其通知值中检索 ADC 转换的结果。

```

/*一个使用ADC的任务。*/
void vADCTask( void *pvParameters )
{
    uint32_t ulADCValue;
    BaseType_t xResult;

    /*触发ADC转换的速率。*/
    const TickType_t xADCConversionFrequency = pdMS_TO_TICKS( 50 );

    for( ; ; )
    {
        /*等待下一个ADC转换结果。*/
        xResult = xTaskNotifyWait(
            /*新的ADC值将覆盖旧的值，因此在等待新的通知值之前不需要清除任何位。*/

```

```

0,

/*未来的 ADC 值将覆盖现有值，因此在退出 xTaskNotifyWait() 之前
无需清除任何位*/
0,
/*任务的通知值（保存最新的 ADC 转换结果）将复制到的变量的地址*/

&ulADCValue,
/*每个 xADCConversionFrequency 滴答应接收新的 ADC 值*/
xADCConversionFrequency * 2 );

if( xResult == pdPASS )
{
    /*收到了一个新的ADC值。现在处理它。*/
    ProcessADCResult ( ulADCValue );
}
else
{
    /*对 xTaskNotifyWait() 的调用未在预期时间内返回，触发 ADC 转换的输入或 ADC
    本身一定有问题。在这里处理错误。*/
}
}
}

/*-----*/

/*每次 ADC 转换完成时执行的中断服务程序*/

void ADC_ConversionEndISR( xADC *pxADCInstance )
{
    uint32_t ulConversionResult;
    BaseType_t xHigherPriorityTaskWoken = pdFALSE, xResult;

    /*读取新的ADC值并清除中断。*/
    ulConversionResult = _ADC_low_level_read( pxADCInstance );

    /*直接发送通知和ADC转换结果到
    vADCTask().*/
    xResult = xTaskNotifyFromISR( xADCTaskToNotify,
                                ulConversionResult, /* ulValue parameter */
                                eSetValueWithoutOverwrite, /* eAction parameter. */
                                &xHigherPriorityTaskWoken );

    /*如果对 xTaskNotifyFromISR() 的调用返回 pdFAIL，则该任务未跟上 ADC 值生成的速率。*/

```

```

    配置ASSERT（），详见第11.2节。*/
    configASSERT ( xResult == pdPASS );
    portYIELD_FROM_ISR (xHighPriorityTaskWoken);
}

```

清单10.13 演示如何使用任务通知向任务传递值的伪代码

10.3.9 直接在应用程序中使用的任务通知

本节通过演示任务通知在包含以下功能的假设应用程序中的使用，增强任务通知的功能：

该应用程序通过慢速互联网连接进行通信，以将数据发送到远程数据服务器并从远程数据服务器请求数据。从现在开始，下文的远程数据服务器被称为云服务器。

请求任务向云服务器请求数据后，必须处于阻塞状态等待接收请求的数据。

发送任务向云服务器发送数据后，必须在阻塞状态等待云服务器正确接收数据的确认

软件设计的原理图如图10.6所示。图10.6：

- 处理与云服务器的多个互联网连接的复杂性被封装在单个 FreeRTOS 任务中。该任务充当 FreeRTOS 应用程序中的代理服务器，称为服务器任务。
- 应用程序任务通过调用CloudRead()从云服务器读取数据。CloudRead()不直接与云服务器通信，而是将读取请求发送到队列中的服务器任务，然后从服务器任务接收请求的数据作为任务通知。
- 应用程序任务通过调用 CloudWrite() 将日期写入云服务器。CloudWrite()不直接与云服务器通信，而是将写入请求发送到队列上的服务器任务，并从服务器任务接收写入操作的结果作为任务通知

CloudRead() 和 CloudWrite() 函数发送到服务器任务的结构如清单 10.14 所示

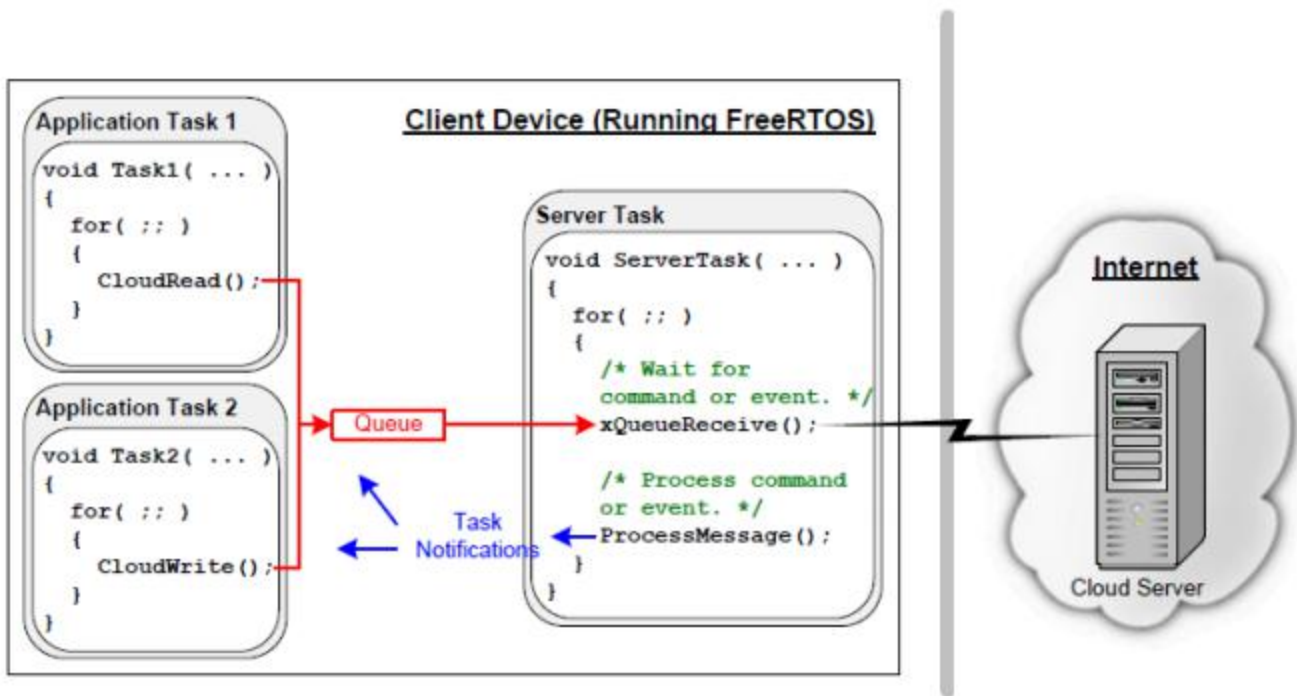


图10.6 从应用程序任务到云服务器，再返回的通信路径

```
typedef enum CloudOperations
{
    eRead, /* Send data to the cloud server. */
    eWrite /* Receive data from the cloud server. */
} Operation_t;

typedef struct CloudCommand
{
    Operation_t eOperation; /*要执行的操作（读取或写）。*/
    uint32_t ulDataID; /*标识正在读取或写入的数据。*/
    uint32_t ulDataValue; /*仅在将数据写入云服务器时使用。*/
    TaskHandle_t xTaskToNotify; /* 执行操作的任务的句柄。*/
} CloudCommand_t;
```

清单10.14 在队列上发送到服务器任务的结构和数据类型

CloudRead() 的伪代码如清单 10.15 所示。该函数将其请求发送到服务器任务，然后调用 xTaskNotifyWait() 在 Blocked 状态下等待，直到收到请求的数据可用的通知。

清单 10.16 中显示了显示服务器任务如何管理读取请求的伪代码。当从云服务器接收到数据时，服务器任务解除对应用程序任务的阻塞，并通过调用 xTaskNotify() 并将 eAction 参数设置为 eSetValueWithOverwrite 将接收到的数据发送到应用程序任务

清单10.16显示了一个简化的场景，因为它假设GetCloudData（）不必等待从云服务器获取值。

```

/* ulDataID标识要读取的数据。pulValue包含要写入从云服务器接收到的数据的变量的地址。*/
 BaseType_t CloudRead ( uint32_t ulDataID, uint32_t *pulValue )
{
    CloudCommand_t xRequest;
    BaseType_t xReturn;

    /*将 CloudCommand_t 结构成员设置为适合此读取请求。*/
    xRequest.eOperation = eRead; /*这是一个要读取数据的请求。*/
    xRequest.ulDataID = ulDataID; /*一个识别要读取的数据的代码。*/
    xRequest.xTaskToNotify = xTaskGetCurrentTaskHandle(); /*调用任务的句柄。*/

    /*通过读取阻塞时间为 0 的通知值，确保没有已挂起的通知，然后将该结构发送到服务器任务。*/

    xTaskNotifyWait( 0, 0, NULL, 0 );
    xQueueSend( xServerTaskQueue, &xRequest, portMAX_DELAY );

    /*等待服务器任务的通知。服务器任务将从云服务器接收到的值直接写入该任务的通知值中，
    因此在进入或退出 xTaskNotifyWait() 函数时无需清除通知值中的任何位。
    接收到的值写入*pulValue，因此pulValue作为通知值写入的地址传递*/

    xReturn = xTaskNotifyWait( 0, /* No bits cleared on entry */
                               0, /* No bits to clear on exit */
                               pulValue, /* Notification value into *pulValue */
                               pdMS_TO_TICKS( 250 ) ); /* Wait 250ms maximum */

    /*如果xReturn为pdPASS，则获得该值。如果xReturn是pdFAIL，则请求将超时。*/
    return xReturn;
}

```

清单10.15 CloudRead() API函数的实现

```

void ServerTask( void *pvParameters )
{
    CloudCommand_t xCommand;
    uint32_t ulReceivedValue;

    for( ;; )

```

```

{
    /*等待从任务接收下一个 CloudCommand_t 结构 */
    xQueueReceive( xServerTaskQueue, &xCommand, portMAX_DELAY );

    switch( xCommand.eOperation ) {
        case eRead:

            /* 从远程云服务器获取请求的数据项 */
            ulReceivedValue = GetCloudData( xCommand.ulDataID );

            /*调用 xTaskNotify() 将通知和从云服务器接收到的值发送到发出请求的任务。
            任务的句柄从CloudCommand_t结构体中获取*/

            xTaskNotify( xCommand.xTaskToNotify, /* The task's handle is in
                                                    the structure */
                        ulReceivedValue, /* Cloud data sent as notification value */
                        eSetValueWithOverwrite );

            break;

            /*其他的switch case写在这里。*/

        }
    }
}

```

清单10.16 处理读取请求的服务器任务

CloudWrite() 的伪代码如清单 10.17 所示。出于演示目的，CloudWrite() 返回一个按位状态代码，其中状态代码中的每一位都分配有唯一的含义。清单 10.17 顶部的 #define 语句显示了四个示例状态位

该任务清除四个状态位，将其请求发送到服务器任务，然后调用 xTaskNotifyWait() 在阻塞状态下等待状态通知。

```

/*CloudWrite()使用的状态位*/
#define SEND_SUCCESSFUL_BIT ( 0x01 << 0 )
#define OPERATION_TIMED_OUT_BIT ( 0x01 << 1 )
#define NO_INTERNET_CONNECTION_BIT ( 0x01 << 2 )
#define CANNOT_LOCATE_CLOUD_SERVER_BIT ( 0x01 << 3 )

/*一个设置了四个状态位的掩码。*/
#define CLOUD_WRITE_STATUS_BIT_MASK ( SEND_SUCCESSFUL_BIT |
                                       OPERATION_TIMED_OUT_BIT |
                                       NO_INTERNET_CONNECTION_BIT |
                                       CANNOT_LOCATE_CLOUD_SERVER_BIT )

```

```

uint32_t CloudWrite ( uint32_t ulDataID, uint32_t ulDataValue )
{
    CloudCommand_t xRequest;
    uint32_t ulNotificationValue;

    /*将CloudCommand_t结构成员设置合适于
    写入请求。*/
    xRequest.eOperation = eWrite; /* This is a request to write data */
    xRequest.ulDataID = ulDataID; /* A code that identifies the data being written */
    xRequest.ulDataValue = ulDataValue; /* Value of the data written to the cloud server. */
    xRequest.xTaskToNotify = xTaskGetCurrentTaskHandle();

    /*通过调用 xTaskNotifyWait() 清除与写操作相关的三个状态位，ulBitsToClearOnExit 参数设置为
    CLOUD_WRITE_STATUS_BIT_MASK，块时间设置为 0。不需要当前通知值，因此 pulNotificationValue
    参数设置为 NULL*/

    xTaskNotifyWait( 0, CLOUD_WRITE_STATUS_BIT_MASK, NULL, 0 );

    /*将请求发送到服务器任务。*/
    xQueueSend( xServerTaskQueue, &xRequest, portMAX_DELAY );

    /*等待服务器任务的通知。服务器任务将按位状态代码写入该任务的通知值，该值写
    入 ulNotificationValue*/

    xTaskNotifyWait( 0,

                     CLOUD_WRITE_STATUS_BIT_MASK,

                     &ulNotificationValue,

                     pdMS_TO_TICKS( 250 )

                     );

    return ( ulNotificationValue & CLOUD_WRITE_STATUS_BIT_MASK );
}

```

清单10.17 CloudWrite() API函数的实现

清单 10.18 中显示了演示服务器任务如何管理写入请求的伪代码。当数据发送到云服务器后，服务器任务会解除对应用程序任务的阻塞，并通过调用 xTaskNotify()（并将 eAction 参数设置为 eSetBits）将按位状态代码发送到应用程序任务。只有 CLOUD_WRITE_STATUS_BIT_MASK 常量定义的位可以在接收任务的通知值中更改，因此接收任务可以将其通知值中的其他位用于其他目的。

清单 10.18 显示了一个简化的场景，因为它假设 SetCloudData() 不必等待从远程云服务器获取确认

```

void ServerTask( void *pvParameters )
{
    CloudCommand_t xCommand;
    uint32_t ulBitwiseStatusCode;

    for( ;; )
    {
        /*等待下一条消息。*/
        xQueueReceive( xServerTaskQueue, &xCommand, portMAX_DELAY );

        /*它是读取请求还是写请求? */
        switch( xCommand.eOperation )
        {
            case eWrite:

                /*将数据发送到远程云服务器。 SetCloudData() 返回一个按位状态代码，仅使用
                CLOUD_WRITE_STATUS_BIT_MASK 定义定义的位（如清单 10.17 所示）*/

                ulBitwiseStatusCode = SetCloudData ( xCommand.ulDataID,
                                                    xCommand.ulDataValue );

                /*向发出写入请求的任务发送通知。
                使用 eSetBits 操作，以便在 ulBitwiseStatusCode 中设置的任何状态位都将在正在通知的任务的通知值中设置。所有其他位保持不变。
                任务的句柄从CloudCommand_t结构体中获取*/

                xTaskNotify( xCommand.xTaskToNotify, /* The task's handle is in the structure. */
                            ulBitwiseStatusCode, /* Cloud data sent as notification value. */
                            eSetBits );

                break;
            /*其他的case*/
        }
    }
}

```

清单10.18 处理发送请求的服务器任务

11 低功耗支持

11.1 节能介绍

FreeRTOS 通过 IDLE 任务挂钩和无滴答空闲模式提供了一种进入低功耗模式的简单方法

通常，通过使用 IDLE 任务挂钩将微控制器置于低功耗状态来减少运行 FreeRTOS 的微控制器的功耗。此方法可实现的节能效果受到定期退出然后重新进入低功耗状态以处理滴答中断的必要性的限制。此外，如果滴答中断的频率太高（从空闲唤醒太频繁），每次滴答进入然后退出低功耗状态所消耗的能量和时间将超过除最轻省电模式之外的所有潜在省电增益。

FreeRTOS 支持低功耗状态，允许微控制器定期进入和退出低功耗状态。FreeRTOS 无滴答空闲模式会在空闲期间（当没有能够执行的应用程序任务时）停止周期性滴答中断，这允许 MCU 保持在深度省电状态，直到发生中断或需要执行以下操作：RTOS 内核将任务转换为就绪状态。然后，当滴答中断重新启动时，它会对 RTOS 滴答计数值进行校正调整。FreeRTOS 的 tickless 模式的原理是当 MCU 执行空闲任务时，使 MCU 进入低功耗模式以节省系统功耗。

11.2 FreeRTOS 睡眠模式

FreeRTOS 支持三种类型的睡眠模式：

1. `eAbortSleep` - 此模式表示任务已准备就绪、上下文切换已挂起或滴答中断已发生但由于调度程序挂起而被挂起。它向 RTOS 发出信号以中止进入睡眠模式
2. `eStandardSleep` - 此模式允许进入睡眠模式，该模式的持续时间不会超过预期的空闲时间
3. `eNoTasksWaitingTimeout` - 当没有任务等待超时进入此模式，因此可以安全地进入只能通过外部中断或复位退出的睡眠模式。

11.3 功能和启用内置无滴答空闲功能

通过在 `FreeRTOSConfig.h` 中将 `configUSE_TICKLESS_IDLE` 定义为 1（对于支持此功能的端口），可以启用内置 Tickless Idle 功能。通过在 `FreeRTOSConfig.h` 中将 `configUSE_TICKLESS_IDLE` 定义为 2，可以为任何 FreeRTOS 端口（包括内置实现的端口）提供用户定义无滴答空闲功能

当启用无滴答空闲功能时，当满足以下两个条件时，内核将调用 `portSUPPRESS_TICKS_AND_SLEEP()` 宏：

1. 空闲任务是唯一能够运行的任务，因为所有的应用程序任务都处于阻塞状态，或者处于挂起状态。
2. 在内核将应用程序任务转换出阻塞状态之前，至少还会经过 n 个完整的时钟周期，其中 n 由 FreeRTOSConfig.h 中的 configEXPECTED_IDLE_TIME_BEFORE_SLEEP 定义设置

11.3.1 portSUPPRESS_TICKS_AND_SLEEP()宏

```
portSUPPRESS_TICKS_AND_SLEEP( xExpectedIdleTime )
```

清单11.1 portSUPPRESS_TICKS_AND_SLEEP宏的原型

portSUPPRESS_TICKS_AND_SLEEP() 中 xExpectedIdleTime 参数的值等于任务进入就绪状态之前的时钟周期总数。因此，该参数值是微控制器可以安全地保持在深度睡眠状态且抑制滴答中断的时间

11.3.2 vPortSuppressTicksAndSleep函数

vPortSuppressTicksAndSleep() 函数在FreeRTOS中定义，可用于实现tickless模式。该函数在 FreeRTOS Cortex-M 端口层中弱定义，可以由应用程序编写者覆盖

```
void vPortSuppressTicksAndSleep( TickType_t xExpectedIdleTime );
```

清单11.2 vPortSuppressTicksAndSleep API函数原型

11.3.3 eTaskConfirmSleepModeStatus函数

API eTaskConfirmSleepModeStatus返回睡眠模式状态，以确定是否可以继续进行睡眠，以及是否可以无限期地进行睡眠。此功能仅在configUSE_TICKLESS_IDLE设置为1时可用。

```
eSleepModeStatus eTaskConfirmSleepModeStatus( void );
```

清单11.3 eTaskConfirmSleepModeStatus API函数原型

如果从 portSUPPRESS_TICKS_AND_SLEEP() 内调用 eTaskConfirmSleepModeStatus() 返回 eNoTasksWaitingTimeout，则微控制器可以无限期地保持在深度睡眠状态。

仅当满足以下条件时，eTaskConfirmSleepModeStatus() 才会返回 eNoTasksWaitingTimeout:

- 软件定时器没有被使用，因此调度程序不会在未来的任何时候执行计时器回调函数。
- 所有应用程序任务要么处于挂起状态，要么处于超时值为 `portMAX_DELAY` 的阻塞状态，因此调度程序不会在未来的任何固定时间将任务从阻塞状态转换出来。

为了避免竞争条件，FreeRTOS 调度程序在调用 `portSUPPRESS_TICKS_AND_SLEEP()` 之前暂停，并在 `portSUPPRESS_TICKS_AND_SLEEP()` 完成时恢复。这确保了应用程序任务无法在微控制器退出低功耗状态和 `portSUPPRESS_TICKS_AND_SLEEP()` 完成其执行之间执行。此外，`portSUPPRESS_TICKS_AND_SLEEP()` 函数需要在计时器停止和进入睡眠模式之间创建一个小的临界区，以确保可以进入睡眠模式。应从此临界区调用 `eTaskConfirmSleepModeStatus()`。

此外，FreeRTOS还为用户提供了在FreeRTOSConfig.h中定义的另外两个接口函数。这些宏允许应用程序编写者分别在单片机进入低功耗状态之前和之后添加其他步骤。

11.3.4 configPRE_SLEEP_PROCESSING配置

```
configPRE_SLEEP_PROCESSING( xExpectedIdleTime )
```

清单11.4 configPRE_SLEEP_PROCESSING宏的原型

在用户能够使单片机进入低功耗模式之前，必须调用`configPRE_SLEEP_PROCESSING()`来配置系统参数，以降低系统功耗，如关闭其他外围时钟，降低系统频率。

11.3.5 configPOST_SLEEP_PROCESSING配置

```
configPOST_SLEEP_PROCESSING( xExpectedIdleTime )
```

清单11.5 configPOST_SLEEP_PROCESSING宏的原型

退出低功耗模式后，用户应调用`configPOST_SLEEP_PROCESSING()`功能，恢复系统的主频率和外围功能。

11.4 实现portSUPPRESS_TICKS_AND_SLEEP() 宏

如果使用的 FreeRTOS 端口不提供 `portSUPPRESS_TICKS_AND_SLEEP()` 的默认实现，则应用程序编写者可以通过在 `FreeRTOSConfig.h` 中定义 `portSUPPRESS_TICKS_AND_SLEEP()` 来提供自己的实现。如果正在使用的 FreeRTOS 端口确实提供了

`portSUPPRESS_TICKS_AND_SLEEP()` 的默认实现，那么应用程序编写者可以通过在 `FreeRTOSConfig.h` 中定义 `portSUPPRESS_TICKS_AND_SLEEP()` 来覆盖默认实现。

以下源代码是应用程序编写者如何实现 `portSUPPRESS_TICKS_AND_SLEEP()` 的示例。该示例是基本的，并且将在内核维护的时间和日历时间之间引入一些滑动。在示例中显示的函数调用中，只有 `vTaskStepTick()` 和 `eTaskConfirmSleepModeStatus()` 是 FreeRTOS API 的一部分。其他功能特定于所用硬件上可用的时钟和省电模式，因此必须由应用程序编写者提供。

```
/*首先定义portSUPPRESS_TICKS_AND_SLEEP()宏。参数是内核接下来需要执行的时间。*/
#define portSUPPRESS_TICKS_AND_SLEEP( xIdleTime ) vApplicationSleep( xIdleTime )

/*定义 portSUPPRESS_TICKS_AND_SLEEP() 调用的函数*/
void vApplicationSleep( TickType_t xExpectedIdleTime )
{
    unsigned long ulLowPowerTimeBeforeSleep, ulLowPowerTimeAfterSleep;

    eSleepModeStatus eSleepStatus;

    /*从将保持可运行的时间源中读取当前时间
    而微控制器则处于低功率状态。*/
    ulLowPowerTimeBeforeSleep = ulGetExternalTime();

    /*停止正在产生滴答中断的计时器。*/
    prvStopTickInterruptTimer();

    /*进入一个不会影响 中断使 MCU 脱离睡眠模式 的临界区
    */
    disable_interrupts();

    /*确保仍然可以进入睡眠模式。*/
    eSleepStatus = eTaskConfirmSleepModeStatus();

    if( eSleepStatus == eAbortSleep )
    {
        /*自执行此宏以来，任务已移出阻塞状态，或者上下文 siwth 处于挂起状态。
        不要进入睡眠状态。重新启动勾选并退出临界区。*/

        prvStartTickInterruptTimer();
        enable_interrupts();
    }
    else
    {
        if( eSleepStatus == eNoTasksWaitingTimeout )
        {
```

```

        /*无需配置中断即可在将来的固定时间使微控制器脱离低功耗状态。*/
        prvSleep();
    }
    else
    {
        /*配置中断，以便在内核下次需要执行时使微控制器脱离低功耗状态。
        中断必须由微控制器处于低功耗状态时保持运行的源生成*/

        vSetWakeTimeInterrupt ( xExpectedIdleTime );

        /*进入低功耗状态。*/
        prvSleep();

        /*确定微控制器实际处于低功耗状态的时间长度，如果微控制器通过
        vSetWakeTimeInterrupt() 调用配置的中断以外的中断退出低功耗模式，则该时间
        将小于 xExpectedIdleTime。
        请注意，调度程序在调用 portSUPPRESS_TICKS_AND_SLEEP() 之前暂停，并在
        portSUPPRESS_TICKS_AND_SLEEP() 返回时恢复。因此，在此函数完成之前不会执
        行其他任务*/

        ulLowPowerTimeAfterSleep = ulGetExternalTime();

        /*纠正内核滴答计数，以考虑到微控制器在其低功耗状态下所花费的时间。
        */
        vTaskStepTick( ulLowPowerTimeAfterSleep - ulLowPowerTimeBeforeSleep );
    }
    /*退出临界区 - 可以在 prvSleep() 调用后立即执行此操作。*/

    enable_interrupts();

    /*重新启动生成滴答中断的定时器*/
    prvStartTickInterruptTimer();
}
}

```

清单11.6 用户定义的`portSUPPRESS_TICKS_AND_SLEEP()`实现示例

11.5 空闲任务钩子函数

空闲任务可以选择调用应用程序定义的钩子（或回调）函数 - 空闲钩子。空闲任务以最低优先级运行，因此只有当没有更高优先级的任务可以运行时，这样的空闲钩子函数才会被执行。这使得 Idle 挂钩函数成为将处理器置于低功耗状态的理想场所。

在没有要执行的处理时提供自动节能功能。仅当 FreeRTOSConfig.h 中的 `configUSE_IDLE_HOOK` 设置为 1 时，才会调用 Idle hook

```
void vApplicationIdleHook( void );
```

清单11.7 vApplicationIdleHook API函数原型

只要空闲任务正在运行，空闲钩子就会被重复调用。最重要的是，空闲钩子函数不会调用任何可能导致其阻塞的 API 函数。此外，如果应用程序使用 vTaskDelete() API 函数，则必须允许空闲任务挂钩定期返回，因为空闲任务负责清理 RTOS 内核分配给已删除任务的资源。

12 开发人员支持

12.1 简介

本章重点介绍了一系列旨在最大化生产力的功能：

- 提供对应用程序如何行为的洞察力。
- 突出了进行优化的机会。
- 在错误发生的点捕获错误。

12.2 configASSERT()

在C中，`configASSERT()`用于验证由程序创建的断言（一个假设）。该断言被写为C表达式，如果该表达式的计算结果为false（0），则认为该断言失败了。例如，清单12.1测试了指针`pxMyPointer`不是NULL的断言。

```
/*测试pxMyPointer不是NULL的断言*/  
configASSERT (pxMyPointer != NULL);
```

清单12.1 使用标准的C `configASSERT()` 宏来检查`pxMyPointer`不是NULL

应用程序编写者通过提供断言宏的实现来指定在断言失败时要采取的操作。

FreeRTOS 源代码不会调用`assert()`，因为并非所有编译FreeRTOS 的编译器都可以使用`assert()`。相反，FreeRTOS 源代码包含对名为 `configASSERT()` 的宏的大量调用，该宏可以由应用程序编写者在 `FreeRTOSConfig.h` 中定义，并且行为与标准 C `assert()` 完全相同

失败的断言必须被视为致命错误。不要尝试执行超过断言失败的行。

使用 `configASSERT()` 可立即捕获和识别许多最常见的错误源，从而提高工作效率。强烈建议在开发或调试 FreeRTOS 应用程序时定义 `configASSERT()`。

定义 `configASSERT()` 将极大地有助于运行时调试，但也会增加应用程序代码的大小，从而减慢其执行速度。如果未提供 `configASSERT()` 的定义，则将使用默认的空定义，并且所有对 `configASSERT()` 的调用将被 C 预处理器完全删除

12.2.1 configASSERT() 定义示例

当应用程序在调试器的控制下执行时，清单 12.2 中所示的 `configASSERT()` 的定义非常有用。它将停止执行在断言失败的行上，因此断言失败的行将是调试器在调试会话暂停时显示的行

```
/*禁用中断，以便滴答中断停止执行，然后进入循环，以便执行不会越过断言失败的行。
如果硬件支持调试中断指令，则可以使用调试中断指令代替 for() 循环
*/

#define configASSERT( x ) if( ( x ) == 0 ) { taskDISABLE_INTERRUPTS(); for(;;); }
```

清单12.2 一个简单的`configASSERT()` 定义，在调试器的控制下执行时很有用

当应用程序不在调试器的控制下执行时，清单 12.3 中所示的 `configASSERT()` 的定义非常有用。它打印出或以其他方式记录断言失败的源代码行。

使用标准 C `__FILE__` 宏来识别断言失败的行，以获取源文件的名称，并使用标准 C `__LINE__` 宏来获取源文件中的行号

```
/*该函数必须在 C 源文件中定义，而不是在 FreeRTOSConfig.h 头文件中 */

void vAssertCalled( const char *pcFile, uint32_t ulLine )
{
    /*在此函数内，pcFile 保存包含检测到错误的行的源文件的名称，ulLine 保存源文件中的行号。
    在进入以下无限循环之前，可以打印出或以其他方式记录 pcFile 和 ulLine 值。*/

    RecordErrorInformationHere( pcFile, ulLine );

    /*禁用中断，以使滴答中断停止执行，然后留在一个循环中，因此执行不会超过断言失败的行。*/
    taskDISABLE_INTERRUPTS();
    for( ;; );
}

/*-----*/

/*以下两行必须放置在 FreeRTOSConfig.h 中。*/

extern void vAssertCalled( const char *pcFile, unsigned long ulLine );
#define configASSERT( x ) if( ( x ) == 0 ) vAssertCalled( __FILE__, __LINE__ )
```

清单12.3 一个`configASSERT()` 定义，它记录了断言失败的源代码行

12.3 FreeRTOS 的 Tracealyzer

Tracealyzer for FreeRTOS 是我们的合作伙伴公司 Percepio 提供的运行时诊断和优化工具。

Tracealyzer for FreeRTOS 捕获有价值的动态行为信息，然后在互连的图形视图中显示捕获的信息。该工具还能够显示多个同步视图。

在分析、故障排除或简单地优化 FreeRTOS 应用程序时，捕获的信息非常宝贵。

Tracealyzer for FreeRTOS 可以与传统调试器并行使用，并以更高级别、基于时间的视角补充调试器的视图。

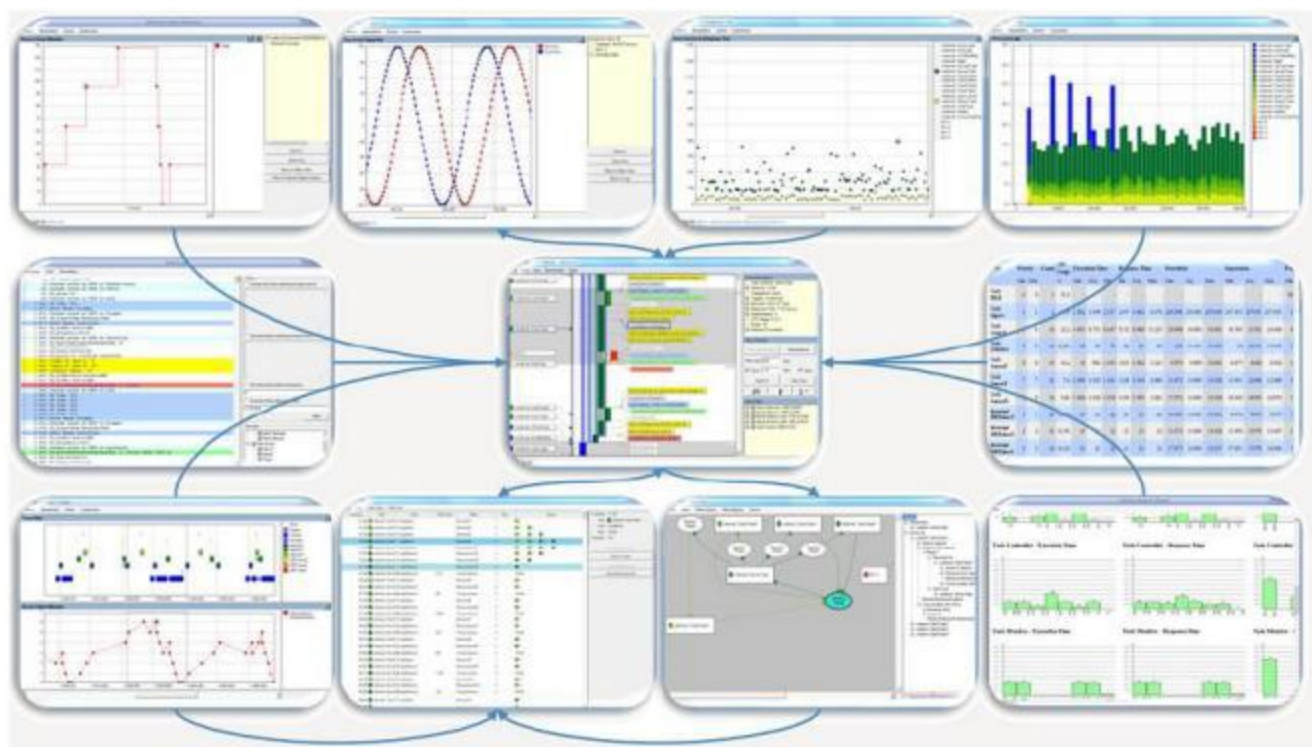


图12.1 FreeRTOS+Trace包括20多个互联视图

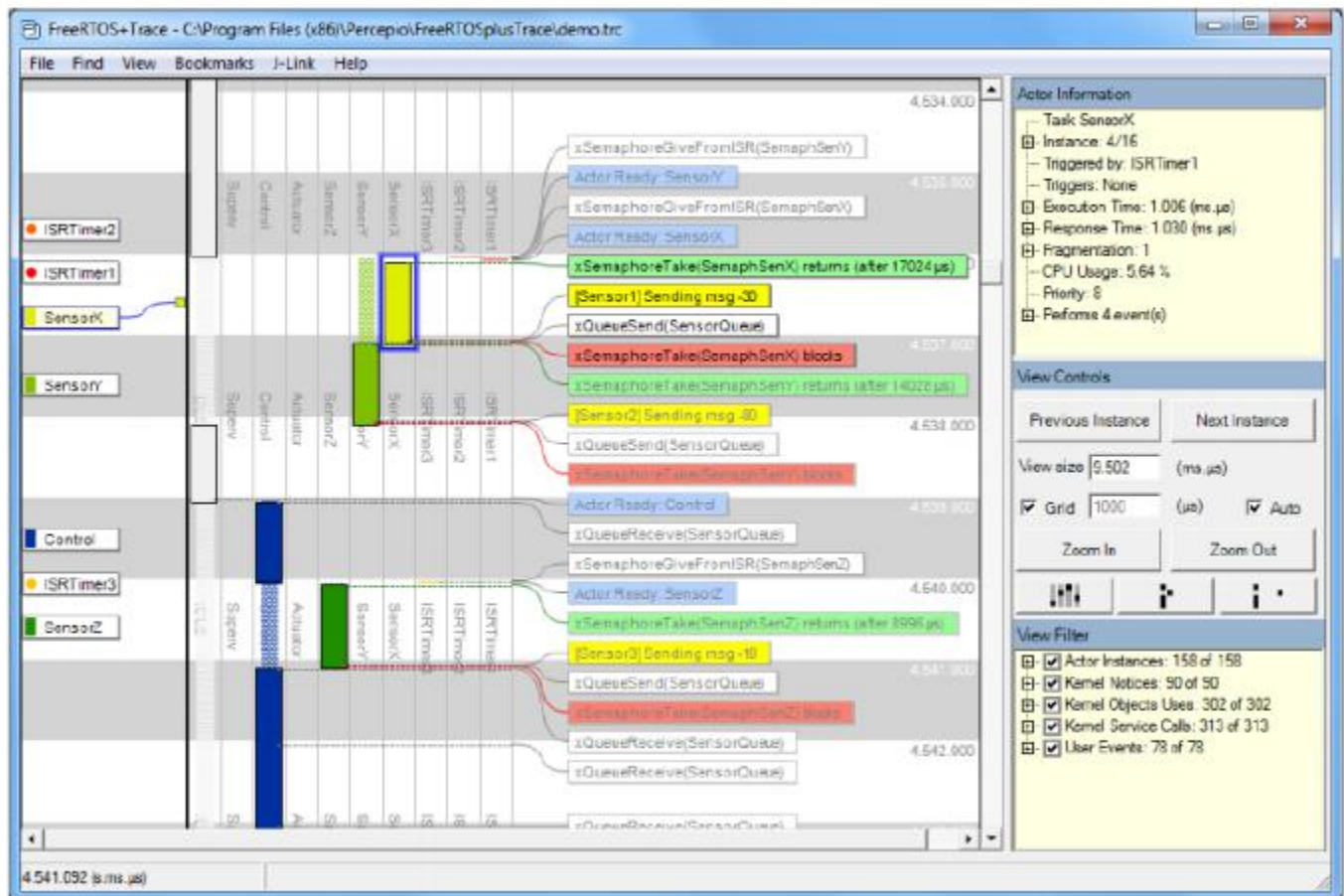


图12.2 FreeRTOS+Trace 主跟踪视图 - 20 多个互连跟踪视图之一

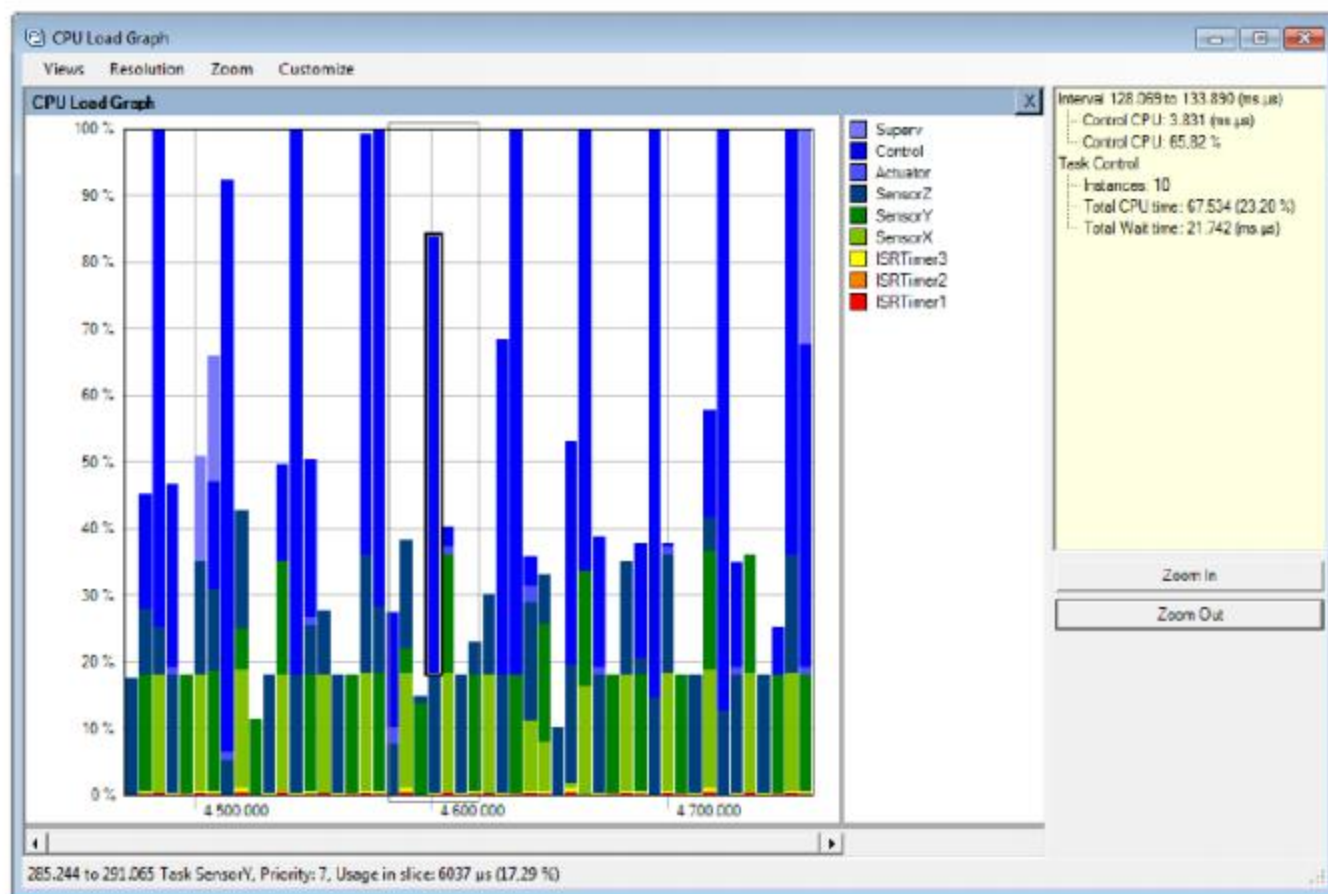


图12.3 FreeRTOS+Trace CPU负载视图——20多个互联的跟踪视图中的一个

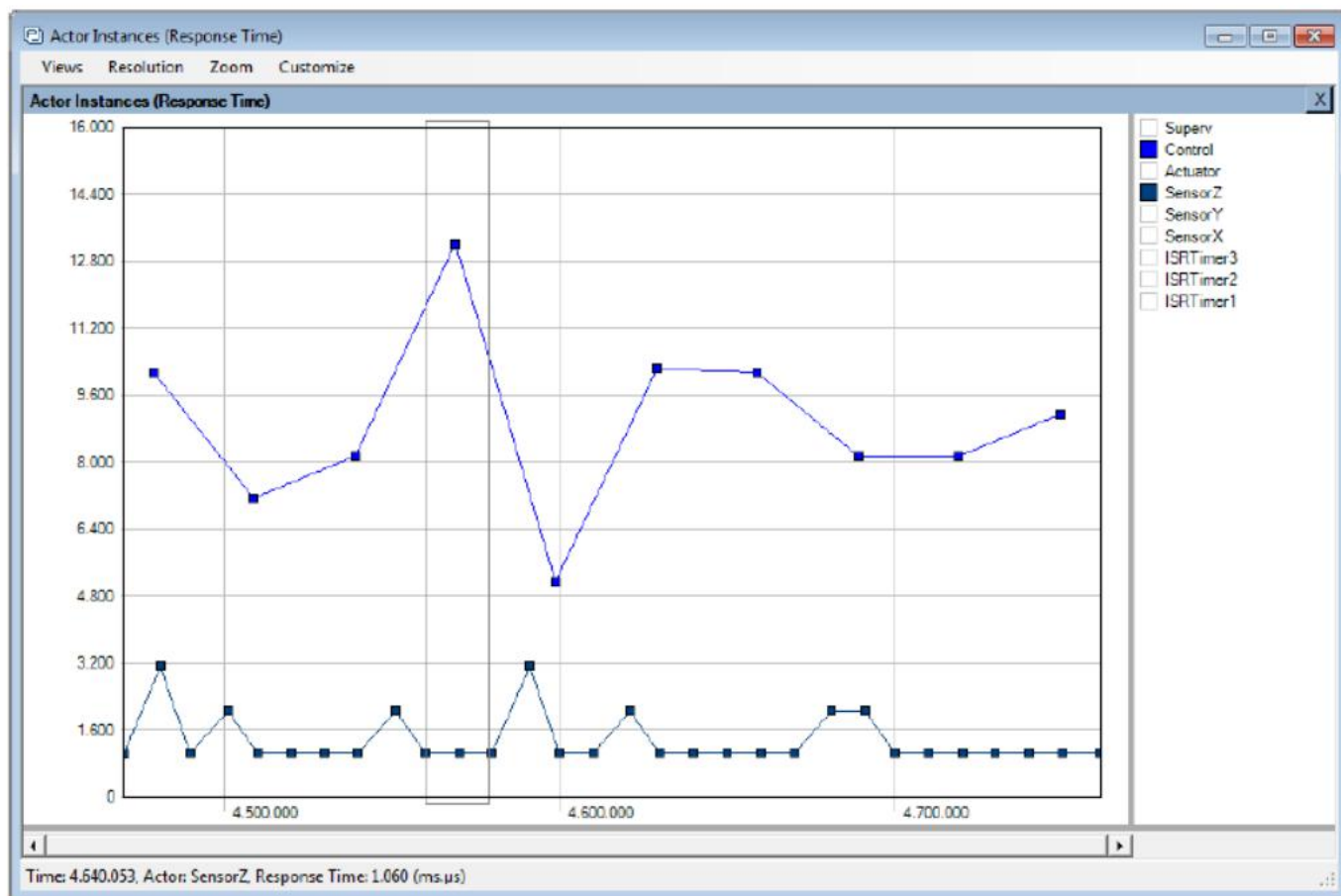


图12.4 FreeRTOS+Trace 响应时间视图-----20多个相互关联的跟踪视图中的一个

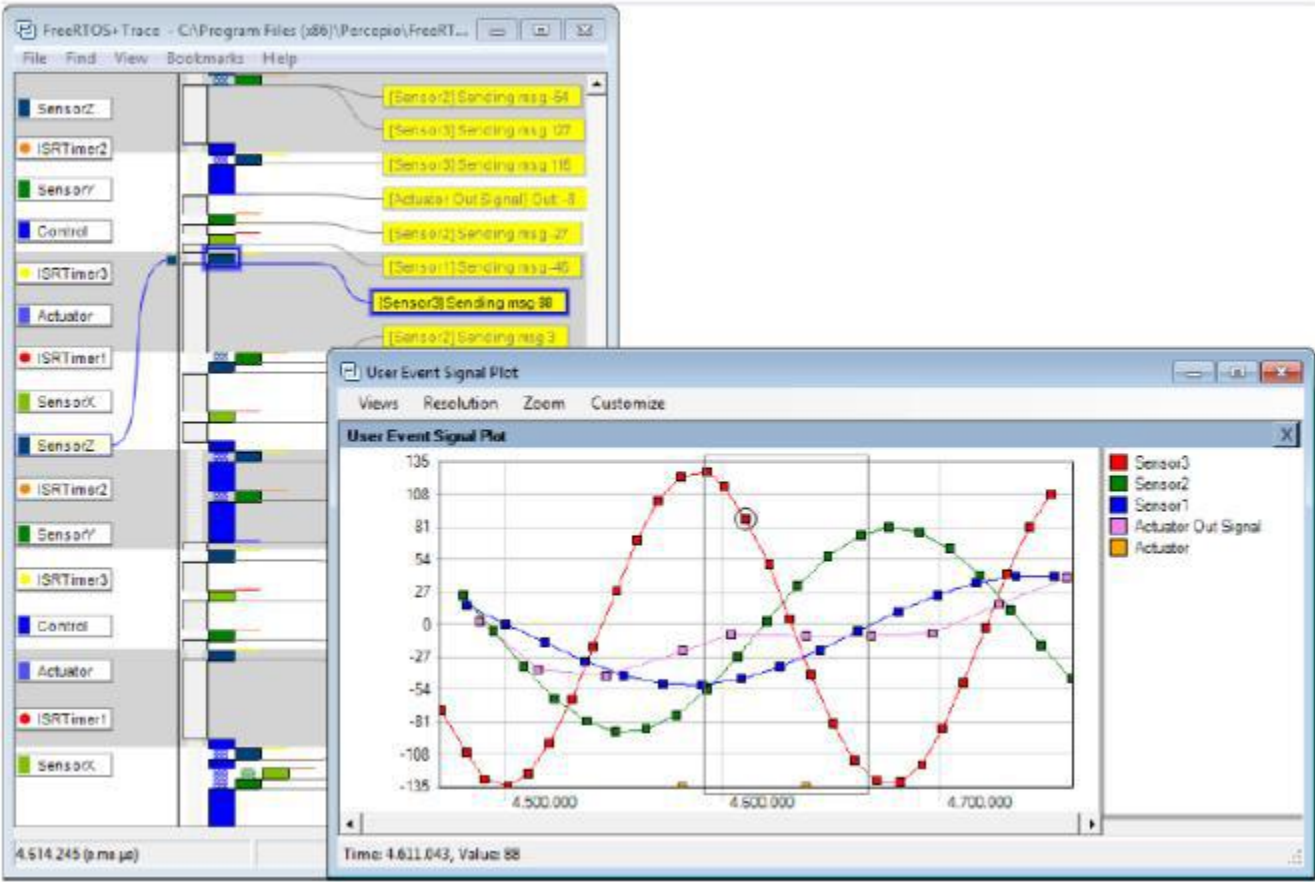


图12.5 FFreeRTOS+Trace 用户事件情节视图——20多个相互关联的跟踪视图中的一个

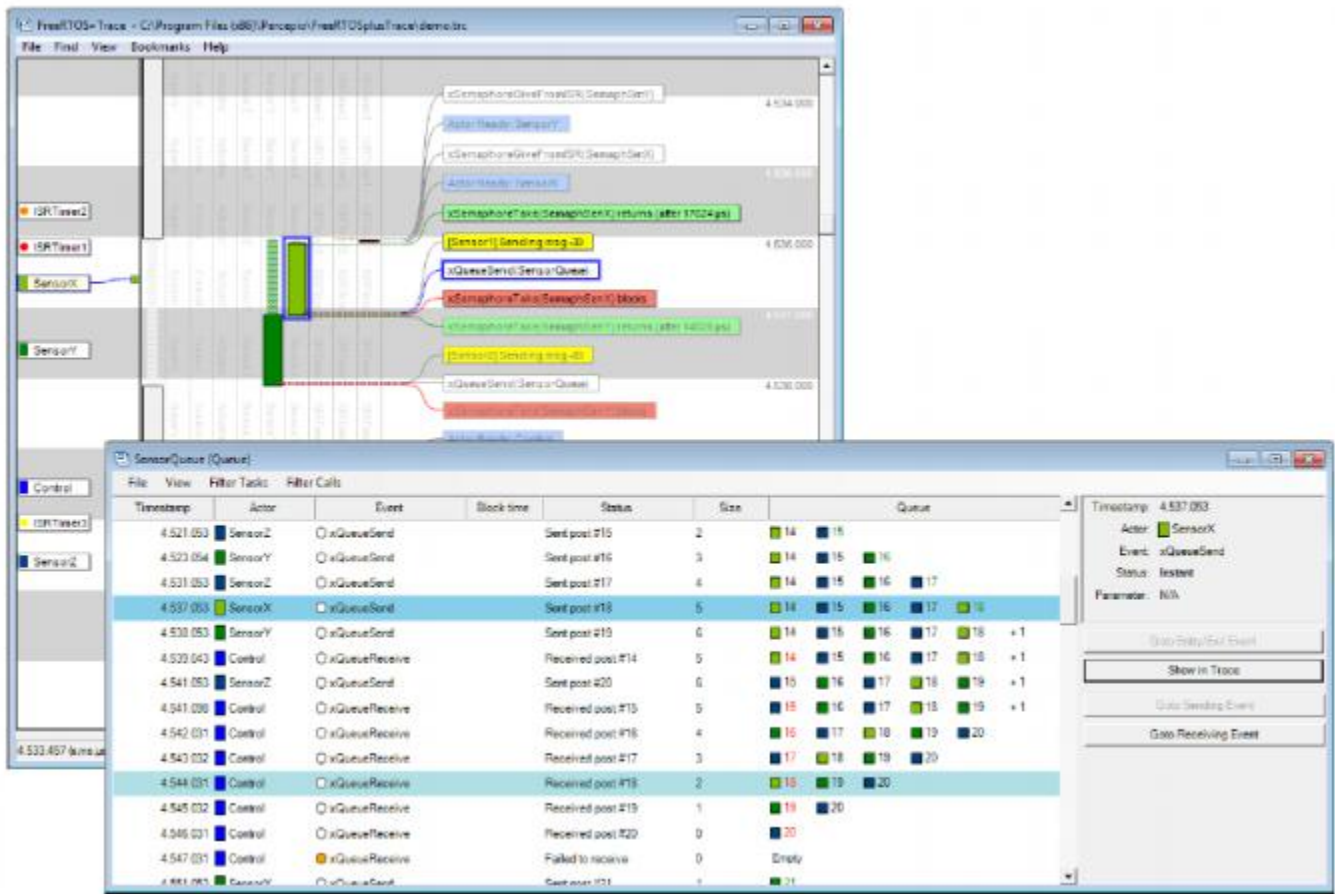


图12.6 FreeRTOS+Trace 内核对象历史记录视图——在20多个相互关联的跟踪视图中的一个

12.4 与调试相关的Hook（回调）功能

12.4.1 Malloc failed hook

malloc失败钩子函数在第3章，堆内存管理中有描述。

定义一个malloc failed hook可以确保在尝试创建任务、队列、信号量或事件组失败时，立即通知应用程序开发人员。

12.4.2 堆栈溢出钩

堆栈溢出挂钩的详细信息在第 13.3 节“堆栈溢出”中提供。

定义堆栈溢出挂钩可确保在任务使用的堆栈量超过分配给该任务的堆栈空间时通知应用程序开发人员。

12.5 查看运行时和任务状态信息

12.5.1 任务运行时统计信息

任务运行时统计信息提供有关每个任务已收到的处理时间量的信息。任务的运行时间是从应用程序启动以来任务处于运行状态的总时间。

运行时统计数据旨在用作项目开发阶段的分析和调试辅助工具。它们提供的信息仅在用作运行时统计时钟的计数器溢出之前有效。

收集运行时统计信息会增加任务上下文切换时间。

要获取二进制运行时统计信息，请调用 `uxTaskGetSystemState()` API 函数。要以人类可读的 ASCII 表形式获取运行时统计信息，请调用 `vTaskGetRunTimeStatistics()` 辅助函数

12.5.2 运行时统计时钟

运行时统计需要测量滴答周期的分数。因此，RTOS 滴答计数不用作运行时统计时钟，而是由应用程序代码提供。建议使运行时统计时钟的频率比滴答中断的频率快 10 到 100 倍。运行时统计时钟越快，统计就越准确，但时间值也就越快溢出。

理想情况下，时间值将由一个自由运行的 32 位外围定时器/计数器生成，其值可以被读取，而没有其他处理开销。如果可用的外设和时钟速度不能使该技术成为可能，那么可替代但效率较低的技术包括：

配置外设以所需的运行时统计时钟频率生成周期性中断，然后使用生成的中断数量计数作为运行时统计时钟。

如果周期性中断仅用于提供运行时统计时钟，则该方法的效率非常低。然而，如果应用程序已经使用了具有合适频率的周期性中断，那么将生成的中断数量的计数添加到现有中断服务例程中是简单且高效的。

通过使用自由运行的 16 位外设定时器的当前值作为 32 位值的最低有效 16 位，并将定时器溢出的次数作为 32 位值的最高有效 16 位来生成 32 位值-位。

通过适当且有些复杂的操作，可以通过将 RTOS 滴答计数与 ARM Cortex-M SysTick 定时器的当前值相结合来生成运行时统计时钟。FreeRTOS 下载中的一些演示项目演示了如何实现这一点

12.5.3 配置一个应用程序以收集运行时统计信息

以下是收集任务运行时统计信息所需的宏的详细信息。最初，这些宏旨在包含在 RTOS 端口层中，这就是宏以“port”为前缀的原因，但事实证明在 `FreeRTOSConfig.h` 中定义它们更为实用

在运行时统计信息中使用的宏的集合

- `configGENERATE_RUN_TIME_STATS`

这个宏必须在 `FreeRTOSConfig.h` 中设置为 1。当此宏设置为 1 时，调度程序将在适当的时间调用本节中详细介绍的其他宏。

- `portCONFIGURE_TIMER_FOR_RUN_TIME_STATS()`

必须提供此宏来初始化用于提供运行时统计时钟的任何外围设备。

- `portGET_RUN_TIME_COUNTER_VALUE()`，或 `portALT_GET_RUN_TIME_COUNTER_VALUE(Time)`

必须提供这两个宏中的一个来返回当前的运行时统计信息时钟值。这是自应用程序首次启动以来以运行时统计时钟单位运行的总时间。

如果使用了第一个宏，则必须定义它来计算当前的时钟值。如果使用了第二个宏，则必须定义它来将它的“时间”参数设置为当前的时钟值。

12.5.4 `uxTaskGetSystemState()` API 函数

`uxTaskGetSystemState()` 提供 FreeRTOS 调度程序控制下每个任务的状态信息快照。该信息以 `TaskStatus_t` 结构数组的形式提供，数组中每个任务都有一个索引。`TaskStatus_t` 由清单 12.5 及下面的内容描述。

```
UBaseType_t uxTaskGetSystemState( TaskStatus_t * const pxTaskStatusArray,
                                   const UBaseType_t uxArraySize,
                                   configRUN_TIME_COUNTER_TYPE * const
                                   pulTotalRunTime );
```

Listing 12.4 `uxTaskGetSystemState()` API 函数原型

注意：`configRUN_TIME_COUNTER_TYPE` 默认为 `uint32_t` 以实现向后兼容性，但如果 `uint32_t` 限制过多，则可以在 `FreeRTOSConfig.h` 中覆盖

参数和返回值

- `pxTaskStatusArray`

一个指向 `TaskStatus_t` 结构数组的指针。

该数组必须为每个任务至少包含一个 `TaskStatus_t` 结构。任务的数量可以使用 `uxTaskGetNumberOfTasks()` API 函数来确定。

`TaskStatus_t` 结构如清单 12.5 所示，`TaskStatus_t` 结构成员在下一个列表中描述。

- `uxArraySize`

`pxTaskStatusArray` 参数指向的数组的大小。大小指定为数组中的索引数（数组中包含的 `TaskStatus_t` 结构的数量），而不是数组中的字节数

• `pulTotalRunTime`

如果 `FreeRTOSConfig.h` 中的 `configGENERATE_RUN_TIME_STATS` 设置为 1，则 `uxTaskGetSystemState()` 将 `*pulTotalRunTime` 设置为自目标启动以来的总运行时间（由应用程序提供的运行时统计时钟定义）

`pulTotalRunTime` 是可选的，如果不需要总运行时间，可以设置为 `NULL`

• 返回值

返回由 `uxTaskGetSystemState()` 填充的 `TaskStatus_t` 结构的数量。
返回值应等于 `uxTaskGetNumberOfTasks()` API 函数返回的数字，但如果 `uxArraySize` 参数中传递的值太小，则返回值为零

```
typedef struct xTASK_STATUS
{
    TaskHandle_t xHandle;
    const char *pcTaskName;
    UBaseType_t xTaskNumber;
    eTaskState eCurrentState;
    UBaseType_t uxCurrentPriority;
    UBaseType_t uxBasePriority;
    configRUN_TIME_COUNTER_TYPE ulRunTimeCounter;
    StackType_t * pxStackBase;
    #if ( ( portSTACK_GROWTH > 0 ) || ( configRECORD_STACK_HIGH_ADDRESS == 1 ) )
        StackType_t * pxTopOfStack;
        StackType_t * pxEndOfStack;
    #endif
    uint16_t usStackHighWaterMark;
    #if ( ( configUSE_CORE_AFFINITY == 1 ) && ( configNUMBER_OF_CORES > 1 ) )
        UBaseType_t uxCoreAffinityMask;
    #endif
} TaskStatus_t;
```

清单12.5 `TaskStatus_t` 结构

`TaskStatus_t` 结构成员

• `xHandle`

结构中的信息所涉及的任务的句柄。

• `pcTaskName`

该任务的人类可读的文本名称。

●xTaskNumber

每个任务都有一个唯一的xTaskNumber值。

如果应用程序在运行时创建并删除任务，那么一个任务可能与之前删除的任务具有相同的句柄。提供了xTaskNumber，以允许应用程序代码和内核感知调试器区分仍然有效的任务和与有效任务具有相同句柄的已删除任务。

●eCurrentState

保存任务状态的枚举类型。 eCurrentState 可以是以下值之一：

- eRunning
- eReady
- eBlocked
- eSuspended
- eDeleted

仅在通过调用 vTaskDelete() 删除任务和空闲任务释放分配给已删除任务的内部数据结构的内存和堆栈之间的短时间内，任务才会被报告为处于 eDeleted 状态。在此之后，该任务将不再以任何方式存在，并且尝试使用其句柄是无效的

●uxCurrentPriority

调用 uxTaskGetSystemState() 时任务运行的优先级。

uxCurrentPriority 只会高于应用程序编写者分配给该任务的优先级，在根据优先级继承机制条件下（第 8.3 节中描述了互斥体（和二进制信号量））暂时为任务分配了更高的优先级

●uxBasePriority

应用程序编写者分配给任务的优先级。仅当 FreeRTOSConfig.h 中的 configUSE_MUTEXES 设置为 1 时，uxBasePriority 才有效

●ulRunTimeCounter

自创建任务以来任务使用的总运行时间。总运行时间以绝对时间的形式提供，该绝对时间使用应用程序编写者提供的时钟来收集运行时统计信息。

仅当 FreeRTOSConfig.h 中的 configGENERATE_RUN_TIME_STATS 设置为 1 时，ulRunTimeCounter 才有效

pxStackBase

指向分配给此任务的堆栈区域的基地址。

pxTopOfStack

指向分配给该任务的堆栈区域的当前顶部地址。仅当堆栈向上增长（即 `portSTACK_GROWTH` 大于零）或 `configRECORD_STACK_HIGH_ADDRESS` 在 `FreeRTOSConfig.h` 中设置为 1 时，字段 `pxTopOfStack` 才有效。

- `pxEndOfStack`

指向分配给该任务的堆栈区域的结束地址。仅当堆栈向上增长（即 `portSTACK_GROWTH` 大于零）或 `configRECORD_STACK_HIGH_ADDRESS` 在 `FreeRTOSConfig.h` 中设置为 1 时，字段 `pxEndOfStack` 才有效。

- `usStackHighWaterMark`

该任务的叠加高水位标记。这是自创建任务以来为任务保留的最小堆栈空间量。它表示任务离溢出堆栈的距离；这个值越接近零，任务就越接近溢出堆栈。指定标记以字节表示。

- `uxCoreAffinityMask`

一个按位值，指示可以运行任务的核心。核心编号从 0 到 `configNUMBER_OF_CORES - 1`。例如，可以在核心 0 和核心 1 上运行的任务将其 `uxCoreAffinityMask` 设置为 0x03。仅当 `FreeRTOSConfig.h` 中 `configUSE_CORE_AFFINITY` 设置为 1 且 `configNUMBER_OF_CORES` 设置为大于 1 时，字段 `uxCoreAffinityMask` 才可用。

12.5.5 `vTaskListTasks()` 助手功能

`vTaskListTasks()` 提供与 `uxTaskGetSystemState()` 类似的任务状态信息，但它将信息显示为人类可读的 ASCII 表，而不是二进制值数组。

`vTaskListTasks()` 是一个处理器密集型函数，会使调度程序长时间挂起。因此，建议仅将该函数用于调试目的，而不是用于生产实时系统。

如果 `FreeRTOSConfig` 中的 `configUSE_TRACE_FACILITY` 设置为 1 并且 `configUSE_STATS_FORMATTING_FUNCTIONS` 设置为大于 0，则 `vTaskListTasks()` 可用

```
void vTaskListTasks( char * pcWriteBuffer, size_t uxBufferLength );
```

Listing 12.6 The `vTaskListTasks()` 函数原型

参数

- `pcWriteBuffer`

指向字符缓冲区的指针，其中写入格式化的和人类可读的表。假设此缓冲区足够大，足以包含生成的报告

每个任务大约有40个字节就足够了。

•`uxBufferLength`

`pcWriteBuffer`的长度。

图12.7显示了由`vTaskListTasks()` 生成的输出示例。在输出中：

- 每一行都提供了关于单个任务的信息。
- 第一列是任务的名称。
- 第二列是任务的状态，其中“X”表示正在运行，“R”表示已准备就绪，“B”表示已阻塞，“S”表示已挂起，“D”表示任务已被删除。只有在调用`vTaskDelete()` 删除任务到空闲任务释放分配给已删除任务的内部数据结构和堆栈的内存之间的短时间内，一个任务才会被报告为处于已删除状态。在此之后，该任务将不再以任何方式存在，并且尝试使用它的句柄是无效的。
- 第三列是该任务的优先级。
- 第四列是任务的堆栈高水位标记。请参见我们的描述。第五列是分配给该任务的唯一编号。请参见 `xTaskNumber` 的描述。

tcpip	R	3	393	0
Tmr Svc	R	3	111	48
QConsB1	R	1	143	3
QProdB5	R	0	144	7
QConsB6	R	0	143	8
PolSEM1	R	0	145	11
PolSEM2	R	0	145	12
GenQ	R	0	155	17
MuLow	R	0	147	18
Rec3	R	0	141	30
SUSP_RX	R	0	148	36
Math1	R	0	167	38
Math2	R	0	167	39

图12.7 由`vTaskListTasks()` 生成的输出示例

注意：
`vTaskListTasks` 的旧版本是 `vTaskList`。 `vTaskList` 假定 `pcWriteBuffer` 的长度为 `configSTATS_BUFFER_MAX_LENGTH`。该函数只是为了向后兼容。

建议新的应用程序使用 `vTaskListTasks` 并显式提供 `pcWriteBuffer` 的长度

```
void vTaskList( signed char *pcWriteBuffer );
```

清单12.7 `vTaskList()` API函数原型

参数

- `pcWriteBuffer`

指向字符缓冲区的指针，其中写入格式化的和人类可读的表。缓冲区必须足够大，以容纳整个表，因为不执行边界检查。

12.5.6 `vTaskGetRunTimeStatistics()` 帮助程序函数

`vTaskGetRunTimeStatistics()` 将收集的运行时统计信息格式化为人类可读的 ASCII 表。

`vTaskGetRunTimeStatistics()` 是一个处理器密集型函数，会使调度程序长时间挂起。因此，建议仅将该函数用于调试目的，而不是用于生产实时系统。

当 `FreeRTOSConfig.h` 中 `configGENERATE_RUN_TIME_STATS` 设置为 1、`configUSE_STATS_FORMATTING_FUNCTIONS` 设置为大于 0 且 `configUSE_TRACE_FACILITY` 设置为 1 时，`vTaskGetRunTimeStatistics()` 可用。

```
void vTaskGetRunTimeStatistics( char * pcWriteBuffer, size_t uxBufferLength );
```

`vTaskGetRunTimeStatistics()` API函数原型

参数

- `pcWriteBuffer`

指向字符缓冲区的指针，其中写入格式化的和人类可读的表。假设此缓冲区足够大，足以包含生成的报告。
每个任务大约有40个字节就足够了。

- `uxBufferLength`

`pcWriteBuffer` 的长度。

图 12.8 显示了 `vTaskGetRunTimeStatistics()` 生成的输出示例。在输出中：

- 第一列是任务名称。
- 第二列是任务在运行状态下所花费的时间。请参见ulRunTimeCounter的描述。
- 第三列是任务在运行状态下所花费的时间占总数的百分比
- 自目标被启动后的时间。显示的百分比次数的总数通常将小于预期的100%，因为统计数据是使用整数计算来收集和计算的，这些整数计算是向下舍入到最接近的整数值。

PolSEM1	994	<1%
PolSEM2	23248	1%
GenQ	194479	16%
MuLow	3690	<1%
Rec3	229450	18%
CNT1	242720	19%
PeekL	94	<1%
CNT_INC	165	<1%
CNT2	243166	20%
SUSP_RX	243192	20%
IDLE	55	<1%

图12.8 函数生成的输出示例输出

注意：
vTaskGetRunTimeStatistics 的旧版本是 vTaskGetRunTimeStats。vTaskGetRunTimeStats 假定 pcWriteBuffer 的长度为 configSTATS_BUFFER_MAX_LENGTH。该函数只是为了向后兼容。建议新应用程序使用 vTaskGetRunTimeStatistics 并显式提供 pcWriteBuffer 的长度

```
void vTaskGetRunTimeStats( signed char *pcWriteBuffer );
```

清单12.9 vTaskGetRunTimeStats() API函数原型

参数

•pcWriteBuffer

指向字符缓冲区的指针，其中写入格式化的和人类可读的表。缓冲区必须足够大，以容纳整个表，因为不执行边界检查。

12.5.7 生成和显示运行时统计数据，一个有效示例

此示例使用假设的 16 位定时器来生成 32 位运行时统计时钟。计数器配置为每次 16 位值达到最大值时生成中断，从而有效地创建溢出中断。中断服务程序对溢出发生的次数进行计数。通过使用溢出发生次数作为 32 位值的两个最高有效字节，以及当前 16 位计数器值作为 32 位值的两个最低有效字节来创建 32 位值。中断服务程序的伪代码如清单 12.10 所示。

```
void TimerOverflowInterruptHandler( void )
{
    /*只是计算一下中断的次数。*/
    ulOverflowCount++;

    /*清除中断。*/
    ClearTimerInterrupt();
}
```

清单12.10 16位计时器溢出中断处理程序，用于计数计时器溢出

清单12.11显示了添加到FreeRTOSConfig.h中的行来启用运行时统计集。

```
/*将configGENERATE_RUN_TIME_STATS设置为1，以启用运行时统计集。当完成此操作时，
   还必须同时定义portCONFIGURE_TIMER_FOR_RUN_TIME_STATS（）和
   portGET_RUN_TIME_COUNTER_VALUE（）或portALT_GET_RUN_TIME_COUNTER_VALUE（x）。*/
#define configGENERATE_RUN_TIME_STATS 1

/* portCONFIGURE_TIMER_FOR_RUN_TIME_STATS（）被定义为调用设置假设的16位计时器的函数
   （没有显示该函数的实现）。*/
void vSetupTimerForRunTimeStats (void);
#define portCONFIGURE_TIMER_FOR_RUN_TIME_STATS() vSetupTimerForRunTimeStats()

/* portALT_GET_RUN_TIME_COUNTER_VALUE() 定义为将其参数设置为当前运行时计数器/时间值
   。返回的时间值是 32 位长，通过将 16 位定时器溢出计数移位到 32 位数字的前两个字节，
   然后将结果与当前 16 位计数器值按位或运算形成。*/

#define portALT_GET_RUN_TIME_COUNTER_VALUE( ulCountValue ) \
{ \
    extern volatile unsigned long ulOverflowCount; \
    \
    /*断开时钟与计数器的连接，以便在使用其值时它不会改变。*/ \
    PauseTimer(); \
    \
    \
    \
}
```

```

/*溢出的数量被转移到返回的32位值中的高两个字节中。*/
ulCountValue = (ulOverflowCount << 16UL );

/*当前计数器值用作返回的32位值中的两个最小有效字节。*/
ulCountValue |= ( unsigned long ) ReadTimerCount();

/*请将时钟重新连接到计数器上。*/
ResumeTimer();

}

```

清单12.11 添加到FreeRTOSConfig.h中的宏，以启用运行时统计信息的集合

清单12.12中所示的任务每隔5秒打印出收集到的运行时统计信息。

```

#define RUN_TIME_STATS_STRING_BUFFER_LENGTH 512

/* 为清楚起见，此代码清单中省略了对 fflush() 的调用。*/
static void prvStatsTask( void *pvParameters )
{
    TickType_t xLastExecutionTime;

    /*用于保存格式化的运行时统计文本的缓冲区需要相当大。因此将其声明为静态以确保
    它不会在任务堆栈上分配。这使得该函数不可重入*/

    static signed char cStringBuffer[ RUN_TIME_STATS_STRING_BUFFER_LENGTH ];

    /*该任务将每隔5秒钟运行一次。*/
    const TickType_t xBlockPeriod = pdMS_TO_TICKS( 5000 );

    /*将 xLastExecutionTime 初始化为当前时间。这是唯一一次需要显式写入该变量。然后，它会在
    vTaskDelayUntil() API 函数内部进行更新。*/

    xLastExecutionTime =xTaskGetTickCount();

    /*与大多数任务一样，这个任务是在一个无限循环中实现的。*/
    for ( ; ; ) {
        /*等待到再次运行此任务。*/
        xTaskDelayUntil ( &xLastExecutionTime, xBlockPeriod );

        /*从运行时统计数据中生成一个文本表。这必须适合于cstring缓冲区数组。*/
        vTaskGetRunTimeStatistics( cStringBuffer, RUN_TIME_STATS_STRING_BUFFER_LENGTH );

        /*请打印出运行时统计数据表的列标题。*/
    }
}

```



```
printf( "\\nTask\\t\\tAbs\\t\\t%%\\n" );

printf( "-----\\n" );

/*打印出运行时统计数据本身。数据表包含多行，因此调用 vPrintMultipleLines() 函数而
不是直接调用 printf()。vPrintMultipleLines() 只需在每一行上单独调用 printf()，以
确保行缓冲按预期工作。*/

vPrintMultipleLines ( cStringBuffer );

}
```

清单12.12 打印出收集到的运行时统计数据的任务

12.6 跟踪钩宏程序

跟踪宏是已放置在FreeRTOS源代码中的关键点上的宏。默认情况下，宏是空的，因此不生成任何代码，也没有运行时开销。通过重写默认的空实现，应用程序编写器可以：

- 将代码插入到FreeRTOS中，而不修改FreeRTOS源文件。
- 通过目标硬件上的任何可用方式，输出详细的执行顺序信息。跟踪宏出现在 FreeRTOS 源代码中足够多的地方，允许使用它们来创建完整而详细的调度器活动跟踪和剖析日志

12.6.1 可用的跟踪钩子宏

在这里详细说明每个宏需要太多的空间。下面的列表详细说明了被认为对应用程序编写器最有用的宏的子集。

下面列表中的许多说明都提到了一个名为 `pxCurrentTCB` 的变量。 `pxCurrentTCB` 是一个 FreeRTOS 私有变量，用于保存处于运行状态的任务句柄，任何从 FreeRTOS/Source/tasks.c 源文件调用的宏都可以使用它。

可以选择最常用的跟踪钩子宏

- traceTASK_INCREMENT_TICK(xTickCount)

在滴答中断期间、滴答计数递增之前调用。 xTickCount 参数将新的滴答计数值传入宏

- traceTASK_SWITCHED_OUT()

在选择运行新任务前调用。此时，pxCurrentTCB 包含即将离开运行状态的任务句柄。

- traceTASK_SWITCHED IN()

在选择任务进行运行后调用。此时，`pxCurrentTCB`包含即将进入“正在运行”状态的任务的句柄。

- `traceBLOCKING_ON_QUEUE_RECEIVE (pxQueue)`

在尝试从空队列中读取数据，或尝试 "take "空信号量或互斥体后，当前执行任务进入阻塞状态前立即调用。`pxQueue` 参数会将目标队列或 semaphore 的句柄传入宏。

- `traceBLOCKING_ON_QUEUE_SEND (pxQueue)`

在当前执行的任务尝试向已满的队列写入后进入阻塞状态之前立即调用。`pxQueue` 参数将目标队列的句柄传入宏

- `traceQUEUE_SEND (pxQueue)`

当队列发送或信号 "给定 "成功时，在 `xQueueSend()`、`xQueueSendToFront()`、`xQueueSendToBack()` 或任何一个信号 "给定 "函数中调用。`pxQueue` 参数会将目标队列或 semaphore 的句柄传入宏

- `traceQUEUE_SEND_FAILED (pxQueue)`

当队列发送或信号 "给定 "操作失败时，在 `xQueueSend()`、`xQueueSendToFront()`、`xQueueSendToBack()` 或任何一个信号 "给定 "函数中调用。如果队列已满，且在指定的阻塞时间内队列一直满，则队列发送或信号 "给予 "将失败。`pxQueue` 参数会将目标队列或寄存器的句柄传入宏

`traceQUEUE_RECEIVE (pxQueue)`

当队列接收或信号 "接收 "成功时，在 `xQueueReceive()` 或任何一个信号 "接收 "函数中调用。`pxQueue` 参数会将目标队列或寄存器的句柄传入宏

`traceQUEUE_RECEIVE_FAILED (pxQueue)`

当队列或寄存器接收操作失败时，在 `xQueueReceive()` 或任何寄存器 "接收 "函数中调用。如果队列或寄存器为空，且在指定的阻塞时间内一直为空，则队列接收或寄存器 "接收 "操作将失败。`pxQueue` 参数会将目标队列或寄存器的句柄传入宏

- `traceQUEUE_SEND_FROM_ISR (pxQueue)`

发送操作成功时从 `xQueueSendFromISR()` 内调用。`pxQueue` 参数将目标队列的句柄传递到宏中。

- `traceQUEUE_SEND_FROM_ISR_FAILED (pxQueue)`

当发送操作失败时从 `xQueueSendFromISR()` 内调用。如果队列已满，发送操作将失败。`pxQueue` 参数将目标队列的句柄传递给宏

- `traceQUEUE_RECEIVE_FROM_ISR (pxQueue)`

当接收操作成功时从 `xQueueReceiveFromISR()` 内调用。 `pxQueue` 参数将目标队列的句柄传递到宏中。

- `traceQUEUE_RECEIVE_FROM_ISR_FAILED (pxQueue)`

当接收操作因队列已为空而失败时，从 `xQueueReceiveFromISR()` 内调用。 `pxQueue` 参数将目标队列的句柄传递给宏

- `traceTASK_DELAY_UNTIL (xTimeToWake)`

在调用任务进入阻塞状态之前立即从 `xTaskDelayUntil()` 中调用

- `traceTASK_DELAY()`

在调用任务进入阻塞状态之前立即从 `vTaskDelay()` 中调用

12.6.2 定义跟踪挂钩宏

每个跟踪宏都有一个默认的空定义。可以通过在 `FreeRTOSConfig.h` 中提供新的宏定义来覆盖默认定义。如果跟踪宏定义变得很长或复杂，那么它们可以在新的头文件中实现，然后该头文件本身包含在 `FreeRTOSConfig.h` 中。

根据软件工程最佳实践，FreeRTOS 维护严格的数据隐藏策略。跟踪宏允许将用户代码添加到 FreeRTOS 源文件中，因此跟踪宏可见的数据类型将不同于应用程序代码可见的数据类型：

- 在FreeRTOS/源代码/任务中。c源文件，一个任务句柄是一个指向数据结构的指针

描述任务（任务的任务控制块或TCB）。在FreeRTOS/源代码/任务之外。c源文件一个任务句柄是一个指向void的指针。

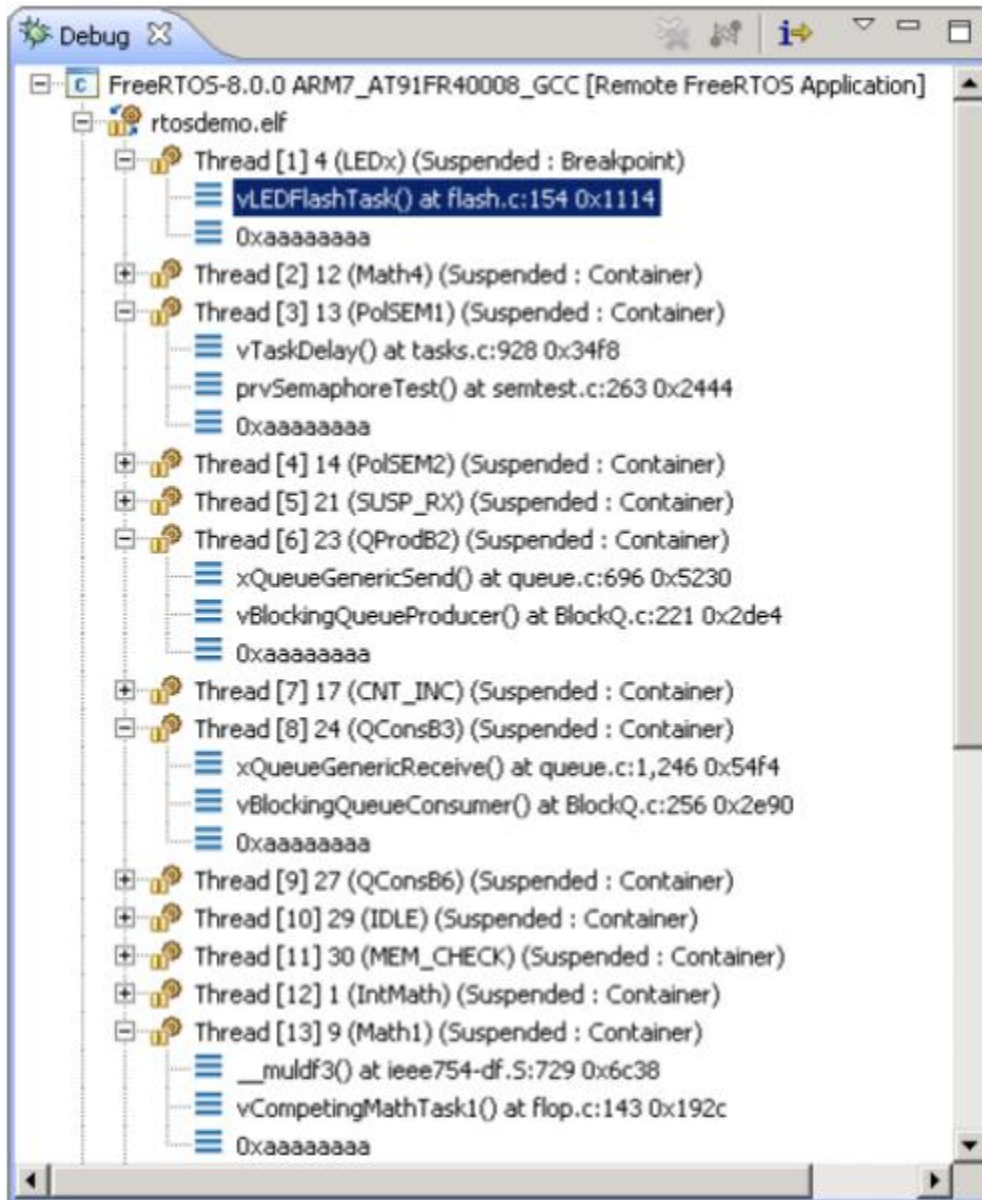
- 在FreeRTOS/源代码/队列中。c源文件，一个队列句柄是一个指向数据结构的指针

描述一个队列。在FreeRTOS/源代码/队列之外。c源文件，一个队列句柄是一个指向void的指针。

如果通常由跟踪宏直接访问私有的FreeRTOS数据结构，则需要非常小心，因为私有的数据结构可能会在FreeRTOS版本之间发生更改。

12.6.3 免费的RTOS感知调试器插件

提供一些 FreeRTOS 感知的插件可用于以下 IDE。此列表可能并不详尽：



- Eclipse (StateViewer)
- Eclipse (ThreadSpy)
- IAR
- ARM DS-5
- Atollic TrueStudio
- Microchip MPLAB
- iSYSTEM WinIDEA
- STM32CubeIDE

13 故障排除

13.1 本章简介及适用范围

本章重点介绍 FreeRTOS 新手用户遇到的最常见问题。首先，它重点关注多年来被证明是最常见的请求支持来源的三个问题：错误的中断优先级分配、堆栈溢出和 `printf()` 的不当使用。然后，它以常见问题解答的形式简要介绍了其他常见错误、其可能的原因及其解决方案。

使用 `configASSERT()` 可立即捕获和识别许多最常见的错误源，从而提高工作效率。强烈建议在开发或调试 FreeRTOS 应用程序时定义 `configASSERT()`。 `configASSERT()` 在 12.2 节中描述

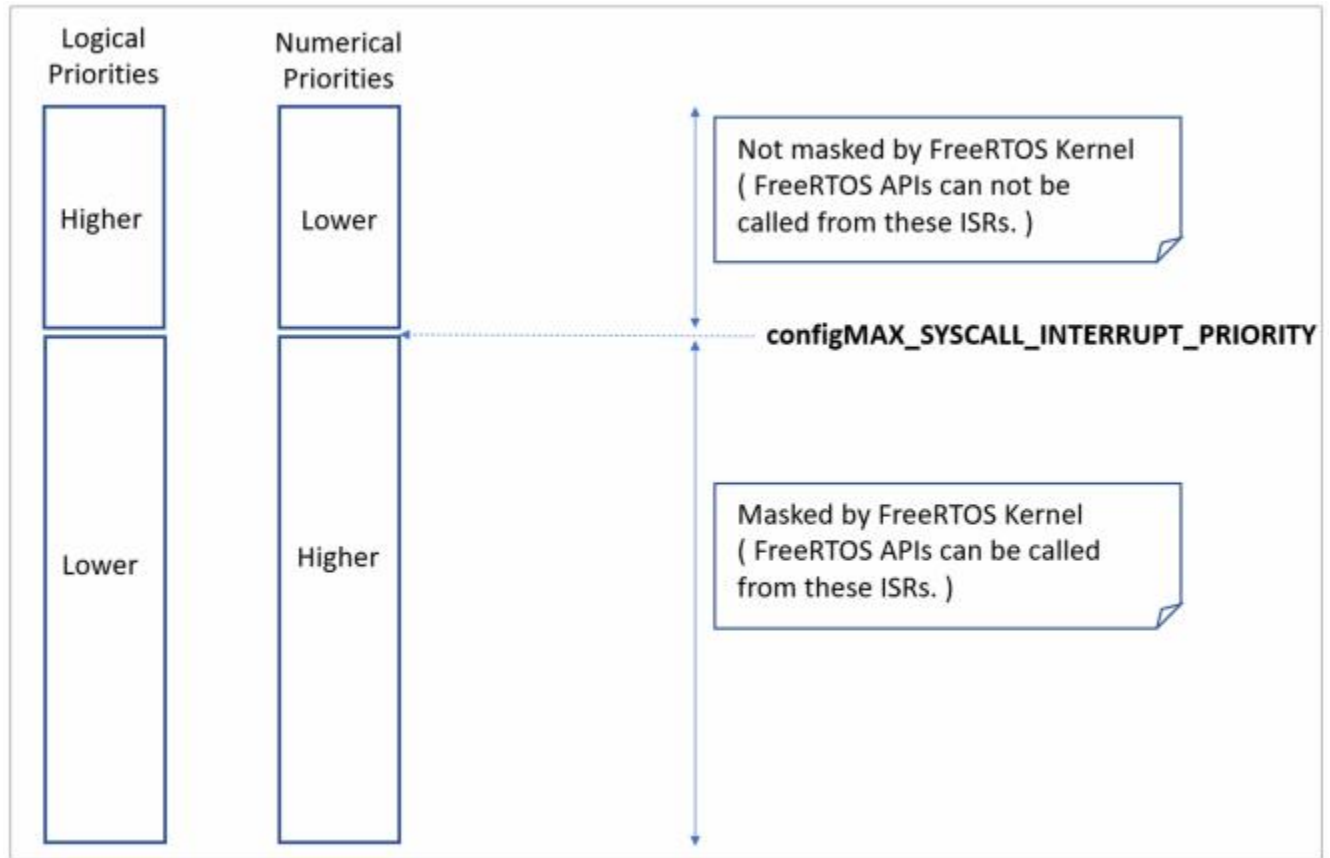
13.2 中断优先级

注意：这是请求支持的首要原因，并且在大多数端口中定义 `configASSERT()` 将立即捕获错误！

如果使用的 FreeRTOS 端口支持中断嵌套，并且中断的服务例程使用 FreeRTOS API，则必须将中断的优先级设置为等于或低于 `configMAX_SYSCALL_INTERRUPT_PRIORITY`，如第 7.8 节“中断嵌套”中所述。如果不这样做将导致临界区无效，进而导致间歇性故障

如果在以下处理器上运行 FreeRTOS，请特别注意：

- 中断优先级默认具有最高可能的优先级，某些 ARM Cortex 处理器（也可能是其他处理器）就是这种情况。在此类处理器上，使用 FreeRTOS API 的中断优先级不能保持未初始化状态
- 数字上的高优先级数字代表逻辑上低的中断优先级，这可能看起来违反直觉，因此会引起混乱。同样，ARM Cortex 处理器以及其他处理器上的情况也是如此
- 例如，在此类处理器上，以优先级 5 执行的中断本身可以被优先级为 4 的中断中断。因此，如果 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 设置为 5，则任何使用 FreeRTOS API 的中断只能分配一个优先级数值大于或等于 5。在这种情况下，中断优先级为 5 或 6 是有效的，但中断优先级为 3 肯定是无效的



- 不同的库实现期望以不同的方式指定中断的优先级。
同样，这与针对 ARM Cortex 处理器的库尤其相关，其中中断优先级在写入硬件寄存器之前会进行位移。有些库会自己执行位移，而其他库则希望在将优先级传递到库函数之前执行位移
- 同一架构的不同实现实现不同数量的中断优先级位。
例如，来自一个制造商的 Cortex-M 处理器可以实现 3 个优先级位，而来自另一制造商的 Cortex-M 处理器可以实现 4 个优先级位
- 定义中断优先级的位可以分为定义抢占优先级的位和定义子优先级的位。
确保分配所有位来指定抢占优先级，以便不使用子优先级。

在一些FreeRTOS端口中，`configMAX_SYSCALL_INTERRUPT_PRIORITY`的另一个名称是 `configMAX_API_CALL_INTERRUPT_PRIORITY`。

13.3 堆栈溢出

堆栈溢出是请求支持的第二大常见来源。FreeRTOS提供了几个特性来帮助捕获和调试与堆栈相关的问题^[28]。

^[28]：这些功能在 FreeRTOS Windows 端口中不可用。

13.3.1 `uxTaskGetStackHighWaterMark()` API函数

每个任务都维护自己的堆栈，其总大小在创建任务时指定。uxTaskGetStackHighWaterMark() 用于查询任务溢出分配给它的堆栈空间的距离。这个值被称为堆栈的“高水位标记”。

```
UBaseType_t uxTaskGetStackHighWaterMark ( TaskHandle_t xTask );
```

清单13.1 uxTaskGetStackHighWaterMark() API函数原型

参数和返回值

•xTask

正在查询其堆栈高水位标记的任务（主题任务）的句柄 - 有关获取任务句柄的信息，请参阅 xTaskCreate() API 函数的 pxCreatedTask 参数。

任务可以通过传递 NULL 代替有效的任务句柄来查询自己的堆栈高水位线

•返回值

任务使用的堆栈量随着任务的执行和中断的处理而增加和减少。
uxTaskGetStackHighWaterMark() 返回自任务开始执行以来可用的最小剩余堆栈空间量。这是当堆栈使用量达到最大（或最深）值时保持未使用的堆栈量。高水位线越接近零，任务就越接近溢出堆栈

可以使用 uxTaskGetStackHighWaterMark2() API 代替 uxTaskGetStackHighWaterMark()，后者仅在返回类型上有所不同。

```
configSTACK_DEPTH_TYPE uxTaskGetStackHighWaterMark2( TaskHandle_t xTask );
```

uxTaskGetStackHighWaterMark2() API函数原型

使用configSTACK_DEPTH_TYPE允许应用程序编写器控制用于堆栈深度的类型。

13.3.2 运行时堆栈检查——概述

FreeRTOS 包括三种可选的运行时堆栈检查机制。这些由 FreeRTOSConfig.h 中的 configCHECK_FOR_STACK_OVERFLOW 编译时配置常量控制。这两种方法都会增加执行上下文切换所需的时间。

堆栈溢出钩子（或堆栈溢出回调）是内核在检测到堆栈溢出时调用的函数。使用堆栈溢出钩子函数：

1. 在 FreeRTOSConfig.h 中将 configCHECK_FOR_STACK_OVERFLOW 设置为 1 、 2 或 3 ， 如以下小节中所述。
2. 提供钩子函数的实现，使用清单 13.3 中所示的确切函数名称和原型

```
void vApplicationStackOverflowHook( TaskHandle_t *pxTask, signed char *pcTaskName );
```

清单13.3 堆栈溢出钩功能原型

提供堆栈溢出钩子是为了更容易地捕获和调试堆栈错误，但是当堆栈溢出发生时，没有真正的方法可以从堆栈溢出中恢复。该函数的参数将已溢出堆栈的任务的句柄和名称传递到钩子函数中。

堆栈溢出钩子从一个中断的上下文中被调用。

一些微控制器在检测到不正确的内存访问时会生成故障异常，并且可能在内核有机会调用堆栈溢出钩子函数之前触发故障。

13.3.3 运行时堆栈检查——方法1

当 configCHECK_FOR_STACK_OVERFLOW 设置为 1 时，选择方法 1。

每次换出任务时，其整个执行上下文都会保存到其堆栈中。这很可能是堆栈使用量达到峰值的时间。当 configCHECK_FOR_STACK_OVERFLOW 设置为 1 时，内核在保存上下文后检查堆栈指针是否保留在有效堆栈空间内。如果发现堆栈指针超出其有效范围，则调用堆栈溢出挂钩。

方法 1 执行速度很快，但可能会错过上下文切换之间发生的堆栈溢出。

13.3.4 运行时堆栈检查——方法2

方法 2 对方法 1 中已描述的检查执行附加检查。当 configCHECK_FOR_STACK_OVERFLOW 设置为 2 时选择该方法

创建任务时，其堆栈会填充已知模式。方法 2 测试任务堆栈空间的最后有效 20 个字节，以验证该模式未被覆盖。如果 20 个字节中的任何一个与预期值发生了变化，就会调用堆栈溢出挂钩函数。

方法 2 的执行速度不如方法 1，但仍然相对较快，因为只测试了 20 个字节。最有可能的是，它会捕获所有堆栈溢出；然而，有可能（但极不可能）错过一些溢出。

13.3.4 运行时堆栈检查——方法3

当configCHECK_FOR_STACK_OVERFLOW设置为3时，选择方法3。

此方法仅适用于选定的端口。如果可用，此方法将启用 ISR 堆栈检查。当检测到 ISR 堆栈溢出时，会触发断言。请注意，在这种情况下不会调用堆栈溢出挂钩函数，因为它特定于任务堆栈而不是 ISR 堆栈

13.4 printf()和sprintf()的使用

通过 printf() 进行日志记录是常见的错误来源，并且应用程序开发人员没有意识到这一点，通常会添加对 printf() 的进一步调用来帮助调试，这样做会加剧问题。

许多交叉编译器供应商将提供适合在小型嵌入式系统中使用的 printf() 实现。即使在这种情况下，实现也可能不是线程安全的，可能不适合在中断服务例程中使用，并且根据输出的定向位置，需要相对较长的执行时间。

如果没有专门为小型嵌入式系统设计的 printf() 实现，并且使用通用 printf() 实现，则必须特别小心，如下所示：

- 仅包含对 printf() 或 sprintf() 的调用即可大量增加应用程序可执行文件的大小。
- printf() 和 sprintf() 可能会调用 malloc()，如果使用除 heap_3 之外的内存分配方案，则该函数可能无效。有关详细信息，请参阅第 3.2 节“内存分配方案示例”
- printf() 和 sprintf() 可能需要比其他方式大很多倍的堆栈

13.4.1 Printf-stdarg.c

许多 FreeRTOS 演示项目使用名为 printf-stdarg.c 的文件，该文件提供了 sprintf() 的最小且堆栈高效的实现，可用于代替标准库版本。在大多数情况下，这将允许为调用 sprintf() 和相关函数的每个任务分配更小的堆栈。

printf-stdarg.c 还提供了一种将 printf() 输出逐个字符定向到端口的机制，虽然速度较慢，但可以进一步减少堆栈使用。

请注意，并非 FreeRTOS 下载中包含的所有 printf-stdarg.c 副本都实现 snprintf()。不实现 snprintf() 的副本只是忽略缓冲区大小参数，因为它们直接映射到 sprintf()。

printf-stdarg.c 是开源的，但由第三方所有，因此与 FreeRTOS 分开许可。

许可条款包含在源文件的顶部

13.5 其他常见的错误来源

13.5.1 症状：向演示中添加一个简单的任务会导致演示崩溃

创建任务需要从堆中获得内存。许多演示应用程序项目将堆的维度完全大到足以创建演示任务——因此，在创建任务之后，将会有剩余的堆不足以用于要添加的任何其他任务、队列、事件组或信号量。

调用 `vTaskStartScheduler()` 时，会自动创建空闲任务（可能还有 RTOS 守护程序任务）。仅当没有足够的堆内存用于创建这些任务时，`vTaskStartScheduler()` 才会返回。包括一个空循环 `[ for(;;); ]` 调用 `vTaskStartScheduler()` 后可以使此错误更容易调试

为了能够添加更多任务，您必须增加堆大小，或删除一些现有的演示任务。
堆大小的增加始终受到可用 RAM 量的限制。有关详细信息，请参阅第 3.2 节“内存分配方案示例”

13.5.2 症状：在中断中使用API函数会导致应用程序崩溃

不要在中断服务例程中使用 API 函数，除非 API 函数的名称以“...FromISR()”结尾。特别是，除非使用中断安全宏，否则不要在中断内创建临界区。有关更多信息，请参阅第 7.2 节“从 ISR 使用 FreeRTOS API”。
在支持中断嵌套的 FreeRTOS 端口中，请勿在已分配高于 `configMAX_SYSCALL_INTERRUPT_PRIORITY` 的中断优先级的中断中使用任何 API 函数。有关更多信息，请参见第 7.8 节“中断嵌套”。

13.5.3 症状：有时应用程序会在中断服务例程中崩溃

首先要检查的是，中断是否不会导致堆栈溢出。有些端口只检查任务中的堆栈溢出，而不检查中断中的堆栈溢出。

中断的定义和使用方式因端口和编译器而异。因此，要检查的第二件事是中断服务例程中使用的语法、宏和调用约定是否与为所使用的端口提供的文档页面上的描述完全一致，并且与随端口提供的演示应用程序中的演示完全一致

如果应用程序运行在使用更小数的字来表示逻辑上高优先级的处理器上，则确保分配给每个中断的优先级考虑到这一点，因为这似乎违反直觉。如果应用程序运行在将每个中断的优先级默认为最大可能优先级的处理器上，请确保每个中断的优先级不保留其默认值。有关更多信息，请参见第 7.8 节“中断嵌套”和第 13.2 节“中断优先级”

13.5.4 症状：尝试启动第一个任务时崩溃

确保已安装 FreeRTOS 中断处理程序。请参阅正在使用的 FreeRTOS 端口的文档页面以获取信息，并参阅为该端口提供的演示应用程序以获取示例。

在启动调度程序之前，某些处理器必须处于特权模式。实现此目的的最简单方法是在调用 `main()` 之前，在 C 启动代码中将处理器置于特权模式

13.5.5 症状：中断被意外禁用，或者临界区未正确嵌套

如果在调度程序启动之前调用 FreeRTOS API 函数，则中断将被故意禁用，并且在第一个任务开始执行之前不会再次重新启用。这样做是为了防止系统在系统初始化期间、调度程序启动之前以及调度程序可能处于不一致状态时尝试使用 FreeRTOS API 函数而导致系统崩溃。

除了调用 `taskENTER_CRITICAL()` 和 `taskEXIT_CRITICAL()` 之外，请勿使用任何方法更改微控制器中断启用位或优先级标志。这些宏保留其调用嵌套深度的计数，以确保仅当调用嵌套完全展开为零时才再次启用中断。请注意，某些库函数本身可能会启用和禁用中断

13.5.6 症状：应用程序甚至在调度程序启动之前就崩溃了

在调度程序启动之前，不得允许执行可能导致上下文切换的中断服务例程。这同样适用于尝试向 FreeRTOS 对象（例如队列或信号量）发送或接收的任何中断服务例程。在调度程序启动之前，上下文切换不会发生。

许多 API 函数只有在调度程序启动后才能调用。最好将 API 使用限制为创建任务、队列和信号量等对象，而不是使用这些对象，直到调用 `vTaskStartScheduler()` 后

13.5.7 症状：在调度程序挂起时或从临界区内部调用API函数会导致应用程序崩溃

调度程序通过调用 `vTaskSuspendAll()` 暂停，并通过调用 `xTaskResumeAll()` 恢复（取消暂停）。通过调用 `taskENTER_CRITICAL()` 进入临界区，并通过调用 `taskEXIT_CRITICAL()` 退出。

不要在调度程序挂起时或从临界区内部调用 API 函数

13.6 其他调试步骤

如果您遇到上述常见原因中未涉及的问题，您可以尝试使用以下一些调试步骤。

定义 `configASSERT()`，在应用程序的 `FreeRTOSConfig` 文件中启用 `malloc` 失败检查和堆栈溢出检查。

检查 FreeRTOS API 的返回值以确保这些值成功。

检查调度程序相关配置（如 `configUSE_TIME_SLICING` 和 `configUSE_PREEMPTION`）是否根据应用程序要求正确设置。

本页提供有关调试 Cortex-M 微控制器上的硬故障的详细信息。